A PRACTICAL, PROJECT-LED CURRICULUM

The Complete JavaScript & React Course

From absolute beginner to production-ready frontend engineer

Build a durable mental model of the web, then turn it into real applications with modern JavaScript, React, APIs, and production practices.

JAVASCRIPT / REACT / APIS / PROJECTS

YOUR LEARNING JOURNEY

What you will learn

A complete pathway from the fundamentals of the browser to shipping accessible, well-structured React applications.

01

Understand the web

Learn how browsers, servers, HTTP, DNS, the DOM, browser APIs, and JSON work together in a real application.

02

Write modern JavaScript

Master variables, control flow, functions, objects, arrays, async code, modules, debugging, and tooling.

03

Work with live data

Use Fetch, promises, async/await, HTTP status codes, storage, forms, and error handling to connect interfaces to APIs.

04

Build with React

Create component-driven interfaces using JSX, props, state, events, effects, forms, refs, context, reducers, and custom hooks.

05

Engineer complete apps

Apply routing, architecture, state management, performance techniques, accessibility, security, testing, and TypeScript fundamentals.

06

Learn by building

Consolidate your skills with guided projects, a capstone inventory and order-management dashboard, assessments, and cheat sheets.

BY THE END: You will be able to plan, build, debug, and improve real frontend applications with confidence.

A textbook, coding course, reference manual, exercise book, project workbook, and interview-preparation resource - combined into one document.

How to use this course: Work through it in order. Every major concept follows the same rhythm: what it is -> why it exists -> a mental model -> syntax -> examples -> common mistakes -> best practices -> exercises -> solutions. Don't skip the exercises - attempt them before revealing the `<details>` solution blocks. Projects appear throughout and increase in difficulty; a large capstone project closes out the React curriculum.

Progress legend: [Beginner] Beginner · [Intermediate] Intermediate · [Advanced] Advanced · [Expert] Expert / Optional Deep Dive

---

Table of Contents

- Part 0 - How The Web Works [Beginner]
- Part 1 - JavaScript From Absolute Beginner Level [Beginner]
- Part 2 - Control Flow [Beginner]
- Part 3 - Functions [Beginner][Intermediate]
- Part 4 - Strings, Numbers, Arrays and Objects [Beginner][Intermediate]
- Part 5 - JavaScript Internal Concepts [Intermediate][Advanced]
- Part 6 - DOM Manipulation [Beginner][Intermediate]
- Part 7 - Forms [Beginner][Intermediate]
- Part 8 - Browser Storage [Beginner][Intermediate]
- Part 9 - Errors and Debugging [Intermediate][Advanced]
- Part 10 - Asynchronous JavaScript [Intermediate][Advanced] (the 20-level Async curriculum)
- Part 11 - APIs [Intermediate][Advanced] (the API Mastery Challenge)
- Part 12 - Modern JavaScript [Intermediate][Advanced]
- Part 13 - Development Tools [Beginner][Intermediate]
- Final JavaScript Assessment [Advanced]
- Part 14 - Introduction to React [Beginner]
- Part 15 - JSX [Beginner]
- Part 16 - Components [Beginner]
- Part 17 - Props [Beginner]
- Part 18 - State [Beginner][Intermediate][Advanced]
- Part 19 - Events in React [Beginner]

- Part 20 - React Forms [Beginner][Intermediate]
- Part 21 - Effects [Intermediate][Advanced]
- Part 22 - API Calls in React [Advanced] (the full loading/error/success model)
- Part 23 - React Hooks [Intermediate][Advanced]
- Part 24 - Refs [Intermediate]
- Part 25 - Context [Intermediate][Advanced]
- Part 26 - Reducers [Intermediate][Advanced]
- Part 27 - Custom Hooks [Advanced]
- Part 28 - Routing [Intermediate][Advanced]
- Part 29 - Application Architecture [Intermediate][Advanced]
- Part 30 - State Management [Advanced]
- Part 31 - Performance [Advanced][Expert]
- Part 32 - Error Handling in React [Intermediate][Advanced]
- Part 33 - Accessibility [Intermediate]
- Part 34 - Security [Intermediate][Advanced]
- Part 35 - Testing [Intermediate][Advanced]
- Part 36 - TypeScript Introduction [Intermediate][Advanced]
- Part 37 - Production Concepts [Intermediate][Advanced]
- The Full Project Catalogue (42 Projects) [Beginner][Intermediate][Advanced][Expert]
  - Tier 1 - Beginner Projects (1-8)
  - Tier 2 - DOM Projects (9-16)
  - Tier 3 - Async/API Projects (17-24)
  - Tier 4 - React Beginner Projects (25-30)
  - Tier 5 - Intermediate React Projects (31-36)
  - Tier 6 - Advanced React Projects (37-42)
- The Capstone Project - Inventory + Order Management Dashboard [Expert]
  - Stages 1-5: Requirements, Data Model, Architecture, Components, Routing
  - Stage 6 - State
  - Stage 9 - Orders

- Stage 12 - Refactoring
- Stage 13 - Testing
- Stage 14 - Performance
- Stage 15 - Production Preparation
  - Final React Assessment [Expert]
  - The Final "Build It Yourself" Challenge [Expert]
  - Cheat Sheets [Beginner][Intermediate][Advanced][Expert] (JS Syntax, Array/Object Methods, DOM, Events, Promises, Async/Await, Fetch/API, HTTP Status, React, React Hooks, Git/npm)
  - Common Errors and Their Meaning [Beginner][Intermediate][Advanced]
  - Final Learning Roadmap [Beginner][Intermediate][Advanced][Expert]

PART 0 - HOW THE WEB WORKS [Beginner]

> *Before you write a single line of JavaScript, you need a mental map of the system JavaScript lives inside. Skipping this is why so many self-taught developers can copy-paste code that works but panic the moment something breaks - they don't know what's actually talking to what.*

0.1 Websites vs Web Applications

What it is

A website is primarily a collection of documents you read. A web application is software you use - it has state, it responds to your actions, it does things.

Why the distinction matters

The tools and techniques you reach for depend on which one you're building. A blog is mostly HTML and CSS with sprinkles of JavaScript. A dashboard, an email client, a to-do app - those are applications, and they need JavaScript to manage state, respond to input, and update the screen without reloading the page.

Mental model

Think of a website as a magazine - you flip through pre-printed pages. A web application is a calculator - you press buttons and it computes and reacts. Most real products today are somewhere on a spectrum between the two, and this course exists specifically to teach you how to build the "calculator" end of that spectrum.

[x] You should now understand: the difference is about behavior, not about which company built it. A blog can have interactive widgets; a "web app" can have static marketing pages. It's a spectrum, not a hard category.

0.2 Frontend and Backend

What it is

- Frontend: everything that runs in the user's browser. This is the code that draws the buttons, handles clicks, shows the layout. HTML, CSS, and JavaScript running client-side.
- Backend: everything that runs on a server, somewhere else, that the frontend talks to. It stores data, enforces business rules, talks to databases, and sends data back when asked.

Why this split exists

The user's browser cannot be trusted. Anyone can open DevTools and change any variable, rewrite any function, or fake any button click. So anything that matters - "does this user actually have ₹500 in their wallet," "is this password correct," "can this user delete this order" - must be checked on a server the user does not control. The frontend's job is to be a nice, fast, responsive interface to that server; the backend's job is to be the source of truth.

Mental model

The frontend is a restaurant's dining room - menus, waitstaff, the visible experience. The backend is the kitchen - where the actual food (data) is prepared and guarded. A customer can shout whatever they want into the dining room, but the kitchen decides what actually gets made.

> *Important: This course is a JavaScript and React (frontend) course. We will talk about backends conceptually wherever necessary (especially in the API sections) but we will not build one. A "backend" in our exercises is usually a public test API or a small mock server.*

0.3 Client and Server

What it is

- Client: the program requesting something. Usually your browser, but could be a mobile app, another server, or a command-line tool like `curl`.
- Server: the program listening for requests and responding to them.

Syntax / how it looks in practice

There's no "syntax" here yet - this is architecture - but here's the shape of the relationship:

```
[ Client (Browser) ]  --- request --->  [ Server ]
[ Client (Browser) ]  <--- response ---  [ Server ]
```

Step-by-step execution

1. You type a URL into your browser, or your JavaScript code calls `fetch(...)`.

2. The browser (the client) packages that into an HTTP request and sends it across the network.
3. A server somewhere receives that request, does some work (reads a database, runs some logic).
4. The server sends back an HTTP response.
5. The browser receives the response and does something with it - renders a page, or hands the data to your JavaScript.

Common mistake

[Advanced] Broken mental model: "My website talks directly to the database."

Why it's wrong: Your frontend JavaScript should never talk directly to a production database. It talks to a server, and the server talks to the database. If your frontend had direct database credentials, anyone could open DevTools, read those credentials out of the JavaScript bundle, and do whatever they want to your data.

[x] Correct mental model: Browser (client) -> API server -> Database. The server is the gatekeeper.

0.4 HTTP, Requests, and Responses

What it is

HTTP (HyperText Transfer Protocol) is the language clients and servers use to talk to each other. It defines the format of requests and responses so that any client can talk to any server, regardless of what they're built with.

Mental model

HTTP is like a standardized order form at a restaurant. It doesn't matter which waiter takes your order or which chef cooks it - as long as everyone fills out the order form (HTTP) the same way, the system works.

Anatomy of a request

```
GET /api/users/42 HTTP/1.1
Host: example.com
Authorization: Bearer abc123
Accept: application/json
```

- Method: `GET` - what kind of action (more on this in Part 11).
- Path: `/api/users/42` - which resource.
- Headers: metadata about the request (`Authorization`, `Accept`, etc.).
- Body (not shown here, used in `POST`/`PUT`/`PATCH`): the actual data being sent.

Anatomy of a response

```
HTTP/1.1 200 OK
Content-Type: application/json

{
```

```
  "id": 42,
  "name": "Ammaar"
}
```

- Status code: `200 OK` - did it work? (Deep dive in Part 11.)
- Headers: metadata about the response.
- Body: the actual data.

We will go far deeper into HTTP in Part 11 - APIs, because it's one of the most important topics in this entire course. For now, just internalize: the web runs on a constant cycle of requests and responses.

0.5 URL and Domain

What it is

A URL (Uniform Resource Locator) is the address of a specific resource on the web.

```
https://example.com:443/products/42?color=red#reviews
└──┘   └─────────┘└┘└──────────┘└──────┘└─────┘
scheme     domain  port      path        query   fragment
```

Part	Meaning
`https://`	The scheme/protocol - how to talk to the server (secure HTTP)
`example.com`	The domain - which server to talk to
`:443`	The port (443 is default for HTTPS, usually hidden)
`/products/42`	The path - which resource on that server
`?color=red`	The query string - extra parameters
`#reviews`	The fragment - a location within the page (handled by the browser, not sent to the server)

Why it exists

Without a shared addressing scheme, there'd be no way to say "I want this specific thing on this specific server." A URL is a complete, unambiguous address.

0.6 DNS (Conceptual)

What it is

Computers route traffic using numeric IP addresses (like `142.250.183.14`), not domain names. DNS (Domain Name System) is the system that translates human-friendly domain names (`example.com`) into IP addresses.

Mental model

DNS is the phonebook of the internet. You know a business by its name; DNS looks up the actual "phone number" (IP address) so your computer knows where to send the request.

Step-by-step execution (simplified)

1. You type `example.com` in the browser.
2. Your computer asks a DNS server: "what's the IP address for `example.com`?"
3. The DNS server responds with something like `93.184.216.34`.
4. Your browser opens a connection to that IP address and sends the HTTP request there.

You don't need to configure DNS to learn JavaScript - but understanding this removes a huge amount of mystery around how "typing a name" turns into "talking to a specific machine somewhere in the world."

0.7 HTML, CSS, and JavaScript - The Three Pillars

What they are

Technology	Job	Analogy
HTML	Structure and content - what is on the page	The skeleton of a body
CSS	Presentation - how it looks	The skin, clothes, styling
JavaScript	Behavior - how it responds and changes	The muscles and nervous system

Why they're separate

Separating structure, style, and behavior means you can change how something looks (CSS) without touching what it means (HTML) or how it behaves (JavaScript). This separation of concerns is a recurring theme throughout this entire course - you'll see it again in React's architecture, in API layers, and in file structure.

How they work together - step by step

1. Browser requests an HTML file.
2. Browser starts parsing the HTML top to bottom, building the DOM (explained next).
3. When the parser hits a `<link rel="stylesheet">`, it fetches that CSS and applies styling rules to the DOM it's building - this produces something called the render tree internally, but you don't need that term day-to-day.
4. When the parser hits a `<script>` tag, by default it pauses HTML parsing, downloads (if external) and executes the JavaScript immediately, then resumes parsing. (This is why `defer` exists - see below.)
5. Once everything is parsed and styled, the browser paints pixels to the screen.

6 From then on, JavaScript can modify the DOM at any time - adding elements, changing text, responding to clicks - and the browser re-renders the parts that changed.

> *Tip: This is the entire reason JavaScript is powerful on the web: it's the only one of the three languages that can run again and again, in response to events, after the page has loaded. HTML and CSS are (mostly) declared once; JavaScript is alive.*

0.8 The JavaScript Engine

What it is

A JavaScript engine is the program that actually reads your JavaScript code and executes it. Browsers have one built in. Chrome and Node.js use V8. Firefox uses SpiderMonkey. Safari uses JavaScriptCore.

Why it matters to you

You don't write engine code, but understanding that "JavaScript" is not magic - it's a program, written mostly in C++, that parses your text file and executes it step by step - demystifies a lot of what's coming, especially in Part 5 (execution context, call stack) and Part 10 (the event loop).

Mental model

Think of the JS engine as a very literal, very fast reader who reads your script top to bottom (with some important exceptions you'll learn about - hoisting, async) and does exactly what each line says, no more, no less.

0.9 The DOM (Document Object Model)

What it is

When the browser parses your HTML, it doesn't just display it - it builds a tree of objects in memory that represents the page. This tree is called the DOM. Every HTML tag becomes a node (an object) in this tree, and JavaScript can read and modify those objects.

Why it exists

HTML is just text. Once parsed into the DOM, the page becomes something JavaScript can interact with as data - read a value, change a class, insert a new paragraph, delete an image - all in response to user actions, without asking the server for a whole new page.

Mental model

If HTML is a recipe (instructions written on paper), the DOM is the actual dish sitting on the counter after the recipe was followed - you can now inspect it, add salt, take a bite out of it, put a new garnish on it. The DOM is the live, in-memory, editable representation of what's on screen.

Simple example

```
<!DOCTYPE html>
<html>
  <body>
```

```
    <h1 id="title">Hello</h1>
  </body>
</html>
```

Becomes (conceptually) this tree:

```
document
 └── html
      └── body
           └── h1 (id="title")
                └── "Hello" (text node)
```

JavaScript can now do:

```
document.getElementById("title").textContent = "Hello, World!";
```

...and the visible page updates instantly, without a server round-trip. We cover DOM manipulation exhaustively in Part 6.

0.10 Browser APIs

What it is

The JavaScript language itself (the syntax - variables, functions, loops) is defined by a specification called ECMAScript. But the browser gives JavaScript extra powers beyond the core language - these are called Browser APIs (or Web APIs). Examples:

- `document` (the DOM API)
- `fetch()` (network requests)
- `localStorage` (browser storage)
- `setTimeout()` (timers)
- `console` (developer tools output)
- `navigator.geolocation` (device location)

Why this distinction matters

These are not part of the JavaScript language. They're provided by the environment JavaScript is running in. This is why `fetch` and `document` exist in browsers but not always in Node.js (Node has its own different set of APIs - `fs` for file access, for example). If someone asks you "is `fetch` a JavaScript feature?", the accurate answer is: no, it's a Browser API that the JavaScript engine has access to.

0.11 APIs (General Concept)

What it is

API stands for Application Programming Interface - a set of rules for how one piece of software can talk to another. A Browser API (above) is one example - the browser exposing functionality to JavaScript. A web API (in the sense used in Part 11) is a server exposing functionality over HTTP so other programs

(like your frontend) can use it.

We will do a full, deep treatment of web APIs in Part 11. For now: an API is a contract. It says "send me a request shaped like this, and I'll send back a response shaped like that."

0.12 JSON

What it is

JSON (JavaScript Object Notation) is a lightweight, text-based data format used to exchange data between a client and a server. Despite the name, it's used by virtually every programming language, not just JavaScript.

Simple example

```
{
  "id": 1,
  "name": "Ammaar",
  "isStudent": true,
  "subjects": ["Physics", "Chemistry", "Maths", "Biology"]
}
```

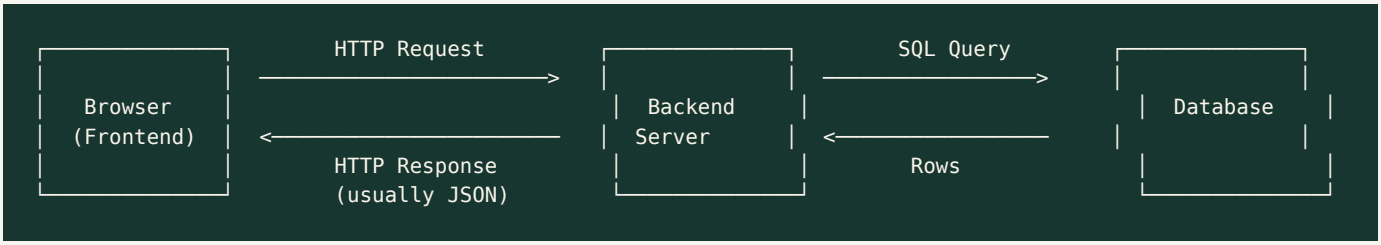
Why it exists

Before JSON became standard, data exchange formats (like XML) were verbose and harder to parse. JSON maps almost directly onto JavaScript object and array syntax, making it trivially easy to convert between "JSON text" and "a JavaScript object you can use in code." We'll use `JSON.stringify()` and `JSON.parse()` heavily starting in Part 8 and constantly from Part 10 onward.

⚠ Common mistake: JSON keys must be in double quotes, and JSON does not support comments, trailing commas, or JavaScript-only values like `undefined` or functions. JSON is a subset of what a JS object literal can express.

0.13 Frontend/Backend Communication

Putting it all together:



Your JavaScript, running in the browser, calls `fetch()`, the browser sends an HTTP request, a server somewhere processes it (possibly talking to a database), and sends back a JSON response, which your JavaScript then uses to update the DOM. This entire pipeline is what Part 10 and Part 11 are about.

0.14 Static Sites vs Dynamic Sites vs SPAs

Type	What it means	Example
Static site	Server sends the exact same pre-built HTML file to every visitor. No per-request computation.	A simple portfolio page
Dynamic site	Server generates the HTML on each request, often pulling from a database (e.g., "show this specific user's profile"). Traditional server-rendered apps (PHP, old-school Django/Rails).	A forum where each page shows different posts
SPA (Single Page Application)	The server sends one HTML shell. JavaScript then takes over - fetching data and rendering the UI in the browser - and subsequent "pages" are handled by JavaScript rewriting the DOM, without full page reloads.	Gmail, Twitter/X, most React apps

Why this matters for this course

React apps are, by default, SPAs. Understanding what an SPA isn't doing (full page reloads for every click) is key to understanding why React exists and why things like client-side routing (Part 28) are a whole topic of their own.

0.15 How the Browser Loads a Page - Full Sequence

Putting every piece above together, here's the full step-by-step:

1. You type a URL or click a link.
2. Browser performs a DNS lookup to find the server's IP address.
3. Browser opens a connection and sends an HTTP GET request for the HTML document.
4. Server sends back an HTTP response containing the HTML.
5. Browser starts parsing HTML top-to-bottom, building the DOM as it goes.
6. When it encounters `<link rel="stylesheet" href="...">`, it fetches the CSS (this can happen in parallel with HTML parsing) and begins applying styles.
7. When it encounters a `<script>` tag (without `defer`/`async`/`type="module"`), it pauses HTML parsing, fetches (if external) and executes the script immediately, then resumes.
8. Once the DOM and CSS are ready, the browser renders pixels to the screen.
9. JavaScript continues running, can attach event listeners, and the page becomes interactive.
10. From here on, any user interaction (clicks, typing, scrolling) can trigger JavaScript to run again, modifying the DOM live.

0.16 Developer Tools

Every modern browser ships with Developer Tools (DevTools). You will live in these tools for your entire career. Open them with `F12` or `Ctrl+Shift+I` (Windows/Linux) / `Cmd+Option+I` (Mac).

Tab	What it's for
Console	Run JavaScript directly, see `console.log()` output, see errors and warnings
Elements	Inspect and live-edit the DOM and CSS - see exactly what HTML/CSS the browser actually built
Network	See every HTTP request the page makes - URLs, methods, status codes, response bodies, timing. You will use this constantly in Part 10 and 11.
Sources	View your JavaScript source files and set breakpoints to pause execution and step through code line by line
Application	Inspect `localStorage`, `sessionStorage`, cookies, and other browser storage (Part 8)

Debugging basics

- `console.log(value)` - print a value to see what it is.
- `console.error(value)` - print as an error (shows in red, useful for distinguishing real problems from normal logs).
- `console.table(arrayOfObjects)` - print an array of objects as a nice table. Extremely useful for arrays of data.
- Breakpoints: click a line number in the Sources tab. When execution reaches that line, it pauses, and you can inspect every variable's current value, then step through line by line (`Step Over`, `Step Into`, `Step Out`).

> *Tip: New developers massively under-use the debugger and massively over-use scattered `console.log()` statements. Both are valid, but learning to set a breakpoint and pause time to inspect a broken program is one of the highest-leverage skills you can build early. We'll practice this explicitly in Part 9.*

Knowledge Check - Part 0

1. What's the practical difference between a website and a web application?
2. Why shouldn't frontend JavaScript talk directly to a database?
3. What are the 4 main visible parts of a URL, and what does each mean?
4. What does DNS actually translate?
5. What's the difference between the JavaScript language and a Browser API? Give one example of each.

6 What is the DOM, and how is it different from the raw HTML text file?

7 What does JSON stand for, and why is it used instead of raw JavaScript objects for data exchange over the network?

8 Explain the difference between a static site, a dynamic site, and an SPA.

9 Where would you look in DevTools to see every HTTP request your page has made, and what would you check there?

10 Why does the browser pause HTML parsing when it hits a normal `<script>` tag?

> View solutions

1 A website is mostly read (documents); a web application has state and responds to user actions like software. It's a spectrum, not a strict binary.

2 Because the browser is fully under the user's control - anyone can read, modify, or fake anything sent from it. Direct DB access would expose credentials and let anyone manipulate data with no server-side checks.

3 Scheme (`https://`), domain (`example.com`), path (`/products/42`), and query string (`?color=red`) - plus an optional fragment (`#reviews`).

4 DNS translates a human-readable domain name (like `example.com`) into a numeric IP address that computers use to route network traffic.

5 The JavaScript language (ECMAScript) defines syntax like variables, functions, and loops, and is engine-independent. A Browser API (like `fetch` or `document`) is functionality provided by the browser environment, not by the JS language spec itself.

6 The DOM is the browser's in-memory, live, object-based tree representation of the page, built by parsing the HTML text. Unlike the static HTML file, the DOM can be read and modified by JavaScript in real time.

7 JSON = JavaScript Object Notation. It's used because it's lightweight, text-based, human-readable, and maps almost directly onto JS object/array syntax, but it's also supported by virtually every other language, making it a universal exchange format.

8 Static: same pre-built HTML sent to everyone. Dynamic: server generates custom HTML per request. SPA: server sends one HTML shell, and JavaScript renders/updates the UI in the browser without full page reloads.

9 The Network tab - you'd check the request URL, method, status code, and response body/timing.

10 Because a `<script>` tag can contain code that modifies the DOM (e.g., `document.write`), so the browser must execute it immediately, in order, before continuing to parse the rest of the HTML - otherwise the DOM state could be inconsistent. (This is also why `defer`/`async` exist - covered in Part 1 - to avoid this blocking behavior when not needed.)

Next: Part 1 - JavaScript From Absolute Beginner Level

PART 1 - JAVASCRIPT FROM ABSOLUTE BEGINNER LEVEL [Beginner]

1.1 Running JavaScript

What it is

JavaScript code has to be executed by something. In this course you'll run it in two places: inside a browser (via a `<script>` tag in an HTML file) and, later, occasionally in Node.js (a JavaScript runtime outside the browser, covered properly in Part 13).

The `<script>` tag

```
<!DOCTYPE html>
<html>
  <head>
    <title>My Page</title>
  </head>
  <body>
    <h1>Hello</h1>

    <script>
      console.log("This runs in the browser console.");
    </script>
  </body>
</html>
```

External JS files

In real projects, you almost never write JavaScript inline. You link an external file:

```
<script src="app.js"></script>

// app.js
console.log("Loaded from an external file.");
```

Why external files

- Your HTML stays clean and focused on structure.
- The browser can cache the JS file separately, so repeat visits are faster.
- You can reuse the same JS file across multiple HTML pages.

`defer`

Recall from Part 0.15: a normal `<script>` tag blocks HTML parsing while it downloads and executes. `defer` fixes this:

```
<script src="app.js" defer></script>
```

`defer` tells the browser: "download this file in the background, but don't execute it until the HTML document has finished being parsed." This is why, in modern practice, you almost always see `<script defer>` in the `<head>`, rather than placing scripts at the bottom of `<body>` (which was the old

workaround for this same problem before `defer` existed).

Attribute	Downloads	Executes	Blocks parsing?
(none)	Immediately, blocking	Immediately after download	[x] Yes
`async`	In background	As soon as it's ready (order not guaranteed vs other scripts)	[x] No (only briefly, to execute)
`defer`	In background	After HTML parsing completes, in document order	[x] No

Modules

```
<script type="module" src="app.js"></script>
```

Marking a script as `type="module"` opts into the ES Modules system (`import`/`export` - full treatment in Part 12). Modules are automatically deferred, run in strict mode by default (see below), and have their own scope (variables don't leak into the global `window` object). Modern frontend development (and all of React) is built on modules.

1.2 console.log and Comments

`console.log`

```
console.log("Hello, world!");
console.log(42);
console.log(true);
console.log([1, 2, 3]);
```

This is your single most-used debugging tool for the rest of your career. It prints a value to the DevTools Console tab (Part 0.16).

Comments

```
// This is a single-line comment

/*
  This is a
  multi-line comment
*/

const age = 17; // inline comment explaining a specific line
```

 Tip: Good comments explain why, not what. `// increment count` above `count++` is useless - the code already says that. `// we cap at 17 because that's the max group size allowed by the venue` is useful, because it explains a reason the code alone can't show.

1.3 Statements, Expressions, and Syntax

What they are

- An expression is any piece of code that produces a value. `5 + 3`, `"hello"`, `isLoggedIn`, `getUser()` are all expressions.
- A statement is an instruction that performs an action. It may contain expressions, but doesn't itself produce a usable value. `if (...) { ... }`, `for (...) { ... }`, `let x = 5;` are statements.

Why the distinction matters

It explains rules that otherwise look arbitrary. For example, you can put an expression wherever a value is expected (inside a function call's parentheses, on the right side of `=`), but you can't do that with a statement - `const x = if (true) {}` is a syntax error, because `if` is a statement, not something that produces a value. This distinction becomes very important later when you learn why `{condition && <Component />}` works inside JSX but a plain `if` block does not (Part 15).

Syntax, case sensitivity, semicolons

- JavaScript is case-sensitive: `myVariable` and `myvariable` are different identifiers.
- JavaScript uses Automatic Semicolon Insertion (ASI) - it will often insert semicolons for you if you omit them. This mostly works, but has sharp edges (returning a value on the line after `return` is a classic bug - the engine inserts a semicolon right after `return`, silently returning `undefined`). This course writes semicolons explicitly, and you should too, as a professional default.

```
// ⚠ This is a classic ASI trap:
function getObject() {
  return
  {
    key: "value"
  };
}

console.log(getObject()); // undefined, NOT { key: "value" }!
```

Why it fails: ASI inserts a semicolon immediately after `return` because `return` and `{` are on different lines, making the function return `undefined` before it even reaches the object.

Correct version:

```
function getObject() {
  return {
    key: "value"
  };
}
```

1.4 Strict Mode

What it is

```
"use strict";
```

A directive that puts JavaScript into a stricter parsing and execution mode, catching common mistakes (like accidentally creating a global variable by forgetting `let`/`const`) and disabling some legacy, error-prone features.

Why it exists

Early JavaScript allowed a lot of sloppy behavior for backward compatibility. Strict mode is an opt-in way to say "treat my mistakes as errors instead of silently doing something weird."

```
"use strict";

x = 10; // [x] ReferenceError: x is not defined (without strict mode, this would silently create a global variable!)
```

You rarely need to type `"use strict"` yourself in modern code - ES Modules (`type="module"`) and all class bodies are automatically in strict mode. Since virtually all real-world JavaScript today (including everything in React) uses modules, you get strict mode for free. It's worth knowing what it is so you understand why certain sloppy patterns simply don't work in modern codebases.

1.5 Variables

What they are

A variable is a named container for a value. Instead of writing `42` everywhere in your code, you give it a name - `age` - and refer to `age` from then on.

Why they exist

Without variables, you couldn't store the result of a computation, remember user input, or refer to the same value in multiple places without repeating literal values (and hoping you never need to change one instance without changing all of them).

```
let score = 0;        // can be reassigned later
const name = "Ammaar"; // cannot be reassigned
var oldStyle = "avoid this in modern code";
```

Declaration vs initialization vs assignment vs reassignment

```
let age;        // declaration (no value yet  -  value is `undefined`)
age = 17;        // assignment (giving it a value)

let score = 0;    // declaration + initialization in one line
score = 10;        // reassignment
```

- Declaration: telling JavaScript "this name exists."
- Initialization: giving it its first value, usually at the same time as declaration.
- Assignment: giving a variable a value (could be the first time or not).
- Reassignment: assigning a new value to a variable that already had one.

`let` vs `const` - when to use which

	let	const
Can be reassigned?	[x] Yes	[x] No
Must be initialized at declaration?	No	[x] Yes

```
const PI = 3.14159; // will never change  -  const
let counter = 0;     // will change  -  let
counter = counter + 1; // [x] fine

PI = 3.14; // [x] TypeError: Assignment to constant variable.
```

 Tip / best practice: Default to `const`. Only use `let` when you know the variable's value genuinely needs to change (a loop counter, a running total, a piece of UI state). This isn't just style - it communicates intent to anyone reading your code (including future-you), and it prevents accidental reassignment bugs. Professional codebases overwhelmingly favor `const`.

⚠ Common misconception: `const` does not mean "the value can never change" - it means "this variable name can never be reassigned to point at a different value." If the value is an object or array, its contents can still change:

```
const person = { name: "Ammaar" };
person.name = "Khatri"; // [x] totally fine  -  we're not reassigning `person` itself
person.age = 17;          // [x] also fine

person = { name: "Someone Else" }; // [x] TypeError  -  this IS reassignment
```

We'll cover this in depth (mutation vs reassignment) in Part 4.

Rules (the language enforces these):

- Must start with a letter, `$`, or `_`.
- Can contain letters, digits, `$`, `_` after the first character.
- Cannot be a reserved word (`let`, `class`, `return`, etc.).
- Case-sensitive.

Conventions (the community enforces these, by agreement, not by the engine):

```
let firstName;        // camelCase  -  standard for variables and functions
const MAX_USERS = 100; // SCREAMING_SNAKE_CASE  -  conventional for true constants
class UserProfile {}   // PascalCase  -  conventional for classes and (later) React components
```

What scope is

Scope determines where in your code a variable is accessible.

```
// Global scope  -  accessible everywhere in this file/module
const appName = "MyApp";

function greet() {
  // Function scope  -  `message` only exists inside this function
  const message = "Hello!";
  console.log(message);
}
```

```
if (true) {
  // Block scope  -  `let`/`const` only exist inside these { } braces
  let insideBlock = "I only exist here";
}

console.log(insideBlock); // [x] ReferenceError: insideBlock is not defined
```

- Global scope: declared outside any function or block. Accessible from anywhere in the file (and, for old-style scripts without modules, attached to the global `window` object in browsers).
- Function scope: variables declared with `var` are scoped to the nearest enclosing function, not block.
- Block scope: variables declared with `let`/`const` are scoped to the nearest enclosing `{ }` - this includes `if` blocks, `for` loops, and plain `{ }` blocks, not just functions.

```
if (true) {
  var x = 10;
}
console.log(x); // 10  -  `var` "leaked" out of the block!

if (true) {
  let y = 10;
}
console.log(y); // [x] ReferenceError  -  `let` respected the block
```

`var` predates the concept of block scope in JavaScript. It only respects function boundaries, not `{ }` blocks. This causes real bugs - most famously inside loops:

```
// Classic var bug
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
// Prints: 3, 3, 3  (NOT 0, 1, 2!)

for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log(j), 0);
}
// Prints: 0, 1, 2  -  as expected
```

Why this happens: `var i` is one single variable, shared across all three loop iterations (because `var` doesn't create a new binding per block). By the time the `setTimeout` callbacks actually run (after the loop has fully finished), `i` has already reached `3`. `let j`, in contrast, creates a fresh binding for every iteration of the loop - each callback captures its own `j`. We'll revisit this exact example with full understanding once we cover closures (Part 3) and the event loop (Part 10) - file it away as a preview of why `var` is considered legacy.

> *Modern rule: never use `var` in new code. Use `const` by default, `let` when reassignment is needed. This course will not use `var` again except to explain legacy code you might encounter.*

What it is

Before executing your code, the JavaScript engine scans through it and "hoists" (moves to the top of their scope, conceptually) variable and function declarations. This doesn't move code around physically - it's about when the engine becomes aware of a name existing.

```
console.log(hoistedVar); // undefined (NOT an error!)
var hoistedVar = 5;

console.log(hoistedLet); // [x] ReferenceError
```

```
let hoistedLet = 5;
```

- `var` declarations are hoisted and initialized to `undefined` immediately - so referencing them before their line doesn't crash, it just gives `undefined`.
- `let` and `const` declarations are also hoisted (the engine knows they exist), but they are not initialized. Accessing them before their declaration line throws a `ReferenceError`.

  What it is

  The Temporal Dead Zone is the span of code between the start of a scope and the line where a `let`/`const` variable is actually declared. During this zone, the variable technically exists (it's been hoisted) but cannot be accessed.

```
{
  // TDZ for `value` starts here
  console.log(value); // [x] ReferenceError: Cannot access 'value' before initialization
  let value = 10;      // TDZ for `value` ends here
  console.log(value); // 10  -  fine now
}
```

  Why it exists

  The TDZ isn't a design flaw - it's a deliberate safety feature. It ensures you can never accidentally read a `let`/`const` variable before it's meaningfully initialized, catching bugs at the exact line where they'd otherwise cause confusing behavior.

  Function hoisting (preview - full detail in Part 3)

```
sayHi(); // [x] works! "Hi!"  -  function declarations are FULLY hoisted

function sayHi() {
  console.log("Hi!");
}
```

  Function declarations (the `function name() {}` form) are hoisted completely, body and all - you can call them before their line in the source. Function expressions assigned to `const`/`let` are not - they follow normal variable hoisting rules (TDZ applies). We revisit this precisely in Part 3.

- Default to `const`.
- Switch to `let` only when the variable's value must change after its initial assignment (loop counters, accumulators, toggled flags, state that will be reassigned).
- Never use `var` in new code.

Knowledge Check - Section 1.1-1.5

1. What's the practical difference between `defer` and a plain `<script>` tag with no attributes?
2. What's the difference between a statement and an expression? Give one example of each.
3. Does `const` mean a value can never change? Explain precisely what it does guarantee.
4. Why does this loop print `3, 3, 3` with `var` but `0, 1, 2` with `let`?

5 What is the Temporal Dead Zone, and why does it exist?

6 Is this valid: `let 2cool = "yes";`? Why or why not?

1 Declare a constant `PI` and a mutable variable `radius`. Compute and log the area of the circle.

2 Predict the output, then run it to check:`console.log(a); var a = 1; console.log(b); let b = 2;`

3 Rewrite this to avoid `var`, fixing the bug:`for (var i = 0; i < 5; i++) { setTimeout(() => console.log("Value:", i), 100); }`

> View solutions

Knowledge Check

1 A plain `<script>` blocks HTML parsing - the browser stops building the DOM, downloads (if external) and executes the script immediately, then resumes parsing. `defer` downloads the script in the background without blocking parsing, and only executes it after the HTML document has been fully parsed (in source order relative to other deferred scripts).

2 A statement is an instruction that performs an action (e.g., `if (x) { ... }`); an expression produces a value (e.g., `5 + 3`). `let x = 5;` is a statement that contains the expression `5`.

3 No - `const` only prevents the variable name from being reassigned to point to a new value. If the value is an object or array, its properties/elements can still be mutated.

4 Because `var` is function-scoped, not block-scoped - there's only ONE `i` shared across all iterations, and by the time the `setTimeout` callbacks run (after the loop finishes), `i` is `3`. `let` creates a fresh binding of `i` for each iteration, so each callback closes over its own separate value.

5 The TDZ is the region of code between the start of a scope and the actual `let`/`const` declaration line, where the variable is hoisted (known to exist) but not yet initialized - accessing it throws a `ReferenceError`. It exists to prevent bugs from reading a variable before it has a meaningful value.

6 No - identifiers cannot start with a digit. `let cool2 = "yes";` would be valid.

Coding Challenges

1

```
const PI = 3.14159;
let radius = 5;
console.log(PI * radius ** 2); // 78.53975
```

1

```
console.log(a); // undefined  -  var is hoisted and initialized to undefined
var a = 1;

console.log(b); // ReferenceError  -  b is in the TDZ
let b = 2;
```

(Note: the `ReferenceError` on the `console.log(b)` line would actually stop execution before reaching later lines, in a real run.)

1

```
for (let i = 0; i < 5; i++) {
  setTimeout(() => console.log("Value:", i), 100);
}
// Prints: Value: 0, Value: 1, Value: 2, Value: 3, Value: 4
```

Simply switching `var` to `let` fixes it, because `let` gives each loop iteration its own independent binding of `i`.

1.6 Data Types

What they are

JavaScript values come in two categories: primitives and objects (reference types).

Primitive types (7 total):

Type	Example	Description
`string`	`"hello"`	Text
`number`	`42`, `3.14`	All numbers - integer and floating point are the same type
`bigint`	`9007199254740993n`	Integers larger than `Number` can safely represent
`boolean`	`true`, `false`	Logical true/false
`undefined`	`undefined`	A variable that has been declared but not assigned a value
`null`	`null`	An intentional "no value"
`symbol`	`Symbol("id")`	A guaranteed-unique value, used for advanced object key scenarios

Reference type:

Type	Example
`object`	`{ }`, `[ ]`, functions, `Date`, etc.

Everything that isn't one of the 7 primitives is technically an `object` in JavaScript - including arrays and functions, which just have special behavior layered on top.

```
typeof "hello";     // "string"
typeof 42;            // "number"
typeof 9007199254740993n; // "bigint"
typeof true;          // "boolean"
typeof undefined;    // "undefined"
typeof null;          // "object"  ⚠ famous historical bug, see below
typeof Symbol("id"); // "symbol"
typeof {};            // "object"
typeof [];            // "object"  (arrays are objects!)
typeof function(){}; // "function" (a special-cased category)
```

⚠ `typeof null` is `"object"` - this is a well-known bug baked into JavaScript since 1995 and never fixed, because fixing it would break the entire web. `null` is a primitive, despite what `typeof` reports. Never rely on `typeof x === "object"` to mean "is not `null`" - always check `x !== null` separately.

This is one of the most important concepts in the entire language.

Primitives are copied by value. When you assign a primitive to a new variable, you get an independent copy.

```
let a = 5;
let b = a; // b gets a COPY of the value 5
b = 10;
console.log(a); // 5  -  a is untouched
console.log(b); // 10
```

Objects (and arrays) are copied by reference. When you assign an object to a new variable, both variables point to the same object in memory.

```
let obj1 = { value: 5 };
let obj2 = obj1; // obj2 points to the SAME object as obj1
obj2.value = 10;
console.log(obj1.value); // 10  -  obj1 is affected too!
```

Mental model

Think of a primitive value as writing a number on a sticky note and handing someone a photocopy. They can scribble on their copy all they want - your original is untouched. An object is like handing someone the key to the same house. If they repaint a wall, and you walk into that house later, you'll see the new paint too - because there's only one house, and you both have keys to it.

We'll return to this constantly - it's the root cause of most "why did my state mysteriously change" bugs in React, and the whole reason immutable update patterns exist (Part 18).

1.7 Type Coercion

What it is

JavaScript is a dynamically, weakly typed language. It will often try to automatically convert values between types when an operation expects a particular type - this is called type coercion.

Implicit coercion

```
console.log("5" + 3);   // "53"   -  number 3 is coerced to a string, then concatenated
console.log("5" - 3);   // 2       -  string "5" is coerced to a number for subtraction
console.log("5" * "2"); // 10      -  both coerced to numbers
console.log(true + 1);  // 2       -  true coerced to 1
console.log(false + 1); // 1       -  false coerced to 0
```

Why `+` behaves differently: `+` is overloaded for both string concatenation and numeric addition. If either operand is a string, JavaScript coerces the other to a string and concatenates. Every other arithmetic operator (`-`, `*`, `/`) has no string meaning, so JavaScript always coerces both operands to numbers.

Explicit conversion

```
Number("42");    // 42
Number("42px"); // NaN  -  can't fully parse
Number("");        // 0
Number(true);    // 1
Number(null);    // 0
Number(undefined); // NaN

String(42);       // "42"
String(true);    // "true"
String(null);    // "null"
String(undefined); // "undefined"

Boolean(1);        // true
Boolean(0);        // false
Boolean("");      // false
Boolean("hello"); // true
Boolean(null);    // false
Boolean(undefined); // false
```

What it is

Every JavaScript value is either "truthy" or "falsy" when evaluated in a boolean context (like an `if` condition). There are exactly 8 falsy values - everything else is truthy.

The complete falsy list:

```
false
0
-0
0n        // BigInt zero
""        // empty string
null
undefined
NaN
```

Everything else - including `"0"` (a string!), `[]` (an empty array!), and `{}` (an empty object!) - is truthy.

```
if ("0") {
  console.log("This runs!"); // yes  -  "0" is a non-empty string, which is truthy
}

if ([]) {
  console.log("This runs too!"); // yes  -  [] is an object, and all objects are truthy
}
```

⚠ Common mistake: Beginners often assume `[]` or `"0"` are falsy because they "mean" zero/empty conceptually. They are not. Only the 8 values listed above are falsy.

```
console.log(0 / 0);           // NaN  -  "Not a Number"
console.log("abc" * 2);      // NaN
console.log(1 / 0);           // Infinity
console.log(-1 / 0);          // -Infinity

console.log(NaN === NaN);   // false !! NaN is never equal to anything, including itself
console.log(Number.isNaN(NaN)); // true  -  the correct way to check for NaN
```

⚠ Common mistake:

```
function isInvalid(value) {
  return value === NaN; // [x] ALWAYS false, no matter what!
}
```

Why it fails: NaN is defined by the IEEE 754 floating point spec to never equal anything, including itself. Comparing with === will never work.

Correct version:

```
function isInvalid(value) {
  return Number.isNaN(value); // [x] correctly checks for NaN
}
```

1.8 Operators

```
5 + 3;   // 8
5 - 3;   // 2
5 * 3;   // 15
5 / 3;   // 1.6666666666666667
5 % 3;   // 2   (remainder / modulo)
5 ** 2; // 25  (exponentiation)

let x = 5;
x += 3; // same as x = x + 3  -> 8
x -= 2; // x = x - 2  -> 6
x *= 2; // x = x * 2  -> 12
x /= 3; // x = x / 3  -> 4
x **= 2; // x = x ** 2 -> 16
```

This is one of the most important distinctions in beginner JavaScript.

- === (strict equality): compares value AND type. No coercion. Use this by default, always.
- == (loose equality): coerces types before comparing. This leads to confusing, hard-to-predict results.

```
5 === 5;      // true
5 === "5";    // false  -  different types, no coercion

5 == "5";     // true  -  "5" is coerced to 5 first
0 == false;   // true  -  false is coerced to 0
"" == false;  // true
null == undefined; // true  (special case)
null === undefined; // false

// The famous WTF examples that convinced the industry to avoid `==`:
"" == 0;         // true
"0" == false;    // true
[] == false;      // true
[] == "";           // true
[1] == 1;          // true
```

Rule for this entire course and for professional practice: always use === and !==. Never use == or !=, with one narrow, explicitly-chosen exception: `value == null` is sometimes used deliberately to check for both `null` and `undefined` in one comparison (because of the `null == undefined` special case above) - but even this is a stylistic choice some teams avoid entirely in favor of being fully explicit. Default to strict equality unless you have a specific, understood reason not to.

```
true && false; // false  -  AND: both must be true
true || false; // true   -  OR: at least one must be true
!true;          // false  -  NOT: flips the boolean
```

Short-circuit evaluation

`&&` and `||` don't just return booleans - they return one of their actual operands, and they stop evaluating as soon as the result is determined.

```
console.log(0 && "hello");       // 0         -  0 is falsy, so && short-circuits and returns 0 without checking the
console.log("hi" && "hello");   // "hello"  -  "hi" is truthy, so && evaluates and returns the right side

console.log(0 || "default");    // "default"  -  0 is falsy, so || moves on to the right side
console.log("value" || "default"); // "value"  -  "value" is truthy, so || short-circuits and returns it
```

Practical use - default values (a very common older pattern):

```
function greet(name) {
  const displayName = name || "Guest";
  console.log(`Hello, ${displayName}!`);
}

greet("Ammaar"); // "Hello, Ammaar!"
greet("");        // "Hello, Guest!"   -  "" is falsy, so it falls through to "Guest"
greet(0);          // "Hello, Guest!"   -  ⚠ 0 is falsy too! This might not be what you wanted.
```

Practical use - conditional execution:

```
isLoggedIn && showDashboard(); // showDashboard() only runs if isLoggedIn is truthy
```

This exact pattern (`condition && expression`) reappears constantly in React JSX (Part 15) for conditional rendering.

What it is and why it exists

`||` has a flaw: it treats all falsy values (`0`, `""`, `false`, `NaN`) as "missing," which isn't always what you want. `??` only falls back when the left side is specifically `null` or `undefined` - not for other falsy values.

```
function greet(name) {
  const displayName = name ?? "Guest";
  console.log(`Hello, ${displayName}!`);
}

greet(0);          // "Hello, 0!"        -  0 is NOT null/undefined, so it's kept
greet("");         // "Hello, !"           -  "" is NOT null/undefined, so it's kept
greet(undefined); // "Hello, Guest!"   -  undefined triggers the fallback
greet(null);       // "Hello, Guest!"   -  null triggers the fallback
```

`||` vs `??` - the rule

Use `||` when...	Use `??` when...
You want ANY falsy value replaced (0, "", false all count as "no value" for your use case)	You only want to replace `null`/`undefined`, and `0`, `""`, or `false` are legitimate, meaningful values

A classic real bug this prevents: a stock-quantity field where `0` is a valid, meaningful count (out of stock) but `||` would incorrectly replace it with a default:

```
const stock = 0;
console.log(stock || 10); // 10  -  WRONG, we lost the real "0 in stock" info
console.log(stock ?? 10); // 0    -  CORRECT, 0 is preserved
```

What it is

Safely access deeply nested properties without manually checking every level for `null`/`undefined`.

```
const user = {
  profile: {
    address: {
      city: "Mumbai"
    }
  }
};

console.log(user.profile.address.city); // "Mumbai"

const user2 = { profile: null };
console.log(user2.profile.address.city); // [x] TypeError: Cannot read properties of null

console.log(user2.profile?.address?.city); // undefined  -  no error!
```

Step-by-step: `?.` checks if the value immediately to its left is `null` or `undefined`. If it is, the entire expression short-circuits and evaluates to `undefined`, without attempting to access anything further to the right. If it's not `null`/`undefined`, it proceeds normally.

Also works for function calls and array access:

```
user2.someMethod?.();    // calls someMethod only if it exists, else undefined
someArray?.[0];               // safely accesses index 0 only if someArray exists
```

⚠ Common mistake: Optional chaining prevents crashes, but the result is still `undefined` - you often still need a fallback:

```
const city = user2.profile?.address?.city ?? "Unknown city";
```

This `?.` + `??` combo is extremely common in real production code, especially when working with API data (Part 11) that might have missing fields.

```
const age = 17;
const category = age >= 18 ? "adult" : "minor";
console.log(category); // "minor"
```

Syntax: `condition ? valueIfTrue : valueIfFalse`. It's an expression (produces a value), unlike `if/else` which is a statement. This is exactly why you'll see ternaries (not `if/else`) used inside JSX in Part 15 - JSX needs expressions, not statements.

⚠ Common mistake - nesting ternaries too deeply:

```
// [Advanced] Hard to read
const label = age < 13 ? "child" : age < 18 ? "teen" : age < 65 ? "adult" : "senior";
```

Best practice: one level of ternary is fine and idiomatic. Beyond that, use `if/else if` chains or a lookup structure - readability wins.

```
let count = 5;
count++; // post-increment: count becomes 6
count--; // post-decrement: count becomes 5

let a = 5;
console.log(a++); // 5  -  logs OLD value, THEN increments
console.log(a);      // 6

let b = 5;
console.log(++b); // 6  -  increments FIRST, then logs NEW value
```

 Tip: In modern code, avoid using x++ embedded inside larger expressions where the pre/post distinction matters - it's a classic source of subtle bugs. Prefer using it as its own statement on its own line.

Just like in mathematics, operators have precedence (order of evaluation). You don't need to memorize the full table, but know the practical rule: when in doubt, use parentheses - both for correctness and for readability.

```
console.log(2 + 3 * 4);    // 14, not 20  -  * has higher precedence than +
console.log((2 + 3) * 4); // 20  -  parentheses force order
```

Knowledge Check - Section 1.6-1.8

1. What's the difference between how primitives and objects are copied when assigned to a new variable?
2. Why does `typeof null` return `"object"`? Is `null` actually an object?
3. List all 8 falsy values in JavaScript.
4. Why is `NaN === NaN` false, and what's the correct way to check if a value is `NaN`?
5. Explain the difference between `||` and `??` with a concrete example where they behave differently.
6. What does `user?.profile?.city` do if `user` is `undefined`? What does it evaluate to?
7. Why does `"5" + 3` produce `"53"` but `"5" - 3` produce `2`?
1. Predict the output of each line, then verify:`console.log(1 + "1"); console.log(1 - "1"); console.log("5" == 5); console.log("5" === 5); console.log(null == undefined); console.log(null === undefined); console.log([] + []); console.log([] + {});`
2. Write a function `getDiscount(couponValue)` that returns `couponValue` if it's provided (including if it's `0`), or defaults to `5` if it's `null`/`undefined`. Use the correct operator.
3. Rewrite this nested ternary as a cleaner `if/else if` chain:`const grade = score >= 90 ? "A" : score >= 80 ? "B" : score >= 70 ? "C" : "F";`

> View solutions

Knowledge Check

1. Primitives are copied by value (an independent copy is made). Objects are copied by reference (the new variable points to the same underlying object in memory).
2. It's a long-standing bug in JavaScript from 1995 that was never fixed because doing so would break existing code across the web. `null` is a primitive value, not an object, despite what `typeof` reports.
3. `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN`.
4. Per the IEEE 754 floating point standard, `NaN` is defined to never equal anything, including itself. Use `Number.isNaN(value)` to correctly check.

5 `||` replaces ANY falsy value (0, "", false, null, undefined, NaN) with the fallback. `??` only replaces `null`/`undefined`. Example: `0 || "default"` gives `"default"`, but `0 ?? "default"` gives `0`.

6 If `user` is `undefined`, `user?.profile` short-circuits immediately to `undefined` without attempting to access `.profile`, and the whole chain evaluates to `undefined` with no error thrown.

7 `+` is overloaded to mean both addition and string concatenation - if either side is a string, JS coerces the other side to a string too. `-` has no string meaning, so JS always coerces both sides to numbers.

Coding Challenges

1

```
console.log(1 + "1");   // "11"  -  number coerced to string
console.log(1 - "1");   // 0     -  string coerced to number
console.log("5" == 5);  // true  -  loose equality coerces
console.log("5" === 5); // false -  strict equality, different types
console.log(null == undefined);  // true  -  special case
console.log(null === undefined); // false -  different types
console.log([] + []);    // ""     -  both arrays coerced to strings ("" + "")
console.log([] + {});    // "[object Object]"  -  [] becomes "", {} becomes "[object Object]"
```

1

```
function getDiscount(couponValue) {
  return couponValue ?? 5;
}

console.log(getDiscount(0));      // 0   -  correctly preserved
console.log(getDiscount(undefined)); // 5
console.log(getDiscount(null));       // 5
console.log(getDiscount(20));      // 20
```

1

```
let grade;
if (score >= 90) {
  grade = "A";
} else if (score >= 80) {
  grade = "B";
} else if (score >= 70) {
  grade = "C";
} else {
  grade = "F";
}
```

Cheat Sheet - JavaScript Syntax Basics

Concept	Syntax
Variable (mutable)	`let x = 5;`
Variable (constant)	`const x = 5;`
String	`"text"`, `'text'`, `` `template` ``
Number	`42`, `3.14`

Concept	Syntax
Boolean	`true`, `false`
Null	`null`
Undefined	`undefined`
Strict equality	`a === b`
Strict inequality	`a !== b`
Logical AND	`a && b`
Logical OR	`a
Nullish coalescing	`a ?? b`
Optional chaining	`a?.b`
Ternary	`condition ? valA : valB`
Comment (line)	`// comment`
Comment (block)	`/* comment */`

Next: Part 2 - Control Flow

PART 2 - CONTROL FLOW [Beginner]

Control flow is how you make a program do different things depending on conditions, or repeat an action. Without it, every program would just be a straight list of instructions executed once, top to bottom - useless for anything real.

2.1 if , else , else if

What it is

```
if (condition) {
  // runs if condition is truthy
} else if (otherCondition) {
  // runs if condition was falsy AND otherCondition is truthy
} else {
  // runs if nothing above matched
}
```

Simple example

```
const temperature = 28;

if (temperature > 30) {
  console.log("It's hot.");
} else if (temperature > 20) {
```

```
  console.log("It's warm.");
} else {
  console.log("It's cold.");
}
// "It's warm."
```

Step-by-step execution

1. JavaScript evaluates `temperature > 30` -> `false`.
2. Moves to the `else if` - evaluates `temperature > 20` -> `true`.
3. Runs that block's body and skips everything else in the chain, even if later conditions would also be true.
4. Never reaches the final `else`.

Nested conditions

```
const age = 25;
const hasLicense = true;

if (age >= 18) {
  if (hasLicense) {
    console.log("You can drive.");
  } else {
    console.log("You need a license first.");
  }
} else {
  console.log("You're too young to drive.");
}
```

 Tip: Deep nesting (3+ levels) is a readability smell. Prefer guard clauses (below) to flatten logic.

What they are

A guard clause exits a function (or loop) early when a condition isn't met, avoiding deep nesting for the "main" logic.

```
// [Advanced] Deeply nested
function processOrder(order) {
  if (order) {
    if (order.items.length > 0) {
      if (order.paymentConfirmed) {
        console.log("Processing order...");
      } else {
        console.log("Payment not confirmed.");
      }
    } else {
      console.log("Order has no items.");
    }
  } else {
    console.log("No order provided.");
  }
}

// [x] Guard clauses  -  flat, reads top-to-bottom
function processOrder(order) {
  if (!order) {
    console.log("No order provided.");
    return;
  }
```

```
  if (order.items.length === 0) {
    console.log("Order has no items.");
    return;
  }
  if (!order.paymentConfirmed) {
    console.log("Payment not confirmed.");
    return;
  }

  console.log("Processing order...");
}
```

Both versions behave identically, but the guard-clause version reads as a checklist of things that must be true, with the "happy path" clearly at the bottom, unindented. This pattern is extremely common in professional code and especially in React components (Part 18 onward), where you'll frequently see `if (loading) return <Spinner />;` at the top of a component.

2.2 switch

What it is

An alternative to long `if/else if` chains when you're comparing one value against many possible exact matches.

```
const day = "Tuesday";

switch (day) {
  case "Monday":
    console.log("Start of the week");
    break;
  case "Tuesday":
  case "Wednesday":
  case "Thursday":
    console.log("Midweek");
    break;
  case "Friday":
    console.log("Almost done");
    break;
  default:
    console.log("Weekend");
}
// "Midweek"
```

Step-by-step: JavaScript compares `day` against each `case` using strict equality (`===`). `"Tuesday"` matches the second case. Because there's no `break` before it, execution falls through into `"Wednesday"` and `"Thursday"`'s shared body (this is intentional grouping here). Once it hits `break`, it exits the `switch` entirely.

⚠ Common mistake - forgetting `break`:

```
const day = "Monday";

switch (day) {
  case "Monday":
    console.log("Start of the week");
    // [x] missing break!
  case "Tuesday":
    console.log("Midweek");
    break;
```

```
  default:
    console.log("Other");
}
// Prints BOTH "Start of the week" AND "Midweek"  -  unintended fall-through!
```

Why it fails: Without `break`, execution continues into the next case's body regardless of whether it matches, until it hits a `break` or the end of the `switch`. This is one of the most common beginner bugs with `switch`.

Best practice: Always include `break` (or `return`, if inside a function) at the end of every case unless you are intentionally grouping cases (like `"Tuesday"`/`"Wednesday"`/`"Thursday"` above).

2.3 Loops

```
for (let i = 0; i < 5; i++) {
  console.log(i);
}
// 0, 1, 2, 3, 4
```

Anatomy: `for (initialization; condition; update) { body }`

1. Initialization runs once, before the loop starts.
2. Condition is checked before every iteration - if falsy, the loop ends.
3. Body runs.
4. Update runs after the body.
5. Back to step 2.

```
let count = 0;
while (count < 3) {
  console.log(count);
  count++;
}
// 0, 1, 2
```

The condition is checked before each iteration. If it's false from the start, the body never runs even once.

```
let n = 10;
do {
  console.log(n);
  n++;
} while (n < 3);
// 10  -  runs ONCE even though the condition was already false
```

Why it exists: `do...while` guarantees the body runs at least once, because the condition is checked after the first iteration. Use this when you always need at least one execution regardless of the condition (e.g., "show a menu, then keep showing it while the user hasn't chosen 'exit'").

```
const fruits = ["apple", "banana", "cherry"];

for (const fruit of fruits) {
  console.log(fruit);
}
```

```
// apple, banana, cherry
```

`for...of` iterates over the values of an iterable (arrays, strings, Maps, Sets - anything implementing the iterator protocol, covered in Part 12). This is the modern, preferred way to loop over array values when you don't need the index.

```
const person = { name: "Ammaar", age: 17, city: "Mumbai" };

for (const key in person) {
  console.log(key, person[key]);
}
// name Ammaar
// age 17
// city Mumbai
```

`for...in` iterates over the enumerable property keys of an object.

⚠ Common mistake - using `for...in` on arrays:

```
const arr = ["a", "b", "c"];
for (const index in arr) {
  console.log(index); // "0", "1", "2"  -  these are STRINGS, not numbers!
}
```

Why this is a problem: `for...in` gives you keys as strings, and technically iterates over all enumerable properties (including inherited ones in edge cases), which can produce surprising results on arrays. Best practice: use `for...of` (or array methods like `.forEach`/`.map`, Part 4) for arrays, and reserve `for...in` for plain objects - or better yet, use `Object.keys()`/`Object.entries()` with `for...of`, covered in Part 4.

2.4 break and continue

```
for (let i = 0; i < 10; i++) {
  if (i === 5) break; // exit the loop entirely
  console.log(i);
}
// 0, 1, 2, 3, 4

for (let i = 0; i < 5; i++) {
  if (i === 2) continue; // skip just this iteration
  console.log(i);
}
// 0, 1, 3, 4
```

- `break` exits the loop entirely, immediately.
- `continue` skips the rest of the current iteration and moves to the next one.

```
outer: for (let i = 0; i < 3; i++) {
  for (let j = 0; j < 3; j++) {
    if (j === 1) continue outer; // skips to the NEXT iteration of the OUTER loop
    console.log(i, j);
  }
}
```

Labels (`outer:`) let `break`/`continue` target an outer loop from inside a nested one. This is a rarely-needed but occasionally very useful escape hatch - most of the time, extracting the inner loop

into its own function and using `return` is cleaner.

For most application code, loop performance differences between `for`, `for...of`, and array methods (`.map`, `.forEach`) are irrelevant - the difference is microseconds on realistic data sizes. Don't prematurely optimize loop choice; optimize for readability first (Part 31 covers real performance concerns). The one practical exception: extremely hot loops over very large datasets (tens of millions of iterations) where a classic `for` loop can be meaningfully faster than `.forEach` due to function call overhead - but this is rare in typical frontend work.

Knowledge Check - Part 2

1. What's the key difference in execution order between `while` and `do...while`?
2. Why does forgetting `break` in a `switch` statement cause a bug, and what does it actually do?
3. What's wrong with using `for...in` to loop over an array, even though it "works"?
4. What's the difference between `break` and `continue`?
5. What is a guard clause, and why is it preferred over deep nesting?

1. Write a function `fizzBuzz(n)` that logs numbers 1 to `n`, but logs `"Fizz"` for multiples of 3, `"Buzz"` for multiples of 5, and `"FizzBuzz"` for multiples of both.
2. Use a `for` loop to sum all even numbers from 1 to 100.
3. Rewrite this using a guard clause:`function canVote(age) { if (age >= 18) { return true; } else { return false; } }`
4. Given an array of numbers, use `for...of` to find and log the first number greater than 50, then stop looping (hint: `break`).

Find and fix the bug:

```
function printEvens(max) {
  for (let i = 0; i <= max; i++) {
    if (i % 2 !== 0) continue;
    if (i = 10) break; // bug is here
    console.log(i);
  }
}
```

> View solutions

Knowledge Check

1. `while` checks the condition before running the body (may run zero times); `do...while` checks it after (always runs at least once).
2. Without `break`, execution "falls through" into the next case's body regardless of whether it matches, continuing until a `break` is hit or the switch ends - causing unintended code to run.

3 `for...in` returns keys as strings (not numbers), and iterates over all enumerable properties, which can include inherited or unexpected properties - it's not guaranteed to give you clean sequential numeric indices the way array-specific tools do.

4 `break` exits the loop completely. `continue` skips only the current iteration and proceeds to the next one.

5 A guard clause exits early when a precondition isn't met, keeping the "main" logic flat and unindented instead of buried inside nested `if` blocks - it's easier to read top-to-bottom.

Coding Challenges

1

```
function fizzBuzz(n) {
  for (let i = 1; i <= n; i++) {
    if (i % 3 === 0 && i % 5 === 0) {
      console.log("FizzBuzz");
    } else if (i % 3 === 0) {
      console.log("Fizz");
    } else if (i % 5 === 0) {
      console.log("Buzz");
    } else {
      console.log(i);
    }
  }
}
```

1

```
let sum = 0;
for (let i = 1; i <= 100; i++) {
  if (i % 2 === 0) {
    sum += i;
  }
}
console.log(sum); // 2550
```

1

```
function canVote(age) {
  return age >= 18;
}
```

1

```
const numbers = [12, 45, 63, 8, 99, 21];
for (const num of numbers) {
  if (num > 50) {
    console.log(num); // 63
    break;
  }
}
```

Debugging Challenge

Two bugs:

- `if (i = 10)` uses assignment (`=`) instead of comparison (`===`). This assigns `10` to `i` (and the expression evaluates to `10`, which is truthy), so the condition is always true, breaking the loop the first even number it reaches... actually since it also reassigns `i`, this compounds the bug further.
- Even fixed to `===`, breaking exactly at `i === 10` may or may not be the intended behavior - but the clear bug is the `=` vs `===` typo.

```
function printEvens(max) {
  for (let i = 0; i <= max; i++) {
    if (i % 2 !== 0) continue;
    if (i === 10) break; // fixed: use strict equality
    console.log(i);
  }
}
```

Next: Part 3 - Functions

PART 3 - FUNCTIONS [Beginner][Intermediate]

> *This is one of the most important sections in the entire course. Functions are the fundamental unit of reuse and abstraction in JavaScript, and misunderstanding them (especially closures and `this`) is the single biggest source of confusion for developers moving from beginner to intermediate. Take your time here.*

3.1 What a Function Is

What it is

A function is a reusable block of code that you define once and can call (execute) as many times as you want, optionally with different inputs each time.

Why it exists

Without functions, you'd have to copy-paste the same logic everywhere you need it. If you ever needed to fix a bug in that logic, you'd have to find and fix every copy. Functions let you write logic once and reuse it - the foundation of the "Don't Repeat Yourself" (DRY) principle.

Mental model

A function is a vending machine. You put something in (arguments/parameters), it does its internal process (the function body), and something comes out (the return value). You don't need to know or care about the machine's internals every time you use it - you just need to know what to put in and what to expect out.

3.2 Three Ways to Write a Function

```
function add(a, b) {
  return a + b;
}

console.log(add(2, 3)); // 5

const add = function (a, b) {
  return a + b;
};

console.log(add(2, 3)); // 5

const add = (a, b) => {
  return a + b;
};

// Shorthand for a single expression body (implicit return):
const addShort = (a, b) => a + b;

console.log(add(2, 3));      // 5
console.log(addShort(2, 3)); // 5
```

Feature	Function Declaration	Function Expression	Arrow Function
Syntax	`function name() {}`	`const name = function() {}`	`const name = () => {}`
Hoisting	Fully hoisted - callable before its line in the source	Not hoisted (follows `const`/`let` rules - TDZ applies)	Not hoisted (same as expressions)
Has its own `this`?	[x] Yes, dynamic (depends on how it's called)	[x] Yes, dynamic	[x] No - inherits `this` from the enclosing scope (lexical `this`)
Has its own `arguments` object?	[x] Yes	[x] Yes	[x] No - inherits from enclosing scope
Can be used as a constructor (`new Fn()`)?	[x] Yes	[x] Yes	[x] No - throws an error
Implicit return (no braces)?	[x] No	[x] No	[x] Yes, for single-expression bodies
Typical use	Named, reusable, top-level utility functions	Assigning a function to a variable/property, especially conditionally	Short callbacks, and anywhere you specifically want lexical `this`

Why these differences exist: Arrow functions were added in ES6 (2015) specifically to solve the recurring pain of `this` behaving unpredictably inside callbacks (explained fully in 3.9). They deliberately have no `this` of their own, so that they transparently "borrow" `this` from whatever scope they were defined in - this alone is why they're now the default choice for most callbacks in modern JavaScript and virtually all of React.

3.3 Parameters, Arguments, and Return Values

Terminology

- Parameters are the named placeholders in the function definition: `function add(a, b)` -> `a` and `b` are parameters.
- Arguments are the actual values you pass when calling the function: `add(2, 3)` -> `2` and `3` are arguments.
- Return value is what the function produces via the `return` keyword. If there's no `return`, the function implicitly returns `undefined`.

```
function multiply(x, y) {   // x, y = parameters
  return x * y;
}

const result = multiply(4, 5); // 4, 5 = arguments; result = 20 (return value)
```

⚠ Common mistake - forgetting `return`:

```
function multiply(x, y) {
  x * y; // [x] no return  -  this expression's value is computed and discarded!
}

console.log(multiply(4, 5)); // undefined
```

Correct version:

```
function multiply(x, y) {
  return x * y;
}

function greet(name = "Guest") {
  console.log(`Hello, ${name}!`);
}

greet("Ammaar"); // "Hello, Ammaar!"
greet();          // "Hello, Guest!"  -  no argument, default kicks in
greet(undefined); // "Hello, Guest!"  -  explicitly passing undefined also triggers the default
greet(null);       // "Hello, null!"   -  ⚠ null does NOT trigger the default!
```

Why `null` doesn't trigger the default: Default parameters only activate when the argument is `undefined` (missing, or explicitly `undefined`). `null` is treated as an intentional value, not a missing one.

```
function sum(...numbers) {
  // `numbers` is a real array containing ALL arguments passed
  return numbers.reduce((total, n) => total + n, 0);
}

console.log(sum(1, 2, 3));       // 6
console.log(sum(1, 2, 3, 4, 5)); // 15
console.log(sum());                // 0
```

Why it exists: Before rest parameters, handling a variable number of arguments required the clunky, array-like-but-not-really-an-array `arguments` object (see 3.9). Rest parameters give you a real array, with all real array methods available, and can be combined with named parameters:

```
function logScores(studentName, ...scores) {
  console.log(studentName, "scored:", scores);
}

logScores("Ammaar", 95, 88, 92); // "Ammaar scored: [95, 88, 92]"
```

⚠ Rule: the rest parameter must be the last parameter. `function f(...rest, last)` is a syntax error.

3.4 Callbacks, First-Class Functions, Higher-Order Functions

What it means

In JavaScript, functions are values, just like numbers or strings. You can:

- Store them in variables.
- Pass them as arguments to other functions.
- Return them from other functions.
- Store them in arrays or objects.

```
const sayHi = function () {
  console.log("Hi!");
};

const functions = [sayHi, () => console.log("Bye!")]; // functions in an array
const obj = { greet: sayHi };                          // a function as an object property
```

This is called having first-class functions, and it's the foundation for everything covered below.

What it is

A callback is a function passed as an argument to another function, to be called back (invoked) later - either immediately, or after some operation completes.

```
function processUserInput(callback) {
  const name = "Ammaar";
  callback(name);
}

processUserInput(function (name) {
  console.log(`Hello, ${name}!`);
});
// "Hello, Ammaar!"
```

Why callbacks matter so much: They're the foundation of asynchronous JavaScript (Part 10), event handling (Part 6), and array methods like `.map`/`.filter`/`.forEach` (Part 4). Understanding callbacks deeply now makes everything from here forward dramatically easier.

What it is

A function that either takes a function as an argument, returns a function, or both.

```
// Takes a function as an argument (this makes processUserInput above a HOF)
```

```
// Returns a function:
function createMultiplier(factor) {
  return function (number) {
    return number * factor;
  };
}

const double = createMultiplier(2);
const triple = createMultiplier(3);

console.log(double(5)); // 10
console.log(triple(5)); // 15
```

`double` and `triple` are functions that "remember" the `factor` they were created with - this is your first real taste of closures, covered fully in 3.8. `Array.prototype.map`, `.filter`, and `.reduce` (Part 4) are all higher-order functions - they take a callback and use it internally.

3.5 Pure Functions and Side Effects

What a pure function is

A function is pure if:

1. Given the same inputs, it always returns the same output.
2. It causes no side effects - it doesn't modify anything outside its own scope (no changing external variables, no mutating arguments, no network requests, no `console.log`, no modifying the DOM).

```
// [x] Pure
function add(a, b) {
  return a + b;
}

// [x] Impure  -  depends on external state
let taxRate = 0.18;
function calculateTotal(price) {
  return price + price * taxRate; // depends on something outside its parameters
}

// [x] Impure  -  has a side effect (mutates external state)
let total = 0;
function addToTotal(amount) {
  total += amount; // modifies something outside the function
}
```

Why pure functions matter

- They're trivially testable - same input, same output, no setup needed.
- They're predictable - no hidden dependencies to track.
- They're safe to run multiple times, reorder, or run in parallel.
- React is built around this idea - components should, as much as possible, be pure functions of their props and state (Part 18 covers this directly). Understanding purity now will make React's rules click much faster later.

Side effects aren't bad - real programs need them (saving data, making network requests, updating the screen). The goal is to be deliberate about where they live, rather than scattering them throughout otherwise-pure logic. React formalizes this exact idea with the `useEffect` hook (Part 21) - the name is not a coincidence.

3.6 Recursion

What it is

A function that calls itself, working toward a base case that stops the recursion.

```
function factorial(n) {
  if (n <= 1) return 1;       // base case  -  stops the recursion
  return n * factorial(n - 1); // recursive case  -  calls itself with a smaller problem
}

console.log(factorial(5)); // 120
```

Step-by-step execution

```
factorial(5)
= 5 * factorial(4)
= 5 * (4 * factorial(3))
= 5 * (4 * (3 * factorial(2)))
= 5 * (4 * (3 * (2 * factorial(1))))
= 5 * (4 * (3 * (2 * 1)))
= 5 * (4 * (3 * 2))
= 5 * (4 * 6)
= 5 * 24
= 120
```

Each call waits for the call it made to return before it can finish its own calculation. This "stack of waiting calls" is a direct preview of the call stack, covered in Part 5.

⚠ Common mistake - missing or unreachable base case:

```
function countDown(n) {
  console.log(n);
  countDown(n - 1); // [x] no base case  -  this never stops!
}

countDown(5); // Eventually: "Maximum call stack size exceeded"
```

Correct version:

```
function countDown(n) {
  if (n < 0) return; // base case
  console.log(n);
  countDown(n - 1);
}
```

When to use recursion: Naturally recursive problems - tree traversal, nested data structures (like recursive comment threads or nested categories/folders), certain mathematical sequences. For simple counting/repetition, a loop is usually clearer and more efficient. Recursion shines when the problem itself has a recursive shape.

3.7 Execution Context and the Call Stack

What an execution context is

Every time a function is called, JavaScript creates a new execution context - a container that tracks that specific call's local variables, its `this` value, and where to resume in the code that called it.

What the call stack is

The call stack is a LIFO (Last In, First Out) structure that tracks all currently active execution contexts.

```
function a() {
  console.log("Inside a");
  b();
  console.log("Back in a");
}

function b() {
  console.log("Inside b");
  c();
  console.log("Back in b");
}

function c() {
  console.log("Inside c");
}

a();
```

Step-by-step call stack trace:

```
1. a() is called       -> stack: [a]
2. "Inside a" logs
3. b() is called        -> stack: [a, b]
4. "Inside b" logs
5. c() is called        -> stack: [a, b, c]
6. "Inside c" logs
7. c() finishes, POPPED -> stack: [a, b]
8. "Back in b" logs
9. b() finishes, POPPED -> stack: [a]
10. "Back in a" logs
11. a() finishes, POPPED -> stack: []
```

Output:

```
Inside a
Inside b
Inside c
Back in b
Back in a
```

Why this matters: This is exactly what a stack trace (Part 9) shows you when an error occurs - the chain of function calls that were active at the moment of the crash, from most recent (top) to oldest (bottom). Understanding the call stack is the difference between reading a stack trace as noise versus reading it as a precise map of "what called what."

⚠ Stack overflow: If functions keep calling other functions (especially via unstoppable recursion, as in 3.6) without ever returning, the stack grows until it hits a memory limit, throwing `RangeError: Maximum call stack size exceeded`.

3.8 Closures and Lexical Scope

This is widely considered the single most important - and most commonly asked in interviews - concept in JavaScript. Read this section slowly.

What it is

"Lexical" means "related to where things are written in the source code." Lexical scoping means a function's access to variables is determined by where it is physically written, not by how or where it's called from.

```
const outerVar = "I'm outside";

function outer() {
  const innerVar = "I'm inside";

  function inner() {
    console.log(outerVar); // [x] accessible  -  inner() is lexically inside outer(), which is inside the global sco
    console.log(innerVar); // [x] accessible  -  inner() is lexically inside outer()
  }

  inner();
}

outer();
```

The scope chain: when JavaScript looks up a variable name, it checks the current scope first, and if not found, walks outward through each enclosing scope (following how the code is nested in the source), all the way to the global scope, until it finds the variable or runs out of scopes (-> `ReferenceError`).

What it is

A closure is what happens when a function "remembers" the variables from its lexical scope, even after the outer function has finished executing.

The classic example

```
function createCounter() {
  let count = 0; // this variable lives inside createCounter's scope

  return function () {
    count++;       // this inner function "closes over" count
    return count;
  };
}

const counter = createCounter(); // createCounter() FINISHES executing here

console.log(counter()); // 1
console.log(counter()); // 2
console.log(counter()); // 3
```

Why this is remarkable: `createCounter()` has already returned - its execution context is, in principle, done. And yet the inner function keeps working, still able to read and modify `count`. Normally, when a function finishes, its local variables should be garbage-collected (freed from memory) because nothing needs them anymore. But because the returned inner function references `count`, JavaScript keeps that variable alive in memory, tucked inside the closure, for as long as the inner function itself might still be called.

Mental model

A closure is like a backpack. When a function is created, it packs a backpack with references to all the variables from its surrounding scope that it might need later. No matter where you carry that function afterward - pass it to another function, return it, store it in an array - it always has that backpack with it.

Step-by-step for `createCounter`:

1. `createCounter()` is called. A new execution context is created with its own `count = 0`.
2. Inside, an anonymous function is defined and returned. This function's closure captures a reference to `count` (not a copy - the actual variable).
3. `createCounter()` finishes and is popped off the call stack. Normally `count` would be garbage collected - but it isn't, because the returned function still references it.
4. `counter` now holds that returned function, backpack and all.
5. Each call to `counter()` accesses and mutates the same `count` variable that lives in its closure.

Each call to `createCounter()` creates a brand-new, independent closure:

```
const counterA = createCounter();
const counterB = createCounter();

console.log(counterA()); // 1
console.log(counterA()); // 2
console.log(counterB()); // 1  -  completely separate count! Not shared with counterA.
```

1. Data privacy (encapsulation)

```
function createBankAccount(initialBalance) {
  let balance = initialBalance; // private  -  can't be accessed directly from outside

  return {
    deposit(amount) {
      balance += amount;
      return balance;
    },
    withdraw(amount) {
      if (amount > balance) {
        console.log("Insufficient funds");
        return balance;
      }
      balance -= amount;
      return balance;
    },
    getBalance() {
      return balance;
    }
  };
}

const account = createBankAccount(1000);
console.log(account.getBalance()); // 1000
account.deposit(500);
console.log(account.getBalance()); // 1500
console.log(account.balance);        // undefined  -  no direct access! `balance` is truly private.
```

2. Function factories (like `createMultiplier` in 3.4)

3. Explaining the `var` loop bug from Part 1, fully now:

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
// 3, 3, 3
```

Now you have all the pieces: there is one single `i` (because `var` is function/global-scoped, not block-scoped). Every arrow function passed to `setTimeout` closes over that same `i`. By the time any of the callbacks actually run (after the synchronous loop has fully finished - see Part 10 for why), `i` is `3` for all of them, because they're all looking at the same variable in the same backpack.

```
for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log(j), 0);
}
// 0, 1, 2
```

`let` creates a new binding of `j` for every iteration of the loop. Each callback's closure captures its own independent `j`, frozen at that iteration's value.

⚠ Common closure mistake:

```
function createButtons() {
  const buttons = [];
  for (var i = 0; i < 3; i++) {
    buttons.push(function () {
      console.log("Button", i, "clicked");
    });
  }
  return buttons;
}

const [btn1, btn2, btn3] = createButtons();
btn1(); // "Button 3 clicked"  [x] expected "Button 0 clicked"
btn2(); // "Button 3 clicked"  [x] expected "Button 1 clicked"
btn3(); // "Button 3 clicked"
```

Why it fails: Same root cause as above - `var i` is one shared variable, and by the time any button function runs, the loop has finished and `i` is `3`.

Correct version:

```
function createButtons() {
  const buttons = [];
  for (let i = 0; i < 3; i++) { // [x] let creates a fresh i per iteration
    buttons.push(function () {
      console.log("Button", i, "clicked");
    });
  }
  return buttons;
}
```

3.9 this

What it is

`this` is a special keyword whose value is determined by how a function is called, not where it's defined (with the notable exception of arrow functions - see below).

The core rule: look at the call site

```
const person = {
  name: "Ammaar",
  greet() {
    console.log(`Hi, I'm ${this.name}`);
  }
};

person.greet(); // "Hi, I'm Ammaar"  -  `this` is `person`, because greet() was called AS person.greet()
```

`this` changes based on how you call the SAME function:

```
const greetFn = person.greet;
greetFn(); // "Hi, I'm undefined" (or throws in strict mode)  -  `this` is no longer `person`!
```

Why: `greetFn()` is called as a plain, unattached function call. There's no object before the dot, so `this` defaults to `undefined` (in strict mode / modules) or the global object (in old-style non-strict scripts). This is the single most common source of `this`-related bugs - extracting a method from its object and calling it separately loses the connection to that object.

Call pattern	`this` value
`obj.method()`	`obj` (whatever is before the dot at the call site)
`fn()` (plain call)	`undefined` in strict mode/modules, global object otherwise
`new Fn()`	The newly created object (covered in Part 5)
`fn.call(obj)`, `fn.apply(obj)`, `fn.bind(obj)`	Explicitly `obj` (covered next)

Arrow functions do not have their own `this`. They capture `this` lexically - from wherever they were physically written - exactly like closures capture variables.

```
const person = {
  name: "Ammaar",
  greetRegular: function () {
    setTimeout(function () {
      console.log(`Hi, I'm ${this.name}`); // [x] "Hi, I'm undefined"
    }, 100);
  },
  greetArrow: function () {
    setTimeout(() => {
      console.log(`Hi, I'm ${this.name}`); // [x] "Hi, I'm Ammaar"
    }, 100);
  }
};

person.greetRegular();
person.greetArrow();
```

Why the difference: The regular `function` passed to `setTimeout` is called as a plain function call internally by the browser - it gets its own fresh `this`, which is `undefined`/global, not `person`. The arrow function has no `this` of its own - it looks outward to its enclosing scope (`greetArrow`'s body), where

`this` correctly refers to `person`.

This is precisely why arrow functions became the default for callbacks in modern JavaScript - they eliminate an entire category of `this`-losing bugs by design.

⚠ Important exception: Because arrow functions don't have their own `this`, never use an arrow function as an object method when you need `this` to refer to that object:

```
const person = {
  name: "Ammaar",
  greet: () => {
    console.log(`Hi, I'm ${this.name}`); // [x] `this` here is NOT `person`  -  it's whatever `this` was in the surr
  }
};

person.greet(); // "Hi, I'm undefined"
```

Best practice: use regular function syntax (`method() {}` shorthand) for object methods that need `this`, and arrow functions for callbacks nested inside those methods.

These three methods exist on every function and let you explicitly control what `this` will be.

```
function introduce(greeting) {
  console.log(`${greeting}, I'm ${this.name}`);
}

const ammaar = { name: "Ammaar" };
const priya = { name: "Priya" };

introduce.call(ammaar, "Hello");  // "Hello, I'm Ammaar"  -  call() takes args individually
introduce.apply(priya, ["Hi"]);   // "Hi, I'm Priya"       -  apply() takes args as an array

const boundIntroduce = introduce.bind(ammaar); // bind() returns a NEW function with `this` permanently locked to `a
boundIntroduce("Hey"); // "Hey, I'm Ammaar"
boundIntroduce("Yo");  // "Yo, I'm Ammaar"  -  this binding never changes again
```

Method	Calls the function immediately?	Arguments format	Returns
`.call(thisArg, a, b, c)`	[x] Yes	Individual arguments	The function's return value
`.apply(thisArg, [a, b, c])`	[x] Yes	An array of arguments	The function's return value
`.bind(thisArg, a, b)`	[x] No	Individual arguments (optional, pre-filled)	A NEW function, permanently bound

Practical use case for `bind`: fixing the exact `this`-losing bug shown above, when you can't use an arrow function for some reason (e.g., older class-based code):

```
class Timer {
  constructor() {
    this.seconds = 0;
    // Without this line, `this` inside tick() would be lost when passed to setInterval:
    this.tick = this.tick.bind(this);
  }
  tick() {
    this.seconds++;
    console.log(this.seconds);
```

```
  }
}
```

3.10 IIFEs (Immediately Invoked Function Expressions)

What it is

A function that is defined and called immediately, in one expression, never to be called again.

```
(function () {
  console.log("This runs immediately!");
})();

// Arrow function version:
(() => {
  console.log("This too!");
})();
```

Why it exists (historically)

Before ES Modules gave every file its own scope (Part 1.1, Part 12), all top-level `var` declarations in separate `<script>` tags shared the same global scope, risking name collisions between different scripts/libraries. Wrapping code in an IIFE created an isolated function scope, so nothing inside leaked into the global namespace.

```
(function () {
  var privateVar = "Nobody outside can see this";
  console.log(privateVar);
})();

console.log(typeof privateVar); // "undefined"  -  successfully hidden
```

Is this still relevant? In modern code using ES Modules, you get this same file-level isolation automatically (Part 12), so IIFEs are far less necessary than they used to be. You should still recognize them, because you'll encounter them in older codebases, some library source code, and certain patterns (like running an async function immediately at the top level before top-level `await` was widely supported).

3.11 Function Hoisting - Precise Rules

Recall from Part 1: function declarations are fully hoisted (name and body); function expressions (including arrow functions) are not.

```
console.log(declared()); // [x] "I work!"  -  fully hoisted

function declared() {
  return "I work!";
}

console.log(expressed()); // [x] ReferenceError: Cannot access 'expressed' before initialization (TDZ)

const expressed = function () {
  return "I don't work yet";
};
```

Best practice: Don't rely on function hoisting for readability's sake, even though it works for declarations. Define functions before you use them wherever practical - it makes code easier to read top-to-bottom regardless of the underlying hoisting mechanics.

Knowledge Check - Part 3

1. Why don't arrow functions have their own `this`? What do they do instead?
2. What is a closure, in your own words? Give an example that doesn't use a counter.
3. Why does `createCounter()` in 3.8 keep working even after the outer function has returned?
4. What's the difference between `.call()`, `.apply()`, and `.bind()`?
5. What makes a function "pure"? Why does purity matter for testability?
6. What's the difference between a parameter and an argument?
7. Why does `null` not trigger a default parameter, but `undefined` does?

1. Write a closure-based function `createIdGenerator()` that returns a function which produces a new unique incrementing ID (starting at 1) every time it's called.
2. Fix this code so each logged message shows the correct index using a closure, without changing the `for` loop's `var` to `let` (hint: wrap in an IIFE inside the loop):`const messages = []; for (var i = 0; i < 3; i++) { messages.push(function () { console.log("Message", i); }); } messages.forEach(fn => fn());`
3. Write a pure function `formatPrice(amount, currency)` that returns a formatted string like `"₹1,500"` given `amount = 1500, currency = "₹"`. (Keep it simple - no need for full locale formatting.)
4. Convert this function declaration into an arrow function, and explain one situation where this conversion would change behavior:`function makeGreeter(greeting) { return function (name) { return `${greeting}, ${name}!`; }; }`

> View solutions

Knowledge Check

1. Arrow functions don't create their own `this` binding - they lexically inherit `this` from the scope in which they were written, exactly like they inherit any other closed-over variable.
2. A closure is a function that retains access to variables from its enclosing (lexical) scope, even after that outer scope has finished executing. Example: a function that returns a "greeter" pre-configured with a name, e.g. `function makeGreeter(name) { return () => console.log("Hi " + name); }`.
3. Because the returned inner function holds a reference to `count` from its enclosing scope - JavaScript keeps that variable alive in memory (in the closure) as long as any function that references it still exists, rather than garbage collecting it the moment `createCounter` finishes.
4. `.call(thisArg, arg1, arg2)` invokes the function immediately with individual arguments. `.apply(thisArg, [arg1, arg2])` invokes immediately with arguments as an array. `.bind(thisArg)`

does NOT invoke immediately - it returns a new function permanently bound to `thisArg`, to be called later.

5 A function is pure if it always returns the same output for the same input and has no observable side effects (no mutating external state, no I/O). This matters for testability because you can test it in complete isolation - same input always verifiably produces the same output, with nothing else to set up or mock.

6 A parameter is the named placeholder in the function's definition. An argument is the actual value passed in when calling the function.

7 Default parameters only activate for `undefined` specifically - `null` is treated as a deliberate, meaningful value the caller chose to pass, not as "nothing was provided."

Coding Challenges

1

```
function createIdGenerator() {
  let id = 0;
  return function () {
    id++;
    return id;
  };
}

const nextId = createIdGenerator();
console.log(nextId()); // 1
console.log(nextId()); // 2
console.log(nextId()); // 3
```

1

```
const messages = [];
for (var i = 0; i < 3; i++) {
  (function (capturedI) {
    messages.push(function () {
      console.log("Message", capturedI);
    });
  })(i); // pass the CURRENT value of i into the IIFE immediately, creating a new capturedI per iteration
}
messages.forEach(fn => fn());
// Message 0, Message 1, Message 2
```

1

```
function formatPrice(amount, currency) {
  return `${currency}${amount.toLocaleString("en-IN")}`;
}

console.log(formatPrice(1500, "₹")); // "₹1,500"
```

1

```
const makeGreeter = (greeting) => (name) => `${greeting}, ${name}!`;
```

This works fine here because neither function uses `this` or `arguments`. It would change behavior if `makeGreeter` needed to be called with `new` as a constructor (arrow functions can't be constructors), or if the inner function needed access to its own `arguments` object (arrow functions have none - they'd

inherit `arguments` from `makeGreeter` instead, which is a common, confusing bug).

Interview Questions - Functions & Closures

Q: What is a closure?
A closure is the combination of a function and references to its surrounding (lexical) scope, bundled together. It allows a function to continue accessing variables from an outer function even after that outer function has returned. Closures are why techniques like private state (data hiding) and function factories work in JavaScript.

Q: Difference between `let`, `const`, and `var`?
`var` is function-scoped and hoisted with an `undefined` initial value, allowing access before declaration. `let` and `const` are block-scoped and live in the Temporal Dead Zone until their declaration line, throwing if accessed early. `const` additionally disallows reassignment (but not mutation of the underlying value if it's an object/array).

Q: What causes `this` to be different inside a regular callback function versus an arrow function callback?
A regular function's `this` is determined dynamically by its call site - a plain, unattached call sets `this` to `undefined` (strict mode) or the global object, losing the connection to whatever object the callback was passed from. An arrow function has no `this` of its own and lexically inherits `this` from its enclosing scope at the point it was defined, which usually preserves the intended object context.

Next: Part 4 - Strings, Numbers, Arrays and Objects

PART 4 - STRINGS, NUMBERS, ARRAYS AND OBJECTS [Beginner][Intermediate]

4.1 Strings

Strings are sequences of characters. In JavaScript, strings are immutable - no string method ever changes the original string; they all return a new string.

```
const name = "Ammaar";
const age = 17;

// Old way (string concatenation):
const oldMsg = "Hello, " + name + ". You are " + age + " years old.";

// Template literal (backticks, `${}`):
const newMsg = `Hello, ${name}. You are ${age} years old.`;

// Multi-line, effortlessly:
const multiLine = `Line one
Line two
Line three`;
```

```
// Any expression works inside ${}:
const total = `Total: ${2 + 3}`; // "Total: 5"
```

 Tip: Template literals (backticks) are the default in all modern JavaScript. Use them instead of `+` concatenation.

```
const str = "  Hello, World!  ";

str.length;                      // 18 (including the spaces)
str.trim();                        // "Hello, World!"  -  removes leading/trailing whitespace (new string)

const clean = "Hello, World!";
clean.charAt(0);                 // "H"
clean[0];                          // "H"  -  bracket access also works for reading
clean.at(-1);                      // "!"  -  .at() supports negative indices (from the end)
clean.at(0);                       // "H"

clean.includes("World");        // true
clean.startsWith("Hello");      // true
clean.endsWith("!");             // true
clean.indexOf("World");         // 7  -  position of first match, -1 if not found
clean.indexOf("xyz");             // -1

clean.slice(0, 5);                // "Hello"  -  extracts from index 0 up to (not including) 5
clean.slice(7);                    // "World!"  -  from index 7 to the end
clean.slice(-6);                  // "World!"  -  negative index counts from the end

clean.substring(0, 5);           // "Hello"  -  similar to slice, but no negative index support

clean.replace("World", "There"); // "Hello, There!"  -  replaces FIRST match only
clean.replaceAll("l", "L");       // "HeLLo, WorLd!"  -  replaces ALL matches

clean.toLowerCase();               // "hello, world!"
clean.toUpperCase();               // "HELLO, WORLD!"

clean.split(", ");                  // ["Hello", "World!"]  -  splits into an array
"a-b-c".split("-");                 // ["a", "b", "c"]
"hello".split("");                    // ["h", "e", "l", "l", "o"]  -  split by character
```

`slice` vs `substring` - the practical difference: `slice` supports negative indices to count from the end; `substring` treats negative numbers as `0` and also swaps its arguments if the first is bigger than the second. Best practice: default to `slice` - it's more predictable and works consistently across both strings and arrays (Part 4.3).

Simple:

```
const greeting = `Hi, ${"Ammaar"}`;
```

Practical - building a search-safe comparison:

```
function matchesSearch(productName, searchTerm) {
  return productName.toLowerCase().includes(searchTerm.toLowerCase().trim());
}

console.log(matchesSearch("Wireless Mouse", "  MOUSE ")); // true
```

Step-by-step: `.trim()` removes stray whitespace the user might have typed, `.toLowerCase()` on both sides makes the comparison case-insensitive, and `.includes()` checks for a partial substring match - exactly the pattern you'll build into real search bars in Part 11's projects.

⚠ Common mistake:

```
const name = "Ammaar";
name[0] = "K"; // [x] silently does nothing  -  strings are immutable!
console.log(name); // still "Ammaar"
```

Correct approach - build a new string:

```
const newName = "K" + name.slice(1);
console.log(newName); // "Kmmaar"
```

4.2 Numbers

```
Number("42");         // 42
Number("42.5");       // 42.5
Number("42px");       // NaN  -  Number() requires the WHOLE string to be numeric

parseInt("42px");     // 42  -  parseInt stops at the first non-digit character
parseInt("42.9px");   // 42  -  parseInt ignores decimals entirely
parseFloat("42.9px"); // 42.9  -  parseFloat keeps the decimal portion

parseInt("101", 2);   // 5  -  second argument is the RADIX (base)  -  here, binary
```

When to use which:

- `Number(x)` - strictest, use when you expect the entire string to be a clean number (e.g., form input validation, Part 7).
- `parseInt(x)` - use when you need to extract a leading integer from a string that might have trailing text (e.g., `"100px"` -> `100`).
- `parseFloat(x)` - same as `parseInt` but preserves decimals.

```
Math.round(4.5);  // 5
Math.round(4.4);  // 4
Math.floor(4.9);  // 4  -  always rounds down
Math.ceil(4.1);    // 5  -  always rounds up
Math.trunc(4.9);  // 4  -  chops off the decimal, no rounding logic
Math.trunc(-4.9); // -4  -  different from Math.floor(-4.9), which is -5!

Math.abs(-5);      // 5
Math.max(1, 5, 3); // 5
Math.min(1, 5, 3); // 1
Math.pow(2, 10);   // 1024 (same as 2 ** 10)
Math.sqrt(64);     // 8

Math.random();      // a float between 0 (inclusive) and 1 (exclusive)
```

Generating a random integer in a range - a pattern you'll use constantly:

```
function randomInt(min, max) {
  // returns an integer between min and max, INCLUSIVE of both
  return Math.floor(Math.random() * (max - min + 1)) + min;
}

console.log(randomInt(1, 6)); // simulates a dice roll: 1-6
```

Step-by-step: `Math.random()` gives `[0, 1)`. Multiplying by `(max - min + 1)` scales it to `[0, range)`. `Math.floor` truncates to a whole number in `[0, range - 1]`. Adding `min` shifts the range to start at `min` instead of `0`.

⚠ Famous gotcha:

```
console.log(0.1 + 0.2); // 0.30000000000000004   -  NOT 0.3!
console.log(0.1 + 0.2 === 0.3); // false
```

Why this happens: Computers store floating-point numbers in binary, and most decimal fractions (like `0.1`) cannot be represented exactly in binary - the same way `1/3` can't be written exactly in decimal (`0.3333...`). This is a property of all languages that use the IEEE 754 floating-point standard (which is nearly all of them), not a JavaScript-specific bug.

Correct approach - never compare floats directly for equality; use a small tolerance, or work in integers (e.g., cents/paise instead of rupees) when precision matters:

```
function isApproximatelyEqual(a, b, epsilon = 0.0001) {
  return Math.abs(a - b) < epsilon;
}

console.log(isApproximatelyEqual(0.1 + 0.2, 0.3)); // true
```

For real currency handling, the professional pattern is to store amounts as integers in the smallest unit (paise instead of rupees) and only convert to decimal for display - this sidesteps floating point issues entirely for money math.

4.3 Arrays

```
const fruits = ["apple", "banana", "cherry"];

fruits[0];        // "apple"  -  zero-indexed
fruits[2];        // "cherry"
fruits.length;    // 3
fruits[fruits.length - 1]; // "cherry"  -  common pattern for "last item"
fruits.at(-1);      // "cherry"  -  modern, cleaner way to get the last item

const arr = [1, 2, 3];

arr.push(4);       // arr is now [1, 2, 3, 4]  -  adds to the END, returns new length
arr.pop();          // arr is now [1, 2, 3]  -  removes from the END, returns removed item
arr.unshift(0);    // arr is now [0, 1, 2, 3]  -  adds to the START, returns new length
arr.shift();         // arr is now [1, 2, 3]  -  removes from the START, returns removed item

arr.splice(1, 1);            // removes 1 item starting at index 1 -> arr is now [1, 3]
arr.splice(1, 0, "a", "b"); // inserts "a","b" at index 1, removes 0 -> arr is now [1, "a", "b", 3]

arr.sort();          // sorts IN PLACE (mutates!)  -  default sort is STRING-based, see below
arr.reverse();       // reverses IN PLACE (mutates!)
```

⚠ Famous `.sort()` gotcha - default sort is alphabetical (string-based), even for numbers:

```
const numbers = [100, 25, 9];
numbers.sort();
console.log(numbers); // [100, 25, 9]  ← NOT [9, 25, 100] !!
```

Why: the default sort converts every element to a string first, then compares them character by character. As strings, `"100"` comes before `"25"` because `'1'` sorts before `'2'`, and `"25"` comes before `"9"` because `'2'` sorts before `'9'` - completely ignoring numeric magnitude.

Correct version - always provide a comparator function for numbers:

```
const numbers = [10, 2, 33, 4];
numbers.sort((a, b) => a - b); // ascending
console.log(numbers); // [2, 4, 10, 33]

numbers.sort((a, b) => b - a); // descending
console.log(numbers); // [33, 10, 4, 2]
```

How the comparator works: For each pair `(a, b)` being compared, if the function returns a negative number, `a` is sorted before `b`. If positive, `b` is sorted before `a`. If `0`, their order is unchanged (relative to each other). `a - b` naturally produces negative when `a < b`, achieving ascending order.

```
const arr = [1, 2, 3, 4, 5];

arr.slice(1, 3);      // [2, 3]  -  new array, original arr untouched
arr.concat([6, 7]);   // [1, 2, 3, 4, 5, 6, 7]  -  new array
arr.includes(3);       // true
arr.indexOf(3);        // 2
arr.join("-");           // "1-2-3-4-5"

const nested = [1, [2, 3], [4, [5, 6]]];
nested.flat();         // [1, 2, 3, 4, [5, 6]]  -  flattens ONE level deep by default
nested.flat(Infinity); // [1, 2, 3, 4, 5, 6]  -  flattens ALL levels

console.log(arr); // [1, 2, 3, 4, 5]  -  UNCHANGED by any of the above
```

⚠ Critical distinction: `.slice()` (array) does not mutate; `.splice()` (array) DOES mutate. This one-letter difference is a very common source of bugs. Read carefully every time.

```
const users = [
  { id: 1, name: "Ammaar", age: 17 },
  { id: 2, name: "Priya", age: 22 },
  { id: 3, name: "Rohan", age: 19 }
];

users.find(u => u.age > 18);       // { id: 2, name: "Priya", age: 22 }  -  first MATCH (the object itself)
users.findIndex(u => u.age > 18); // 1  -  the INDEX of the first match
users.find(u => u.age > 100);     // undefined  -  no match found
users.findIndex(u => u.age > 100); // -1  -  no match found

users.some(u => u.age > 18);       // true  -  does AT LEAST ONE element match?
users.every(u => u.age > 18);      // false  -  do ALL elements match?
```

`.find()` vs `.filter()`: `.find()` returns the first matching element itself (or `undefined`). `.filter()` (below) returns an array of all matching elements. Use `.find()` when you expect (or only care about) one result - like "find the user with this ID" - and `.filter()` when you want every match.

4.4 The Big Three: map , filter , reduce - Deep Dive

These three methods are the backbone of modern JavaScript data transformation and appear constantly in React (rendering lists, deriving state, transforming API responses). Understanding them deeply is non-negotiable.

What it is

Runs a callback once for each element. Does not return a new array - it returns `undefined`. Used purely for side effects (like logging, or pushing into an external array - though `.map` is almost always better for that).

```
const nums = [1, 2, 3];
nums.forEach((num, index) => {
  console.log(`Index ${index}: ${num}`);
});
// Index 0: 1
// Index 1: 2
// Index 2: 3

const result = nums.forEach(n => n * 2);
console.log(result); // undefined  -  forEach NEVER returns anything useful
```

⚠ Common mistake: trying to "build a new array" with `.forEach()` and `.push()`, when `.map()` (below) does this more idiomatically and safely.

What it is

Calls a callback on every element and builds a new array from the callback's return values. The new array is always the same length as the original.

Mental model

Picture an assembly line: each item on the conveyor belt goes through the exact same transformation station, and comes out the other side as a new item, in the same order, onto a new belt (the original belt is untouched).

```
Input:  [ 1,     2,     3,     4  ]
             |      |      |      |
          ×2     ×2     ×2     ×2     (the callback)
             |      |      |      |
Output: [ 2,     4,     6,     8  ]
```

Simple example:

```
const numbers = [1, 2, 3, 4];
const doubled = numbers.map(n => n * 2);
console.log(doubled);  // [2, 4, 6, 8]
console.log(numbers); // [1, 2, 3, 4]  -  original UNCHANGED
```

Practical example - transforming API-shaped data (a pattern you'll use constantly from Part 11 onward):

```
const apiUsers = [
  { id: 1, first_name: "Ammaar", last_name: "Khatri" },
  { id: 2, first_name: "Priya", last_name: "Sharma" }
];

const displayUsers = apiUsers.map(user => ({
  id: user.id,
  fullName: `${user.first_name} ${user.last_name}`
}));

console.log(displayUsers);
// [
//   { id: 1, fullName: "Ammaar Khatri" },
//   { id: 2, fullName: "Priya Sharma" }
// ]
```

⚠ Syntax trap: when the callback's body is a single expression that returns an object literal, you must wrap it in parentheses - `(u => ({ ... }))` - otherwise JavaScript interprets the `{ }` as a function body block, not an object.

```
// [x] Wrong  -  JS thinks { id: user.id } is a function body, "id:" is parsed as a label
const broken = apiUsers.map(user => { id: user.id });
console.log(broken); // [undefined, undefined] !!

// [x] Correct  -  parentheses tell JS this { } is an object literal
const fixed = apiUsers.map(user => ({ id: user.id }));
```

What it is

Calls a callback on every element; keeps the element in the new array only if the callback returns truthy. The new array can be any length from 0 to the original length.

Mental model

Picture a bouncer at a club door checking each person (element) against a rule (the callback). Only those who pass get let through to the new list.

```
Input:  [ 1,     2,     3,     4,     5  ]
              |      |      |      |      |
        is even?  no    yes    no    yes    no    (the callback)
              |      |      |      |      |
Output:       [      2,           4      ]

const numbers = [1, 2, 3, 4, 5, 6];
const evens = numbers.filter(n => n % 2 === 0);
console.log(evens); // [2, 4, 6]

const products = [
  { name: "Mouse", inStock: true, price: 500 },
  { name: "Keyboard", inStock: false, price: 1200 },
  { name: "Monitor", inStock: true, price: 8000 }
];

const availableProducts = products.filter(p => p.inStock);
console.log(availableProducts);
// [{ name: "Mouse", ... }, { name: "Monitor", ... }]
```

Combining `.filter()` and `.map()` - extremely common pattern:

```
const expensiveInStockNames = products
  .filter(p => p.inStock && p.price > 1000)
  .map(p => p.name);

console.log(expensiveInStockNames); // ["Monitor"]
```

What it is

`.reduce()` "reduces" an entire array down to a single value - which could be a number, a string, an object, or even a new array. It does this by running a callback (the "reducer") on each element, carrying forward an accumulator value between each call.

Syntax

```
array.reduce((accumulator, currentElement, index, array) => {
  // must return the NEW accumulator value
}, initialValue);
```

Mental model

Picture a snowball rolling downhill, picking up snow (data) as it goes. The `accumulator` is the snowball itself - it starts as `initialValue`, and at each step, the reducer function decides how much of the current element gets "stuck" onto it, returning the updated snowball to carry into the next step.

Simple example - summing numbers:

```
const numbers = [1, 2, 3, 4];

const sum = numbers.reduce((accumulator, current) => {
  return accumulator + current;
}, 0); // 0 is the initial value of the accumulator

console.log(sum); // 10
```

Step-by-step execution:

```
Initial accumulator: 0

Step 1: accumulator=0, current=1 -> returns 0+1=1
Step 2: accumulator=1, current=2 -> returns 1+2=3
Step 3: accumulator=3, current=3 -> returns 3+3=6
Step 4: accumulator=6, current=4 -> returns 6+4=10

Final result: 10
```

Practical example - counting occurrences (building an object from an array):

```
const words = ["apple", "banana", "apple", "cherry", "banana", "apple"];

const counts = words.reduce((acc, word) => {
  acc[word] = (acc[word] || 0) + 1;
  return acc;
}, {}); // start with an empty object

console.log(counts); // { apple: 3, banana: 2, cherry: 1 }
```

Step-by-step: For each word, `acc[word] || 0` reads the current count (or `0` if this is the first time seeing that word), adds 1, and stores it back into `acc` under that key. Because objects are reference types (Part 1.6), mutating `acc` and returning the same object reference on each iteration works correctly here.

Practical example - grouping data (a pattern you'll use in the capstone project):

```
const orders = [
  { id: 1, status: "pending" },
  { id: 2, status: "shipped" },
  { id: 3, status: "pending" },
  { id: 4, status: "delivered" }
];

const grouped = orders.reduce((acc, order) => {
  if (!acc[order.status]) {
    acc[order.status] = [];
  }
  acc[order.status].push(order);
  return acc;
}, {});

console.log(grouped);
// {
//   pending: [{id:1,...}, {id:3,...}],
```

```
//   shipped: [{id:2,...}],
//   delivered: [{id:4,...}]
// }
```

⚠ Common mistake - forgetting to return the accumulator:

```
const sum = numbers.reduce((acc, n) => {
  acc + n; // [x] no return!
}, 0);

console.log(sum); // undefined  -  the accumulator becomes undefined after the very first iteration
```

Why it fails: without `return`, the callback implicitly returns `undefined` every time, so on the second iteration, `accumulator` is already `undefined`, and `undefined + n` is `NaN`, cascading into garbage from there.

⚠ Common mistake - forgetting the initial value:

```
const numbers = [];
const sum = numbers.reduce((acc, n) => acc + n); // [x] no initial value, and the array is empty!
// TypeError: Reduce of empty array with no initial value
```

Best practice: Always provide an initial value for `.reduce()`. It removes an entire class of edge-case bugs with empty arrays, and makes the accumulator's starting shape explicit and readable.

Method	Use when you want to...	Output
`.map()`	Transform every element into something else, 1-to-1	Array of the SAME length
`.filter()`	Keep only some elements, based on a condition	Array of 0 to original length
`.reduce()`	Combine everything into a SINGLE value (number, object, string, even another array/shape)	Any single value
`.forEach()`	Just DO something for each element (no new data structure)	`undefined` (side effects only)

 Tip: If you find yourself using `.reduce()` to build something that `.map()` or `.filter()` could do more simply, prefer the simpler tool. `.reduce()` is the most powerful but also the least readable at a glance - reserve it for when you genuinely need to accumulate into a single combined value.

Knowledge Check - Section 4.3-4.4

1. Why does `[100, 25, 9].sort()` NOT produce `[9, 25, 100]` by default? How do you fix it?
2. What's the difference between `.slice()` and `.splice()` on arrays?
3. What does `.map()` guarantee about the length of the array it returns, that `.filter()` does not?
4. Why must you always provide an initial value to `.reduce()`?
5. When would you use `.find()` instead of `.filter()`?

6 What does this produce, and why: `[1,2,3].map(n => { n * 2 })`?

1 Given `const nums = [5, 12, 8, 130, 44];`, use `.filter()` and `.map()` (chained) to get an array of just the numbers greater than 10, doubled.

2 Given an array of product objects `{ name, price }`, use `.reduce()` to compute the total price of all products.

3 Given `const words = ["hi", "hello", "hey", "yo"];`, use `.reduce()` to find the longest word (as a single string, not an array).

4 Use `.reduce()` to flatten this array of arrays into one flat array, without using `.flat()`: `[[1,2],[3,4],[5]]`.

> View solutions

Knowledge Check

1 Because the default `.sort()` converts elements to strings and compares them alphabetically/lexicographically, not numerically - `"100"` sorts before `"25"` since `'1' < '2'`. Fix: `arr.sort((a, b) => a - b)` for ascending numeric order.

2 `.slice(start, end)` returns a new array (non-mutating) containing a shallow copy of a portion of the original. `.splice(start, deleteCount, ...items)` mutates the original array in place, removing and/or inserting elements, and returns the removed elements.

3 `.map()` always returns an array of exactly the same length as the input (one output per input, even if that output is `undefined`). `.filter()` can return anywhere from an empty array to the full original length, depending on how many elements pass the test.

4 Without an initial value, calling `.reduce()` on an empty array throws a `TypeError`, and even on a non-empty array, omitting it means the first array element is used as the starting accumulator and iteration begins from the second element - which is often not what you intend, and is far less explicit/readable.

5 Use `.find()` when you expect (or only care about) a single matching element - e.g., finding a specific user by ID. Use `.filter()` when you want every element that matches.

6 It produces `[undefined, undefined, undefined]`. The arrow function's `{ n * 2 }` is parsed as a block-bodied function with no `return` statement, so `n * 2` is computed and discarded, and the function implicitly returns `undefined` for every element.

Coding Challenges

1

```
const nums = [5, 12, 8, 130, 44];
const result = nums.filter(n => n > 10).map(n => n * 2);
console.log(result); // [24, 260, 88]
```

1

```
const products = [
  { name: "Mouse", price: 500 },
  { name: "Keyboard", price: 1200 },
  { name: "Monitor", price: 8000 }
```

```
];
const total = products.reduce((sum, p) => sum + p.price, 0);
console.log(total); // 9700
```

1

```
const words = ["hi", "hello", "hey", "yo"];
const longest = words.reduce((longestSoFar, word) => {
  return word.length > longestSoFar.length ? word : longestSoFar;
}, "");
console.log(longest); // "hello"
```

1

```
const nested = [[1, 2], [3, 4], [5]];
const flat = nested.reduce((acc, subArray) => acc.concat(subArray), []);
console.log(flat); // [1, 2, 3, 4, 5]
```

4.5 Copying Arrays: Mutating vs Non-Mutating, Shallow Copies

Why this matters

Because arrays (and objects) are reference types (Part 1.6), assigning one to a new variable doesn't copy it - both names point to the same data. This is especially critical in React (Part 18), where you must never mutate state directly.

```
const original = [1, 2, 3];
const notACopy = original;    // same array in memory!
notACopy.push(4);
console.log(original);          // [1, 2, 3, 4]  -  original was affected!

const realCopy = [...original]; // spread syntax creates a NEW array
realCopy.push(5);
console.log(original); // [1, 2, 3, 4]  -  untouched this time
console.log(realCopy); // [1, 2, 3, 4, 5]
```

Ways to copy an array:

```
const arr = [1, 2, 3];

const copy1 = [...arr];           // spread syntax  -  most common, modern default
const copy2 = arr.slice();          // .slice() with no arguments copies the whole array
const copy3 = Array.from(arr);   // Array.from also works
```

⚠ These are all shallow copies. If the array contains objects, the objects themselves are still shared by reference:

```
const users = [{ name: "Ammaar" }];
const copiedUsers = [...users]; // shallow copy  -  the ARRAY is new, but the OBJECT inside is shared

copiedUsers[0].name = "Changed!";
console.log(users[0].name); // "Changed!"  -  the original object was mutated too!
```

Deep copy (when you truly need one):

```
const deepCopy = structuredClone(users); // modern built-in, handles nested structures properly
deepCopy[0].name = "Totally Different";
```

```
console.log(users[0].name); // "Changed!"  -  untouched this time, because deepCopy's objects are independent
```

We'll return to this exact concept (shallow vs deep, mutation vs immutable updates) with full force in Part 18, because it's the #1 source of React state bugs.

4.6 Objects

```
const person = {
  name: "Ammaar",
  age: 17,
  isStudent: true
};

// Dot notation  -  use when you know the property name ahead of time
person.name; // "Ammaar"

// Bracket notation  -  use when the property name is dynamic (a variable) or not a valid identifier
const key = "age";
person[key];      // 17
person["is-active"]; // needed if the key has characters dot notation can't express

const user = {
  name: "Ammaar",
  address: {
    city: "Mumbai",
    pincode: "400001"
  }
};

console.log(user.address.city); // "Mumbai"

const calculator = {
  value: 0,
  add(n) {          // shorthand method syntax
    this.value += n;
    return this;    // returning `this` enables "method chaining"
  },
  subtract(n) {
    this.value -= n;
    return this;
  }
};

calculator.add(10).subtract(3);
console.log(calculator.value); // 7

const propName = "score";
const dynamicObj = {
  [propName]: 100 // the KEY itself is computed from a variable
};
console.log(dynamicObj); // { score: 100 }

const car = { brand: "Toyota", model: "Corolla" };

car.year = 2024;         // adding a new property
car.model = "Camry";       // modifying an existing property
delete car.brand;        // removing a property

console.log(car); // { model: "Camry", year: 2024 }
```

```
const scores = { math: 95, science: 88, english: 91 };

Object.keys(scores);   // ["math", "science", "english"]
Object.values(scores); // [95, 88, 91]
Object.entries(scores); // [["math", 95], ["science", 88], ["english", 91]]

// Looping over an object cleanly with Object.entries + for...of:
for (const [subject, score] of Object.entries(scores)) {
  console.log(`${subject}: ${score}`);
}

const defaults = { theme: "light", fontSize: 14 };
const userPrefs = { fontSize: 18 };

const merged1 = Object.assign({}, defaults, userPrefs); // older syntax
const merged2 = { ...defaults, ...userPrefs };              // modern spread syntax  -  PREFERRED

console.log(merged2); // { theme: "light", fontSize: 18 }
```

Step-by-step: properties are applied left to right - later sources overwrite earlier ones for the same key. This is exactly how you'll merge default props with overrides in React (Part 17), and it's the foundation of immutable state updates (Part 18).

⚠ Critical: `Object.assign({}, ...)` and `{ ...obj }` both start with an empty `{}` as the FIRST argument/spread. Forgetting this mutates the first real object instead of creating a new one:

```
// [x] WRONG  -  mutates defaults directly!
const broken = Object.assign(defaults, userPrefs);
console.log(defaults); // { theme: "light", fontSize: 18 }  -  defaults got mutated!

const config = { api: { baseUrl: "https://api.example.com" } };
console.log(config.api?.timeout ?? 5000); // 5000  -  api.timeout doesn't exist, falls back safely
```

4.7 Destructuring

What it is

A syntax for extracting values out of arrays or objects into individual variables, in a single concise statement.

```
const user = { name: "Ammaar", age: 17, city: "Mumbai" };

const { name, age } = user;
console.log(name); // "Ammaar"
console.log(age);   // 17

// Renaming while destructuring:
const { name: userName } = user;
console.log(userName); // "Ammaar"

// Default values (used when the property is undefined):
const { country = "India" } = user;
console.log(country); // "India"  -  user.country doesn't exist, so the default kicks in

// Nested destructuring:
const { address: { city } } = { address: { city: "Mumbai" } };
console.log(city); // "Mumbai"
```

Practical example - function parameters (extremely common in React props, Part 17):

```
function printUser({ name, age }) {
  console.log(`${name} is ${age} years old`);
}

printUser(user); // "Ammaar is 17 years old"

const colors = ["red", "green", "blue"];

const [first, second] = colors;
console.log(first, second); // "red" "green"

const [, , third] = colors; // skip elements with empty commas
console.log(third); // "blue"

// Swapping variables  -  a classic destructuring party trick:
let a = 1, b = 2;
[a, b] = [b, a];
console.log(a, b); // 2 1
```

⚠ Important: you'll see this exact syntax the moment you learn `useState` in Part 18: `const [count, setCount] = useState(0);` is array destructuring - `useState` returns a 2-element array, and this syntax pulls out both pieces in one line.

4.8 Spread and Rest Syntax

Same `...` syntax, two different meanings depending on context.

```
// Spreading an array into a new array:
const a = [1, 2, 3];
const b = [...a, 4, 5]; // [1, 2, 3, 4, 5]

// Spreading into function arguments:
function sum3(a, b, c) { return a + b + c; }
const nums = [1, 2, 3];
console.log(sum3(...nums)); // 6  -  spreads array elements as individual arguments

// Spreading an object into a new object:
const base = { a: 1, b: 2 };
const extended = { ...base, c: 3 }; // { a: 1, b: 2, c: 3 }

// Rest in function parameters (seen in Part 3.3):
function sum(...numbers) {
  return numbers.reduce((t, n) => t + n, 0);
}

// Rest in destructuring:
const [first, ...rest] = [1, 2, 3, 4];
console.log(first); // 1
console.log(rest);   // [2, 3, 4]

const { name, ...otherFields } = { name: "Ammaar", age: 17, city: "Mumbai" };
console.log(name);         // "Ammaar"
console.log(otherFields); // { age: 17, city: "Mumbai" }
```

How to tell them apart: if `...` appears where a collection is being built or called (in an array/object literal, or a function call), it's spread - expanding values out. If it appears where a collection is being destructured or defined (in a parameter list or a destructuring pattern), it's rest - gathering values in.

Knowledge Check - Section 4.6-4.8

1. What's the difference between dot notation and bracket notation for accessing object properties, and when must you use bracket notation?
2. Why does `Object.assign(defaults, userPrefs)` mutate `defaults`, while `Object.assign({}, defaults, userPrefs)` does not?
3. What does `const { a, ...rest } = obj` do?
4. Is `[...array]` a deep copy or a shallow copy? What's the practical implication?
5. Write the destructuring syntax to extract `city` from `{ user: { address: { city: "Mumbai" } } }` in one line.

1. Given `const settings = { theme: "dark", notifications: true, volume: 50 };`, write a function `updateSetting(settings, key, value)` that returns a new settings object with just that one key changed, without mutating the original.
2. Use array destructuring with default values to safely extract the first two items from a potentially short array: `const [a = "default-a", b = "default-b"] = someArray;`. Test it with `[]`, `["only-one"]`, and `["one", "two"]`.
3. Given two objects `{ name: "Ammaar" }` and `{ age: 17 }`, combine them into one object using spread syntax.

> View solutions

Knowledge Check

1. Dot notation (`obj.key`) is used when you know the exact, static property name and it's a valid identifier. Bracket notation (`obj[key]` or `obj["some-key"]`) must be used when the property name is stored in a variable, computed dynamically, or contains characters (like hyphens or spaces) that aren't valid in dot notation.
2. `Object.assign(target, ...sources)` uses its FIRST argument as the target it copies properties INTO, mutating it directly. Passing `{}` as the first argument gives it a fresh, empty object to mutate instead of one of your real objects, effectively producing a new merged object without touching the originals.
3. It destructures `obj`, extracting the value of `a` into its own variable, and collects ALL remaining properties (everything except `a`) into a new object called `rest`.
4. Shallow copy - only the top-level array structure is new; if the array contains objects, those nested objects are still shared references with the original array.
5. `const { user: { address: { city } } } = { user: { address: { city: "Mumbai" } } };`

Coding Challenges

1.

```
function updateSetting(settings, key, value) {
  return { ...settings, [key]: value };
}
```

```
const newSettings = updateSetting(settings, "volume", 80);
console.log(newSettings); // { theme: "dark", notifications: true, volume: 80 }
console.log(settings);      // { theme: "dark", notifications: true, volume: 50 }  -  untouched
```

1

```
function testDestructure(someArray) {
  const [a = "default-a", b = "default-b"] = someArray;
  console.log(a, b);
}

testDestructure([]);                    // "default-a" "default-b"
testDestructure(["only-one"]);         // "only-one" "default-b"
testDestructure(["one", "two"]);       // "one" "two"
```

1

```
const merged = { ...{ name: "Ammaar" }, ...{ age: 17 } };
console.log(merged); // { name: "Ammaar", age: 17 }
```

Cheat Sheet - Array Methods

Method	Mutates?	Returns
`.push(x)`	[x] Yes	New length
`.pop()`	[x] Yes	Removed element
`.shift()`	[x] Yes	Removed element
`.unshift(x)`	[x] Yes	New length
`.splice()`	[x] Yes	Array of removed elements
`.sort()`	[x] Yes	The same array (sorted)
`.reverse()`	[x] Yes	The same array (reversed)
`.slice()`	[x] No	New array (portion)
`.concat()`	[x] No	New array
`.map()`	[x] No	New array (transformed)
`.filter()`	[x] No	New array (subset)
`.reduce()`	[x] No	Single accumulated value
`.forEach()`	[x] No	`undefined`
`.find()`	[x] No	First matching element or `undefined`
`.findIndex()`	[x] No	Index or `-1`
`.includes()`	[x] No	Boolean

Method	Mutates?	Returns
`.indexOf()`	[x] No	Index or `-1`
`.some()`	[x] No	Boolean
`.every()`	[x] No	Boolean
`.join()`	[x] No	String
`.flat()`	[x] No	New flattened array

Cheat Sheet - Object Methods

Method	What it does
`Object.keys(obj)`	Array of keys
`Object.values(obj)`	Array of values
`Object.entries(obj)`	Array of `[key, value]` pairs
`Object.assign(target, ...sources)`	Merges sources into target (mutates target!)
`Object.freeze(obj)`	Prevents any further mutation of `obj`
`{ ...obj }`	Shallow copy / merge (spread)
`structuredClone(obj)`	Deep copy

Next: Part 5 - JavaScript Internal Concepts

PART 5 - JAVASCRIPT INTERNAL CONCEPTS [Intermediate][Advanced]

> *This section covers what separates "can write JavaScript" from "understands JavaScript." Interviewers love these topics precisely because they reveal whether you've built a real mental model or just pattern-matched syntax.*

5.1 Recap: Execution Context, Lexical Environment, Scope Chain, Call Stack

We covered execution contexts and the call stack in Part 3.7, and lexical scope/closures in Part 3.8. Quick consolidated recap:

- Execution context: created every time a function runs; holds its local variables, its `this`, and a reference to the outer lexical environment.
- Lexical environment: the "record" of what variables are in scope, plus a link to the parent scope - this link is what forms the scope chain.
- Scope chain: when a variable isn't found in the current scope, JavaScript walks outward through each parent lexical environment until it finds it or reaches global scope.
- Call stack: the LIFO record of currently active execution contexts.

5.2 Stack vs Heap (Conceptual Model)

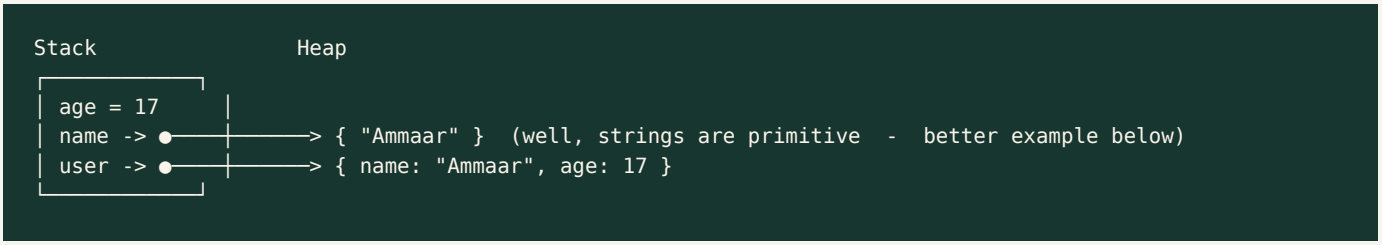
What it is

JavaScript engines use two different memory regions:

- The stack: fast, organized memory used for primitive values and the call stack itself. Values here have a known, fixed size, so the engine can allocate and free them extremely efficiently, in strict last-in-first-out order.
- The heap: a larger, less organized region used for objects, arrays, and functions - anything whose size isn't fixed ahead of time. Reference-type variables (Part 1.6) store a pointer to their actual data in the heap, while that pointer itself lives on the stack.

Mental model

The stack is like a stack of labeled boxes on a table - quick to add and remove from the top. The heap is like a massive warehouse - you can store things of any size, but you need an address (reference) to find them again, and that address is what actually sits in your neat stack of boxes.



This is why primitives copy by value (the stack value itself is copied) and objects copy by reference (only the pointer/address is copied, both variables end up pointing at the same heap location) - the exact behavior you learned in Part 1.6, now with the underlying mechanism.

5.3 Garbage Collection (Basics)

What it is

JavaScript automatically frees memory that's no longer reachable from your code - you never manually allocate or free memory like you would in C. The dominant algorithm modern engines use is called mark-and-sweep: periodically, the engine starts from a set of "roots" (global variables, currently-executing functions' local variables) and marks everything reachable from them. Anything left unmarked is considered garbage and its memory is reclaimed.

Why this matters practically

Closures (Part 3.8) keep variables alive precisely because they remain reachable - as long as a function that references a variable still exists somewhere reachable, that variable can't be garbage collected. This is usually exactly what you want, but it's also how memory leaks happen in long-running applications: if you keep a reference around forever (e.g., an event listener that's never removed, referencing data that should have been freed), that data can never be garbage collected. We'll revisit this practically in Part 6 (removing event listeners) and Part 21 (`useEffect` cleanup).

5.4 this , Binding, Constructors, Prototypes

We covered `this`, `call`/`apply`/`bind` fully in Part 3.9. Here we extend into object-oriented patterns.

```
function Person(name, age) {
  this.name = name;
  this.age = age;
}

Person.prototype.greet = function () {
  console.log(`Hi, I'm ${this.name}`);
};

const ammaar = new Person("Ammaar", 17);
ammaar.greet(); // "Hi, I'm Ammaar"
```

What `new` actually does, step by step:

1. Creates a brand-new empty object: `{}`.
2. Sets that new object's internal prototype link to `Person.prototype`.
3. Calls `Person` with `this` bound to that new object.
4. If `Person` doesn't explicitly return an object of its own, `new` automatically returns the newly created object.

What it is

Every JavaScript object has an internal link to another object called its prototype. When you access a property that doesn't exist directly on an object, JavaScript automatically looks it up on the object's prototype, then that prototype's prototype, and so on - this chain is the prototype chain, and it's how inheritance works at the language's core, beneath the `class` syntax.

```
const animal = {
  makeSound() {
    console.log("Some generic sound");
  }
};

const dog = Object.create(animal); // dog's prototype is `animal`
dog.bark = function () {
  console.log("Woof!");
};

dog.bark();      // "Woof!"  -  own property
dog.makeSound(); // "Some generic sound"  -  found via the prototype chain!
```

Why it exists: it's a memory-efficient way to share behavior (like methods) across many objects without duplicating that behavior onto every single instance. All arrays share `.map`, `.filter`, etc. from

`Array.prototype` - they aren't copied onto every array you create.

What it is

`class` syntax is primarily syntactic sugar over the prototype/constructor-function system above - it doesn't introduce a fundamentally new object model, but it's dramatically more readable and is the modern standard.

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  greet() {
    console.log(`Hi, I'm ${this.name}, age ${this.age}`);
  }
}

const ammaar = new Person("Ammaar", 17);
ammaar.greet(); // "Hi, I'm Ammaar, age 17"

class Student extends Person {
  constructor(name, age, school) {
    super(name, age); // calls Person's constructor
    this.school = school;
  }

  greet() {
    super.greet();       // calls Person's greet() first
    console.log(`I study at ${this.school}`);
  }
}

const student = new Student("Ammaar", 17, "Mumbai School");
student.greet();
// "Hi, I'm Ammaar, age 17"
// "I study at Mumbai School"
```

Why `super()` is mandatory in a subclass constructor: if a `constructor` is defined in a class that `extends` another, it must call `super(...)` before it can use `this` - this ensures the parent class's initialization runs first, so the inherited fields exist before the subclass adds its own.

```
class MathHelper {
  static square(n) {
    return n * n;
  }
}

MathHelper.square(5); // 25  -  called on the CLASS itself, not an instance
// const m = new MathHelper(); m.square(5); // [x] would NOT work  -  static methods aren't on instances
```

Why they exist: for utility functions logically related to a class but that don't need any particular instance's data - like a helper or a factory function.

```
class BankAccount {
  #balance = 0; // # prefix makes this truly private  -  inaccessible from outside the class

  constructor(initialBalance) {
    this.#balance = initialBalance;
  }
```

```
  deposit(amount) {
    this.#balance += amount;
  }

  getBalance() {
    return this.#balance;
  }
}

const account = new BankAccount(1000);
account.deposit(500);
console.log(account.getBalance()); // 1500
console.log(account.#balance);       // [x] SyntaxError  -  truly inaccessible from outside
```

This is the modern, language-enforced equivalent of the closure-based privacy pattern shown in Part 3.8's bank account example - both achieve encapsulation, but `#fields` do it at the class-syntax level rather than via closures.

```
class Rectangle {
  constructor(width, height) {
    this.width = width;
    this.height = height;
  }

  get area() {
    return this.width * this.height;
  }

  set area(value) {
    console.log("Cannot set area directly  -  it's computed.");
  }
}

const rect = new Rectangle(4, 5);
console.log(rect.area); // 20  -  accessed like a PROPERTY, not called like a method (no parentheses!)
rect.area = 100;           // triggers the setter  -  logs the warning
```

Why they exist: getters/setters let you expose computed or validated values through plain property syntax (`obj.area`), hiding the fact that logic is running behind the scenes - useful for validation, computed properties, and controlling how a value can be changed.

5.5 Object-Oriented Programming vs Functional Programming

What they are, briefly

	Object-Oriented Programming (OOP)	Functional Programming (FP)
Core unit	Objects bundling data + behavior (methods)	Pure functions transforming data
State	Often mutable, held inside objects	Favors immutability - new data instead of mutation
Example	`class`, `this`, inheritance	`map`/`filter`/`reduce`, pure functions, composition

Where this course lands

Modern JavaScript, and especially React, leans heavily functional: components are (ideally pure) functions, state updates are immutable (Part 18), and array transformations use `map`/`filter`/`reduce` rather than manual mutation loops. You'll still use `class` occasionally (Error subclasses in Part 9, and you may encounter class components in legacy React code), but the idioms this course builds toward - and that the React team recommends - are predominantly functional. Understanding both styles, rather than treating one as "correct," makes you a better engineer: OOP shines for modeling real-world entities with encapsulated behavior; FP shines for predictable, testable data transformations - which is most of what a frontend does with API data.

Knowledge Check - Part 5

1. Why do primitive values copy "by value" but objects copy "by reference"? Explain using the stack/heap model.
2. What four things does the `new` keyword do when you call `new Person(...)`?
3. What is the prototype chain, and why does it exist?
4. Why must a subclass constructor call `super()` before using `this`?
5. What's the difference between a regular class method and a `static` method?
6. Name one advantage of functional programming's emphasis on immutability, specifically in the context of an application's UI needing to detect when something changed.

> View solutions

1. Primitive values are stored directly on the stack, which holds fixed-size data efficiently - copying a primitive copies its actual value. Objects/arrays are stored in the heap, and the variable on the stack only holds a pointer (address) to that heap location - copying the variable only copies the pointer, so both variables end up referencing the same underlying heap data.
2. It creates a new empty object, links that object's prototype to the constructor function's `.prototype`, calls the constructor function with `this` bound to the new object, and (if the constructor doesn't explicitly return its own object) returns the newly created object automatically.
3. The prototype chain is the linked series of prototype objects JavaScript searches through when a property isn't found directly on an object. It exists so that behavior (like methods) can be shared across many objects/instances without duplicating that behavior onto each one individually - a memory-efficient inheritance mechanism.
4. Because the parent class's constructor is responsible for setting up the base object's initial state; `this` isn't properly initialized in a subclass until the parent constructor (via `super()`) has run, so the language enforces calling it first.
5. A regular method is called on instances of the class (`new ClassName().method()`) and has access to that instance's `this`. A `static` method is called directly on the class itself (`ClassName.method()`) and has no access to any particular instance's data - it's used for utility/helper functionality logically grouped with the class.

6 With immutable updates, "did this data change?" can be answered with a cheap reference comparison (`oldValue !== newValue`) instead of deeply inspecting every property for differences. This is exactly how React efficiently decides whether a component needs to re-render (Part 18, Part 31) - mutating data in place would make that cheap check impossible, since the reference would stay the same even though the contents changed.

Interview Questions - JavaScript Internals

Q: What is the event loop? (Full treatment in Part 10 - flagged here as a preview since it's frequently asked alongside these topics.) The event loop is the mechanism that allows JavaScript - a single-threaded language - to handle asynchronous operations without blocking. It continuously checks whether the call stack is empty, and if so, pushes the next queued callback (from the task queue or microtask queue) onto the stack to be executed.

Q: What's the difference between the stack and the heap?
The stack stores fixed-size data (primitives, and the call stack of function calls) in a fast, last-in-first-out structure. The heap stores dynamically-sized data (objects, arrays, functions) and is accessed via references/pointers stored on the stack.

Q: Explain prototypal inheritance in your own words.
Every object has an internal link to another object (its prototype). When a property lookup fails on the object itself, JavaScript automatically continues the search up through the chain of prototypes. `class`/`extends` is syntactic sugar over this same underlying mechanism.

Next: Part 6 - DOM Manipulation

PART 6 - DOM MANIPULATION [Beginner][Intermediate]

> *Recall from Part 0.9: the DOM is the browser's live, in-memory, editable tree representation of your page. This section teaches you to read and write that tree directly with JavaScript - the exact skill React (starting Part 14) exists to abstract away. Understanding why React exists requires first understanding what doing this by hand actually feels like.*

6.1 Selecting Elements

What it is

Before you can change anything on a page, you need to find (select) the element(s) you want to change. The DOM API gives you several ways to do this.

```
<div id="app">
  <p class="message">Hello</p>
  <p class="message">World</p>
</div>
```

```
// By ID  -  returns a single element (or null)
const app = document.getElementById("app");

// By class name  -  returns a LIVE HTMLCollection (array-like, but not a real array)
const messages = document.getElementsByClassName("message");

// Modern, flexible  -  accepts ANY valid CSS selector
const firstMessage = document.querySelector(".message");   // first match only
const allMessages = document.querySelectorAll(".message"); // ALL matches, as a static NodeList
```

Method	Returns	Selector type	Live or static?
`getElementById(id)`	Single element or `null`	ID only (no `#`)	N/A
`getElementsByClassName(cls)`	HTMLCollection	Class name only (no `.`)	Live - updates automatically as the DOM changes
`querySelector(css)`	Single element or `null`	Any CSS selector	N/A
`querySelectorAll(css)`	NodeList	Any CSS selector	Static - a snapshot, doesn't auto-update

Why `querySelector`/`querySelectorAll` won: they accept the exact same selector syntax you already know from CSS (`.class`, `#id`, `div > p`, `[data-active="true"]`, etc.), making them dramatically more flexible than the older, selector-type-specific methods. Modern best practice: default to `querySelector`/`querySelectorAll` unless you have a specific reason to need a live collection.

⚠ Common mistake - treating a `NodeList` like an array carelessly:

```
const messages = document.querySelectorAll(".message");
messages.map(el => el.textContent); // [x] TypeError: messages.map is not a function
```

Why it fails: `NodeList` has `.forEach()` built in (in modern browsers), but not `.map()`, `.filter()`, or `.reduce()` - it's array-like, not a real array.

Correct version:

```
const messages = document.querySelectorAll(".message");
const texts = Array.from(messages).map(el => el.textContent); // convert to a real array first
// or:
const texts2 = [...messages].map(el => el.textContent); // spread also works
```

6.2 Reading and Writing Content

```
const el = document.querySelector(".message");

el.textContent; // reads ALL text content, including hidden elements, ignoring HTML tags
el.innerText;      // reads only VISIBLE text (respects CSS like display:none), slower (triggers layout)
el.innerHTML;       // reads/writes the actual HTML markup as a string

el.textContent = "New text"; // sets text  -  any HTML-looking characters are treated as LITERAL TEXT, safely
el.innerHTML = "<strong>Bold</strong>"; // sets HTML  -  actually parses and renders the tag
```

`textContent` vs `innerText` vs `innerHTML` - when to use which:

	Includes hidden text?	Parses HTML?	Performance
`textContent`	[x] Yes	[x] No - treated as plain text	Fast
`innerText`	[x] No	[x] No	Slower (forces a layout recalculation)
`innerHTML`	[x] Yes	[x] Yes - actually creates elements	Fast to read, risky to write

⚠ Security concern with `innerHTML` - Cross-Site Scripting (XSS):

```
// [Advanced] DANGEROUS if `userComment` comes from user input or an API you don't fully trust:
el.innerHTML = userComment;
```

Why this is dangerous: if `userComment` contains something like `<img src=x onerror="stealCookiesAndSendToAttacker()">`, that script will actually execute in your page, with full access to whatever your page can access (cookies, tokens, the DOM). This is called Cross-Site Scripting (XSS), and it's one of the most common and dangerous web vulnerabilities. We cover this fully in Part 34 (Security), but the rule from day one is: never set `innerHTML` with content you didn't write yourself or haven't sanitized. Default to `textContent` for any dynamic, user-provided, or API-provided text.

```
// [x] Safe  -  always renders as plain text, even if it contains "<script>" characters
el.textContent = userComment;
```

6.3 Attributes

```
const link = document.querySelector("a");

link.getAttribute("href");             // reads the attribute's raw value
link.setAttribute("href", "https://example.com");
link.removeAttribute("target");
link.hasAttribute("href");                // true/false

<div id="product" data-product-id="42" data-in-stock="true"></div>

const product = document.getElementById("product");

product.dataset.productId; // "42"    -  note: kebab-case in HTML becomes camelCase in JS!
product.dataset.inStock;    // "true"  -  ⚠ this is a STRING, not a real boolean!

product.dataset.newField = "hello"; // adds data-new-field="hello" to the actual HTML
```

⚠ Common mistake: all `dataset` values are strings, even ones that "look like" booleans or numbers.

```
if (product.dataset.inStock) { // [x] ALWAYS truthy  -  it's the STRING "true" or "false", and both are non-empty st
  console.log("In stock!"); // this runs even when data-in-stock="false" !!
}
```

Correct version:

```
if (product.dataset.inStock === "true") { // [x] explicit string comparison
  console.log("In stock!");
}
```

6.4 CSS: Style, className, classList

```
const box = document.querySelector(".box");

// Direct inline style (use sparingly  -  see best practice below):
box.style.backgroundColor = "blue";
box.style.fontSize = "16px"; // note: camelCase for hyphenated CSS properties, and units are required

// Reading/replacing the ENTIRE class attribute:
box.className;            // "box active" (a string)
box.className = "box"; // ⚠ OVERWRITES all classes, not just adds one!

// classList  -  the modern, safe way to manage classes:
box.classList.add("active");
box.classList.remove("hidden");
box.classList.toggle("open");         // adds if absent, removes if present
box.classList.toggle("open", true);  // force it to be present
box.classList.contains("active");    // true/false
```

 Best practice: Prefer `classList` over directly setting `element.style`. Keep your CSS rules in `.css` files, and use JavaScript only to toggle which class names are applied. This keeps styling concerns in CSS (separation of concerns, Part 0.7) and avoids scattering inline style logic throughout your JavaScript - it's also exactly the pattern that translates most directly into how React handles conditional styling (Part 15-17).

6.5 Creating and Removing Elements

```
// Creating a new element:
const newParagraph = document.createElement("p");
newParagraph.textContent = "I'm new here!";
newParagraph.classList.add("message");

const container = document.querySelector("#app");

container.appendChild(newParagraph); // adds as the LAST child (older API)
container.append(newParagraph);          // modern equivalent  -  also accepts plain strings, and multiple arguments
container.prepend(newParagraph);        // adds as the FIRST child

const existingEl = document.querySelector(".message");
existingEl.before(newParagraph);          // inserts immediately BEFORE existingEl
existingEl.after(newParagraph);           // inserts immediately AFTER existingEl

// Removing:
existingEl.remove(); // modern, simple  -  removes the element from the DOM entirely
```

Practical example - building a list from data:

```
const fruits = ["Apple", "Banana", "Cherry"];
const list = document.querySelector("#fruit-list");

fruits.forEach(fruit => {
  const li = document.createElement("li");
  li.textContent = fruit;
  list.append(li);
});
```

⚠ Common mistake - building HTML with string concatenation and `innerHTML` instead of `createElement`:

```
// [Advanced] Works, but revisits the XSS risk from 6.2 if `fruit` ever comes from user input
list.innerHTML += `<li>${fruit}</li>`;
```

Why `createElement` is generally safer and better: setting `.textContent` on a created element (as in the correct example above) never interprets its input as HTML, so it's immune to injection regardless of what the string contains. `innerHTML +=` also has a subtle performance problem: it destroys and rebuilds the entire subtree every time (because `+=` reads the current `innerHTML`, concatenates a string, and reassigns the whole thing), losing any existing event listeners or state on the existing children.

6.6 Events

Events are how your JavaScript finds out that something happened - a click, a key press, a form submission, the page finishing loading. This is the mechanism that turns a static page into an interactive one.

What it is

```
element.addEventListener("eventType", callbackFunction);

const button = document.querySelector("#myButton");

button.addEventListener("click", function (event) {
  console.log("Button was clicked!");
});

// Arrow function version (preferred in modern code, per Part 3.9):
button.addEventListener("click", (event) => {
  console.log("Clicked!", event);
});
```

Why `addEventListener` over `onclick="..."` (inline HTML) or `element.onclick = fn`:

- You can attach multiple listeners for the same event to the same element (`onclick = fn` can only hold ONE function - assigning a second overwrites the first).
- It keeps behavior (JavaScript) separate from structure (HTML) - recall the separation-of-concerns principle from Part 0.7.
- It supports fine-grained control over event phase (capturing vs bubbling, see 6.7) and easy removal via `removeEventListener`.

Every event listener callback receives an event object describing what happened.

```
button.addEventListener("click", (event) => {
  console.log(event.type);          // "click"
  console.log(event.target);        // the actual element that triggered the event
  console.log(event.currentTarget); // the element the LISTENER is attached to
  console.log(event.clientX, event.clientY); // mouse coordinates (for mouse events)
});
```

Category	Events
Mouse	`click`, `dblclick`, `mousedown`, `mouseup`, `mouseover`, `mouseout`, `mousemove`

Category	Events
Keyboard	`keydown`, `keyup` (note: `keypress` is deprecated - avoid it)
Form	`submit`, `input`, `change`, `focus`, `blur`
Window/Document	`DOMContentLoaded`, `load`, `resize`, `scroll`
Touch/Pointer	`touchstart`, `touchend`, `pointerdown`, `pointermove` (pointer events unify mouse + touch)

```
const textInput = document.querySelector("#name");

textInput.addEventListener("input", (e) => {
  console.log("Fires on EVERY keystroke:", e.target.value);
});

textInput.addEventListener("change", (e) => {
  console.log("Fires only when the field LOSES FOCUS after a change:", e.target.value);
});
```

When to use which: use `input` for live feedback (character counters, live search, live validation as you'll build in Part 7). Use `change` when you only care about the final, committed value (e.g., a `<select>` dropdown, where `change` fires immediately on selection anyway).

```
document.addEventListener("keydown", (e) => {
  console.log(e.key);        // "Enter", "a", "Shift", "ArrowDown", etc.  -  human-readable
  console.log(e.code);      // "KeyA", "Enter", "ShiftLeft"  -  physical key, layout-independent

  if (e.key === "Enter") {
    console.log("Enter was pressed!");
  }
});

<form id="signupForm">
  <input type="email" name="email" />
  <button type="submit">Sign up</button>
</form>

const form = document.querySelector("#signupForm");

form.addEventListener("submit", (event) => {
  event.preventDefault(); // ⚠ CRUCIAL  -  without this, the browser does a FULL PAGE RELOAD
  console.log("Form submitted via JavaScript instead!");
});
```

Why `preventDefault()` is essential here: a `<form>`'s default browser behavior, on submit, is to navigate to a new page (a full HTTP request/response cycle, reloading everything - recall Part 0.15). In an SPA (Part 0.14) or any app that wants to handle the submission with JavaScript (validate it, send it via `fetch`, Part 11), you must call `preventDefault()` to stop that default full-page-reload behavior.

6.7 Event Target, Bubbling, Capturing, Propagation

What it is

When an event fires on an element, it doesn't just affect that one element - it travels through the DOM tree in a defined sequence called propagation, which has three phases:

```
1. CAPTURING phase: window -> document -> html -> body -> ... -> target (event travels DOWN)
2. TARGET phase: the event reaches the actual element that was interacted with
3. BUBBLING phase: target -> ... -> body -> html -> document -> window (event travels back UP)
```

Mental model

Imagine dropping a pebble into a pond at a specific point (the target). Ripples spread outward from that point (bubbling) - every ancestor element "feels" the event, in order, from the innermost parent outward to the outermost. Capturing is the mirror image - imagine ripples converging inward toward the target before it's hit.

```
<div id="outer">
  <div id="inner">
    <button id="btn">Click</button>
  </div>
</div>

document.getElementById("outer").addEventListener("click", () => console.log("outer"));
document.getElementById("inner").addEventListener("click", () => console.log("inner"));
document.getElementById("btn").addEventListener("click", () => console.log("btn"));

// Clicking the button logs, in order:
// "btn"
// "inner"
// "outer"
```

Why: by default, `addEventListener` listens during the bubbling phase. The click starts at `btn` (the target), then bubbles up through `inner`, then `outer`, triggering each attached listener along the way, in that order.

`event.target` vs `event.currentTarget`:

```
document.getElementById("outer").addEventListener("click", (e) => {
  console.log("target:", e.target.id);           // "btn"  -  whatever was ACTUALLY clicked
  console.log("currentTarget:", e.currentTarget.id); // "outer"  -  the element THIS listener is attached to
});

document.getElementById("inner").addEventListener("click", (e) => {
  e.stopPropagation(); // stops the event from bubbling any further up
  console.log("inner  -  and it stops here");
});
// Clicking the button now only logs "btn" and "inner"  -  "outer" never fires

element.addEventListener("click", handler, true); // third argument `true` = capturing phase
// or, more explicitly:
element.addEventListener("click", handler, { capture: true });
```

This is rarely needed in typical application code, but it's useful to know it exists - some libraries and advanced patterns (like detecting a click before it reaches an inner element that might stop propagation) rely on it.

What it is

Instead of attaching a separate listener to every individual child element, attach one listener to a common parent, and use `event.target` to figure out which specific child was actually interacted with.

Why it exists

```
// [Advanced] Without delegation  -  must attach (and later, remove!) a listener on EVERY item,
// and this breaks for any items added to the list LATER:
document.querySelectorAll(".list-item").forEach(item => {
  item.addEventListener("click", () => console.log("Clicked:", item.textContent));
});
```

Problems with this approach: if items are added to the list dynamically after this code runs, they won't have a listener attached - you'd have to remember to re-run this setup every time the list changes. It's also more memory-intensive with hundreds of items.

Practical example - event delegation, the correct pattern:

```
const list = document.querySelector("#item-list");

list.addEventListener("click", (event) => {
  // Use closest() to find the nearest ancestor (or self) matching a selector,
  // starting from whatever was actually clicked:
  const item = event.target.closest(".list-item");

  if (!item) return; // clicked somewhere in the list but NOT on an item (e.g., empty space)

  console.log("Clicked:", item.textContent);
});

// This SINGLE listener correctly handles items added to the list at ANY point in the future  -
// even ones that don't exist yet when this code runs!
```

Step-by-step: because of event bubbling, a click on any `.list-item` (or anything inside one) eventually bubbles up to `#item-list`, where our one listener catches it. `event.target.closest(".list-item")` walks up from the actual click target to find the nearest `.list-item` ancestor (or the element itself, if it matches), correctly identifying which item was clicked, regardless of whether the click landed on the item itself or a nested child (like an icon inside it).

 Tip: Event delegation is a professional-grade pattern used constantly in real applications, especially for dynamically generated lists. It's also conceptually the ancestor of how React's own internal event system works (React attaches most listeners at a single root and uses synthetic dispatching) - understanding delegation now will make that internal behavior far less mysterious later.

```
function handleClick() {
  console.log("clicked");
}

button.addEventListener("click", handleClick);
button.removeEventListener("click", handleClick); // must pass the SAME function reference
```

⚠ Common mistake:

```
button.addEventListener("click", () => console.log("clicked"));
button.removeEventListener("click", () => console.log("clicked")); // [x] does NOTHING!
```

Why it fails: each arrow function literal creates a brand-new function, even if it looks identical. `removeEventListener` needs the exact same function reference that was originally passed to `addEventListener` - you must store the function in a named variable to be able to remove it later.

> *Why this matters (preview of Part 5.3 and Part 21): forgetting to remove listeners that are no longer needed is a classic cause of memory leaks - the listener (and anything it closes over) stays reachable and alive forever, even after the element it was meant for is gone. This exact concern is why* `useEffect`*'s cleanup function (Part 21) exists in React.*

6.8 Practical DOM Project - Interactive To-Do List (Vanilla JS)

Requirements

Build a to-do list where you can:

- Type a task and press Enter (or click "Add") to add it to the list.
- Click a task to toggle it as completed (strikethrough styling).
- Click a "delete" button on a task to remove it.

Concepts tested: element selection, creating elements, event listeners, event delegation, `classList`, `preventDefault`.

Suggested structure:

```
<div id="app">
  <form id="todo-form">
    <input type="text" id="todo-input" placeholder="Add a task..." />
    <button type="submit">Add</button>
  </form>
  <ul id="todo-list"></ul>
</div>
```

Attempt this yourself before reading the solution below. Think about: where does event delegation help here? What data do you need to track for each to-do item?

> View solution

```
const form = document.querySelector("#todo-form");
const input = document.querySelector("#todo-input");
const list = document.querySelector("#todo-list");

form.addEventListener("submit", (event) => {
  event.preventDefault(); // stop the full-page-reload default behavior

  const text = input.value.trim();
  if (!text) return; // guard clause  -  ignore empty submissions

  addTodoToDOM(text);
  input.value = ""; // clear the input for the next entry
  input.focus();
});

function addTodoToDOM(text) {
  const li = document.createElement("li");
  li.className = "todo-item";

  const span = document.createElement("span");
  span.textContent = text;
  span.className = "todo-text";

  const deleteBtn = document.createElement("button");
  deleteBtn.textContent = "Delete";
  deleteBtn.className = "delete-btn";

  li.append(span, deleteBtn); // append() accepts multiple arguments
```

```
  list.append(li);
}

// Event delegation  -  ONE listener on the parent handles clicks for ALL current and future items:
list.addEventListener("click", (event) => {
  if (event.target.classList.contains("delete-btn")) {
    event.target.closest(".todo-item").remove();
    return;
  }

  if (event.target.classList.contains("todo-text")) {
    event.target.classList.toggle("completed");
  }
});

.completed {
  text-decoration: line-through;
  color: gray;
}
```

Why this architecture: the delegation pattern on `#todo-list` means we never have to attach a new listener every time a to-do is added - a single listener handles clicks for every item, present and future, which is both simpler and more efficient than attaching individual listeners in `addTodoToDOM`.

Knowledge Check - Part 6

1. Why is `textContent` generally safer than `innerHTML` when displaying dynamic content?
2. What's wrong with checking `if (element.dataset.isActive)` to test a boolean data attribute?
3. Explain the three phases of event propagation, in order.
4. What's the difference between `event.target` and `event.currentTarget`?
5. Why does event delegation handle dynamically-added elements correctly, when individually-attached listeners don't?
6. Why does `removeEventListener` sometimes silently fail to remove a listener that was clearly added with `addEventListener`?

1. Build a character counter: as a user types into a `<textarea>`, display "X characters remaining" out of a max of 200, live.
2. Using event delegation, build a list where clicking any `<li>` toggles a `"selected"` class on it (and only it - clicking one shouldn't affect others).
3. Write a function that safely inserts user-provided text into the page without any XSS risk, regardless of what the text contains.

```
const items = document.querySelectorAll(".item");
items.forEach(item => {
  item.addEventListener("click", () => {
    item.style.display = "none";
  });
});
// Later, more .item elements are added to the page dynamically, but clicking them does nothing.
```

> View solutions

Knowledge Check

1. `textContent` always treats its input as plain text - it never parses or executes HTML/scripts, so it can't be exploited for Cross-Site Scripting (XSS), even if the text happens to contain characters that look like HTML tags.

2. `dataset` values are always strings. `element.dataset.isActive` will be truthy for BOTH `"true"` and `"false"` (and any other non-empty string), because both are non-empty strings. You must explicitly compare: `element.dataset.isActive === "true"`.

3. Capturing (event travels from `window` down to the target), target (the event reaches the actual element interacted with), and bubbling (the event travels back up from the target to `window`).

4. `event.target` is the actual element the event originated from (what was actually clicked/interacted with). `event.currentTarget` is the element the currently executing listener is attached to - these can differ when a listener on a parent handles an event that actually happened on a child (due to bubbling).

5. Because the listener is attached to a stable parent element, not to the individual children. Any click on a child (existing or future) bubbles up to that parent, where the one listener is already waiting - no new listener needs to be attached when new children are added.

6. Because `removeEventListener` requires the exact same function reference originally passed to `addEventListener`. Passing a new anonymous/arrow function (even one with identical code) to `removeEventListener` does not match the original reference, so nothing is removed.

Coding Challenges

1

```
const textarea = document.querySelector("#bio");
const counter = document.querySelector("#counter");
const MAX = 200;

textarea.addEventListener("input", () => {
  const remaining = MAX - textarea.value.length;
  counter.textContent = `${remaining} characters remaining`;
});
```

1

```
document.querySelector("#list").addEventListener("click", (event) => {
  const li = event.target.closest("li");
  if (!li) return;
  li.classList.toggle("selected");
});
```

1

```
function safeInsert(container, text) {
  container.textContent = text; // never parses HTML  -  always safe
}
```

Debugging Challenge

The bug: listeners were attached individually with `items.forEach(...)` at the time the code ran, so only the elements that existed at that moment got a listener. Items added to the DOM later were never included in that `querySelectorAll` snapshot (which, recall from 6.1, is static, not live) and never got a listener attached.

Fix - use event delegation on a stable parent instead:

```
const container = document.querySelector("#items-container");
container.addEventListener("click", (event) => {
  const item = event.target.closest(".item");
  if (!item) return;
  item.style.display = "none";
});
// Now works correctly for ALL .item elements, including ones added after this code runs.
```

Cheat Sheet - DOM

Task	Code
Select one element	`document.querySelector(css)`
Select all matches	`document.querySelectorAll(css)`
Read/write text safely	`el.textContent`
Read/write HTML (risky)	`el.innerHTML`
Get/set attribute	`el.getAttribute(name)` / `el.setAttribute(name, val)`
Data attributes	`el.dataset.someName`
Add/remove/toggle class	`el.classList.add/remove/toggle(name)`
Create element	`document.createElement(tag)`
Insert element	`parent.append(el)` / `el.before(x)` / `el.after(x)`
Remove element	`el.remove()`
Listen for event	`el.addEventListener(type, fn)`
Stop bubbling	`event.stopPropagation()`
Prevent default action	`event.preventDefault()`

Cheat Sheet - Events

Event	Fires when...
`click`	Element is clicked
`input`	Value changes, on every keystroke

Event	Fires when...
`change`	Value is committed (e.g., field loses focus, or a select changes)
`submit`	A form is submitted
`keydown` / `keyup`	A key is pressed / released
`focus` / `blur`	Element gains / loses focus
`DOMContentLoaded`	HTML is fully parsed (doesn't wait for images/styles)
`load`	Entire page, including all resources, has loaded

Next: Part 7 - Forms

PART 7 - FORMS [Beginner][Intermediate]

7.1 Reading Input Values

```
<form id="signupForm">
  <input type="text" id="username" />
  <input type="email" id="email" />
  <input type="password" id="password" />
  <input type="checkbox" id="agree" />
  <button type="submit">Sign up</button>
</form>

const username = document.querySelector("#username");
const agree = document.querySelector("#agree");

username.value;   // reads the current text value (always a string, for text/email/password inputs)
agree.checked;      // reads a boolean for checkboxes/radio buttons  -  NOT .value!
```

⚠ Common mistake: trying to read `.checked` on a text input, or `.value` to check if a checkbox is ticked - `.value` on a checkbox always returns its (usually meaningless, default `"on"`) attribute value, not whether it's checked. Use `.checked` specifically for checkboxes and radio buttons.

7.2 Handling Submission and Validation

```
const form = document.querySelector("#signupForm");

form.addEventListener("submit", (event) => {
  event.preventDefault(); // recall from Part 6.6  -  stops the full page reload

  const username = document.querySelector("#username").value.trim();
  const email = document.querySelector("#email").value.trim();
  const password = document.querySelector("#password").value;
  const agree = document.querySelector("#agree").checked;

  const errors = [];

  if (username.length < 3) {
    errors.push("Username must be at least 3 characters.");
  }
  if (!isValidEmail(email)) {
    errors.push("Please enter a valid email address.");
  }
  if (password.length < 8) {
    errors.push("Password must be at least 8 characters.");
  }
  if (!agree) {
    errors.push("You must agree to the terms.");
  }

  if (errors.length > 0) {
    showErrors(errors);
    return; // guard clause  -  stop here, don't proceed to "submit"
  }

  console.log("Form is valid! Proceeding...");
});

function showErrors(errors) {
  const errorBox = document.querySelector("#errors");
  errorBox.textContent = ""; // clear previous errors
  errors.forEach(msg => {
    const p = document.createElement("p");
    p.textContent = msg; // textContent  -  safe, recall Part 6.2
    p.className = "error-message";
    errorBox.append(p);
  });
}
```

7.3 Regular Expressions - Basics for Validation

What it is

A regular expression (regex) is a pattern-matching language for checking whether text matches a certain shape.

```
const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

function isValidEmail(email) {
  return emailPattern.test(email); // .test() returns true/false
}

console.log(isValidEmail("ammaar@example.com")); // true
console.log(isValidEmail("not-an-email"));           // false
```

Reading this pattern, piece by piece:

- `^` - start of the string.
- `[^\s@]+` - one or more characters that are NOT whitespace (`\s`) and NOT `@`.
- `@` - a literal `@` character.
- `[^\s@]+` - one or more non-whitespace, non-`@` characters (the domain).
- `\.` - a literal dot (escaped, because `.` normally means "any character" in regex).
- `[^\s@]+` - the part after the dot (e.g., `com`).
- `$` - end of the string.

⚠ Important, honest caveat: fully correct email validation via regex alone is notoriously difficult - the official email spec allows far more edge cases than most patterns account for. Best practice for real applications: use a simple, reasonable client-side regex (like above) for immediate user feedback, but always perform the authoritative validation on the server (recall Part 0.3 - never trust the client), and the most reliable "is this email real" check is sending a confirmation email, not a regex.

Password strength pattern (example):

```
// At least 8 characters, one uppercase, one lowercase, one digit:
const strongPasswordPattern = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d).{8,}$/;

function isStrongPassword(password) {
  return strongPasswordPattern.test(password);
}
```

 Tip: you don't need to memorize regex syntax. In real work, you'll look up or reuse well-tested patterns for common cases like this. What matters is understanding what a regex is checking and being able to read one carefully enough to verify it does what you think it does.

7.4 FormData

What it is

A browser API that reads all the values out of a `<form>` element at once, keyed by each input's `name` attribute - much more convenient than manually querying every field.

```
<form id="profileForm">
  <input type="text" name="fullName" />
  <input type="email" name="email" />
  <input type="file" name="avatar" />
</form>

const form = document.querySelector("#profileForm");

form.addEventListener("submit", (event) => {
  event.preventDefault();

  const formData = new FormData(form);

  console.log(formData.get("fullName")); // reads a single field by name
  console.log(formData.get("email"));
```

```
  // Converting to a plain object (works well when there are no duplicate field names):
  const data = Object.fromEntries(formData);
  console.log(data); // { fullName: "...", email: "...", avatar: File }

  // Iterating over every field:
  for (const [key, value] of formData) {
    console.log(key, value);
  }
});
```

Why `FormData` matters especially for Part 11: `FormData` objects can be passed directly as the `body` of a `fetch()` request (Part 11.9) - this is the standard way to send file uploads over HTTP, because `FormData` automatically encodes itself as `multipart/form-data`, the format required for binary file data.

7.5 File Inputs

```
<input type="file" id="avatar" accept="image/*" />
<input type="file" id="documents" multiple accept=".pdf,.docx" />

const avatarInput = document.querySelector("#avatar");

avatarInput.addEventListener("change", (event) => {
  const file = event.target.files[0]; // .files is a FileList, even for a single file input

  if (!file) return; // user cancelled the file picker

  console.log(file.name);   // "photo.jpg"
  console.log(file.size);   // size in bytes
  console.log(file.type);    // "image/jpeg"
});

const documentsInput = document.querySelector("#documents");
documentsInput.addEventListener("change", (event) => {
  const files = Array.from(event.target.files); // convert FileList to a real array
  files.forEach(file => console.log(file.name));
});
```

Previewing an image before upload - a very common pattern:

```
avatarInput.addEventListener("change", (event) => {
  const file = event.target.files[0];
  if (!file) return;

  const reader = new FileReader();
  reader.onload = (e) => {
    document.querySelector("#preview").src = e.target.result; // a base64 data URL
  };
  reader.readAsDataURL(file);
});
```

Step-by-step: `FileReader` reads a `File` object's raw bytes asynchronously (notice the `onload` callback - we haven't formally covered async yet, that's all of Part 10, but this is a natural early taste of it). `readAsDataURL` converts the file into a base64-encoded string that can be dropped directly into an `<img src="...">` for an instant local preview, with no server round-trip needed.

7.6 Complete Form Project - Signup Form with Live Validation

Requirements

- Username, email, password, confirm-password fields.
- Live validation feedback as the user types (`input` event, Part 6.6).
- Submit button disabled until the form is valid.
- On successful submit, show a success message and don't reload the page.

View solution

```
<form id="signupForm" novalidate>
  <div>
    <input type="text" id="username" placeholder="Username" />
    <span class="error" id="usernameError"></span>
  </div>
  <div>
    <input type="email" id="email" placeholder="Email" />
    <span class="error" id="emailError"></span>
  </div>
  <div>
    <input type="password" id="password" placeholder="Password" />
    <span class="error" id="passwordError"></span>
  </div>
  <div>
    <input type="password" id="confirmPassword" placeholder="Confirm password" />
    <span class="error" id="confirmError"></span>
  </div>
  <button type="submit" id="submitBtn" disabled>Sign up</button>
  <p id="successMessage" style="display:none;">Account created!</p>
</form>

const fields = {
  username: document.querySelector("#username"),
  email: document.querySelector("#email"),
  password: document.querySelector("#password"),
  confirmPassword: document.querySelector("#confirmPassword")
};

const errors = {
  username: document.querySelector("#usernameError"),
  email: document.querySelector("#emailError"),
  password: document.querySelector("#passwordError"),
  confirmPassword: document.querySelector("#confirmError")
};

const submitBtn = document.querySelector("#submitBtn");
const form = document.querySelector("#signupForm");
const successMessage = document.querySelector("#successMessage");

function validate() {
  let isValid = true;

  // Username
  if (fields.username.value.trim().length < 3) {
    errors.username.textContent = "At least 3 characters required.";
    isValid = false;
  } else {
    errors.username.textContent = "";
```

```
  }

  // Email
  const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!emailPattern.test(fields.email.value.trim())) {
    errors.email.textContent = "Enter a valid email.";
    isValid = false;
  } else {
    errors.email.textContent = "";
  }

  // Password
  if (fields.password.value.length < 8) {
    errors.password.textContent = "At least 8 characters required.";
    isValid = false;
  } else {
    errors.password.textContent = "";
  }

  // Confirm password
  if (fields.confirmPassword.value !== fields.password.value) {
    errors.confirmPassword.textContent = "Passwords don't match.";
    isValid = false;
  } else {
    errors.confirmPassword.textContent = "";
  }

  submitBtn.disabled = !isValid;
  return isValid;
}

// Attach live validation to every field:
Object.values(fields).forEach(field => {
  field.addEventListener("input", validate);
});

form.addEventListener("submit", (event) => {
  event.preventDefault();
  if (!validate()) return; // guard clause  -  should be unreachable since the button is disabled, but never trust t

  form.style.display = "none";
  successMessage.style.display = "block";
});
```

Why validate on every `input` event AND on submit: the `input` listeners give live feedback and control the disabled state of the button. But relying only on a disabled button is fragile (what if the disabled attribute gets removed by a browser extension, or the check has a bug?) - re-validating on submit is a defensive habit, and it's the same "defense in depth" principle behind never trusting client-side-only validation for anything that matters (Part 34).

Knowledge Check - Part 7

1. Why does `.value` return `"on"` (or similar) for a checkbox instead of a useful true/false? What should you use instead?
2. What does `FormData` give you that manually querying each input doesn't?
3. Why is `event.target.files[0]`, not `event.target.value`, the way to access an uploaded file?

4 Why is client-side email regex validation not sufficient on its own?

> View solutions

1 `.value` on a checkbox reflects its `value` HTML attribute (which defaults to `"on"` if unset), not its checked state - it exists for when a checkbox is submitted as part of traditional form data. Use `.checked` to read whether it's actually ticked.

2 `FormData` automatically collects every field's current value, keyed by its `name` attribute, in one object-like structure - including files - without you having to manually `querySelector` each field individually. It's also directly usable as a `fetch()` request body for `multipart/form-data` submissions.

3 `<input type="file">` doesn't store its selected file(s) in `.value` (in fact, for security reasons, `.value` only ever exposes a fake placeholder path, not real file access) - actual `File` objects are exposed through the `.files` `FileList` property instead.

4 Because it's trivial for anyone to bypass client-side JavaScript entirely (disable it, or send a request directly, recall Part 0.3) - real validation must also happen on the server, where it can't be circumvented.

Next: Part 8 - Browser Storage

PART 8 - BROWSER STORAGE [Beginner][Intermediate]

8.1 localStorage and sessionStorage

What they are

Both are simple key-value storage mechanisms built into the browser, tied to a specific website's origin (domain). They only store strings.

	`localStorage`	`sessionStorage`
Persists across browser restarts?	[x] Yes - until explicitly cleared	[x] No - cleared when the tab closes
Shared across tabs of the same site?	[x] Yes	[x] No - each tab has its own
Typical use	Remembering preferences, cached data, a saved to-do list	Data specific to one browsing session (e.g., a multi-step form's progress)

Basic API (identical for both):

```
localStorage.setItem("username", "Ammaar"); // store
localStorage.getItem("username");                // "Ammaar"  -  retrieve
localStorage.removeItem("username");           // delete one key
localStorage.clear();                                 // delete EVERYTHING for this site
```

```
localStorage.key(0);                                    // get the key name at a given index
localStorage.length;                                     // number of stored items
```

8.2 Storing Non-String Data with JSON.stringify / JSON.parse

⚠ Critical limitation: `localStorage`/`sessionStorage` can only store strings. Attempting to store an object or array directly silently converts it to the useless string `"[object Object]"`.

```
const user = { name: "Ammaar", age: 17 };

localStorage.setItem("user", user); // [x] WRONG
console.log(localStorage.getItem("user")); // "[object Object]"  -  data is LOST
```

Correct version - always serialize objects/arrays with `JSON.stringify`, and parse them back with `JSON.parse`:

```
localStorage.setItem("user", JSON.stringify(user)); // [x] "{"name":"Ammaar","age":17}"

const raw = localStorage.getItem("user");
const parsedUser = JSON.parse(raw);
console.log(parsedUser.name); // "Ammaar"  -  a real object again
```

Defensive reading (a value might not exist yet, or might be corrupted):

```
function getStoredUser() {
  const raw = localStorage.getItem("user");
  if (!raw) return null; // key doesn't exist yet

  try {
    return JSON.parse(raw);
  } catch (error) {
    console.error("Corrupted user data in localStorage:", error);
    return null;
  }
}
```

We'll formalize this try/catch pattern fully in Part 9 - this is a preview of why it's necessary even for something as simple as reading storage.

8.3 Cookies (Conceptual Overview)

What they are

Cookies are small pieces of data, also stored per-origin, but with a key difference from `localStorage`: cookies are automatically sent to the server with every matching HTTP request (recall Part 0.4). This makes them useful for server-side session management (like "is this user logged in?"), but also means they add overhead to every request and have a much smaller size limit (~~4KB) than~~ ~~`localStorage`~~ (5-10MB).

```
document.cookie = "theme=dark; max-age=3600"; // basic, clunky API  -  rarely used directly in modern apps
```

In modern practice, cookies are typically set and managed by the server (via HTTP response headers), especially for authentication tokens, because a cookie marked `HttpOnly` cannot be read by JavaScript at all - a deliberate security measure against XSS attacks stealing session tokens (covered fully in Part 34). For general client-side data storage that doesn't need to be sent to a server automatically, `localStorage` is almost always the better, simpler choice.

8.4 IndexedDB (Overview)

What it is

A full, transactional, object-oriented database built into the browser, capable of storing much larger amounts of structured data (including binary data like files/blobs) than `localStorage`, with support for indexes and complex queries.

When to reach for it: `localStorage` is fine for simple key-value data under a few MB (settings, small lists, cached API responses). If you need to store large datasets, work offline with structured/queryable data, or store binary files client-side, `IndexedDB` is the right tool - though its raw API is notoriously verbose, so in practice most teams use a wrapper library over it. For this course's purposes: know it exists and what it's for; you won't need its raw API for any exercises here.

8.5 Project - Persistent To-Do Application

Requirements

Extend the to-do list from Part 6.8 so that the list survives a page refresh, by saving to `localStorage` every time it changes, and loading from `localStorage` on page load.

Concepts tested: `localStorage`, `JSON.stringify`/`JSON.parse`, DOM manipulation, event delegation, defensive coding.

Attempt this yourself first - think about: what's the actual data your UI represents (a list of `{ text, completed }` objects), separate from the DOM elements themselves? What has to happen every time that data changes?

View solution

```
const form = document.querySelector("#todo-form");
const input = document.querySelector("#todo-input");
const list = document.querySelector("#todo-list");

const STORAGE_KEY = "todos";

// The actual DATA  -  the DOM is just a rendering of this array.
let todos = loadTodos();

function loadTodos() {
  const raw = localStorage.getItem(STORAGE_KEY);
  if (!raw) return [];
  try {
    return JSON.parse(raw);
  } catch {
    return []; // corrupted data  -  fall back to empty rather than crashing
  }
}

function saveTodos() {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(todos));
}

function render() {
  list.innerHTML = ""; // clear and rebuild  -  simple, if not maximally efficient (a tradeoff worth naming: see not
```

```
  todos.forEach((todo, index) => {
    const li = document.createElement("li");
    li.className = "todo-item";
    li.dataset.index = index; // link the DOM element back to its data index

    const span = document.createElement("span");
    span.textContent = todo.text;
    span.className = todo.completed ? "todo-text completed" : "todo-text";

    const deleteBtn = document.createElement("button");
    deleteBtn.textContent = "Delete";
    deleteBtn.className = "delete-btn";

    li.append(span, deleteBtn);
    list.append(li);
  });
}

form.addEventListener("submit", (event) => {
  event.preventDefault();
  const text = input.value.trim();
  if (!text) return;

  todos.push({ text, completed: false });
  saveTodos();
  render();

  input.value = "";
  input.focus();
});

list.addEventListener("click", (event) => {
  const li = event.target.closest(".todo-item");
  if (!li) return;
  const index = Number(li.dataset.index);

  if (event.target.classList.contains("delete-btn")) {
    todos.splice(index, 1);
  } else if (event.target.classList.contains("todo-text")) {
    todos[index].completed = !todos[index].completed;
  }

  saveTodos();
  render();
});

render(); // initial render on page load, using whatever was loaded from localStorage
```

Design note: rebuilding the entire list with `innerHTML = ""` and re-creating every element on every change is simple and correct, but not maximally efficient for very large lists (it destroys and recreates DOM nodes that didn't actually change). This exact inefficiency - manually keeping a JavaScript data array in sync with the DOM, and the tension between simplicity and performance when re-rendering - is precisely the problem React was built to solve. Keep this project in mind when you reach Part 14; you'll rebuild this exact app in React and feel directly why it's an improvement.

Knowledge Check - Part 8

1. Why can't you store a plain JavaScript object directly in `localStorage`?
2. What's the key practical difference between `localStorage` and `sessionStorage`?

3. Why are cookies automatically sent with every HTTP request, while `localStorage` is not? Why does this matter for what each is good for?

4. When would you reach for IndexedDB instead of `localStorage`?

> View solutions

1. Browser storage APIs only support string values. Storing an object directly triggers JavaScript's default string coercion, producing the useless `"[object Object]"` - you must explicitly serialize it with `JSON.stringify()` first.

2. `localStorage` persists indefinitely (until explicitly cleared) and is shared across all tabs of the same origin. `sessionStorage` is cleared when the tab closes and is isolated per-tab.

3. Cookies are automatically attached to every HTTP request to a matching domain, which is exactly what's needed for server-side session/auth mechanisms (the server needs to see the cookie on each request to know who's asking) - but this adds overhead to every request. `localStorage` is purely client-side and never automatically transmitted, making it better suited for data the server doesn't need to see on every request.

4. When you need to store large amounts of structured, queryable data (beyond a few MB), need to store binary data like files, or need proper database features (indexes, transactions) client-side - `localStorage`'s simple key-value string model doesn't scale to those needs.

Next: Part 9 - Errors and Debugging

PART 9 - ERRORS AND DEBUGGING [Intermediate][Advanced]

> *This section is deliberately thorough. Error handling is one of the biggest gaps between "tutorial code" and production code, and it's the direct foundation for Part 10 (async error handling) and Part 11 (API error handling) - arguably the two most important sections in this entire course. Do not rush through this.*

9.1 Types of Errors

Syntax errors

The code is not even valid JavaScript - the engine can't parse it at all. This is caught before any code runs.

```
function broken( {
  console.log("this will never run");
} // [x] SyntaxError: missing ')'
```

Runtime errors

The code is syntactically valid, but something goes wrong while it's executing.

```
const user = null;
console.log(user.name); // [x] TypeError: Cannot read properties of null (reading 'name')
```

Logical errors

The code runs without crashing, but produces the wrong result - the most dangerous category, because nothing tells you it happened. You only find out through testing, or (worse) a user reporting broken behavior.

```
function calculateDiscount(price, percent) {
  return price * percent; // [x] Logical error  -  should be price * (percent / 100)
}

console.log(calculateDiscount(1000, 10)); // 10000  -  wildly wrong, but no error is thrown!
```

9.2 Stack Traces

When a runtime error is thrown and not caught, JavaScript produces a stack trace - a printout of the call stack (Part 3.7) at the moment of the crash.

```
function processOrder() {
  validateOrder();
}
function validateOrder() {
  checkInventory();
}
function checkInventory() {
  throw new Error("Item out of stock");
}

processOrder();

Uncaught Error: Item out of stock
    at checkInventory (app.js:8)
    at validateOrder (app.js:5)
    at processOrder (app.js:2)
    at app.js:11
```

How to read this - bottom-to-top is chronological, top-to-bottom is "closest to the crash": the topmost line is exactly where the error was thrown (`checkInventory`, line 8). Each line below it shows who called whom, working backward through the call stack, down to the original top-level call. This is the single most useful piece of debugging information you'll have - it tells you the exact path the program took to reach the failure, not just where it failed.

9.3 The Debugger and Breakpoints (Recap + Practice)

Recall from Part 0.16: the Sources tab lets you set breakpoints - click a line number, and execution pauses there whenever it's reached. Once paused:

- Step Over: run the current line, don't descend into function calls it makes.
- Step Into: descend into a function call on the current line.
- Step Out: finish the current function and return to its caller.

- The Scope panel shows every variable in the current execution context (and, thanks to the scope chain, every outer scope too) with their live current values.
- The Call Stack panel shows the exact same information as a stack trace, but interactively - click any frame to inspect that function's variables at that point in execution.

 Debugging methodology - a repeatable process, not guesswork:

1. Reproduce the bug reliably - know the exact steps that trigger it.
2. Read the error message and stack trace carefully - don't skim past it. It usually tells you far more than it seems to at first glance.
3. Form a hypothesis about what's wrong, based on the error and your understanding of the code.
4. Set a breakpoint at the point where you suspect the problem starts, and inspect actual variable values - don't assume; verify.
5. Narrow down - if the bug isn't where you expected, use the call stack to trace backward/forward until you find the exact line where behavior diverges from what you expected.
6. Fix, then verify - re-run the exact reproduction steps to confirm the fix actually works, and think about whether the same class of bug could exist elsewhere.

```
console.log("Normal message");
console.warn("Something looks off");     // yellow, with a warning icon
console.error("Something broke");           // red, with an error icon  -  also captures a stack trace!
console.table([{ id: 1, name: "A" }, { id: 2, name: "B" }]); // renders as an actual table  -  great for arrays of o
console.group("User validation");
console.log("Checking username...");
console.log("Checking email...");
console.groupEnd();                                 // groups related logs, collapsible in DevTools
```

9.4 try , catch , finally

What it is

A structure for explicitly handling runtime errors, rather than letting them crash your program.

```
try {
  // code that might throw an error
} catch (error) {
  // runs ONLY if something in the try block threw
}
```

Simple example

```
try {
  const result = JSON.parse("{ invalid json }");
  console.log(result);
} catch (error) {
  console.log("Failed to parse JSON:", error.message);
}
// "Failed to parse JSON: Unexpected token i in JSON at position 2"
```

Step-by-step execution: JavaScript runs the `try` block. The moment `JSON.parse` throws (because the string isn't valid JSON), execution immediately jumps to the `catch` block - any code after the failing line, inside the `try` block, is skipped entirely. The thrown error object is made available as the `catch` block's parameter (`error`, though you can name it anything).

```
try {
  console.log("Trying...");
  throw new Error("Oops");
} catch (error) {
  console.log("Caught:", error.message);
} finally {
  console.log("This ALWAYS runs  -  error or not.");
}
// "Trying..."
// "Caught: Oops"
// "This ALWAYS runs  -  error or not."
```

Why `finally` exists: it runs regardless of whether an error was thrown, caught, or even if the `try`/`catch` block returns early. It's the correct place to put "cleanup" logic that must happen no matter what - closing a connection, hiding a loading spinner, resetting a flag. We'll use this exact pattern constantly starting in Part 10 (`finally { setLoading(false); }` is one of the most common lines you'll write in this entire course).

9.5 What an Error Object Contains

```
try {
  null.someProperty;
} catch (error) {
  console.log(error.name);     // "TypeError"
  console.log(error.message); // "Cannot read properties of null (reading 'someProperty')"
  console.log(error.stack);     // the full stack trace, as a string
  console.log(error instanceof TypeError); // true
  console.log(error instanceof Error);       // true  -  TypeError extends Error
}
```

Built-in error types you'll commonly see:

Type	Typical cause
`TypeError`	Using a value in a way inconsistent with its type (calling a non-function, reading a property of `null`/`undefined`)
`ReferenceError`	Referencing a variable that doesn't exist / isn't in scope
`SyntaxError`	Invalid syntax - often from `JSON.parse` on malformed JSON
`RangeError`	A value is outside its valid range (e.g., invalid array length, exceeded call stack)

9.6 throw and Custom Errors

Throwing your own errors

```
function withdraw(balance, amount) {
  if (amount > balance) {
```

```
    throw new Error("Insufficient funds");
  }
  return balance - amount;
}

try {
  withdraw(100, 500);
} catch (error) {
  console.log(error.message); // "Insufficient funds"
}
```

Why throw your own errors: it lets you fail loudly and explicitly at the exact point something is wrong, with a clear message, instead of letting invalid data silently propagate further into your program and cause a more confusing failure somewhere else, later, disconnected from the actual root cause.

What it is

```
class ValidationError extends Error {
  constructor(message, field) {
    super(message); // sets this.message, per the Error base class
    this.name = "ValidationError"; // overrides the default "Error" name
    this.field = field;               // custom extra data specific to this error type
  }
}

function validateAge(age) {
  if (age < 0) {
    throw new ValidationError("Age cannot be negative", "age");
  }
  return age;
}

try {
  validateAge(-5);
} catch (error) {
  if (error instanceof ValidationError) {
    console.log(`Validation failed on field "${error.field}": ${error.message}`);
  } else {
    console.log("Unexpected error:", error);
  }
}
// 'Validation failed on field "age": Age cannot be negative'
```

Why custom error classes matter: they let calling code distinguish between different failure types using `instanceof`, and respond differently to each - e.g., show a validation error inline near a form field, but show a generic "something went wrong" toast for an unexpected `TypeError`. This pattern scales directly into Part 11, where you'll distinguish network errors from HTTP errors from parsing errors, each needing different handling.

9.7 When to Catch, and When to Let Errors Propagate

What "propagate" means

If an error is thrown inside a function and there's no `try/catch` around that specific call, the error bubbles up the call stack - exactly like event bubbling (Part 6.7), but for errors - until either some caller catches it, or it reaches the top and crashes the program (or, in a browser, gets logged as an uncaught error).

```
function level3() {
  throw new Error("Something broke deep down");
}
function level2() {
  level3(); // no try/catch here  -  the error passes straight through
}
function level1() {
  try {
    level2();
  } catch (error) {
    console.log("Caught at level1:", error.message);
  }
}

level1(); // "Caught at level1: Something broke deep down"
```

The judgment call - when SHOULD you catch an error, versus letting it propagate?

- Catch it at the point where you can actually do something meaningful in response - show a specific error message to the user, retry the operation, fall back to a default value, or log it with useful context.
- Let it propagate if the current function has no useful way to handle it - it doesn't know enough about the broader context to decide what the "right" response is. A low-level utility function (like one that parses a date string) usually shouldn't swallow an error and return `null` silently - that hides real problems from whoever eventually needs to know. It's usually better to let the error bubble up to a place that has enough context to respond properly (like the UI layer that can show something to the user).

⚠ Anti-pattern - the silent catch:

```
// [Advanced] DANGEROUS  -  swallows the error completely, with no trace it ever happened
function getConfig() {
  try {
    return JSON.parse(localStorage.getItem("config"));
  } catch (error) {
    return {}; // silently returns an empty object, hiding that something went wrong
  }
}
```

Why this is dangerous: if `config` was corrupted, this function returns an empty object with zero indication anything failed - no log, nothing. Debugging a downstream symptom of this (a mysteriously missing config value) later becomes far harder, because the actual root cause was silently discarded.

Better version - at minimum, log what happened, even if you still want to gracefully fall back:

```
function getConfig() {
  try {
    return JSON.parse(localStorage.getItem("config"));
  } catch (error) {
    console.error("Failed to parse stored config, using defaults:", error);
    return {};
  }
}
```

9.8 Nested try / catch

```
try {
  try {
    JSON.parse("{ bad json");
  } catch (innerError) {
    console.log("Inner catch:", innerError.message);
    throw new Error("Re-thrown with more context: config was invalid");
  }
} catch (outerError) {
  console.log("Outer catch:", outerError.message);
}
// "Inner catch: Unexpected token b in JSON at position 2"
// "Outer catch: Re-thrown with more context: config was invalid"
```

Why re-throw: sometimes the inner `catch` block can add useful context (or attempt a specific recovery for a specific known failure) but still can't fully resolve the situation - re-throwing (optionally as a new, more descriptive error) lets a higher-level `catch` make the final decision, without losing the original error's information.

9.9 Handling Errors Inside Functions - Return Values vs Throwing

Two legitimate strategies exist for signaling failure from a function:

```
// Strategy A: throw  -  forces the caller to handle it (via try/catch) or let it propagate
function parseAge(input) {
  const age = Number(input);
  if (Number.isNaN(age)) {
    throw new Error(`"${input}" is not a valid age`);
  }
  return age;
}

// Strategy B: return a special value (null, or a result object)  -  caller must remember to check it
function parseAgeSafe(input) {
  const age = Number(input);
  if (Number.isNaN(age)) {
    return null;
  }
  return age;
}
```

When to use which:

- Throw when the failure represents a genuine, exceptional programming error or an unrecoverable state that the immediate caller likely can't (and shouldn't silently) ignore - you want to force handling.
- Return a sentinel value (`null`, `undefined`, or a `{ success, data, error }` shape) when the "failure" is actually a normal, expected outcome of calling the function (e.g., "user not found" from a lookup function is often not exceptional - it's a completely valid, expected result), and you don't want every call site burdened with `try/catch` for a common case.

  We'll see this exact tension resolved concretely in Part 10 and 11 with `fetch()` - and crucially, learn why `fetch()`'s own design choice here (not throwing for HTTP error statuses) is one of the most commonly misunderstood things in all of async JavaScript.

Knowledge Check - Part 9

1. What's the difference between a syntax error, a runtime error, and a logical error? Which is hardest to detect, and why?
2. Why does code after a `throw` inside a `try` block never execute?
3. What does `finally` guarantee that `catch` alone does not?
4. What is a stack trace, and how do you read the order of its lines?
5. Give a concrete example of when you should let an error propagate rather than catching it immediately.
6. Why is silently catching an error and returning a default value without logging considered dangerous?
1. Write a function `safeDivide(a, b)` that throws a custom `Error` with a clear message if `b` is `0`, and otherwise returns `a / b`. Call it inside a `try/catch` and log an appropriate message either way.
2. Create a custom error class `NotFoundError extends Error` that takes an `id` in its constructor and sets a `.name` of `"NotFoundError"`. Write a function `findUserById(users, id)` that throws this error if no user with that ID exists.
3. Rewrite this to add proper error handling with try/catch/finally, ensuring `"Cleanup done"` always logs:`function riskyParse(json) { console.log("Starting..."); const data = JSON.parse(json); console.log("Cleanup done"); return data; }`

> View solutions

Knowledge Check

1. A syntax error means the code isn't even valid JavaScript and is caught before any execution begins. A runtime error occurs while the code is executing (e.g., calling a method on `null`). A logical error produces no crash at all - the code runs "successfully" but computes the wrong result. Logical errors are hardest to detect because nothing signals they happened; they only surface through testing or incorrect real-world behavior.
2. Because `throw` immediately halts normal execution and transfers control straight to the nearest enclosing `catch` block (or propagates up if there is none) - any remaining code in the `try` block is skipped entirely, exactly like a `return` statement immediately exits a function.
3. `finally` guarantees its code runs regardless of whether an error was thrown, caught, or not - even if the `try` or `catch` block contains a `return` statement. `catch` alone only runs if an error was actually thrown.
4. A stack trace is a printout of the call stack at the moment an error was thrown, showing the chain of function calls that led to the failure. The topmost line is where the error was actually thrown; each line below shows the function that called the one above it, working back to the original top-level call.
5. When the current function doesn't have enough context to meaningfully respond to the failure - e.g., a low-level data-parsing utility that doesn't know whether the caller wants to retry, show an error message, or fall back to a default; it's better to let that decision happen closer to the UI/application layer that does have that context.

6 Because it hides that something went wrong, discarding all information about the actual root cause. A downstream symptom of the silently-swallowed error (like missing data) becomes much harder to debug later, since there's no trace the original failure ever happened.

Coding Challenges

1

```
function safeDivide(a, b) {
  if (b === 0) {
    throw new Error("Cannot divide by zero");
  }
  return a / b;
}

try {
  console.log(safeDivide(10, 2)); // 5
  console.log(safeDivide(10, 0)); // throws
} catch (error) {
  console.error("Division failed:", error.message);
}
```

1

```
class NotFoundError extends Error {
  constructor(id) {
    super(`User with id ${id} was not found`);
    this.name = "NotFoundError";
    this.id = id;
  }
}

function findUserById(users, id) {
  const user = users.find(u => u.id === id);
  if (!user) {
    throw new NotFoundError(id);
  }
  return user;
}

const users = [{ id: 1, name: "Ammaar" }];

try {
  findUserById(users, 99);
} catch (error) {
  if (error instanceof NotFoundError) {
    console.log(`Could not find user ${error.id}`);
  } else {
    throw error; // re-throw anything unexpected  -  don't silently swallow unrelated errors
  }
}
```

1

```
function riskyParse(json) {
  console.log("Starting...");
  try {
    const data = JSON.parse(json);
    return data;
  } catch (error) {
    console.error("Failed to parse JSON:", error.message);
    return null;
  } finally {
```

```
    console.log("Cleanup done"); // now ALWAYS runs, success or failure
  }
}
```

Interview Questions - Errors

Q: Why use `try/catch` instead of just checking values before using them?
Both have their place. Explicit checks (guard clauses, Part 2.1) are better when you're checking something you fully control and expect to sometimes be true/false as a normal outcome. `try/catch` is essential when the failure can come from unpredictable sources you don't fully control - parsing external data, network calls, or any operation whose internal failure modes you can't (or shouldn't have to) enumerate in advance. In practice, production code uses both together.

Q: What's the difference between `throw`ing an error and returning `null`/`undefined` to signal failure?
Throwing forces the caller to explicitly handle the failure (via try/catch) or have it propagate loudly - it can't be silently ignored. Returning a sentinel value like `null` is easy to forget to check, silently continuing with bad data, but is appropriate when "no result" is itself a normal, expected outcome rather than an exceptional one.

Next: Part 10 - Asynchronous JavaScript

PART 10 - ASYNCHRONOUS JAVASCRIPT [Intermediate][Advanced][Advanced]

This is, by design, one of the biggest and most detailed sections in the entire course. If you only deeply master one part of this document, make it this one - everything from Part 11 onward (APIs) and most of real-world React (Part 22 onward) is built directly on top of what you learn here. Go slowly. Re-run every example yourself.

Async Level 1 - Understanding Synchronous Code

What "asynchronous" actually means

Before we can define "asynchronous," we need to be crystal clear on "synchronous" - because async is defined in contrast to it.

Synchronous code runs one line at a time, in order, and each line must finish before the next one starts.

```
console.log("A");
console.log("B");
console.log("C");

A
B
C
```

This is completely unsurprising - it's how every program you've written so far in this course has worked. But consider what happens if one of those steps takes a long time - say, waiting for a server on the other side of the world to respond to a network request (recall Part 0.4). If JavaScript is strictly synchronous, and running one line strictly waits for the previous line to finish, wouldn't the entire browser freeze while we wait for that response? The user couldn't scroll, couldn't click anything, couldn't even see spinner animations update - a completely frozen tab, possibly for seconds.

This is exactly what "blocking" means, and it's the core problem asynchronous programming exists to solve.

Blocking vs Non-blocking

- Blocking: an operation that halts all further code execution until it completes.
- Non-blocking: an operation that gets started, but control immediately returns to the rest of your program, which keeps running - the operation's result (when ready) is handled later, without freezing anything in the meantime.

Mental model - a restaurant kitchen

Imagine a single chef (JavaScript is famously single-threaded - it can only do one thing at a time) taking orders. If the chef takes an order, then stands at the stove doing nothing else while a 20-minute dish cooks, ignoring every other customer until it's done, that's blocking. A well-run kitchen instead puts the dish on the stove, sets a timer, and immediately moves on to take the next order - and comes back to the first dish when the timer goes off. That's non-blocking: expensive operations run "in the background," and the chef (JavaScript) stays free to do other work in the meantime.

[x] You should now understand: JavaScript itself only ever does one thing at a time (single-threaded), but the environment it runs in (the browser, or Node.js) provides mechanisms to hand off slow operations, let JavaScript keep running other code, and be notified later when those slow operations finish. That handoff mechanism is what the rest of Part 10 explains, piece by piece.

Async Level 2 - setTimeout

What it is

`setTimeout` is a Browser API (recall Part 0.10 - not part of the JavaScript language itself) that schedules a function to run after at least a specified delay, without blocking anything in the meantime.

```
console.log("First");

setTimeout(() => {
  console.log("Third (after the delay)");
}, 1000);

console.log("Second");

First
Second
Third (after the delay)
```

Why "Second" logs before "Third," even though `setTimeout` appears before it in the source code: `setTimeout` doesn't pause anything. It hands its callback function off to the browser, saying "run this in

at least 1000ms," and JavaScript immediately continues to the next line (`console.log("Second")`) without waiting. Only after the synchronous code finishes (and after the delay elapses) does the callback actually run. This is your first real taste of non-blocking behavior - we'll build the precise mental model for why this ordering happens in Level 4 with a full diagram.

⚠ Common misconception: `setTimeout(fn, 0)` does not mean "run immediately." It means "run as soon as possible, but only after the current synchronous code has completely finished" - even a `0`ms delay still goes through the same hand-off mechanism.

```
console.log("A");
setTimeout(() => console.log("B"), 0);
console.log("C");
// A
// C
// B  ← still last, even with a 0ms delay!
```

Async Level 3 - Callbacks

What it is (recap and extension of Part 3.4)

A callback is simply a function passed to another function, to be invoked later. In the async context, callbacks are how you tell an asynchronous operation "when you're done, run this."

```
function fetchUserData(callback) {
  setTimeout(() => {
    const userData = { name: "Ammaar", age: 17 };
    callback(userData); // "calling back" once the (simulated) slow operation is done
  }, 1000);
}

fetchUserData((user) => {
  console.log("Got user:", user.name);
});

console.log("This logs BEFORE the user data, because fetchUserData is non-blocking!");

This logs BEFORE the user data, because fetchUserData is non-blocking!
Got user: Ammaar
```

The "error-first callback" pattern (a historical convention worth recognizing):

```
function fetchUserData(callback) {
  setTimeout(() => {
    const success = Math.random() > 0.3;
    if (success) {
      callback(null, { name: "Ammaar" }); // (error, data)  -  error is null on success
    } else {
      callback(new Error("Failed to fetch user"), null); // (error, data)  -  data is null on failure
    }
  }, 1000);
}

fetchUserData((error, user) => {
  if (error) {
    console.error("Error:", error.message);
    return;
  }
  console.log("Got user:", user.name);
});
```

This "error-first" convention was the dominant way to handle both success and failure in callback-based async code, especially in Node.js. It's largely been superseded by Promises (Level 5+), but you'll still encounter it in older code and some libraries, and understanding it helps you appreciate exactly what problem Promises were designed to fix.

Async Level 4 - Callback Hell, and the JavaScript Runtime Model

The problem - nesting

What happens when one async operation needs to happen after another, which needs to happen after another?

```
getUser(userId, (user) => {
  getOrders(user.id, (orders) => {
    getOrderDetails(orders[0].id, (details) => {
      getShippingInfo(details.shippingId, (shipping) => {
        console.log("Finally have shipping info:", shipping);
        // [Advanced] This is "callback hell" / "the pyramid of doom"
      });
    });
  });
});
```

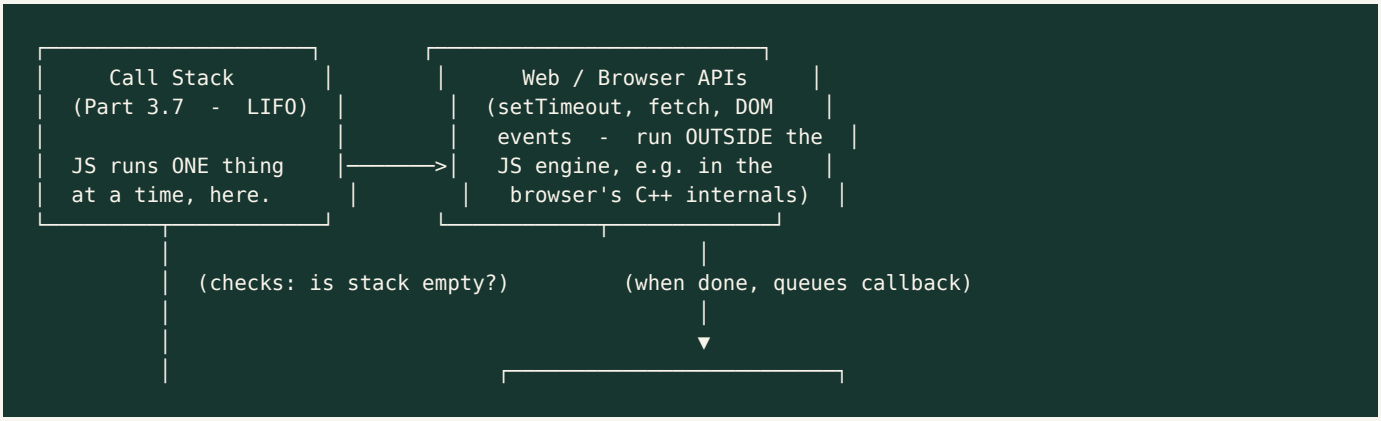
Why this is a real problem, not just an aesthetic one:

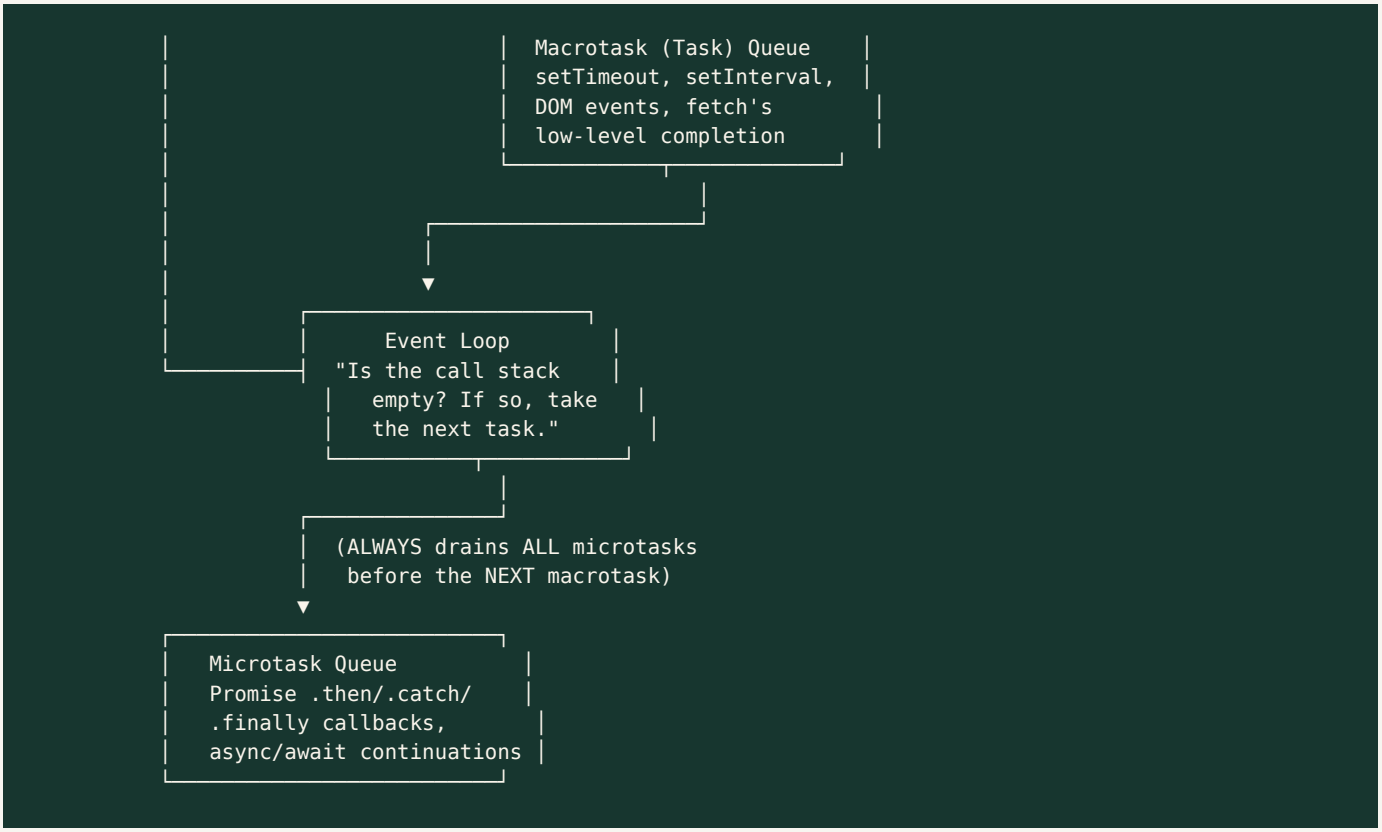
- Readability collapses - the actual sequence of steps is buried in ever-deeper indentation.
- Error handling becomes a nightmare - each level needs its own error check (recall error-first callbacks above), and there's no single place to handle "anything in this chain failed."
- Control flow is hard to reason about - what if you need to run two of these in parallel, or handle a timeout across the whole chain? The nested-callback shape makes this awkward and error-prone.

This exact, widely-felt pain point is the direct historical motivation for Promises (Level 5) and, later, async/await (Level 8) - both exist specifically to let you write asynchronous code that reads like synchronous, top-to-bottom code, while still being non-blocking underneath.

> *This is the single most important diagram in this entire course. Study it carefully - it explains everything about async JavaScript's ordering behavior, including surprising results you'll otherwise just have to memorize.*

The four pieces:





The Event Loop's actual algorithm, in plain English:

1. Run everything currently on the call stack until it's empty (this is your normal, synchronous code, top to bottom).
2. Once the call stack is empty: check the microtask queue. If there's anything there, run all of it - one microtask can queue another, and the loop keeps draining until the microtask queue is completely empty - before moving on.
3. Only once the microtask queue is fully empty does the event loop take one task from the macrotask (task) queue and push it onto the call stack to run.
4. Repeat from step 1, forever (this is literally called "the event loop" because it loops, checking this over and over).

Why the microtask/macrotask distinction matters - a concrete example:

```
console.log("1: synchronous");

setTimeout(() => console.log("2: macrotask (setTimeout)"), 0);

Promise.resolve().then(() => console.log("3: microtask (Promise)"));

console.log("4: synchronous");

1: synchronous
4: synchronous
3: microtask (Promise)
2: macrotask (setTimeout)
```

Step-by-step, using the diagram above:

1. `console.log("1...")` runs immediately (call stack).

2. `setTimeout(...)` hands its callback to the Web API, which will queue it in the macrotask queue once its (0ms) timer elapses.
3. `Promise.resolve().then(...)` queues its callback in the microtask queue (Promises are covered fully starting Level 5 - for now, just trust that `.then()` callbacks go into the microtask queue).
4. `console.log("4...")` runs immediately (call stack).
5. Call stack is now empty. The event loop checks the microtask queue first - finds the `.then()` callback, runs it -> logs `"3..."`.
6. Microtask queue is now empty. Only now does the event loop check the macrotask queue, finds the `setTimeout` callback, runs it -> logs `"2..."`.

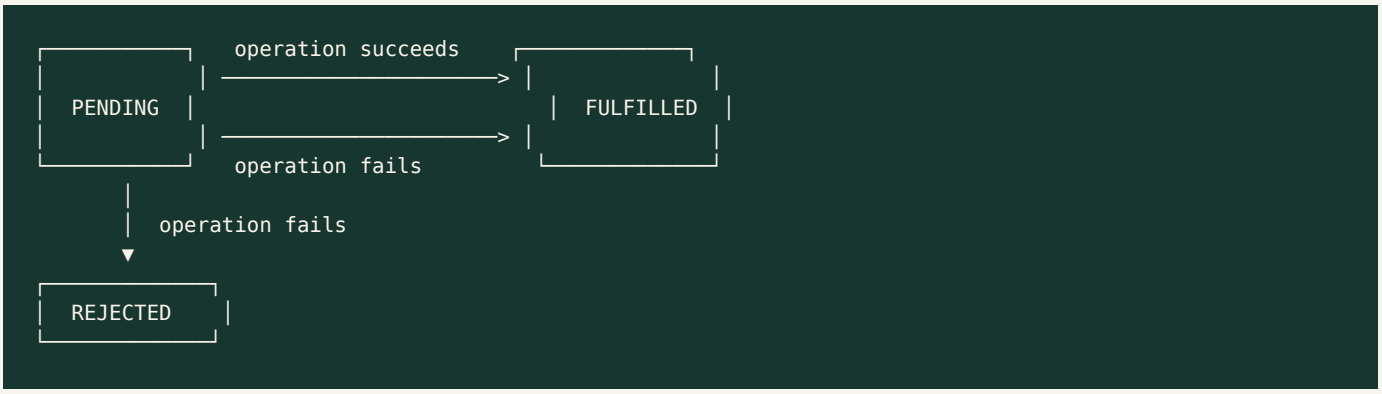
   The takeaway rule to memorize: microtasks (Promises) always run before the next macrotask (setTimeout), no matter which was scheduled "first" in the source code, as long as both were queued during the same synchronous execution. This exact quirk is one of the most commonly asked "predict the output" interview questions in JavaScript.

Async Level 5 - Promises: The Fundamentals

Why Promises exist

A Promise is a first-class JavaScript object representing the eventual result (or failure) of an asynchronous operation. It exists specifically to solve callback hell (Level 4) by giving asynchronous operations a proper return value you can chain, compose, and handle errors on centrally - rather than passing a callback function into the operation and hoping it's called correctly.

The three states of a Promise



- Pending: the initial state - neither fulfilled nor rejected yet. The operation is still in progress.
- Fulfilled: the operation completed successfully, and the Promise now has a resulting value.
- Rejected: the operation failed, and the Promise now has a reason (usually an `Error`) for the failure.

  Crucially: a Promise can only transition from `pending` to EITHER `fulfilled` OR `rejected` - once - and then it's permanently "settled." It can never go back to pending, and it can never switch from fulfilled to rejected or vice versa.

```
const myPromise = new Promise((resolve, reject) => {
  // this function (the "executor") runs IMMEDIATELY, synchronously, when `new Promise(...)` is called
```

```
  const succeeded = true; // simulate an outcome

  if (succeeded) {
    resolve("The operation succeeded!"); // transitions the promise to FULFILLED with this value
  } else {
    reject(new Error("The operation failed!")); // transitions the promise to REJECTED with this reason
  }
});
```

What `resolve` and `reject` are: they're functions provided by the Promise constructor itself, which you call to manually control the Promise's outcome. `resolve(value)` fulfills the promise with `value`. `reject(reason)` rejects it with `reason` (conventionally an `Error` object, though technically it can be any value - always use an `Error` in practice, so consumers reliably get a `.message` and a stack trace).

```
myPromise
  .then((result) => {
    console.log("Success:", result);
  })
  .catch((error) => {
    console.log("Failure:", error.message);
  })
  .finally(() => {
    console.log("This runs regardless of success or failure.");
  });
```

- `.then(onFulfilled)`: registers a callback to run when the Promise is fulfilled, receiving the resolved value.
- `.catch(onRejected)`: registers a callback to run when the Promise is rejected, receiving the rejection reason.
- `.finally(onSettled)`: registers a callback that runs when the Promise settles, regardless of outcome - and receives no argument, because it doesn't know (or care) which one happened. (This mirrors `try/catch/finally` from Part 9.4 very deliberately - the parallel is not a coincidence, as you'll see explicitly in Level 10.)

A realistic simulated async operation, wrapped in a Promise:

```
function fetchUser(id) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (id === 1) {
        resolve({ id: 1, name: "Ammaar" });
      } else {
        reject(new Error(`User with id ${id} not found`));
      }
    }, 1000);
  });
}

fetchUser(1)
  .then((user) => console.log("Found:", user.name))
  .catch((error) => console.log("Error:", error.message));

fetchUser(99)
  .then((user) => console.log("Found:", user.name))
  .catch((error) => console.log("Error:", error.message));

(after 1 second)
Found: Ammaar
Error: User with id 99 not found
```

Async Level 6 - Promise Chaining

What it is

If a `.then()` callback returns a value, `.then()` itself returns a new Promise that resolves with that value - allowing you to chain multiple asynchronous steps in sequence, each depending on the previous one's result, without nesting.

```
function double(n) {
  return new Promise((resolve) => {
    setTimeout(() => resolve(n * 2), 500);
  });
}

double(1)
  .then((result) => {
    console.log(result); // 2
    return double(result); // returning a PROMISE here  -  the chain waits for it!
  })
  .then((result) => {
    console.log(result); // 4
    return double(result);
  })
  .then((result) => {
    console.log(result); // 8
  });
```

Step-by-step: each `.then()` waits for the previous Promise to settle, runs its callback, and - critically - if that callback returns another Promise, the chain automatically waits for that Promise too before moving to the next `.then()`. This is precisely how Promises solve the nesting problem from Level 4: instead of nesting deeper and deeper, each step chains flatly, one after another.

Rewriting the callback-hell example from Level 4 with Promise chaining:

```
getUser(userId)
  .then((user) => getOrders(user.id))
  .then((orders) => getOrderDetails(orders[0].id))
  .then((details) => getShippingInfo(details.shippingId))
  .then((shipping) => {
    console.log("Finally have shipping info:", shipping);
  })
  .catch((error) => {
    console.log("Something failed somewhere in the chain:", error.message);
    // ONE catch handles a failure from ANY step above!
  });
```

This is the single biggest advantage over nested callbacks: one `.catch()` at the end handles an error from any step in the chain - you don't need to duplicate error-handling logic at every single level.

Async Level 7 - Promise Error Propagation

How errors travel through a chain

```
Promise.resolve(1)
  .then((value) => {
    throw new Error("Something broke at step 1");
  })
```

```
  .then((value) => {
    console.log("This step is SKIPPED entirely!");
    return value;
  })
  .catch((error) => {
    console.log("Caught:", error.message); // "Caught: Something broke at step 1"
  })
  .then(() => {
    console.log("Chain continues normally after the catch!");
  });
```

Why the second `.then()` is skipped: once a rejection occurs anywhere in the chain (whether from an explicit `reject()`/`throw`, or a Promise the chain is waiting on rejecting), control jumps directly to the nearest `.catch()`, skipping every `.then()` in between - exactly like `throw` skips to the nearest `catch` in synchronous `try/catch` (Part 9.4). This parallel is intentional and important.

⚠ Common mistake - a `.catch()` in the middle of a chain "swallows" the error and lets the chain continue as if nothing happened, which is sometimes exactly what you want, but is a frequent source of confusion:

```
fetchUser(99)
  .catch((error) => {
    console.log("Caught early:", error.message);
    return { id: 0, name: "Guest" }; // returning a value here RECOVERS the chain!
  })
  .then((user) => {
    console.log("Continuing with:", user.name); // "Continuing with: Guest"  -  runs normally!
  });
```

Why this "recovers": returning a normal value from inside a `.catch()` block produces a new, fulfilled Promise - the failure has been "handled," and the chain proceeds as if that step had succeeded with the returned value. This is a legitimate and useful pattern (providing a fallback value) but must be used deliberately, understanding that it stops the error from propagating any further down the chain.

Async Level 7.5 - Static Promise Methods: resolve , reject , all , allSettled , race , any

```
Promise.resolve(42);   // immediately creates an already-FULFILLED promise with value 42
Promise.reject(new Error("fail")); // immediately creates an already-REJECTED promise
```

Why these exist: useful for normalizing a value that might already be a Promise, or might be a plain value, into always being a Promise - so calling code can treat both cases identically with `.then()`.

What it does: takes an array of Promises and returns a single Promise that fulfills with an array of all their results, in the same order - but only if every single one succeeds. If any one rejects, `Promise.all()` immediately rejects with that first failure's reason, without waiting for the others.

```
Promise.all([
  fetchUser(1),
  fetchUser(2),
  fetchUser(3)
])
  .then((users) => {
    console.log("All succeeded:", users); // [user1, user2, user3]
  })
  .catch((error) => {
```

```
    console.log("At least one failed:", error.message); // fails fast on the FIRST rejection
  });
```

When to use it: when you need all of several independent operations to succeed for your logic to proceed meaningfully - e.g., loading a dashboard that requires user data, settings, AND notifications all at once; if any one fails, showing a partial dashboard doesn't make sense.

What it does: takes an array of Promises and returns a Promise that always fulfills (never rejects) once every input Promise has settled - regardless of whether each one individually succeeded or failed - giving you an array of outcome objects.

```
Promise.allSettled([
  fetchUser(1),
  fetchUser(99), // this one will reject
  fetchUser(3)
]).then((results) => {
  console.log(results);
  // [
  //   { status: "fulfilled", value: { id: 1, name: "Ammaar" } },
  //   { status: "rejected", reason: Error("User with id 99 not found") },
  //   { status: "fulfilled", value: { id: 3, name: "..." } }
  // ]

  results.forEach((result) => {
    if (result.status === "fulfilled") {
      console.log("Success:", result.value);
    } else {
      console.log("Failed:", result.reason.message);
    }
  });
});
```

When to use it: when you want to attempt several independent operations and handle each outcome individually, without one failure derailing the others - e.g., sending analytics events to three different third-party services, where one failing shouldn't stop you from knowing about the other two.

What it does: returns a Promise that settles (fulfills OR rejects) as soon as the first of the input Promises settles - whichever gets there first, wins, and the rest are ignored (though they keep running in the background; they just don't affect the result).

```
function timeout(ms) {
  return new Promise((_, reject) => {
    setTimeout(() => reject(new Error("Timed out!")), ms);
  });
}

Promise.race([
  fetchUser(1),  // resolves after 1000ms in our earlier example
  timeout(500)     // rejects after 500ms
])
  .then((user) => console.log("Got user:", user))
  .catch((error) => console.log("Race lost:", error.message)); // "Race lost: Timed out!"
```

When to use it: the classic use case is exactly this - implementing a timeout for an operation that doesn't have one built in, by racing it against a Promise that rejects after a deadline. We'll build this into a reusable pattern in Part 11.

What it does: returns a Promise that fulfills as soon as the first input Promise fulfills (ignoring rejections along the way) - and only rejects if all of them reject.

```
Promise.any([
  fetchUser(99), // rejects
  fetchUser(1),   // fulfills
  fetchUser(1)    // also fulfills
]).then((user) => {
  console.log("First success:", user.name); // whichever succeeds first
});
```

`race` vs `any` - the key difference: `race` settles on whichever Promise finishes first, period (success or failure). `any` specifically waits for the first success, ignoring failures unless everything fails. Use `race` for timeouts (where a failure should end the race). Use `any` for "try several sources and use whichever one responds successfully first" (e.g., querying several redundant servers/mirrors for the same data).

Method	Settles when...	Result
`Promise.all()`	ALL fulfill, OR any ONE rejects (fails fast)	Array of values, or the first rejection reason
`Promise.allSettled()`	ALL have settled (always fulfills itself)	Array of `{status, value/reason}` objects
`Promise.race()`	The FIRST one settles (success or failure)	That one's value or reason
`Promise.any()`	The FIRST one fulfills, OR all reject	That value, or an `AggregateError` if all failed

---

Async Level 8 & 9 - async Functions and await

What `async` means

Putting `async` before a function declaration does two things:

1. It makes the function always return a Promise, automatically - even if you `return` a plain value.
2. It allows you to use the `await` keyword inside that function's body.

```
async function test() {
  return 10;
}

test().then((value) => console.log(value)); // 10
console.log(test()); // Promise {<fulfilled>: 10}  -  NOT the number 10 itself!
```

Why `test()` returns a Promise even though we wrote `return 10`: `async` functions automatically wrap whatever you return in a resolved Promise. `return 10;` inside an `async function` behaves exactly like `return Promise.resolve(10);` would in a normal function. This is precisely why you can call `.then()` on the result.

What `await` means

`await` can only be used inside an `async` function (with one exception - top-level `await` in modules, covered below). It pauses the execution of that `async` function (and only that function - nothing else in your program is blocked!) until the Promise it's waiting on settles, then "unwraps" the result.

```
function delay(ms, value) {
  return new Promise((resolve) => setTimeout(() => resolve(value), ms));
}

async function example() {
  console.log("Before await");
  const result = await delay(1000, "Done!");
  console.log("After await:", result);
}

example();
console.log("This logs immediately, BEFORE 'After await'!");

Before await
This logs immediately, BEFORE 'After await'!
(1 second later)
After await: Done!
```

Step-by-step, precisely:

1. `example()` is called. Its execution context is pushed onto the call stack.
2. `"Before await"` logs synchronously.
3. `await delay(1000, "Done!")` calls `delay`, which immediately returns a pending Promise.
4. Here's the crucial part: hitting `await` pauses `example()`'s execution and - critically - hands control back to whatever called `example()`, allowing the rest of the program to continue running in the meantime. `example()` itself is essentially "parked," waiting to be resumed.
5. `console.log("This logs immediately...")` runs, because `example()` gave up control at the `await` point.
6. 1000ms later, `delay`'s internal `setTimeout` fires, resolving its Promise with `"Done!"`.
7. This schedules `example()`'s continuation as a microtask (recall Level 4's event loop diagram - `await` continuations behave like `.then()` callbacks under the hood).
8. Once the call stack is empty and it's the microtask queue's turn, `example()` resumes exactly where it left off - `result` is now `"Done!"`, and `console.log("After await:", result)` runs.

The critical insight: `await` does not block the browser or the rest of your program. It only pauses the single async function it's inside of, at that exact line, while everything else continues normally. This is what makes `async/await` "syntactic sugar over Promises" - it lets asynchronous code read like sequential, synchronous code, while the underlying non-blocking Promise machinery from Levels 5-7 still does all the actual work.

```
// [x] Inside an async function
async function fn() {
  const result = await somePromise();
}

// [x] NOT at the top level of a regular script (historically)
const result = await somePromise(); // SyntaxError, outside a module or async function
```

Recall from Part 1.1: files loaded with `type="module"` (or, equivalently, any file in a modern bundled project like the Vite setups in Part 13) get one special exception - `await` can be used at the very top level of the module, outside any `async function`:

```
// Inside an ES module (type="module")
const response = await fetch("https://api.example.com/data");
const data = await response.json();
console.log(data);
```

Why this is allowed here but not in a plain script: ES Modules already have well-defined, asynchronous-friendly loading semantics (other modules that `import` this one will simply wait for it to finish before proceeding) - the module system was specifically designed to accommodate this. A plain, non-module `<script>` has no such waiting mechanism, so top-level `await` there would have unclear, unsafe semantics, and is disallowed.

Async Level 10 - async / await with try / catch

This combination is, per the course's own stated priorities, one of the most important patterns to master completely. It is the standard, professional way virtually all real-world asynchronous error handling is written today.

The basic pattern

```
async function getData() {
  try {
    const response = await fetch(url);
    const data = await response.json();
    return data;
  } catch (error) {
    console.error(error);
  }
}
```

Explaining every single line:

- `async function getData()` - this function will return a Promise, and can use `await` inside.
- `try {` - begin a block where thrown errors (or rejected Promises being `await`ed) will be caught.
- `const response = await fetch(url);` - starts a network request (full detail in Part 11), and pauses `getData()` until the server responds (or the request fails at the network level).
- `const data = await response.json();` - the response body is read and parsed as JSON; this is also asynchronous (reading a network response body takes time), so it's also `await`ed.
- `return data;` - since `getData` is `async`, this automatically wraps `data` in a resolved Promise.
- `catch (error) {` - if either `await` line above throws/rejects (a Promise rejecting while being `await`ed is converted into a thrown exception, exactly as if you'd written `throw` - this is the crucial mechanical link between Promises and `try/catch`), execution jumps here.
- `console.error(error);` - logs whatever went wrong.

Why "a rejected `await`ed Promise behaves exactly like `throw`" is the single most important fact in this entire section: it means all the `try/catch/finally` knowledge from Part 9 applies directly and completely to asynchronous code, with `async/await`. You don't need to learn a separate error-handling system for async code - it's the exact same `try/catch` you already know, just now also catching failed awaited operations.

Here's the critical, easy-to-miss problem: `fetch()` does NOT reject for ordinary HTTP error statuses like `404 Not Found` or `500 Internal Server Error`. It only rejects for true network-level failures (DNS lookup failure, no internet connection, CORS being blocked, the request being aborted). This means the code above will happily continue past `await fetch(url)` even if the server responded with "404: this resource doesn't exist," treating it as if it were a completely normal, successful response - and only fail later (confusingly) at `response.json()`, or worse, not fail at all and just process garbage/error-page data as if it were valid.

We fix this properly and thoroughly in Part 11 - the concept is introduced here because it's fundamentally an error-handling concept, not just an API concept. For now, the corrected pattern:

```
async function getData() {
  try {
    const response = await fetch(url);

    if (!response.ok) { // response.ok is true only for HTTP status 200-299  -  full detail in Part 11
      throw new Error(`HTTP error: ${response.status}`);
    }

    const data = await response.json();
    return data;

  } catch (error) {
    console.error("Failed:", error);
    throw error; // re-throw  -  recall from Part 9.8, this lets the CALLER decide how to ultimately handle it
  }
}
```

Why `!response.ok` is manually `throw`n: since `fetch()` won't do this for us, we must explicitly check the status and `throw` ourselves - converting an HTTP-level failure into the same `catch`-able exception path as a network-level failure, so our error handling is consistent regardless of which layer actually failed.

This is a distinction that trips up even experienced developers - there are (at least) four distinct categories of things that can go wrong in a single `fetch` + `json()` sequence:

Failure type	Example cause	Where it shows up
Network failure	No internet connection, DNS failure, CORS blocked	`fetch()` itself rejects
HTTP failure	Server responds, but with an error status (404, 500, etc.)	`fetch()` resolves successfully! You must manually check `response.ok`
JSON parsing failure	Server claims JSON but sends malformed/empty text	`response.json()` rejects
Application-level failure	Response is valid JSON, HTTP status is 200, but the DATA itself signals an error (e.g., `{ success: false, message: "Invalid credentials" }`)	Neither `fetch()` nor `response.json()` will catch this - you must check it yourself, in your own code, after parsing

We build a complete, production-grade helper function that correctly handles all four categories in Part 11.6.

```
async function loadUserProfile() {
  try {
    setLoading(true);       // (conceptual - real implementation with React state comes in Part 22)
    setError(null);          // clear any previous error before trying again

    const data = await fetchSomething();
    setData(data);
  } catch (error) {
    setError(error.message);
  } finally {
    setLoading(false);      // ALWAYS runs - success or failure - so the UI never gets stuck "loading" forever
  }
}
```

Why `finally` is essential here, specifically: if you instead put `setLoading(false)` at the end of the `try` block, it would never run if an error was thrown, leaving your UI permanently showing a loading spinner even though the operation has actually finished (with an error). `finally`'s "runs no matter what" guarantee (Part 9.4) is precisely the property needed to correctly manage a loading state across both success and failure paths. This exact three-state pattern (loading / success / error) is the backbone of Part 22 (API calls in React) - internalize it now.

Async Level 11 - Sequential vs Parallel await

Sequential (each `await` waits for the previous to fully finish before starting the next):

```
async function sequential() {
  console.time("sequential");
  const a = await taskA(); // waits ~1000ms
  const b = await taskB(); // THEN starts, waits ANOTHER ~1000ms
  console.timeEnd("sequential"); // ~2000ms total
  return [a, b];
}
```

Parallel (both operations START at the same time, and we wait for both together):

```
async function parallel() {
  console.time("parallel");
  const [a, b] = await Promise.all([
    taskA(), // both start executing IMMEDIATELY, at the same time
    taskB()
  ]);
  console.timeEnd("parallel"); // only ~1000ms total - the SLOWER of the two, not the sum!
  return [a, b];
}
```

Why the performance difference matters so much: in the sequential version, `taskB()` doesn't even start until `taskA()` has completely finished - even though, in this example, `taskB` has no actual dependency on `taskA`'s result. This wastes time for no reason. In the parallel version, both promises are created (and their underlying operations begin) on the same synchronous line, and `Promise.all` simply waits for whichever one takes longer.

The rule for deciding which to use: use sequential `await` only when a later operation genuinely depends on the result of an earlier one (e.g., you need a user's ID from step 1 before you can fetch their orders in step 2). Use `Promise.all` (parallel) whenever multiple operations are independent of each other - there's no reason to make the user wait for the sum of both delays when they could wait for only the longer one.

```
// [x] Correctly sequential  -  orders genuinely depend on the user's id
const user = await getUser(userId);
const orders = await getOrders(user.id); // needs user.id  -  MUST wait for the line above

// [x] Correctly parallel  -  completely independent pieces of data
const [settings, notifications] = await Promise.all([
  getSettings(userId),
  getNotifications(userId)
]);
```

Async Level 12 - HTTP Errors (Preview - full depth in Part 11)

Covered thoroughly in Level 10 above and expanded fully with real HTTP status codes in Part 11.3-11.6. The key takeaway to carry forward: a resolved `fetch()` Promise does not mean success - you must check `response.ok`/`response.status` yourself.

Async Level 13 & 14 - Parallel Requests and Promise.all

Covered fully in Level 7.5 and Level 11 above. We'll apply this directly to real `fetch()` calls in Part 11's projects (e.g., loading a dashboard that needs data from three different endpoints simultaneously).

Async Level 15 - Cancellation with AbortController

What it is

A browser API that lets you cancel an in-progress asynchronous operation - most commonly, a `fetch()` request - that you no longer need the result of.

Why it exists - the problem it solves

Imagine a search-as-you-type feature. Every keystroke fires a new `fetch()` request. If the user types quickly, you might have 5 requests in flight at once, and - because of network timing variability - there's no guarantee they resolve in the order they were sent. A slow, early request for `"a"` could resolve after a fast, later request for `"apple"`, overwriting the correct results with stale, irrelevant ones. This is a race condition, covered fully in Level 18. `AbortController` lets you explicitly cancel outdated requests before this can happen.

```
const controller = new AbortController();

fetch(url, { signal: controller.signal })
  .then((response) => response.json())
  .then((data) => console.log(data))
  .catch((error) => {
    if (error.name === "AbortError") {
      console.log("Request was cancelled  -  this is expected, not a real failure!");
    } else {
      console.error("A real error occurred:", error);
    }
  });

// Later, to cancel:
controller.abort();
```

Step-by-step: `new AbortController()` creates a controller object with a `.signal` property and an `.abort()` method. Passing `{ signal: controller.signal }` to `fetch` links this specific request to that controller. Calling `controller.abort()` at any point causes the `fetch` Promise to reject with a special `AbortError` - which is why the `.catch()` block specifically checks `error.name === "AbortError"` to distinguish "we cancelled this on purpose" from "something actually went wrong."

We build the full, practical debounced-search pattern using `AbortController` in Part 22, once we have real React state to tie it to.

Async Level 16 - Timeouts

Implementing a timeout using `Promise.race()` (recap from Level 7.5), now combined with `AbortController` for a proper, complete cancellation:

```
async function fetchWithTimeout(url, timeoutMs = 5000) {
  const controller = new AbortController();
  const timeoutId = setTimeout(() => controller.abort(), timeoutMs);

  try {
    const response = await fetch(url, { signal: controller.signal });
    clearTimeout(timeoutId); // success  -  cancel the pending timeout, we don't need it anymore
    return response;
  } catch (error) {
    clearTimeout(timeoutId);
    if (error.name === "AbortError") {
      throw new Error(`Request timed out after ${timeoutMs}ms`);
    }
    throw error; // some other real error  -  re-throw as-is
  }
}
```

Why `clearTimeout` appears in both the success path and the catch block: if the request finishes (success OR a non-timeout error) before the timeout fires, we must cancel the pending `setTimeout` - otherwise it would still fire later and call `controller.abort()` on a request that's already finished, which is harmless here but is wasted work and a general good habit (an un-cleared timer is a small, classic memory-leak pattern, echoing Part 5.3 and Part 6.7).

Async Level 17 - Retries and Exponential Backoff

What it is

A pattern for automatically re-attempting a failed asynchronous operation, on the theory that some failures (like a temporary network blip, or a server that's briefly overloaded) are transient and likely to succeed on a subsequent attempt.

```
async function fetchWithRetry(url, maxRetries = 3) {
  for (let attempt = 1; attempt <= maxRetries; attempt++) {
    try {
      const response = await fetch(url);
      if (!response.ok) {
        throw new Error(`HTTP ${response.status}`);
      }
      return await response.json(); // success  -  exit the loop entirely
    } catch (error) {
      console.log(`Attempt ${attempt} failed: ${error.message}`);
```

```
      if (attempt === maxRetries) {
        throw error; // out of retries  -  give up and propagate the final failure
      }

      // Exponential backoff: wait longer after each successive failure
      const delayMs = 2 ** attempt * 1000; // 2s, 4s, 8s, ...
      console.log(`Retrying in ${delayMs}ms...`);
      await new Promise((resolve) => setTimeout(resolve, delayMs));
    }
  }
}
```

Why "exponential backoff" specifically, instead of retrying immediately or at a fixed interval: if a server is struggling under load, immediately retrying (or retrying at a fixed short interval) from potentially many clients simultaneously can make the overload worse - a phenomenon sometimes called a "retry storm." Waiting progressively longer between attempts (`2s`, then `4s`, then `8s`...) gives the server breathing room to recover, and reduces the odds that all failing clients hammer it again at exactly the same moment.

⚠ Why blindly retrying can be dangerous - an important, easy-to-miss caveat: retries are safe for operations that are naturally idempotent (doing them twice has the same effect as doing them once) - a `GET` request (Part 11.2) is a great example. Retries are dangerous for non-idempotent operations - e.g., automatically retrying a `POST /createOrder` request that actually succeeded on the server, but whose response was lost due to a network blip, could accidentally create the order twice, charging a customer twice for one purchase. Rule: only build automatic retry logic for genuinely idempotent operations, or ones that have server-side protection against duplicate effects (like an idempotency key), unless you fully understand and accept the risk.

Async Level 18 - Race Conditions

What it is

A race condition occurs when the correctness of your program depends on the timing or order of asynchronous operations completing - and that order isn't guaranteed.

A concrete, realistic example - the search-as-you-type bug (preview of a bug you'll actually fix in Part 22):

```
let latestResults = null;

function search(query) {
  fetch(`/api/search?q=${query}`)
    .then((res) => res.json())
    .then((data) => {
      latestResults = data; // ⚠ whichever request finishes LAST wins, regardless of which was typed last!
      renderResults(latestResults);
    });
}

search("a");      // fired first  -  say it's slow, takes 800ms
search("ap");     // fired second  -  fast, takes 200ms
search("app");   // fired third  -  fast, takes 200ms
```

If `search("a")`'s request (800ms) resolves after `search("app")`'s (200ms), the UI ends up displaying results for `"a"` - the OLDEST, most irrelevant query - simply because its network response happened to arrive last in wall-clock time, even though it was the first one the user typed.

Three ways to fix this (in increasing sophistication):

1. `AbortController` (Level 15): cancel the previous request every time a new one starts, so only the latest one can ever resolve.
2. A request ID / sequence check: tag each request with an incrementing counter, and when a response comes back, only apply it if it's still the most recent request that was sent, discarding stale ones.
3. Debouncing (below): don't even fire the request until the user has paused typing, dramatically reducing how many overlapping requests exist in the first place.

We implement all three approaches concretely with real React state in Part 22.

Async Level 19 - Async in React (Preview)

Fully covered starting Part 22. The short preview: React components need special care around async operations because a component can unmount (be removed from the screen) while an async operation it started is still in flight - and updating state on an unmounted component is a bug. `useEffect`'s cleanup function (Part 21) combined with `AbortController` is the standard solution.

Async Level 20 - Production API Architecture (Preview)

Fully covered in Part 11 and Part 29. The short preview: real applications centralize their async/fetch logic into a dedicated service layer (not scattered `fetch()` calls throughout UI components), with a single, reusable, well-tested request helper (built incrementally in Part 11.6) that consistently handles errors, timeouts, retries, and authentication headers in one place.

Knowledge Check - Part 10

1. In your own words, what does "asynchronous" mean, and why does it matter that JavaScript is single-threaded?
2. Walk through, step by step, why `setTimeout(fn, 0)` still logs after synchronous code that comes after it in the source.
3. What are the three states a Promise can be in, and can it ever move backward between them?
4. Why does `Promise.all()` reject immediately if even one input Promise rejects, while `Promise.allSettled()` never rejects at all?
5. What does `async` do to a function's return value, even if you write `return 5;`?
6. Precisely, what happens to the rest of a program when an `async` function hits an `await`?
7. Why does an `await`ed rejected Promise get caught by a regular `try/catch` block?
8. Why does `fetch()` NOT reject for a 404 or 500 response? What must you do instead?
9. Give a concrete example of when sequential `await` is correct, and one where it would be a performance mistake compared to `Promise.all`.

10 What problem does `AbortController` solve that a plain `try/catch` around a `fetch` call does not?

11 Explain, in your own words, what a race condition is, using the search-as-you-type example.

12 Why is exponential backoff preferred over immediately retrying a failed request?

13 Why can automatically retrying a failed `POST` request be dangerous?

14 In the event loop diagram, why do ALL queued microtasks run before the NEXT single macrotask, rather than one of each alternating?

15 What is callback hell, and what specific problem (beyond just "ugly indentation") does it cause for error handling?

1 Predict the exact console output order, then verify: `console.log("1"); setTimeout(() => console.log("2"), 0); Promise.resolve().then(() => console.log("3")); console.log("4");`

2 Convert this callback-based function into one that returns a Promise instead: `function getRandomNumber(callback) { setTimeout(() => { callback(Math.floor(Math.random() * 100)); }, 500); }`

3 Using the Promise-based version from #2, write an `async` function that gets three random numbers sequentially and logs their sum.

4 Rewrite #3 to get the three random numbers in parallel instead, using `Promise.all`.

5 Write an `async` function `safeJsonParse(jsonString)` that returns `{ success: true, data }` on success, or `{ success: false, error: message }` on failure - using `try/catch`, never letting the function itself throw.

6 Given this function that sometimes rejects: `function unreliableTask() { return new Promise((resolve, reject) => { setTimeout(() => { Math.random() > 0.5 ? resolve("Success!") : reject(new Error("Failed!")); }, 300); }); }` Write an `async` function that calls `unreliableTask()` up to 3 times, stopping as soon as it succeeds, and throwing if all 3 attempts fail.

7 Using `Promise.allSettled()`, write code that calls `unreliableTask()` five times in parallel and logs how many succeeded and how many failed.

8 Rewrite this Promise chain using `async/await`: `function getData() { return fetchStep1() .then((a) => fetchStep2(a)) .then((b) => fetchStep3(b)) .catch((error) => { console.error(error); throw error; }); }`

9 Write a function `delay(ms)` that returns a Promise resolving after `ms` milliseconds, with no resolved value needed - then use it with `await` to log `"Start"`, wait 1 second, then log `"End"`.

10 Using `Promise.race()`, implement a `withTimeout(promise, ms)` function that rejects with `"Timeout!"` if `promise` doesn't settle within `ms` milliseconds, otherwise resolves/rejects with `promise`'s own outcome.

11 Debug this code - it's supposed to log each number 1 second apart (1, then 2, then 3), but doesn't. Fix it: `async function logNumbers() { [1, 2, 3].forEach(async (n) => { await delay(1000); console.log(n); }); }`

12 Explain (in a comment) why the following throws an unhandled Promise rejection warning, and fix it: `async function risky() { throw new Error("Oops"); } risky(); // called with no .catch() and no try/catch around it`

13 Write an `async` function that fetches data with a 3-attempt retry using exponential backoff (reuse the pattern from Level 17), and logs each attempt number.

14 Using `AbortController`, write a function `fetchWithCancel(url)` that returns both the fetch Promise and a `cancel()` function the caller can invoke to abort it.

15 "Predict the output" - a genuinely tricky one combining several concepts: `console.log("start"); async function asyncFn() { console.log("async fn start"); await null; console.log("async fn end"); } asyncFn(); Promise.resolve().then(() => console.log("promise 1")); console.log("end");`

> View solutions

Knowledge Check

1 Asynchronous means an operation can be started, and its completion handled later, without blocking the rest of the program from running in the meantime. This matters because JavaScript can only execute one thing at a time (single-threaded) - without async mechanisms, any slow operation (like a network request) would freeze the entire program/browser tab until it finished.

2 `setTimeout` always hands its callback off to the browser's Web APIs, which queues it as a macrotask once its timer elapses. Even with a 0ms delay, the callback can only run once the current synchronous code has fully finished AND the microtask queue is empty - it never runs "immediately" or interrupts currently-running code.

3 Pending, fulfilled, and rejected. Once a Promise moves from pending to either fulfilled or rejected, it is permanently "settled" and can never change state again.

4 `Promise.all()` is designed for "I need ALL of these to succeed" scenarios - a single failure means the overall goal can't be achieved, so it fails fast. `Promise.allSettled()` is designed for "tell me the outcome of each, regardless of success or failure" scenarios, so it always waits for every Promise and never itself rejects - failures are reported as data (a `{status: "rejected", reason}` object), not as a rejection of the whole `allSettled` call.

5 It automatically wraps the returned value in a resolved Promise - `return 5;` inside an `async function` behaves like `return Promise.resolve(5);`.

6 Execution of that specific `async` function pauses at the `await` line, and control is handed back to whatever called it - allowing the rest of the program (anything after the call to the async function, and the rest of the event loop) to continue running normally in the meantime. Nothing else is blocked.

7 Because `async/await` is built so that a rejected `await`ed Promise is converted into a thrown exception at that exact point - this is a deliberate design choice so that the same `try/catch` syntax and mental model from synchronous code works, unmodified, for asynchronous code too.

8 `fetch()` only rejects for network-level failures (no connection, DNS failure, CORS block, etc.) - it considers receiving ANY HTTP response, including error statuses like 404/500, a "successful" fetch, because technically the request-response cycle completed. You must manually check `response.ok` (or `response.status`) and explicitly `throw` if it indicates failure.

9 Sequential is correct when a later step needs data produced by an earlier step - e.g., fetching a user, then using that user's ID to fetch their orders. It's a performance mistake when steps are independent of each other - e.g., fetching a user's profile and their notification count separately; awaiting them sequentially wastes time waiting for the sum of both delays instead of just the longer one via `Promise.all`.

10 A plain `try/catch` can only handle a request's eventual success or failure - it can't proactively stop an in-flight request that's no longer needed (e.g., because a newer request has superseded it), which can lead to race conditions where a stale, outdated response overwrites more recent, correct data. `AbortController` lets you explicitly cancel the outdated request.

11 A race condition happens when correctness depends on unpredictable timing of async operations. In search-as-you-type, multiple requests for different queries are in flight simultaneously, and because network timing isn't guaranteed to match send order, an older, slower request can resolve AFTER a newer, faster one - overwriting correct, current results with stale, irrelevant ones purely due to arrival timing.

12 Immediately retrying (or retrying at a fixed short interval) can worsen an already-struggling server's load, especially if many clients are all retrying at once ("retry storm"). Exponential backoff spaces out retries with increasing delays, giving the server room to recover and reducing the chance of many clients hammering it simultaneously.

13 If the original request actually succeeded on the server but its response was lost in transit (making it look like a failure to the client), blindly retrying a non-idempotent operation like `POST /createOrder` could cause the action to happen a second time - e.g., creating (and charging for) a duplicate order.

14 This is a deliberate design choice in the event loop specification: it ensures that all pending "reactions" to already-completed work (Promise callbacks) are fully processed before any new macrotask work (like a timer or a UI event) is allowed to run, keeping Promise-chain-driven logic as up-to-date and consistent as possible before the event loop moves on.

15 Callback hell is the deeply nested pyramid of callbacks-within-callbacks that results from chaining several sequential asynchronous operations using the plain callback pattern. Beyond unreadable indentation, it specifically makes error handling extremely painful - because each nested callback typically needs its own separate error check, there's no single, centralized place to catch a failure from any step in the sequence, unlike a single `.catch()` at the end of a Promise chain or a single `try/catch` around `await`ed steps.

Coding Challenges

1

```
1
4
3
2
```

(Synchronous code runs first: "1", "4". Then the microtask queue is drained: "3". Only then does the macrotask queue's setTimeout callback run: "2".)

1

```
function getRandomNumber() {
  return new Promise((resolve) => {
```

```
    setTimeout(() => {
      resolve(Math.floor(Math.random() * 100));
    }, 500);
  });
}
```

1

```
async function sumThreeSequential() {
  const a = await getRandomNumber();
  const b = await getRandomNumber();
  const c = await getRandomNumber();
  console.log("Sum:", a + b + c);
}
sumThreeSequential();
```

1

```
async function sumThreeParallel() {
  const [a, b, c] = await Promise.all([
    getRandomNumber(),
    getRandomNumber(),
    getRandomNumber()
  ]);
  console.log("Sum:", a + b + c);
}
sumThreeParallel();
```

1

```
async function safeJsonParse(jsonString) {
  try {
    const data = JSON.parse(jsonString);
    return { success: true, data };
  } catch (error) {
    return { success: false, error: error.message };
  }
}
```

1

```
async function tryUpToThreeTimes() {
  for (let attempt = 1; attempt <= 3; attempt++) {
    try {
      const result = await unreliableTask();
      console.log(`Succeeded on attempt ${attempt}`);
      return result;
    } catch (error) {
      console.log(`Attempt ${attempt} failed`);
      if (attempt === 3) {
        throw new Error("All 3 attempts failed");
      }
    }
  }
}
```

1

```
async function runFiveInParallel() {
  const results = await Promise.allSettled([
    unreliableTask(), unreliableTask(), unreliableTask(), unreliableTask(), unreliableTask()
  ]);
```

```
  const succeeded = results.filter(r => r.status === "fulfilled").length;
  const failed = results.filter(r => r.status === "rejected").length;

  console.log(`${succeeded} succeeded, ${failed} failed`);
}
```

1

```
async function getData() {
  try {
    const a = await fetchStep1();
    const b = await fetchStep2(a);
    const c = await fetchStep3(b);
    return c;
  } catch (error) {
    console.error(error);
    throw error;
  }
}
```

1

```
function delay(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

async function run() {
  console.log("Start");
  await delay(1000);
  console.log("End");
}
run();
```

1

```
function withTimeout(promise, ms) {
  const timeoutPromise = new Promise((_, reject) => {
    setTimeout(() => reject(new Error("Timeout!")), ms);
  });
  return Promise.race([promise, timeoutPromise]);
}
```

1

```
// BUG: .forEach does not wait for async callbacks  -  it fires all three
// iterations essentially simultaneously, not one after another.
// FIX: use a for...of loop, which DOES respect await inside it.
async function logNumbers() {
  for (const n of [1, 2, 3]) {
    await delay(1000);
    console.log(n);
  }
}
```

1

```
// An async function that throws produces a REJECTED promise.
// Calling risky() with no .then()/.catch() and no surrounding try/catch
// leaves that rejection completely unhandled, which most environments
// warn about loudly (and in some environments, can crash the process).
async function risky() {
  throw new Error("Oops");
}
```

```
risky().catch((error) => console.error("Handled:", error.message)); // fixed
```

1

```
async function fetchDataWithRetry(url, maxAttempts = 3) {
  for (let attempt = 1; attempt <= maxAttempts; attempt++) {
    console.log(`Attempt ${attempt}`);
    try {
      const response = await fetch(url);
      if (!response.ok) throw new Error(`HTTP ${response.status}`);
      return await response.json();
    } catch (error) {
      if (attempt === maxAttempts) throw error;
      const delayMs = 2 ** attempt * 1000;
      await new Promise((resolve) => setTimeout(resolve, delayMs));
    }
  }
}
```

1

```
function fetchWithCancel(url) {
  const controller = new AbortController();
  const promise = fetch(url, { signal: controller.signal });
  return {
    promise,
    cancel: () => controller.abort()
  };
}

// Usage:
const { promise, cancel } = fetchWithCancel("/api/data");
// cancel(); // could be called later, e.g., if the user navigates away
```

1

```
start
async fn start
end
async fn end
promise 1
```

Explanation: `asyncFn()` runs synchronously up until its first `await` - so `"async fn start"` logs immediately (synchronously) when `asyncFn()` is called. `await null` still yields control back (awaiting ANY value, even a non-Promise, still queues a microtask continuation), so `asyncFn` pauses there. Control returns to the top-level code, which registers the `Promise.resolve().then(...)` microtask, then logs `"end"` synchronously. NOW the call stack is empty, and the microtask queue is processed in the order its entries were queued: `asyncFn`'s continuation (`"async fn end"`) was queued first (from the `await null`), so it runs before `"promise 1"` (queued second).

Interview Questions - Asynchronous JavaScript

Q: What is the event loop?
The event loop is the mechanism that allows single-threaded JavaScript to handle asynchronous operations without blocking. It continuously checks whether the call stack is empty; when it is, it first fully drains the microtask queue (Promise callbacks, async/await continuations), and only then takes the next single task from the macrotask queue (setTimeout callbacks, DOM events) to execute.

Q: What is a Promise?
An object representing the eventual result of an asynchronous operation, which exists in one of three states - pending, fulfilled, or rejected - and once settled (fulfilled or rejected), never changes state again. Promises let you attach `.then()`/`.catch()`/`.finally()` handlers and chain sequential asynchronous steps without nested callbacks.

Q: Difference between `Promise.all` and `Promise.allSettled`?
`Promise.all` resolves with an array of all results only if every input Promise succeeds, and rejects immediately (fail-fast) on the first failure. `Promise.allSettled` always resolves once every input has settled, returning an array describing each one's individual outcome (fulfilled or rejected), and never itself rejects.

Q: What does `async` do?
It marks a function as always returning a Promise (auto-wrapping any returned value in a resolved Promise, and any thrown error in a rejected one), and enables the use of `await` within that function's body.

Q: What does `await` do?
It pauses execution of the enclosing `async` function until the awaited Promise settles, then either returns its resolved value or throws its rejection reason (catchable via `try/catch`) - without blocking the rest of the program.

Q: Why use `try/catch` with `async/await` instead of `.catch()`?
Both work and are valid; it's largely a style/context choice. `try/catch` tends to read more naturally when you have several sequential `await` steps that should all be covered by one error-handling block, matching how synchronous error handling already looks. `.catch()` is often preferred at the very end of a Promise chain (`.then().then().catch()`), or when you specifically want to handle a Promise's rejection outside of an `async function` context.

Next: Part 11 - APIs

PART 11 - APIs [Intermediate][Advanced][Advanced]

> *The second of the two "biggest and most detailed" sections in this course. Everything here builds directly on Part 10 - if any of Part 10's async/await/try-catch material felt shaky, revisit it before continuing, because we're about to apply it constantly, to real network requests.*

11.1 What Is an API?

Real-world analogy

Imagine a restaurant. You (the client) don't walk into the kitchen and cook your own food. You give the waiter (the API) an order, written on a form the kitchen understands (a request). The kitchen (the server) prepares it and the waiter brings it back (a response). You never need to know how the kitchen works internally - you just need to know the menu (what you're allowed to ask for) and how to place an order correctly.

Technical definition

An API (Application Programming Interface) is a defined contract that lets one piece of software request functionality or data from another, without needing to know its internal implementation. A web API specifically means this contract is exposed over HTTP (recall Part 0.4) - you send an HTTP request to a specific address, formatted a specific way, and get an HTTP response back.

Core vocabulary

Term	Meaning
Client	The program making the request (your frontend JavaScript, in this course)
Server	The program receiving the request and sending back a response
Endpoint	A specific URL path that performs a specific function (e.g., `/api/users/42`)
Base URL	The root address of the API (e.g., `https://api.example.com`)
Route	The path + method combination that maps to a specific server-side handler
Headers	Metadata sent with a request or response (content type, authentication, etc.)
Body	The actual data payload of a request or response
Authentication	Proving who is making the request

11.2 HTTP Methods

Each HTTP method signals the intent of a request. Servers are free to implement them however they want, but there's a strong, near-universal convention:

Method	Typical meaning	Has a body?	Idempotent? (recall Part 10, Level 17)
GET	Retrieve data. Should never change anything on the server.	[x] No	[x] Yes - repeating it has no additional effect
POST	Create a new resource.	[x] Yes	[x] No - repeating it typically creates duplicates
PUT	Replace an existing resource entirely with new data.	[x] Yes	[x] Yes - repeating it with the same data leaves the same end result
PATCH	Partially update an existing resource (only some fields).	[x] Yes	⚠ Depends on implementation - often not strictly idempotent
DELETE	Remove a resource.	[x] Usually not	[x] Yes - deleting something already deleted has no further effect

`PUT` vs `PATCH` - the practical difference: `PUT` conceptually says "here is the complete new version of this resource, replace everything." `PATCH` says "here are just the fields that changed, leave the rest alone." Sending `{ "name": "New Name" }` via `PUT` to a user resource could, depending on server implementation, wipe out every other field not included in the request (because `PUT` means "replace entirely"); the same payload via `PATCH` should only touch `name`.

11.3 HTTP Status Codes

Status code categories

Range	Category	Meaning
1xx	Informational	Request received, processing continues (rarely seen directly in frontend work)
2xx	Success	The request was successfully received, understood, and accepted
3xx	Redirection	Further action needed to complete the request (usually handled automatically by the browser)
4xx	Client Error	The request itself was invalid, unauthorized, or otherwise the client's fault
5xx	Server Error	The server failed to fulfill a valid request - the server's fault

Individual codes you must know:

Code	Name	Meaning
200	OK	Standard success
201	Created	A new resource was successfully created (typical POST success)
204	No Content	Success, but there's deliberately no response body (typical DELETE success)
400	Bad Request	The request is malformed or invalid in some way the server can detect
401	Unauthorized	You're not authenticated - you need to log in / provide valid credentials
403	Forbidden	You ARE authenticated, but you don't have permission for this specific action
404	Not Found	The requested resource doesn't exist
409	Conflict	The request conflicts with the current state of the resource (e.g., a username that's already taken)
422	Unprocessable Entity	The request is well-formed, but fails validation rules (e.g., an invalid email format)
429	Too Many Requests	You've been rate-limited - you're sending requests too quickly
500	Internal Server Error	Something broke on the server, unrelated to your request's validity
502	Bad Gateway	A server acting as a proxy/gateway got an invalid response from an upstream server
503	Service Unavailable	The server is temporarily overloaded or down for maintenance

401 vs 403 - a very commonly confused pair: 401 means "I don't know who you are (or your credentials are invalid) - log in." 403 means "I know exactly who you are, and you're not allowed to do this." A logged-out user hitting an admin page gets 401; a logged-in non-admin user hitting an admin page gets 403.

11.4 The Fetch API - From First Principles

```
fetch(url);
```

This alone starts a GET request and returns a Promise. On its own, this is rarely useful - you need to actually read the response.

```
fetch("https://api.example.com/users")
  .then((response) => response.json())
  .then((data) => console.log(data))
  .catch((error) => console.error(error));
```

Step-by-step: `fetch(url)` returns a Promise that resolves with a `Response` object once headers have arrived - notably, before the full body has necessarily been downloaded. `response.json()` is itself asynchronous (reading and parsing the body takes additional time), which is why it also returns a Promise, requiring its own `.then()`.

```
async function getUsers() {
  try {
    const response = await fetch("https://api.example.com/users");
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error(error);
  }
}
```

This is functionally identical to the `.then()` version above - recall from Part 10 that `async/await` is syntactic sugar over Promises. From this point forward, this course uses `async/await` as the default style for all API code, as is standard in modern professional practice.

11.5 GET Requests with Query Parameters

```
async function searchProducts(query, category) {
  const params = new URLSearchParams({ q: query, category });
  const response = await fetch(`https://api.example.com/products?${params}`);
  return response.json();
}

searchProducts("laptop", "electronics");
// requests: https://api.example.com/products?q=laptop&category=electronics
```

Why use `URLSearchParams` instead of manually building the string with template literals: it correctly URL-encodes special characters for you (spaces, `&`, `=`, non-ASCII characters) - something that's easy to get subtly wrong by hand and causes confusing bugs when a search term happens to contain a character with special meaning in a URL.

```
const params = new URLSearchParams({ q: "shoes & socks" });
console.log(params.toString()); // "q=shoes+%26+socks"  -  the & is safely encoded, not breaking the query string
```

11.6 POST, PUT, PATCH, DELETE - Sending Data

```
async function createUser(userData) {
  const response = await fetch("https://api.example.com/users", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify(userData)
  });

  if (!response.ok) {
    throw new Error(`Failed to create user: ${response.status}`);
  }

  return response.json();
}

createUser({ name: "Ammaar", email: "ammaar@example.com" });
```

Explaining every property of the options object, in full:

- `method: "POST"` - overrides the default `GET`, telling the server this request intends to create something.
- `headers` - an object of metadata. `"Content-Type": "application/json"` tells the server how to interpret the body - specifically, "the body you're about to read is a JSON string, please parse it as such." Without this header, many servers won't correctly parse the body, even if it looks like valid JSON.
- `body: JSON.stringify(userData)` - the actual data being sent. `fetch` bodies must be strings (or specific binary/FormData types) - you cannot pass a raw JavaScript object. `JSON.stringify` converts our object into its JSON text representation (recall Part 0.12).

```
async function replaceUser(id, fullUserData) {
  const response = await fetch(`https://api.example.com/users/${id}`, {
    method: "PUT",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(fullUserData) // the COMPLETE new representation
  });
  if (!response.ok) throw new Error(`Failed to replace user: ${response.status}`);
  return response.json();
}

async function updateUserPartial(id, changedFields) {
  const response = await fetch(`https://api.example.com/users/${id}`, {
    method: "PATCH",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(changedFields) // ONLY the fields that changed
  });
  if (!response.ok) throw new Error(`Failed to update user: ${response.status}`);
  return response.json();
}

async function deleteUser(id) {
  const response = await fetch(`https://api.example.com/users/${id}`, {
    method: "DELETE"
  });

  if (!response.ok) {
    throw new Error(`Failed to delete user: ${response.status}`);
```

```
  }

  // Recall status 204 "No Content" from 11.3  -  many DELETE endpoints return
  // an empty body, so calling response.json() here would THROW (empty
  // string isn't valid JSON)! Check for 204 before trying to parse:
  if (response.status === 204) {
    return null;
  }

  return response.json();
}

async function getPrivateData(token) {
  const response = await fetch("https://api.example.com/me", {
    headers: {
      "Authorization": `Bearer ${token}`
    }
  });
  return response.json();
}
```

What a "Bearer token" conceptually is: a string (usually issued by the server after a successful login) that proves the request comes from an authenticated user, without needing to send a username/password on every single request. The server checks this token on each incoming request and identifies who's making it. This course doesn't build a full authentication system, but you'll recognize this exact header pattern in virtually every real-world API you work with professionally.

11.7 Reading Different Response Body Types

```
const response = await fetch(url);

await response.json();   // parses the body as JSON -> a JS object/array
await response.text();     // returns the body as a raw string (HTML, plain text, CSV, etc.)
await response.blob();    // returns the body as binary data (images, files, etc.)
```

⚠ Critical rule: you can only read a `Response` body ONCE. Calling `.json()` and then `.text()` on the same response object will throw an error on the second call, because the underlying data stream has already been consumed. If you genuinely need the raw text AND parsed JSON, read `.text()` once and `JSON.parse()` that string yourself.

11.8 CORS (Conceptual Overview)

What it is

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that restricts JavaScript running on one origin (domain) from freely making requests to a different origin, unless that other origin explicitly allows it.

Why it exists

Without CORS, any malicious website you visit could silently use your logged-in browser session to make requests to your bank's API, your email provider, etc., on your behalf, potentially reading or modifying your data - because cookies and session state are often automatically included with requests. CORS is the browser's way of asking the target server, "hey, is it okay for a page from

`attacker-site.com` to talk to you?" before allowing the response to actually reach the requesting page's JavaScript.

How it (conceptually) works

1. Your frontend, running on `https://myapp.com`, makes a `fetch()` request to `https://api.example.com`.
2. This is a cross-origin request (different domain than the page itself).
3. For certain requests (see "preflight" below), the browser first sends an automatic `OPTIONS` request - a preflight request - asking the server "will you allow a real request like this, from this origin?"
4. The server responds with headers like `Access-Control-Allow-Origin: https://myapp.com` (or `*` for "anyone"), explicitly permitting it.
5. If the server allows it, the browser proceeds with the actual request. If not, the browser blocks the response from reaching your JavaScript (the request may have still technically happened on the network, but your code never sees the result) and you see a CORS error in the console.

Preflight requests, briefly: these happen automatically, before the browser sends certain "non-simple" requests (e.g., ones with custom headers like `Authorization`, or methods other than `GET`/`POST`/`HEAD` with simple content types). You don't write preflight logic yourself - the browser handles it - but you'll see these `OPTIONS` requests in the Network tab (Part 0.16), and understanding what they are prevents confusion when debugging.

⚠ A critical misconception to correct: CORS is enforced by the browser, on behalf of the requesting page - it is not a security measure that protects the server from malicious clients in general (a non-browser client, like `curl` or a Node.js script, is completely unaffected by CORS, because there's no browser enforcing it). CORS exists specifically to protect users' browsers from malicious websites making unauthorized cross-origin requests using the user's own browser context.

11.9 API Keys, Environment Variables, and Why Frontend Secrets Don't Work

⚠ Critical security principle: never put a secret API key directly in frontend JavaScript code.

```
// [Advanced] NEVER DO THIS
const response = await fetch(`https://api.example.com/data?apiKey=sk_live_abc123secret`);
```

Why this is a serious security problem: everything in your frontend JavaScript bundle is downloaded, in full, to every single visitor's browser - recall from Part 0.3 that the browser (and everything sent to it) is fully under the user's control. Anyone can open DevTools' Sources tab, or just view the page source / network requests, and read that key directly out of your code. From that point, they can use your key as if it were their own - potentially running up massive bills on your account, or accessing data they shouldn't.

The correct architecture:

```
Browser (Frontend)  ──request (no secret)──>  Your Backend Server  ──request (WITH secret key)──>  External API
                    <──response──────────────                       <──response──────────────────
```

Your own backend server (which you control, and which never sends its source code to the client) holds the real secret key, makes the request to the external API on the frontend's behalf, and returns only the necessary data back to the frontend. The frontend never sees the secret at all.

Environment variables are how secrets are kept out of your source code even on the server side - they're values set in the server's runtime environment (not committed to your Git repository, recall Part 13), read at runtime via something like `process.env.API_KEY` in Node.js. We revisit this concretely in Part 37 (Production Concepts), including how frontend build tools handle environment variables differently (and why even those aren't for secrets - see the warning there).

11.10 Building a Robust API Request Helper - Incrementally

This is the practical culmination of Part 9, Part 10, and everything above in Part 11. We build one function up, step by step, exactly the way you'd actually develop it - starting simple, and adding real-world robustness one justified step at a time.

```
async function apiRequest(url, options = {}) {
  const response = await fetch(url, options);
  return response.json();
}
```

What's wrong with this: doesn't check `response.ok` (recall Part 10, Level 10/12) - a 404 or 500 response is silently treated as success, and whatever error page or error JSON the server sent back gets returned as if it were valid data.

```
async function apiRequest(url, options = {}) {
  try {
    const response = await fetch(url, options);

    if (!response.ok) {
      const message = `Request failed: ${response.status}`;
      throw new Error(message);
    }

    return await response.json();

  } catch (error) {
    console.error("API request failed:", error);
    throw error; // re-throw so the CALLER can still decide how to handle it (recall Part 9.7)
  }
}
```

Improvement: now correctly treats HTTP error statuses as failures. Still has problems, though:

1. The error message is generic (`"Request failed: 404"`) - it doesn't surface whatever useful error message the server itself might have included in its response body.
2. Calling `.json()` on a `204 No Content` response (recall 11.6) will throw, even though 204 represents success.
3. If the server's error response isn't valid JSON at all, `.json()` throws a confusing, unrelated-looking error.

```
async function apiRequest(url, options = {}) {
  try {
    const response = await fetch(url, options);
```

```
    if (!response.ok) {
      let message = `HTTP ${response.status}`;

      // Attempt to get a more specific message from the server's response body,
      // but don't let a SECOND failure (bad JSON) hide the original one:
      try {
        const errorData = await response.json();
        if (errorData.message) {
          message = errorData.message;
        }
      } catch {
        // response body wasn't valid JSON, or was empty  -  that's fine,
        // we just fall back to the generic HTTP status message above
      }

      throw new Error(message);
    }

    if (response.status === 204) {
      return null; // no body to parse  -  this IS success, just with nothing to return
    }

    return await response.json();

  } catch (error) {
    console.error("API request failed:", error);
    throw error;
  }
}
```

Why the inner `try/catch` with an empty `catch {}`: this is a deliberate, narrow use of a "swallowed" error (recall the warning about silent catches in Part 9.7) - but it's justified here specifically because we have a clear, safe fallback (`message` stays as the generic HTTP status string) and we're not hiding the original, more important error (the HTTP failure itself, which is still thrown right after). This is the one legitimate pattern for an intentionally-empty catch: when failure of the inner operation is expected and has an obviously correct fallback.

```
async function apiRequest(url, options = {}) {
  const { timeout = 10000, ...fetchOptions } = options;

  const controller = new AbortController();
  const timeoutId = setTimeout(() => controller.abort(), timeout);

  try {
    const response = await fetch(url, {
      ...fetchOptions,
      signal: controller.signal
    });

    clearTimeout(timeoutId);

    if (!response.ok) {
      let message = `HTTP ${response.status}`;
      try {
        const errorData = await response.json();
        if (errorData.message) message = errorData.message;
      } catch {}
      throw new Error(message);
    }

    if (response.status === 204) return null;
```

```
    return await response.json();

  } catch (error) {
    clearTimeout(timeoutId);

    if (error.name === "AbortError") {
      throw new Error(`Request timed out after ${timeout}ms`);
    }

    console.error("API request failed:", error);
    throw error;
  }
}
```

This is now the exact final function the course spec targets as the mastery goal. Notice how each version was a direct, understandable, justified response to a specific limitation of the previous one - this is exactly how you should approach building your own abstractions in real projects: start simple, and add complexity only when a concrete problem demonstrates you need it, not preemptively.

```
// GET
const users = await apiRequest("https://api.example.com/users");

// POST
const newUser = await apiRequest("https://api.example.com/users", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ name: "Ammaar" })
});

// With a custom timeout
const data = await apiRequest("https://slow-api.example.com/data", { timeout: 3000 });
```

11.11 Rate Limiting and 429 Responses

If a server responds with `429 Too Many Requests`, it's telling you to slow down. Well-behaved clients respect this, often by reading a `Retry-After` header the server may include, and waiting at least that long before trying again - combining naturally with the retry/backoff pattern from Part 10, Level 17. Blindly retrying immediately after a `429` (or worse, retrying in a tight loop) can get your application's requests blocked entirely, or your API key/IP address temporarily banned.

11.12 UI Patterns for API State

Every API interaction in a real UI should account for these states, at minimum:

- Loading indicators: show a spinner/skeleton while a request is in flight, so the user knows something is happening (built in code starting Part 22, using the `loading`/`error`/`data` pattern from Part 10, Level 10).
- Disabled buttons: disable a submit/action button while its request is pending, to prevent duplicate submissions (recall the non-idempotent `POST` danger from Part 10, Level 17 - a user double-clicking "Buy Now" before the button disables could genuinely create two orders).
- Retry UI: on failure, give the user an actual button to retry, rather than forcing a full page refresh.

- User-friendly error messages: `"HTTP 500"` means nothing to a non-technical user. Translate known error conditions into plain language ("Something went wrong on our end - please try again in a moment.") while still logging the technical detail for debugging.

11.13 API Projects

Attempt each project's requirements yourself before opening the solution. Each uses a free, public, no-authentication-required test API so you can run these immediately.

Requirements: Fetch a random user from `https://randomuser.me/api/` and display their photo, name, and email. Include a "New Random User" button.

Concepts tested: basic `fetch`, `async/await`, DOM rendering, loading states.

View solution

```
<div id="app">
  <div id="user-card"></div>
  <button id="refresh-btn">New Random User</button>
</div>

const userCard = document.querySelector("#user-card");
const refreshBtn = document.querySelector("#refresh-btn");

async function loadRandomUser() {
  userCard.textContent = "Loading...";
  try {
    const response = await fetch("https://randomuser.me/api/");
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    const data = await response.json();
    const user = data.results[0];

    userCard.innerHTML = ""; // clear
    const img = document.createElement("img");
    img.src = user.picture.large;
    const name = document.createElement("h2");
    name.textContent = `${user.name.first} ${user.name.last}`;
    const email = document.createElement("p");
    email.textContent = user.email;

    userCard.append(img, name, email);
  } catch (error) {
    userCard.textContent = "Failed to load user. Please try again.";
    console.error(error);
  }
}

refreshBtn.addEventListener("click", loadRandomUser);
loadRandomUser(); // initial load
```

Requirements: Let the user type a city name; fetch and display current weather. (Use any free weather API of your choice - e.g., Open-Meteo, which requires no API key.)

Concepts tested: form handling (Part 7), query parameters, error states for "city not found."

View solution (using Open-Meteo's free geocoding + forecast APIs)

```
const form = document.querySelector("#weather-form");
const cityInput = document.querySelector("#city-input");
const result = document.querySelector("#weather-result");

form.addEventListener("submit", async (event) => {
  event.preventDefault();
  const city = cityInput.value.trim();
  if (!city) return;

  result.textContent = "Loading...";

  try {
    // Step 1: geocode the city name to coordinates
    const geoRes = await fetch(
      `https://geocoding-api.open-meteo.com/v1/search?name=${encodeURIComponent(city)}&count=1`
    );
    const geoData = await geoRes.json();

    if (!geoData.results || geoData.results.length === 0) {
      result.textContent = `Could not find "${city}".`;
      return;
    }

    const { latitude, longitude, name } = geoData.results[0];

    // Step 2: fetch weather for those coordinates  -  a SEQUENTIAL await, because
    // this genuinely depends on the result of step 1 (recall Part 10, Level 11)
    const weatherRes = await fetch(
      `https://api.open-meteo.com/v1/forecast?latitude=${latitude}&longitude=${longitude}¤t_weather=true`
    );
    const weatherData = await weatherRes.json();

    result.innerHTML = "";
    const heading = document.createElement("h3");
    heading.textContent = name;
    const temp = document.createElement("p");
    temp.textContent = `${weatherData.current_weather.temperature}°C`;
    result.append(heading, temp);

  } catch (error) {
    result.textContent = "Something went wrong. Please try again.";
    console.error(error);
  }
});
```

Requirements: Search a GitHub username via `https://api.github.com/users/{username}` and display avatar, bio, follower count, and public repo count. Handle "user not found" (`404`) explicitly.

Concepts tested: explicit status-code branching, not just `response.ok`.

View solution

```
async function findGithubUser(username) {
  const resultBox = document.querySelector("#result");
  resultBox.textContent = "Searching...";

  try {
    const response = await fetch(`https://api.github.com/users/${username}`);

    if (response.status === 404) {
      resultBox.textContent = `No GitHub user found named "${username}".`;
      return;
    }
```

```
    if (!response.ok) {
      throw new Error(`HTTP ${response.status}`);
    }

    const user = await response.json();

    resultBox.innerHTML = `
      <img src="${user.avatar_url}" width="80" />
      <h3>${user.name ?? user.login}</h3>
      <p>${user.bio ?? "No bio."}</p>
      <p>${user.followers} followers · ${user.public_repos} repos</p>
    `;
    // Note: this specific innerHTML use is acceptable here ONLY because every
    // interpolated value comes from GitHub's own trusted API response, not raw
    // unsanitized user input  -  recall the XSS warning from Part 6.2. If this
    // were displaying arbitrary user-submitted text, textContent would be required.

  } catch (error) {
    resultBox.textContent = "Something went wrong.";
    console.error(error);
  }
}
```

Requirements: A search box that queries an API as the user types, but debounced (waits for a pause in typing) to avoid firing a request on every keystroke.

Concepts tested: debouncing, `AbortController` for race conditions (Part 10, Level 15/18).

View solution

```
function debounce(fn, delayMs) {
  let timeoutId;
  return function (...args) {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => fn.apply(this, args), delayMs);
  };
}

let currentController = null;

async function performSearch(query) {
  if (currentController) {
    currentController.abort(); // cancel any previous in-flight request  -  fixes the race condition
  }
  currentController = new AbortController();

  try {
    const response = await fetch(`https://api.example.com/search?q=${encodeURIComponent(query)}`, {
      signal: currentController.signal
    });
    const data = await response.json();
    renderResults(data);
  } catch (error) {
    if (error.name === "AbortError") return; // expected  -  a newer search superseded this one
    console.error(error);
  }
}

const debouncedSearch = debounce(performSearch, 400);

document.querySelector("#search-input").addEventListener("input", (event) => {
  const query = event.target.value.trim();
```

```
  if (query) debouncedSearch(query);
});
```

Why both debouncing AND `AbortController` together: debouncing reduces how many requests get fired in the first place (only after a 400ms typing pause). `AbortController` handles the remaining edge case - even with debouncing, a user could pause, trigger a search, then resume typing and trigger another before the first one's response arrives; cancelling the stale one guarantees correctness regardless of network timing.

Requirements: Display a list of items with "Next" / "Previous" buttons, fetching a new page of results from an API on each click.

Concepts tested: managing page state, disabling buttons appropriately at boundaries.

> View solution (using JSONPlaceholder's `/posts` endpoint, which supports `_page`/`_limit`)

```
let currentPage = 1;
const PAGE_SIZE = 10;

async function loadPage(page) {
  const list = document.querySelector("#post-list");
  list.textContent = "Loading...";

  try {
    const response = await fetch(
      `https://jsonplaceholder.typicode.com/posts?_page=${page}&_limit=${PAGE_SIZE}`
    );
    const posts = await response.json();

    list.innerHTML = "";
    posts.forEach(post => {
      const li = document.createElement("li");
      li.textContent = post.title;
      list.append(li);
    });

    document.querySelector("#page-label").textContent = `Page ${page}`;
    document.querySelector("#prev-btn").disabled = page === 1;
    // (A real API would tell you the total count/last page; here we just
    // disable "Next" if a page comes back with fewer than PAGE_SIZE items)
    document.querySelector("#next-btn").disabled = posts.length < PAGE_SIZE;

  } catch (error) {
    list.textContent = "Failed to load posts.";
  }
}

document.querySelector("#prev-btn").addEventListener("click", () => {
  currentPage--;
  loadPage(currentPage);
});

document.querySelector("#next-btn").addEventListener("click", () => {
  currentPage++;
  loadPage(currentPage);
});

loadPage(currentPage);
```

Requirements: Instead of Next/Previous buttons, automatically load the next page of results when the user scrolls near the bottom of the page.

Concepts tested: the `scroll` event, throttling scroll handling, appending (not replacing) results.

View solution

```
let currentPage = 1;
let isLoading = false;

async function loadMorePosts() {
  if (isLoading) return; // prevent overlapping requests while one is already in flight
  isLoading = true;

  try {
    const response = await fetch(
      `https://jsonplaceholder.typicode.com/posts?_page=${currentPage}&_limit=10`
    );
    const posts = await response.json();

    const list = document.querySelector("#post-list");
    posts.forEach(post => {
      const li = document.createElement("li");
      li.textContent = post.title;
      list.append(li); // APPEND, not replace  -  this is the key difference from Project 5
    });

    currentPage++;
  } catch (error) {
    console.error("Failed to load more posts:", error);
  } finally {
    isLoading = false;
  }
}

window.addEventListener("scroll", () => {
  const nearBottom =
    window.innerHeight + window.scrollY >= document.body.offsetHeight - 300;

  if (nearBottom) {
    loadMorePosts();
  }
});

loadMorePosts(); // initial load
```

Why the `isLoading` guard is essential: the `scroll` event fires many times per second while scrolling. Without this guard, a user scrolling quickly near the bottom could trigger dozens of overlapping `loadMorePosts()` calls before the first one even finishes, causing duplicate content and wasted requests.

Requirements: Build a simple notes app where you can Create, Read, Update, and Delete notes via a REST API (use `https://jsonplaceholder.typicode.com` - note it fakes writes but doesn't persist them, which is fine for practicing the request patterns).

Concepts tested: all four HTTP write methods, optimistic vs pessimistic UI updates.

View solution (core logic)

```
const BASE_URL = "https://jsonplaceholder.typicode.com/posts";

async function fetchNotes() {
  const response = await fetch(`${BASE_URL}?_limit=5`);
```

```
  return response.json();
}

async function createNote(note) {
  const response = await fetch(BASE_URL, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(note)
  });
  return response.json();
}

async function updateNote(id, updates) {
  const response = await fetch(`${BASE_URL}/${id}`, {
    method: "PATCH",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(updates)
  });
  return response.json();
}

async function deleteNote(id) {
  await fetch(`${BASE_URL}/${id}`, { method: "DELETE" });
}

// Example usage tying it together:
async function init() {
  const notes = await fetchNotes();
  renderNotes(notes);
}

function renderNotes(notes) {
  const list = document.querySelector("#notes-list");
  list.innerHTML = "";
  notes.forEach(note => {
    const li = document.createElement("li");
    li.textContent = note.title;

    const deleteBtn = document.createElement("button");
    deleteBtn.textContent = "Delete";
    deleteBtn.addEventListener("click", async () => {
      await deleteNote(note.id);
      li.remove(); // update the UI after confirming the server accepted the delete
    });

    li.append(deleteBtn);
    list.append(li);
  });
}

init();
```

Optimistic vs pessimistic updates - worth understanding conceptually: the code above is pessimistic - it waits for the server to confirm the delete before updating the UI. An optimistic update would remove the item from the UI immediately (assuming it'll succeed), and only revert it if the server request later fails - this feels faster to the user, at the cost of needing rollback logic for the failure case. Both are legitimate patterns; optimistic updates are common for high-confidence, low-risk actions (liking a post), while pessimistic updates are safer for higher-stakes actions (deleting data, processing payments).

Requirements: Let a user select an image file and "upload" it (to any test endpoint that accepts `multipart/form-data`, or simulate one), showing upload progress conceptually.

Concepts tested: `FormData` (Part 7.4), file inputs.

View solution

```
const form = document.querySelector("#upload-form");
const fileInput = document.querySelector("#file-input");
const status = document.querySelector("#upload-status");

form.addEventListener("submit", async (event) => {
  event.preventDefault();

  const file = fileInput.files[0];
  if (!file) {
    status.textContent = "Please select a file first.";
    return;
  }

  const formData = new FormData();
  formData.append("file", file);
  formData.append("uploadedBy", "Ammaar");

  status.textContent = "Uploading...";

  try {
    const response = await fetch("https://httpbin.org/post", {
      method: "POST",
      body: formData
      // NOTE: no "Content-Type" header set manually  -  the browser sets the correct
      // multipart/form-data boundary automatically when the body is a FormData object.
      // Setting it manually yourself is a classic mistake that breaks the upload.
    });

    if (!response.ok) throw new Error(`HTTP ${response.status}`);

    status.textContent = "Upload successful!";
  } catch (error) {
    status.textContent = "Upload failed.";
    console.error(error);
  }
});
```

⚠ Common mistake worth calling out: manually setting `headers: { "Content-Type": "multipart/form-data" }` when sending a `FormData` body. This breaks the upload, because `multipart/form-data` requires a unique, randomly-generated "boundary" string that only the browser knows how to correctly generate and attach - manually setting the header prevents the browser from adding it.

Requirements: Build a dashboard that loads data from three different endpoints in parallel (e.g., user info, posts, and todos from JSONPlaceholder) and displays a summary of each.

Concepts tested: `Promise.all` (Part 10, Level 7.5/11) for genuinely independent parallel requests.

View solution

```
async function loadDashboard(userId) {
  const dashboard = document.querySelector("#dashboard");
  dashboard.textContent = "Loading dashboard...";

  try {
    const [userRes, postsRes, todosRes] = await Promise.all([
```

```
      fetch(`https://jsonplaceholder.typicode.com/users/${userId}`),
      fetch(`https://jsonplaceholder.typicode.com/posts?userId=${userId}`),
      fetch(`https://jsonplaceholder.typicode.com/todos?userId=${userId}`)
    ]);

    if (!userRes.ok || !postsRes.ok || !todosRes.ok) {
      throw new Error("One or more dashboard requests failed");
    }

    const [user, posts, todos] = await Promise.all([
      userRes.json(),
      postsRes.json(),
      todosRes.json()
    ]);

    const completedTodos = todos.filter(t => t.completed).length;

    dashboard.innerHTML = `
      <h2>${user.name}</h2>
      <p>${posts.length} posts</p>
      <p>${completedTodos} / ${todos.length} todos completed</p>
    `;

  } catch (error) {
    dashboard.textContent = "Failed to load dashboard.";
    console.error(error);
  }
}

loadDashboard(1);
```

Why `Promise.all` (not sequential `await`) is correct here: the user's info, posts, and todos are completely independent of each other - none needs the result of another to be fetched. Fetching them sequentially would make the user wait for the sum of all three request times; fetching them in parallel means they wait only for the slowest single one (recall Part 10, Level 11).

Requirements: Simulate a login flow: a form posts credentials to a (mock) endpoint, receives a token, stores it, and includes it as a Bearer token on subsequent requests. Handle `401` responses by prompting re-login.

Concepts tested: tying together forms, POST, localStorage, Authorization headers, and status-code-specific handling - a synthesis of nearly everything in Parts 7, 8, 10, and 11.

> View solution (using a mock/simulated token endpoint - conceptual pattern)

```
async function login(username, password) {
  const response = await fetch("https://api.example.com/login", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ username, password })
  });

  if (!response.ok) {
    throw new Error(response.status === 401 ? "Invalid credentials" : `HTTP ${response.status}`);
  }

  const { token } = await response.json();
  localStorage.setItem("authToken", token); // recall Part 8.2  -  persisting across page loads
  return token;
}
```

```
async function authenticatedRequest(url, options = {}) {
  const token = localStorage.getItem("authToken");

  const response = await fetch(url, {
    ...options,
    headers: {
      ...options.headers,
      Authorization: `Bearer ${token}`
    }
  });

  if (response.status === 401) {
    // Token expired or invalid  -  clear it and force re-login
    localStorage.removeItem("authToken");
    throw new Error("Session expired. Please log in again.");
  }

  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }

  return response.json();
}

// Usage:
document.querySelector("#login-form").addEventListener("submit", async (event) => {
  event.preventDefault();
  const username = document.querySelector("#username").value;
  const password = document.querySelector("#password").value;

  try {
    await login(username, password);
    console.log("Logged in!");
  } catch (error) {
    console.error("Login failed:", error.message);
  }
});
```

Security note, honestly stated: storing an auth token in `localStorage` (as shown here for simplicity and to demonstrate the pattern) is readable by any JavaScript running on the page - including injected malicious scripts from an XSS vulnerability (Part 6.2, and fully in Part 34). Many production systems instead use `HttpOnly` cookies (Part 8.3) for auth tokens specifically because they're inaccessible to JavaScript entirely. This tradeoff is covered in full in Part 34 - flagged honestly here rather than presented as an unqualified best practice.

Knowledge Check - Part 11

1. What's the practical difference between `401` and `403`?
2. Why must you set `Content-Type: application/json` when sending a JSON body with `fetch`?
3. Why does calling `response.json()` twice on the same response throw an error?
4. Explain, in your own words, what CORS protects against, and who enforces it.
5. Why is it never safe to put a secret API key directly in frontend JavaScript, even if you don't share your source code publicly?

6 Why does the `apiRequest` helper's inner `try { await response.json() } catch {}` NOT count as a dangerous "silent catch" the way Part 9.7 warned about?

7 In the debounced search project, why is `AbortController` still needed even with debouncing already in place?

> View solutions

1 `401 Unauthorized` means the server doesn't know who you are, or your credentials are invalid/missing - you need to authenticate. `403 Forbidden` means the server DOES know who you are, but you don't have permission to perform this specific action.

2 It tells the server how to interpret the raw bytes of the request body - without it, the server may not correctly parse your JSON-formatted body, even though it looks correct to you, because it doesn't know to treat it as JSON rather than plain text or some other format.

3 A `Response` body is backed by a readable stream that can only be consumed once - once `.json()` (or `.text()`/`.blob()`) has read and consumed that stream, there's nothing left to read a second time.

4 CORS protects browser users from malicious websites silently using their authenticated browser session to make unauthorized cross-origin requests to other sites on their behalf. It is enforced by the browser itself, checking whether the target server explicitly permits requests from the requesting page's origin.

5 Because the entire frontend JavaScript bundle - including any embedded keys - is downloaded to every visitor's browser and is fully inspectable via DevTools, view-source, or the Network tab, regardless of whether you've shared your original source code repository publicly.

6 Because the fallback behavior is well-understood and safe (falling back to a generic HTTP status message), and - critically - the original, more important error (the actual HTTP failure) is still thrown immediately after, so nothing meaningful is actually being hidden; only a secondary, non-essential parsing attempt is being allowed to fail silently.

7 Debouncing only reduces how often requests are sent, but doesn't guarantee responses arrive in the same order requests were sent - a user could still pause just long enough to trigger one debounced search, then resume typing and trigger a second before the first's (slower) response arrives, recreating the exact race condition AbortController is needed to prevent.

Cheat Sheet - Fetch/API

```
// GET
const res = await fetch(url);
const data = await res.json();

// POST
await fetch(url, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(payload)
});

// With auth
await fetch(url, { headers: { Authorization: `Bearer ${token}` } });

// With timeout/cancellation
const controller = new AbortController();
setTimeout(() => controller.abort(), 5000);
await fetch(url, { signal: controller.signal });

// Always check response.ok before parsing!
if (!res.ok) throw new Error(`HTTP ${res.status}`);
```

Cheat Sheet - HTTP Status Codes

Code	Meaning
200	OK
201	Created
204	No Content
400	Bad Request
401	Unauthorized (who are you?)
403	Forbidden (I know you, you can't do this)
404	Not Found
409	Conflict
422	Unprocessable Entity (validation failed)
429	Too Many Requests
500	Internal Server Error
503	Service Unavailable

Next: Part 12 - Modern JavaScript

PART 12 - MODERN JAVASCRIPT [Intermediate][Advanced]

Many of these topics have already been used throughout the course (destructuring, spread, template literals, classes, Promises, async/await) - this section consolidates them under the "ES Modules and beyond" umbrella and introduces the remaining pieces: modules themselves, iterators/generators, and the Set/Map/WeakMap/WeakSet family.

12.1 ES Modules

What they are

Recall from Part 1.1: marking a script `type="module"` opts into the ES Modules system. A module is simply a JavaScript file that can export values (functions, objects, primitives) for other files to import, with its own isolated scope (nothing leaks to the global object by default).

```
// mathUtils.js
export function add(a, b) {
  return a + b;
}

export function subtract(a, b) {
  return a - b;
}

export const PI = 3.14159;

// app.js
import { add, subtract, PI } from "./mathUtils.js";

console.log(add(2, 3));      // 5
console.log(PI);                 // 3.14159
```

Renaming on import:

```
import { add as sum } from "./mathUtils.js";
console.log(sum(2, 3)); // 5
```

Importing everything as a namespace object:

```
import * as MathUtils from "./mathUtils.js";
console.log(MathUtils.add(2, 3)); // 5

// UserCard.js
export default function UserCard(name) {
  return `Card for ${name}`;
}

// app.js
import UserCard from "./UserCard.js"; // no curly braces, and can be named ANYTHING on import
```

Named vs default exports - when to use which:

	Named exports	Default export
Syntax	`export { a, b }` or `export const a = ...`	`export default X`
Per file	Multiple allowed	Only ONE allowed
Import syntax	`import { a, b } from "..."` (name must match)	`import AnyName from "..."` (name is your choice)
Best for	A module exposing several related utilities	A module whose entire purpose IS one specific thing (very common for a single React component per file - Part 16)

 Tip: Modern convention (and this is genuinely debated across teams, so treat it as a strong default rather than an absolute rule) increasingly favors named exports even for single-purpose files, because named exports are easier for tools to auto-import correctly and rename consistently across a whole codebase. You'll see both conventions in real projects - this course uses whichever fits the example, and you'll pick up your team's convention on the job.

```
button.addEventListener("click", async () => {
  const { showChart } = await import("./chartModule.js"); // only loaded when actually needed!
  showChart();
});
```

Why this matters: `import(...)` as a function (rather than the static `import` statement at the top of a file) returns a Promise and loads the module on demand, at runtime - instead of bundling it into your initial page load. This is the foundation of code splitting, covered fully with React-specific tooling in Part 31 (Performance) - it lets you ship a smaller initial bundle and load additional code only when a user actually needs it (e.g., only loading a heavy charting library if the user navigates to a page that uses charts).

	Plain `<script>`	`<script type="module">`
Scope	Shared global scope across scripts	Each module has its own isolated scope
Strict mode	Only if `"use strict"` is added manually	Automatic
Loading	Blocks parsing by default (Part 1.1)	Automatically deferred
`import`/`export`	Not available	Available
Top-level `await`	Not available	Available (Part 10, Level 9)

12.2 Iterators and the Iterator Protocol (Basics)

What it is

An object is "iterable" if it implements a specific protocol that lets it work with `for...of` (Part 2.3), spread syntax (Part 4.8), and destructuring (Part 4.7). Arrays, strings, Maps, and Sets are all built-in

iterables.

The core idea (conceptual, not something you'll typically hand-write): an iterable object has a special method that returns an iterator - an object with a `.next()` method that, each time it's called, returns `{ value, done }`, where `done` becomes `true` once there's nothing left to iterate.

```
const numbers = [10, 20, 30];
const iterator = numbers[Symbol.iterator](); // getting the underlying iterator

console.log(iterator.next()); // { value: 10, done: false }
console.log(iterator.next()); // { value: 20, done: false }
console.log(iterator.next()); // { value: 30, done: false }
console.log(iterator.next()); // { value: undefined, done: true }
```

Why this matters practically: it's why `for...of`, spread (`[...iterable]`), and destructuring all work uniformly across arrays, strings, Maps, and Sets - they're all built on this same shared protocol. You'll almost never implement this protocol by hand, but recognizing it removes the mystery of why seemingly different data structures all support the same iteration syntax.

12.3 Generators (Basics)

What they are

A special kind of function (declared with `function*`) that can pause and resume its execution, `yield`ing multiple values over time instead of computing and returning just one.

```
function* countUpTo(max) {
  for (let i = 1; i <= max; i++) {
    yield i; // pauses here, "returning" i, until .next() is called again
  }
}

const counter = countUpTo(3);
console.log(counter.next()); // { value: 1, done: false }
console.log(counter.next()); // { value: 2, done: false }
console.log(counter.next()); // { value: 3, done: false }
console.log(counter.next()); // { value: undefined, done: true }

// Generators are iterable, so for...of works directly:
for (const num of countUpTo(3)) {
  console.log(num); // 1, 2, 3
}
```

Why they exist: generators let you express a sequence of values lazily - each value is only computed when actually requested, which is useful for potentially-infinite sequences, or expensive computations you don't want to run until/unless they're needed. They're also the low-level mechanism that historically powered early async/await-like libraries before `async`/`await` became a native language feature. For this course's purposes: recognize the syntax and the core idea (pausable functions that produce a sequence of values over time) - you're unlikely to write custom generators often in typical React application code, but you'll recognize the pattern if you encounter it in library internals.

12.4 Set

What it is

A collection of unique values (no duplicates allowed), with fast membership checking.

```
const uniqueNumbers = new Set([1, 2, 2, 3, 3, 3]);
console.log(uniqueNumbers); // Set(3) {1, 2, 3}  -  duplicates automatically removed

uniqueNumbers.add(4);
uniqueNumbers.has(2);    // true
uniqueNumbers.delete(1);
uniqueNumbers.size;       // 3 (note: .size, not .length!)

// Converting back to an array:
const uniqueArray = [...uniqueNumbers]; // [2, 3, 4]
```

Practical use - deduplicating an array (an extremely common real-world pattern):

```
function removeDuplicates(array) {
  return [...new Set(array)];
}

console.log(removeDuplicates([1, 2, 2, 3, 1, 4])); // [1, 2, 3, 4]
```

12.5 Map

What it is

Similar to a plain object, but with important advantages: keys can be of any type (not just strings/symbols), it maintains insertion order reliably, and it has a real `.size` property.

```
const userRoles = new Map();

userRoles.set("ammaar", "admin");
userRoles.set("priya", "editor");

userRoles.get("ammaar");    // "admin"
userRoles.has("priya");      // true
userRoles.delete("priya");
userRoles.size;               // 1

// Iterating:
for (const [username, role] of userRoles) {
  console.log(username, role);
}
```

`Map` vs plain object - when to use which:

	Plain Object	Map
Key types	Strings/Symbols only (numbers get coerced to strings)	ANY value, including objects, functions
Guaranteed insertion order	Mostly, with edge cases	[x] Always guaranteed
Size	`Object.keys(obj).length`	`.size`
JSON-serializable directly	[x] Yes	[x] No - needs manual conversion

 Tip: for most everyday application data (especially anything that will be sent as JSON, recall Part 0.12), plain objects remain the default choice. Reach for `Map` specifically when you need non-string

keys, guaranteed insertion order under heavy mutation, or frequent additions/removals where a `Map`'s performance characteristics are more predictable.

12.6 WeakSet and WeakMap (Conceptual Overview)

What they are

Variants of `Set`/`Map` that hold their entries with "weak" references - meaning the garbage collector (Part 5.3) is still free to collect an object even while it's a key in a `WeakMap` or a member of a `WeakSet`, as long as nothing else references it.

Why this matters (conceptually): a regular `Map` or `Set` holding a reference to an object keeps that object alive in memory forever (or until explicitly removed) - even if every other part of your program has stopped using it, because the `Map`/`Set` itself still counts as a reference. This can cause memory leaks in long-running applications. `WeakMap`/`WeakSet` don't have this problem - they don't prevent their entries from being garbage collected, making them useful for scenarios like attaching metadata to an object without accidentally keeping that object alive past its natural lifetime. You'll rarely reach for these directly in typical application code (they show up more in library/framework internals), but knowing they exist - and precisely why - rounds out your understanding of Part 5.3's garbage collection concepts.

Knowledge Check - Part 12

1. What's the key structural difference between a named export and a default export?
2. Why does a dynamic `import()` return a Promise, unlike a static `import` statement?
3. What guarantee does a `Set` give you that a plain array does not?
4. Name two advantages `Map` has over a plain object for storing key-value data.
5. Why doesn't a `WeakMap` prevent its keys from being garbage collected, while a regular `Map` does?

Given an array of user objects with possibly duplicate `id` values, write a function using a `Map` that returns an array of unique users (keeping only the first occurrence of each `id`).

> View solutions

Knowledge Check

1. A file can have multiple named exports (each imported using its exact exported name in curly braces) but only one default export (imported under any name the importer chooses, no curly braces).
2. Dynamic `import()` loads the module's code over the network/filesystem at runtime, which is inherently an asynchronous operation - the module might not be available instantly, so it must be awaited like any other async operation, recall Part 10.
3. A `Set` automatically guarantees every value it contains is unique - adding a duplicate value has no effect, whereas a plain array happily allows duplicate values.

4 Any type of value can be used as a `Map` key (not just strings), and `Map` guarantees reliable insertion-order iteration; a `Map` also has a direct `.size` property rather than requiring `Object.keys(obj).length`.

5 A `WeakMap` holds "weak" references to its keys - it doesn't count as a reference that prevents the garbage collector from reclaiming that object's memory once nothing else in the program references it, unlike a regular `Map`, whose entries are "strong" references that keep referenced objects alive for as long as the Map itself exists.

Coding Challenge

```
function uniqueUsersById(users) {
  const seen = new Map();
  users.forEach(user => {
    if (!seen.has(user.id)) {
      seen.set(user.id, user);
    }
  });
  return [...seen.values()];
}
```

Next: Part 13 - Development Tools

PART 13 - DEVELOPMENT TOOLS [Beginner][Intermediate]

13.1 Terminal Basics

You'll live in a terminal (command line) for the rest of your development career. The essentials:

```
pwd                  # print working directory (where am I?)
ls                    # list files in the current directory
cd my-folder         # change directory
cd ..                  # go up one level
mkdir new-folder    # create a folder
touch file.js         # create an empty file (Mac/Linux)
rm file.js              # delete a file
rm -rf folder-name  # delete a folder and everything inside it  -  ⚠ irreversible, use carefully
```

13.2 Node.js

What it is

Recall from Part 0.8: browsers run JavaScript via an engine like V8. Node.js takes that same V8 engine and lets it run outside the browser, directly on your computer, with a different set of built-in APIs (file system access, for instance, instead of `document`/DOM APIs). Node is what lets you run JavaScript build tools, dev servers, and (in full-stack contexts) backend servers.

```
node --version   # check your installed Node version
node app.js         # run a JavaScript file with Node
```

13.3 npm and package.json

What npm is

npm (Node Package Manager) is both a command-line tool and a public registry of JavaScript packages (reusable code libraries) that ships with Node.js.

`package.json`

Every Node/npm project has a `package.json` file - a manifest describing the project: its name, version, dependencies, and the scripts you can run.

```
{
  "name": "my-app",
  "version": "1.0.0",
  "scripts": {
    "dev": "vite",
    "build": "vite build"
  },
  "dependencies": {
    "react": "^18.2.0"
  },
  "devDependencies": {
    "vite": "^5.0.0"
  }
}
```

`dependencies` vs `devDependencies`:

	`dependencies`	`devDependencies`
Needed at...	Runtime - the actual shipped application needs it	Development time only - building, testing, linting
Example	`react`, `react-dom`	`vite`, `eslint`, testing libraries

`npm install`

```
npm install              # installs everything listed in package.json into node_modules/
npm install react       # adds react as a new dependency AND installs it
npm install -D vite      # -D flag = install as a devDependency
npm uninstall react
```

The folder where every installed package's actual code lives. It can grow to be enormous (thousands of files) because packages depend on other packages, which depend on others, and so on. `node_modules` should never be committed to Git (see `.gitignore` below) - it's always regenerated locally from `package.json`.

Records the exact version of every single package (including nested dependencies) that was actually installed, ensuring that every developer on a team - and your production server - installs the exact same dependency tree, not just versions that are merely "compatible" per `package.json`'s ranges. This file should be committed to Git.

Package versions follow the pattern `MAJOR.MINOR.PATCH` (e.g., `18.2.0`):

- MAJOR: breaking changes - code that worked before might not work anymore.

- MINOR: new features, but backward-compatible.
- PATCH: bug fixes, backward-compatible.

```
"react": "^18.2.0"   // ^ allows MINOR and PATCH updates (18.x.x), but not 19.0.0
"react": "~18.2.0"   // ~ allows only PATCH updates (18.2.x)
"react": "18.2.0"     // exact version only, no updates

npm run dev      # runs the "dev" script defined in package.json's "scripts" section
npm run build
```

`npm start` and `npm test` are special-cased and can be run without the word `run` (`npm start`, `npm test`), a historical convention.

13.4 Git Basics

What it is

Git is a version control system - it tracks every change made to your project's files over time, letting you save checkpoints (commits), see history, undo mistakes, and collaborate with others without overwriting each other's work.

```
git init                        # start tracking a new project with Git
git status                       # see what's changed since the last commit
git add file.js                # stage a specific file for the next commit
git add .                         # stage ALL changed files
git commit -m "Add login form"  # save a checkpoint with a descriptive message
git log                            # view commit history
```

A branch is an independent line of development - you can work on a new feature on its own branch without affecting the main codebase, then merge it back in once it's ready.

```
git branch                          # list branches
git branch feature-login          # create a new branch
git checkout feature-login       # switch to it
git checkout -b feature-login   # create AND switch in one command
git merge feature-login             # (from main) merge feature-login's changes in
```

13.5 GitHub Basics

GitHub is a cloud-hosted service for storing Git repositories, enabling collaboration, code review (via Pull Requests), and is the de facto standard place open-source and professional codebases live.

```
git remote add origin https://github.com/username/repo.git
git push origin main       # upload your local commits to GitHub
git pull origin main         # download others' commits from GitHub
```

A file listing patterns for files/folders Git should never track - most importantly `node_modules/` (regenerable, huge) and any file containing secrets (like `.env` files, Part 37).

```
node_modules/
dist/
.env
.DS_Store
```

13.6 ESLint and Prettier

ESLint: a linter - analyzes your code for potential bugs, anti-patterns, and style inconsistencies before you even run it (e.g., catching an unused variable, or a `==` where `===` was likely intended, recall Part 1.8).

Prettier: an automatic code formatter - enforces consistent formatting (indentation, quote style, line length) automatically, so teams never argue about style in code review; it just gets auto-formatted the same way for everyone.

Why both matter, together: ESLint catches potential bugs and code-quality issues; Prettier handles pure formatting/style. Most modern projects run both, often automatically on save or as a pre-commit check, so code quality and consistency are enforced without manual nagging.

13.7 Vite and Bundlers/Dev Servers (Conceptual)

What a bundler/dev server does, conceptually

Recall Part 12.1: real applications are split across many module files (`import`/`export`). Browsers can load ES modules directly, but doing so for hundreds of small files in production is slow (many separate network requests) and doesn't support things like JSX (Part 15) or TypeScript (Part 36) natively. A bundler takes your many source files and combines/transforms them into a small number of optimized files ready for the browser. A dev server additionally serves your app locally during development, with fast reloading as you save changes.

Vite is the current, widely-recommended tool for this in the React ecosystem (superseding older tools like Create React App, which is no longer the recommended starting point). It provides:

- An extremely fast local dev server, using native ES modules during development (only transforming files on demand, as they're requested).
- A production `build` command that bundles and optimizes everything for deployment (fully covered in Part 37).
- Built-in support for JSX and TypeScript out of the box.

```
npm create vite@latest my-app -- --template react
cd my-app
npm install
npm run dev
```

This is the exact command sequence we'll use to start the React portion of this course, beginning in Part 14.

Knowledge Check - Part 13

1. What's the difference between `dependencies` and `devDependencies` in `package.json`?
2. Why should `node_modules/` never be committed to Git, while `package-lock.json` should be?
3. What does the `^` prefix mean in a version like `^18.2.0`?

4 What's the practical difference between what ESLint checks for and what Prettier checks for?

5 Why can't browsers simply load hundreds of raw, unbundled source files directly in production, the way a bundler's output is served instead?

View solutions

1 `dependencies` are packages the actual running application needs (like React itself). `devDependencies` are packages only needed during development/building (like a bundler or a linter) - they're not included in what actually ships to end users.

2 `node_modules/` is enormous and entirely regenerable from `package.json` by running `npm install` - committing it would bloat the repository pointlessly. `package-lock.json` records exact installed versions of every dependency (including nested ones), ensuring everyone on a team and your production environment install the identical dependency tree - it's small and essential for reproducibility.

3 It allows automatic updates to MINOR and PATCH versions (new features and bug fixes) but disallows MAJOR version updates (which may include breaking changes).

4 ESLint analyzes code for potential bugs, anti-patterns, and logical code-quality issues. Prettier only concerns itself with formatting/style (indentation, quotes, spacing) - it doesn't understand or flag logical problems.

5 Loading hundreds of separate files means hundreds of separate network requests, which is slow - and browsers can't natively process file types like JSX or TypeScript without a transformation step first. A bundler combines and optimizes files into a small number of browser-ready files, and performs necessary transformations along the way.

Next: Final JavaScript Assessment

FINAL JAVASCRIPT ASSESSMENT [Advanced]

Before moving on to React, this assessment checks whether you've genuinely internalized the JavaScript foundation the rest of the course builds on. Attempt every question and task honestly before checking answers - if several of these feel shaky, it's worth revisiting the relevant Part before continuing.

Section A - Theory (Short Answer)

1 Explain the difference between `let`, `const`, and `var`, covering scope AND hoisting behavior for each.

2 What is a closure? Why does it matter for writing functions that "remember" data?

3 Explain the difference between `==` and `===`, and state the rule this course follows for which to use.

4 What does `this` refer to inside a regular function versus an arrow function?

5 What's the difference between `.map()`, `.filter()`, and `.reduce()`? When would you use each?

6 Describe, precisely, what happens step-by-step when an `async` function hits an `await` on a pending Promise.

7 Why does `fetch()` not reject for a `404` response, and what must you do to correctly detect that failure?

8 What's the difference between a named export and a default export in ES Modules?

9 Explain event bubbling and why event delegation relies on it.

10 What's the difference between `null` and `undefined`?

Section B - Coding Tasks

Task 1 - Variables & Functions
Write a function `createInventoryTracker()` using a closure that returns an object with `addItem(name, quantity)`, `removeItem(name)`, and `getTotal()` methods, tracking a private inventory count.

Task 2 - Arrays & Objects
Given:

```
const orders = [
  { id: 1, customer: "Ammaar", total: 1500, status: "delivered" },
  { id: 2, customer: "Priya", total: 800, status: "pending" },
  { id: 3, customer: "Ammaar", total: 2200, status: "delivered" },
  { id: 4, customer: "Rohan", total: 500, status: "cancelled" }
];
```

Write code that:
(a) Returns the total revenue from `"delivered"` orders only.
(b) Returns an array of unique customer names.
(c) Returns the orders grouped by status, as an object keyed by status.

Task 3 - DOM & Events
Build a live character counter for a `<textarea>` with a max of 100 characters, using event delegation if there are multiple textareas on the page sharing this behavior.

Task 4 - Closures
Predict the output, then explain why:

```
function makeCounters() {
  const counters = [];
  for (let i = 0; i < 3; i++) {
    counters.push(() => i);
  }
  return counters;
}
const [a, b, c] = makeCounters();
console.log(a(), b(), c());
```

Task 5 - Classes
Write a `Rectangle` class with `width` and `height`, a getter `area`, a `static` method `fromSquare(side)`

that returns a `Rectangle` with equal width/height, and a private `#validate()` method called in the constructor that throws if width or height is negative.

Task 6 - Promises
Write a function `fetchAll(urls)` that fetches all URLs in `urls` in parallel, returning an array of `{ url, success, data }` objects for EVERY url - regardless of whether individual requests succeed or fail (hint: `Promise.allSettled`).

Task 7 - async/await + try/catch
Write an `async` function `getUserOrThrow(id)` that:

- Fetches `https://api.example.com/users/${id}`.
- Throws a custom `NotFoundError` if the status is 404.
- Throws a generic `Error` for any other non-ok status.
- Returns the parsed JSON on success.

Task 8 - API error handling
Using the `apiRequest` helper built in Part 11.10, write a function `getUserSafely(id)` that calls `apiRequest` and returns `null` instead of throwing if the user isn't found, but still lets any OTHER kind of error propagate normally.

Task 9 - Modules
Given a file `validators.js` that needs to export a default function `validateForm` AND named helper functions `isValidEmail` and `isValidPhone`, write both the export statements in `validators.js` and the corresponding import statement in a file that needs all three.

Task 10 - Debugging
Find and fix every bug in this code (there are at least 3):

```
async function loadProducts() {
  const response = fetch("https://api.example.com/products");
  const data = response.json();
  return data
}

loadProducts().then(products => {
  console.log(products.map(p => { p.name }));
});
```

> View all answers

Section A

1 `var` is function-scoped (ignores block boundaries like `if`/`for`) and hoisted with an initial value of `undefined`, so it can be referenced (as `undefined`) before its declaration line. `let` and `const` are block-scoped and are hoisted into the Temporal Dead Zone - referencing them before their declaration throws a `ReferenceError`. `const` additionally forbids reassignment of the variable itself (though object/array contents it points to can still be mutated); `let` allows reassignment.

2 A closure is a function bundled together with references to variables from its surrounding (lexical) scope, which remain accessible even after the outer function has finished executing. This matters

because it lets you create functions with private, persistent state (like a counter or encapsulated data) without needing global variables or class instances.

3 `==` performs type coercion before comparing, producing frequently confusing results (`"" == false` is `true`, for example). `===` compares both value and type with no coercion. This course's rule: always use `===`/`!==`.

4 Inside a regular function, `this` is determined dynamically by how the function is called (the call site) - an unattached call gives `undefined`/global object. An arrow function has no `this` of its own; it lexically inherits `this` from whatever scope it was physically written inside.

5 `.map()` transforms every element 1-to-1 into a new array of the same length. `.filter()` keeps only elements passing a test, producing an array of 0 to the original length. `.reduce()` combines every element into a single accumulated value (a number, object, string, etc.). Use `.map()` for transformation, `.filter()` for selection, `.reduce()` for combination/aggregation.

6 Execution of that specific async function pauses at the `await` line, and control returns to whatever called the async function, allowing the rest of the program to continue running. Once the awaited Promise settles, the async function's continuation is queued as a microtask; once the call stack is empty and it's the microtask queue's turn, execution resumes exactly where it left off, with `await`'s expression now evaluating to the resolved value (or throwing, if rejected).

7 `fetch()` only rejects for network-level failures (no connection, DNS failure, CORS block) - it considers receiving any HTTP response, including error statuses, a "successful" fetch. You must manually check `response.ok` (or `response.status`) and explicitly `throw` an error yourself if it indicates failure.

8 A file can have multiple named exports, each imported by its exact name in curly braces (`import { a, b } from "..."`). A file can have only one default export, importable under any name the importing file chooses, without curly braces.

9 Event bubbling means an event fired on an element travels back up through all of its ancestor elements after reaching the target. Event delegation relies on this by attaching a single listener to a stable ancestor and using `event.target` (often with `.closest()`) to determine which specific descendant actually triggered the event - working correctly even for descendants added to the DOM after the listener was attached.

10 `undefined` means a variable has been declared but never assigned a value (or a property/array index doesn't exist). `null` is an intentional, explicit representation of "no value," assigned deliberately by code.

Section B

Task 1:

```
function createInventoryTracker() {
  const inventory = {};

  return {
    addItem(name, quantity) {
      inventory[name] = (inventory[name] || 0) + quantity;
    },
    removeItem(name) {
      delete inventory[name];
    },
    getTotal() {
```

```
      return Object.values(inventory).reduce((sum, qty) => sum + qty, 0);
    }
  };
}

const tracker = createInventoryTracker();
tracker.addItem("Pens", 50);
tracker.addItem("Notebooks", 20);
console.log(tracker.getTotal()); // 70
```

Task 2:

```
// (a)
const deliveredRevenue = orders
  .filter(o => o.status === "delivered")
  .reduce((sum, o) => sum + o.total, 0);
// 3700

// (b)
const uniqueCustomers = [...new Set(orders.map(o => o.customer))];
// ["Ammaar", "Priya", "Rohan"]

// (c)
const groupedByStatus = orders.reduce((acc, order) => {
  if (!acc[order.status]) acc[order.status] = [];
  acc[order.status].push(order);
  return acc;
}, {});
```

Task 3:

```
document.addEventListener("input", (event) => {
  if (!event.target.matches("textarea.counted")) return;

  const textarea = event.target;
  const max = 100;
  const remaining = max - textarea.value.length;

  const counterEl = textarea.nextElementSibling; // assumes a counter element follows each textarea
  if (counterEl) counterEl.textContent = `${remaining} characters left`;
});
```

Task 4:

```
0 1 2
```

Because `let` creates a fresh binding of `i` for each loop iteration (Part 3.8), each arrow function pushed into `counters` closes over its own independent `i`, capturing the value it had at that specific iteration - unlike `var`, which would have produced `3 3 3`.

Task 5:

```
class Rectangle {
  #width;
  #height;

  constructor(width, height) {
    this.#width = width;
    this.#height = height;
    this.#validate();
  }

  #validate() {
```

```
    if (this.#width < 0 || this.#height < 0) {
      throw new Error("Width and height must be non-negative");
    }
  }

  get area() {
    return this.#width * this.#height;
  }

  static fromSquare(side) {
    return new Rectangle(side, side);
  }
}

const square = Rectangle.fromSquare(5);
console.log(square.area); // 25
```

Task 6:

```
async function fetchAll(urls) {
  const results = await Promise.allSettled(urls.map(url => fetch(url).then(r => r.json())));

  return results.map((result, index) => ({
    url: urls[index],
    success: result.status === "fulfilled",
    data: result.status === "fulfilled" ? result.value : null
  }));
}
```

Task 7:

```
class NotFoundError extends Error {
  constructor(id) {
    super(`User ${id} not found`);
    this.name = "NotFoundError";
  }
}

async function getUserOrThrow(id) {
  const response = await fetch(`https://api.example.com/users/${id}`);

  if (response.status === 404) {
    throw new NotFoundError(id);
  }

  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }

  return response.json();
}
```

Task 8:

```
async function getUserSafely(id) {
  try {
    return await apiRequest(`https://api.example.com/users/${id}`);
  } catch (error) {
    if (error.message.includes("404")) {
      return null;
    }
    throw error; // anything else propagates normally
  }
}
```

Task 9:

```
// validators.js
export default function validateForm(data) { /* ... */ }
export function isValidEmail(email) { /* ... */ }
export function isValidPhone(phone) { /* ... */ }

// app.js
import validateForm, { isValidEmail, isValidPhone } from "./validators.js";
```

Task 10 - bugs and fixes:

```
async function loadProducts() {
  const response = await fetch("https://api.example.com/products"); // BUG 1: missing await  -  response was a Promi
  if (!response.ok) throw new Error(`HTTP ${response.status}`);        // BUG 3 (added): no error handling for a fai
  const data = await response.json();                                          // BUG 2: missing await  -  data wa
  return data;
}

loadProducts()
  .then(products => {
    console.log(products.map(p => p.name)); // BUG 4: arrow function body used { } without a return, so .map produce
  })
  .catch(error => console.error("Failed to load products:", error)); // BUG 3 continued: no .catch() on the promise
```

Next: Part 14 - Introduction to React

PART 14 - INTRODUCTION TO REACT [Beginner]

14.1 Why React Exists

Recall the pain point

Go back to the persistent to-do app you built in Part 8.5. Every single time the data changed, you had to:

1. Manually figure out which DOM elements needed to change.
2. Manually write `createElement`, `textContent`, `classList`, `append`, `remove` calls to update them.
3. Manually keep your JavaScript data array and the actual DOM in sync, by hand, every single time.

For a to-do list, this is manageable. Now imagine a real application: a dashboard with dozens of interdependent pieces of data, deeply nested UI, live updates from a server, and dozens of components that all need to reflect the current data correctly, all the time, without ever getting out of sync. Hand-writing and maintaining that DOM-synchronization logic across a large application becomes a genuine engineering nightmare - it's not that it's impossible with vanilla JavaScript, it's that it doesn't scale to real application complexity without becoming extremely error-prone and hard to reason about.

What React actually is

React is a JavaScript library (not a full framework - an important, if slightly technical, distinction) for building user interfaces, built around one core idea: you describe what the UI should look like for a given set of data (state), and React figures out how to make the actual DOM match that description - you never manually touch the DOM yourself.

This is the single most important conceptual shift moving from Parts 0-13 into React.

Imperative (what you've done so far - vanilla DOM manipulation): you give the computer a sequence of steps to perform. "Create an `<li>`. Set its text. Append it to the list. If the item's completed, add a class."

```
// Imperative  -  HOW to update the DOM, step by step
const li = document.createElement("li");
li.textContent = todo.text;
if (todo.completed) li.classList.add("completed");
list.appendChild(li);
```

Declarative (React's model): you describe what the UI should look like for the current data, and let the underlying system figure out how to make the actual DOM match that description.

```
// Declarative  -  WHAT the UI should look like, given `todo`
<li className={todo.completed ? "completed" : ""}>{todo.text}</li>
```

Mental model

Imperative code is like giving someone turn-by-turn driving directions ("turn left, then right, then continue for 2km"). Declarative code is like just giving them the destination address and letting their GPS figure out the turns. You stop thinking about how to mutate the DOM step by step, and start thinking only about what the UI should be, given the current data - React handles translating that description into the actual, efficient DOM changes needed.

14.2 Component-Based Architecture

What it is

Instead of one big page description, React applications are built from small, independent, reusable components - each responsible for a piece of the UI, with its own logic and (often) its own visual output - that you compose together like building blocks.

```
App
 ├── Header
 ├── ProductList
 │    ├── ProductCard
 │    ├── ProductCard
 │    └── ProductCard
 └── Footer
```

Why this matters: each component can be built, tested, and reasoned about independently. A `ProductCard` component doesn't need to know anything about the `Header` - it just needs to know how to render one product, given that product's data. This mirrors good engineering practice generally (recall separation of concerns, Part 0.7) and directly enables reuse - the same `ProductCard` component renders every single product in the list, just fed different data each time.

14.3 State-Driven UI

The core React mental model, stated precisely:

> *UI = a function of State.*

Given the same state (data) at any point in time, a component should render the same output. When the state changes, React re-runs the component's rendering logic and updates the DOM to match the new output - you never manually say "now go update this specific DOM node"; you just update the data, and describe what the UI looks like for any given data, and React handles the rest.

This directly echoes the pure functions concept from Part 3.5 - a React component is, ideally, a pure function of its inputs (props and state, both covered fully starting Part 17-18): same inputs in, same UI description out, with side effects (Part 3.5, and formally Part 21) handled separately and deliberately.

14.4 Rendering and Reconciliation (Conceptual)

What "rendering" means in React

When we say a component "renders," we mean it runs its function body and produces a description of what the UI should look like - NOT that it directly touches the real DOM.

What "reconciliation" means (conceptual, high level - you don't need deep internals for this course)

React keeps an internal, lightweight representation of what the UI should look like (often called the "virtual DOM," though the precise internal mechanism has evolved over React's history and the exact term matters less than the concept). When state changes and a component re-renders, React compares ("diffs") the new description against the previous one, and calculates the minimal set of actual changes needed to update the real DOM to match - then applies only those specific changes, rather than tearing down and rebuilding everything from scratch.

Why this matters practically, even without deep internals: it's why React can be declarative (you just describe the current desired UI, seemingly "from scratch," every render) while still being efficient (it doesn't actually rebuild the entire DOM every time - only what actually changed). You'll return to this understanding with much more precision in Part 31 (Performance), once you have real components and state to reason about concretely.

14.5 React DOM

What it is

React itself is a general UI library - its core concepts (components, state, props) aren't inherently tied to web browsers. ReactDOM is the specific package that knows how to take React's output and render it into an actual browser DOM (recall Part 0.9). This separation is why React's same core concepts also power React Native (mobile apps) - a different "renderer" target, same component model.

```
import { createRoot } from "react-dom/client";

const root = createRoot(document.getElementById("root"));
root.render(<App />);
```

This is the code that "mounts" your entire React application into a single real DOM element (conventionally `<div id="root"></div>` in your `index.html`) - from that point forward, React owns and manages everything inside that element.

14.6 Creating a React Project

Recall Part 13.7 - we use Vite, the current standard tool for starting new React projects.

```
npm create vite@latest my-react-app -- --template react
cd my-react-app
npm install
npm run dev
```

This opens a local dev server (typically at `http://localhost:5173`) with your new React app running, and hot module reloading - your changes appear in the browser almost instantly as you save files, without a full page refresh or losing your app's current state where possible.

```
my-react-app/
├── node_modules/         (installed packages  -  never edit or commit, Part 13.3)
├── public/                    (static files served as-is, e.g., favicon)
├── src/
│   ├── App.jsx              (your root/main component)
│   ├── main.jsx             (the entry point  -  mounts App into the DOM)
│   ├── App.css
│   └── index.css
├── index.html               (the single HTML page  -  the "shell" for your SPA, recall Part 0.14)
├── package.json
└── vite.config.js
```

`main.jsx` - the entry point:

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";
import "./index.css";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

What `<StrictMode>` does: a development-only tool that helps catch certain classes of bugs early (like accidental side effects in rendering) by deliberately double-invoking certain functions during development to surface inconsistencies - it has zero effect on your production build. You don't need to fully understand its mechanics yet; just know it's there deliberately, as a development safety net, and you shouldn't remove it without a specific reason.

`index.html` - notice how minimal it is:

```
<!DOCTYPE html>
<html>
  <head>
    <title>My React App</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
```

```
  </body>
</html>
```

This is the entire "shell" - recall Part 0.14's definition of an SPA: one HTML file, with an essentially empty `<body>` except for a single mounting point, and everything else rendered by JavaScript after the page loads. This is architecturally identical to what you learned conceptually in Part 0, now made concrete.

Knowledge Check - Part 14

1. What's the core difference between imperative and declarative UI code? Give an example of each for the same outcome.
2. State the "UI = a function of State" idea in your own words.
3. What problem does component-based architecture solve compared to one large monolithic page description?
4. What's the difference between React itself and ReactDOM?
5. Why does a Vite-generated `index.html` contain almost no actual content?

> View solutions

1. Imperative code specifies the exact sequence of steps to manually perform to reach a desired result (e.g., "create this element, set its text, append it"). Declarative code specifies only the desired end result/description, letting the underlying system figure out how to achieve it (e.g., `<li>{text}</li>`, with React determining the actual DOM operations needed).
2. Given the same state (data) at any point in time, a component's rendered output should be the same - updating the UI means updating the underlying data (state), not manually finding and mutating specific DOM elements yourself.
3. It lets you build, reason about, test, and reuse small, independent pieces of UI logic and markup, rather than managing one large, tangled description of an entire page's structure and behavior all at once.
4. React is the core library defining the concepts of components, state, and how they should be described/composed - it isn't inherently tied to any particular rendering target. ReactDOM is the specific package that knows how to take React's output and actually render it into a real browser DOM.
5. Because a React application is a Single Page Application (recall Part 0.14) - nearly the entire UI is generated by JavaScript at runtime and injected into a single mounting element, rather than being written directly as static HTML markup ahead of time.

Next: Part 15 - JSX

PART 15 - JSX [Beginner]

15.1 What JSX Is

What it is

JSX (JavaScript XML) is a syntax extension for JavaScript that lets you write HTML-like markup directly inside your JavaScript code.

```
const element = <h1>Hello, World!</h1>;
```

Why it exists

At first glance, this looks like it violates the separation-of-concerns principle from Part 0.7 - mixing markup into JavaScript. React's team made a deliberate, considered argument for this: rendering logic and markup are inherently coupled - what a UI looks like is fundamentally tied to the data and logic that produces it. Instead of separating by technology (all HTML here, all JS there), React separates by component - each component owns both its markup and its logic together, and that becomes the real unit of separation and reuse (recall Part 14.2).

Critical fact: JSX is not valid JavaScript on its own. It must be compiled/transformed into regular JavaScript function calls before it can run in a browser. Vite (Part 14.6) handles this transformation automatically and instantly during development.

```
// What you write:
const element = <h1 className="title">Hello</h1>;

// What it compiles to (conceptually  -  modern React uses an automatic,
// slightly different internal call, but this is the classic illustrative form):
const element = React.createElement("h1", { className: "title" }, "Hello");
```

Why this matters to understand: it demonstrates that JSX is just syntactic sugar - a friendlier way to write function calls that build up a description of UI elements (recall Part 14.4's "description," not direct DOM manipulation). This is also why JSX has rules that sometimes surprise people coming from HTML - it's compiled to JavaScript function calls, and must obey JavaScript's syntax rules.

15.2 JavaScript Expressions Inside JSX - { }

What it is

Any JavaScript expression (recall Part 1.3 - something that produces a value) can be embedded inside JSX using curly braces.

```
const name = "Ammaar";
const age = 17;

const element = <p>Hello, {name}. You are {age} years old.</p>;

const element2 = <p>2 + 2 = {2 + 2}</p>;

function getGreeting(hour) {
  return hour < 12 ? "Good morning" : "Good evening";
}

const element3 = <p>{getGreeting(9)}</p>;
```

⚠ Critical rule - only expressions, never statements: recall from Part 1.3 that a statement (like `if`, `for`, `const x = 5;`) performs an action, while an expression produces a value. JSX's `{ }` can only contain

expressions.

```
// [x] This is a SyntaxError  -  `if` is a statement, not an expression
const element = <div>{if (isLoggedIn) { "Welcome" }}</div>;
```

Why this specific rule exists: recall that JSX compiles to function calls (`React.createElement(...)` or equivalent). A function call's arguments must be values - you can pass `someExpression` as an argument, but you cannot pass `if (x) { ... }` as an argument, because a statement doesn't produce a value at all. This single fact explains nearly every "why can't I just put normal JavaScript control flow directly in JSX" question beginners run into - and it's exactly why the patterns below (ternaries, `&&`) exist: they're all expressions.

15.3 Attributes in JSX

```
const element = <img src={imageUrl} alt="A description" width={200} />;
```

- String literal attributes use normal quotes: `alt="A description"`.
- Dynamic (JavaScript-derived) attribute values use `{ }`, not quotes: `width={200}`, `src={imageUrl}`.

```
// [x] Wrong  -  `class` is a reserved JavaScript keyword (for actual JS classes, Part 5)
<div class="container"></div>

// [x] Correct
<div className="container"></div>
```

Why: since JSX compiles to JavaScript, and `class` is already a reserved word in JavaScript (used for `class` declarations, Part 5.4), React had to pick a different attribute name for the equivalent of HTML's `class` attribute - `className` was chosen to directly mirror the DOM's own underlying `element.className` property (recall Part 6.4).

```
<button onClick={handleClick} tabIndex={0}></button>
<div style={{ backgroundColor: "blue", fontSize: "16px" }}></div>
```

The double curly braces for `style`, explained: the outer `{ }` is JSX's "this is a JavaScript expression" syntax (Part 15.2). The inner `{ }` is an actual JavaScript object literal - `style` in JSX takes a real JS object of camelCased CSS properties (recall Part 6.4's `element.style.backgroundColor` pattern), not a CSS string.

15.4 Fragments

The problem: JSX requires a single root element

```
// [x] Error: JSX expressions must have one parent element
function Component() {
  return (
    <h1>Title</h1>
    <p>Description</p>
  );
}
```

Why this restriction exists: recall that JSX compiles to a function call - `React.createElement(type, props, children)` - a single function call can only return one value, so a component must return one single root element (or, as you'll see next, a Fragment) - never multiple sibling elements at the top

level.

Solution 1 - wrap in a `<div>`:

```
function Component() {
  return (
    <div>
      <h1>Title</h1>
      <p>Description</p>
    </div>
  );
}
```

Problem with always doing this: it adds an extra, often meaningless `<div>` to your actual HTML output, which can interfere with CSS layout (especially with CSS Grid/Flexbox, where extra wrapper elements can break intended layouts) and adds unnecessary DOM nesting.

Solution 2 - Fragments (the preferred approach when no real wrapper is needed):

```
import { Fragment } from "react";

function Component() {
  return (
    <Fragment>
      <h1>Title</h1>
      <p>Description</p>
    </Fragment>
  );
}

// Shorthand syntax (much more common in practice):
function Component() {
  return (
    <>
      <h1>Title</h1>
      <p>Description</p>
    </>
  );
}
```

What a Fragment does: it satisfies JSX's "single root element" requirement, but renders nothing extra into the actual DOM - the `<h1>` and `<p>` end up as direct siblings in the real HTML output, with no wrapping element at all.

15.5 Comments in JSX

```
function Component() {
  return (
    <div>
      {/* This is a JSX comment  -  note the curly braces! */}
      <p>Visible content</p>
      {
        // A regular JS comment ALSO works, but only inside an existing { } expression block
      }
    </div>
  );
}
```

⚠ Plain HTML-style `<!-- comment -->` does not work in JSX - it would be interpreted as an attempt to render an actual (invalid) element.

15.6 Children

Anything nested between a component/element's opening and closing tags becomes its children:

```
<div>
  <h1>I am a child of div</h1>
  <p>So am I</p>
</div>
```

We formalize `children` as a special prop in Part 16.4/17, once we're building actual custom components.

15.7 Conditional Rendering

```
function Notification({ hasUnread }) {
  return (
    <div>
      <h2>Inbox</h2>
      {hasUnread && <span className="badge">New!</span>}
    </div>
  );
}
```

Step-by-step: recall from Part 1.8 that `a && b` evaluates to `a` if `a` is falsy (short-circuiting), or `b` if `a` is truthy. If `hasUnread` is `true`, the expression evaluates to the `<span>` element, which JSX renders. If `hasUnread` is `false`, the expression evaluates to `false` itself - and React specifically knows to render nothing for the values `false`, `null`, `undefined`, and `true` (this is a deliberate, useful special case in how React interprets JSX children).

⚠ The classic `&&` gotcha with the number `0`:

```
function Cart({ itemCount }) {
  return (
    <div>
      {itemCount && <p>{itemCount} items in cart</p>}
    </div>
  );
}

// When itemCount is 0:
// {0 && <p>...</p>} evaluates to 0 (since 0 is falsy, && short-circuits and returns 0 itself)
// And React DOES render the literal number 0 onto the page! Unlike false/null/undefined, 0 is a valid, renderable v
```

Why this happens: unlike `false`/`null`/`undefined` (which React deliberately renders as nothing), `0` is a valid, meaningful thing to render (imagine wanting to display a literal `0` price, for example) - so React renders it. This means `itemCount && <p>...</p>` will literally display a stray `"0"` on the page when `itemCount` is `0`, which is almost never what you want.

Correct version - force a real boolean, or use a ternary (below):

```
{itemCount > 0 && <p>{itemCount} items in cart</p>}
// or
{Boolean(itemCount) && <p>{itemCount} items in cart</p>}

function Greeting({ isLoggedIn }) {
  return (
    <div>
```

```
      {isLoggedIn ? <p>Welcome back!</p> : <p>Please log in.</p>}
    </div>
  );
}
```

Use a ternary when you need to render one of two different things (recall Part 1.8 - this reads as "value if true : value if false"). Use `&&` when you want to render something or nothing at all.

We already established the core reason in 15.2: `if` is a statement, and `{ }` in JSX requires expressions. But you absolutely CAN use `if` in your component's function body, before the `return`, to decide which JSX to return:

```
function StatusMessage({ status }) {
  if (status === "loading") {
    return <p>Loading...</p>;
  }

  if (status === "error") {
    return <p>Something went wrong.</p>;
  }

  return <p>Data loaded successfully!</p>;
}
```

Why this works, when `if` inside `{ }` doesn't: here, `if` is being used in normal JavaScript function body territory, outside of any JSX expression - it's deciding which entire JSX expression the `return` statement will produce, rather than trying to be embedded inside an existing JSX expression. This "early return" pattern is extremely common and idiomatic in React - it's the direct application of the guard clause pattern from Part 2.1 to component rendering logic, and you'll build entire loading/error/success UI flows this way starting in Part 22.

15.8 Rendering Lists

The naive attempt

```
const fruits = ["Apple", "Banana", "Cherry"];

function FruitList() {
  return (
    <ul>
      {fruits.map((fruit) => <li>{fruit}</li>)}
    </ul>
  );
}
```

This actually works and renders correctly - but produces a console warning: `Warning: Each child in a list should have a unique "key" prop.`

```
function FruitList() {
  return (
    <ul>
      {fruits.map((fruit, index) => (
        <li key={index}>{fruit}</li>
      ))}
    </ul>
  );
}
```

Why `key` is necessary - connecting back to reconciliation (Part 14.4): when a list re-renders (say, an item is added, removed, or reordered), React needs a reliable way to match each JSX element in the new render to the corresponding element from the previous render, so it can figure out the minimal actual DOM changes needed (move this one, remove that one, add this new one) rather than blindly re-creating every single item from scratch. The `key` is that reliable identifier.

⚠ Why using the array `index` as the `key` is a common but risky shortcut: it works fine for static lists that never reorder, get items inserted in the middle, or get items removed from the middle. But if the list can change order or have items removed/inserted anywhere except the very end, using `index` as `key` can cause React to incorrectly match the wrong DOM elements to the wrong data, leading to bugs - most notoriously, form input values or component-local state "sticking" to the wrong list item after a reorder/removal, because React thinks "position 2's `<li>`" is the same element it was before, when the data at position 2 has actually changed.

Correct, robust version - always prefer a stable, unique ID from your actual data when one exists:

```
const fruits = [
  { id: "a1", name: "Apple" },
  { id: "b2", name: "Banana" },
  { id: "c3", name: "Cherry" }
];

function FruitList() {
  return (
    <ul>
      {fruits.map((fruit) => (
        <li key={fruit.id}>{fruit.name}</li>
      ))}
    </ul>
  );
}
```

Best practice, precisely stated: use `index` as a `key` ONLY when the list is guaranteed to be static (never reordered, filtered, or spliced) - otherwise, always use a stable unique identifier from the actual data. This single rule prevents an entire category of subtle React bugs you will otherwise absolutely encounter.

⚠ Common mistake - putting `key` on the wrong element:

```
// [x] key is on the WRONG element  -  it must be on the OUTERMOST element returned by the .map() callback
{fruits.map((fruit) => (
  <li>
    <span key={fruit.id}>{fruit.name}</span>
  </li>
))}

// [x] Correct  -  key belongs on the outermost/root element of each iteration
{fruits.map((fruit) => (
  <li key={fruit.id}>
    <span>{fruit.name}</span>
  </li>
))}
```

Knowledge Check - Part 15

1 Why is JSX not valid JavaScript on its own, and what has to happen before it can run in a browser?

2 Why must `{ }` in JSX contain only expressions, never statements?

3 Why is `className` used instead of `class` in JSX?

4 What problem do Fragments solve, and why is wrapping everything in an extra `<div>` not always a good alternative?

5 Explain the `0`-rendering gotcha with `&&` in JSX, and how to avoid it.

6 Why does React need a `key` prop when rendering a list, connecting your answer to reconciliation?

7 Why is using array index as `key` risky specifically for reorderable or filterable lists, but fine for a truly static list?

1 Given `const user = { name: "Ammaar", isAdmin: true };`, write JSX that renders the name, and conditionally renders an "Admin" badge only if `isAdmin` is true, using `&&`.

2 Write a component `PriceTag({ price })` that renders `"Free"` if `price` is exactly `0`, or `` `₹${price}` `` otherwise - being careful to correctly handle the case where `price` is `0` (recall the `&&` gotcha).

3 Given an array of `{ id, title, completed }` task objects, write the JSX to render them as a `<ul>` of `<li>`s, with a strikethrough style applied (via conditional `className`) to completed tasks, using the correct `key`.

> View solutions

Knowledge Check

1 JSX is a syntax extension, not valid JavaScript syntax by itself - browsers cannot parse `<h1>Hello</h1>` embedded directly in JS. It must be compiled/transformed (by a tool like Vite, via Babel/esbuild under the hood) into plain JavaScript function calls (conceptually, `React.createElement(...)` or an equivalent) before it can run.

2 Because JSX compiles down to function calls, and a function call's arguments must be values (the result of evaluating an expression) - a statement like `if` or `for` doesn't produce a value at all, so it cannot be validly used where an expression/argument is expected.

3 Because `class` is already a reserved JavaScript keyword used for actual class declarations (Part 5.4) - since JSX compiles to JavaScript, it can't reuse that same word for the HTML class attribute, so React chose `className`, mirroring the DOM's own `element.className` property.

4 JSX requires each component to return a single root element; wrapping multiple sibling elements is otherwise necessary. A plain `<div>` wrapper adds an extra, often unwanted element to the actual rendered DOM, which can interfere with CSS layout (especially Grid/Flexbox) and adds unnecessary nesting. Fragments satisfy the single-root requirement without adding any actual extra DOM element.

5 `{itemCount && <Component />}` relies on `&&` returning its left operand when falsy - but `0` is a "valid," renderable value in React (unlike `false`/`null`/`undefined`, which React deliberately renders as nothing), so when `itemCount` is `0`, the literal number `0` gets rendered onto the page instead of nothing. Avoid it by ensuring the condition is a true boolean, e.g. `itemCount > 0 && ...`.

6 key gives React a stable way to match elements in a re-rendered list back to the corresponding elements from the previous render, so it can compute the minimal actual DOM changes needed (moving, adding, or removing specific elements) during reconciliation, rather than assuming every item changed and re-creating the whole list from scratch.

7 When a list can be reordered, filtered, or have items removed/inserted anywhere except the end, an item's array index can point to different underlying data across renders even though the index itself stays the same - causing React to incorrectly match old DOM state (like input values or component-local state) to the wrong data. For a genuinely static list whose items and order never change, the index reliably corresponds to the same data every time, so it's safe.

Coding Challenges

1

```
<div>
  <p>{user.name}</p>
  {user.isAdmin && <span className="badge">Admin</span>}
</div>
```

1

```
function PriceTag({ price }) {
  return <span>{price === 0 ? "Free" : `₹${price}`}</span>;
}
```

1

```
<ul>
  {tasks.map((task) => (
    <li key={task.id} className={task.completed ? "completed" : ""}>
      {task.title}
    </li>
  ))}
</ul>
```

Next: Part 16 - Components

PART 16 - COMPONENTS [Beginner]

16.1 What a Function Component Is

What it is

A React component, in modern React, is simply a JavaScript function that returns JSX describing what should appear on screen.

```
function Welcome() {
  return <h1>Hello, welcome to my app!</h1>;
}
```

That's it - that's a complete, valid, fully functional React component. Recall Part 14.3: a component describes UI as a function of its inputs.

16.2 Component Naming - Why Capitalization Matters

⚠ Critical rule: component names MUST start with a capital letter.

```
function welcome() { return <h1>Hi</h1>; } // [x] lowercase  -  React treats this as an HTML tag name, not a compone

<welcome /> // React looks for an HTML element called "welcome"  -  doesn't exist, renders nothing/errors

function Welcome() { return <h1>Hi</h1>; } // [x] correct
<Welcome />
```

Why this specific rule exists, precisely: JSX needs a way to distinguish "this is a built-in HTML element" (`<div>`, `<h1>`, `<button>`) from "this is a custom component I wrote" (`<Welcome>`, `<ProductCard>`). React's compiler uses a simple, consistent rule to tell them apart: lowercase tag names are treated as built-in HTML elements; capitalized tag names are treated as references to a component variable/function in scope. This isn't a style preference - it's how the compiler actually decides what to render.

16.3 Importing and Exporting Components

Recall Part 12.1's module system - this is exactly how components are organized across files in real projects: one component per file is the overwhelmingly standard convention.

```
// Welcome.jsx
export default function Welcome() {
  return <h1>Hello!</h1>;
}

// App.jsx
import Welcome from "./Welcome.jsx";

export default function App() {
  return (
    <div>
      <Welcome />
    </div>
  );
}
```

16.4 Composition - Building UIs from Smaller Pieces

What it is

Composition means building complex UIs by combining simpler components together, rather than building one enormous component that does everything.

```
function Avatar({ src }) {
  return <img src={src} className="avatar" />;
}

function UserName({ name }) {
  return <span className="username">{name}</span>;
}
```

```
function UserCard() {
  return (
    <div className="user-card">
      <Avatar src="/ammaar.jpg" />
      <UserName name="Ammaar Khatri" />
    </div>
  );
}
```

Why this matters: each piece (`Avatar`, `UserName`) is independently simple, testable, and - crucially - reusable. `Avatar` could be used in `UserCard`, in a comments list, in a navigation bar - anywhere an avatar image is needed, without duplicating its markup/logic.

Every React application forms a tree of components, exactly mirroring the DOM tree concept from Part 0.9, but at the level of your own component abstractions:

```
App
 └── UserCard
      ├── Avatar
      └── UserName
```

Understanding this tree structure is essential for later topics: how data flows down through props (Part 17), why Context exists to avoid passing data through every level (Part 25), and how React decides what needs to re-render (Part 31).

16.5 When to Extract a New Component - A Practical Decision Framework

There's no rigid formula, but here are the practical signals experienced developers use:

- Reused in multiple places -> almost always extract it, even if currently small.
- A distinct, nameable "thing" in the UI (a card, a button, a modal, a table row) -> extracting it improves readability, even without reuse yet.
- A section has its own internal state or logic that's unrelated to its parent's concerns -> extracting it isolates that complexity.
- A component's function body is getting long and hard to scan -> breaking it into smaller named pieces (even if only used once) often improves readability, the same way extracting a well-named helper function improves a long imperative function (recall Part 3's principles about small, focused functions).

  ⚠ Common mistake - premature extraction: splitting every tiny piece of markup into its own component "just in case," before there's a real reuse need or complexity reason, can make a codebase harder to navigate (too many tiny files to jump between) rather than easier. Extract when there's a concrete reason (reuse, clarity, isolated logic) - not reflexively.

16.6 children - Deep Dive

What it is

Recall Part 15.6: anything nested between a component's opening and closing tags is passed to that component as a special prop called `children`.

```
function Card({ children }) {
  return <div className="card">{children}</div>;
}

function App() {
  return (
    <Card>
      <h2>Card Title</h2>
      <p>Some card content.</p>
    </Card>
  );
}
```

Step-by-step: whatever JSX is written between `<Card>` and `</Card>` (here, the `<h2>` and `<p>`) is collected and passed into `Card` as `props.children` (or, using destructuring - Part 17 - directly as `children`). `Card` then decides where within its own markup that content should appear, by placing `{children}` there.

Why this is powerful: it lets you build genuinely reusable "wrapper" or "layout" components (`Card`, `Modal`, `Layout`, `Panel`) that provide consistent structure/styling but don't need to know or care about the specific content they'll wrap - that content is supplied by whoever uses the component, entirely from the outside.

Practical example - a reusable `Modal` shell:

```
function Modal({ children, onClose }) {
  return (
    <div className="modal-overlay">
      <div className="modal-box">
        <button onClick={onClose}>×</button>
        {children}
      </div>
    </div>
  );
}

// Usage  -  completely different content, same Modal shell:
<Modal onClose={handleClose}>
  <h2>Confirm Deletion</h2>
  <p>Are you sure you want to delete this item?</p>
</Modal>

<Modal onClose={handleClose}>
  <h2>Edit Profile</h2>
  <form>{/* ... */}</form>
</Modal>
```

Knowledge Check - Part 16

1. Why must a component's name start with a capital letter? What would happen if it didn't?
2. What does "composition" mean in the context of React, and why does it matter for reuse?
3. Give two concrete signals that suggest a piece of JSX should be extracted into its own component.
4. What is `children`, and where does its value come from?
5. Why is `children` useful specifically for building "wrapper"/layout-style components?

View solutions

1. React's JSX compiler uses capitalization to distinguish custom components from built-in HTML elements - lowercase tags are treated as HTML element names, capitalized tags are treated as references to component variables/functions. A lowercase component name would be (incorrectly) interpreted as an attempt to render a non-existent HTML tag with that name, not as a call to your component function.

2. Composition means building complex UI by combining smaller, simpler components together. It matters for reuse because each small component (like an `Avatar` or `Button`) can be written once and used in many different contexts/parent components, rather than duplicating similar markup/logic repeatedly.

3. Any two of: the markup/logic is needed in more than one place; it represents a distinct, nameable UI "thing" (a card, a modal); it has its own internal state/logic unrelated to its parent; the parent component's code is getting long and hard to scan.

4. `children` is a special prop automatically populated with whatever JSX content was nested between a component's opening and closing tags when it was used. Its value comes directly from the calling code's JSX, not from the component itself.

5. Because it lets a component define reusable structure/styling (like a card border, a modal overlay, consistent padding) without needing to know anything about the specific content it will contain - that content is supplied externally, by whatever code uses the component, making the wrapper genuinely reusable across many different content types.

Next: Part 17 - Props

PART 17 - PROPS [Beginner]

17.1 What Props Are

What it is

"Props" (short for properties) are how a parent component passes data down to a child component - conceptually identical to how you pass arguments into a regular JavaScript function (Part 3.3), because that's essentially what's happening under the hood.

```
function Greeting(props) {
  return <h1>Hello, {props.name}!</h1>;
}

function App() {
  return <Greeting name="Ammaar" />;
}
```

Step-by-step: `App` renders `<Greeting name="Ammaar" />`. React calls the `Greeting` function, passing it a single object: `{ name: "Ammaar" }`. Inside `Greeting`, that object is received as the `props` parameter (you can name this parameter anything, but `props` is the near-universal convention), and `props.name`

reads the specific value passed in.

17.2 Destructuring Props (Standard Modern Practice)

```
// Instead of props.name, props.age everywhere:
function Greeting({ name, age }) {
  return <h1>Hello, {name}. You are {age}.</h1>;
}
```

Recall Part 4.7 - this is exactly the object destructuring you already learned, applied directly to the function's parameter. This is the standard, idiomatic way to write components in modern React - you'll see it in virtually every real-world codebase, far more often than explicit `props.xyz` access.

17.3 Passing Different Data Types as Props

```
function ProductCard({ name, price, inStock, tags, onAddToCart }) {
  return (
    <div>
      <h3>{name}</h3>
      <p>₹{price}</p>
      {inStock ? <p>In stock</p> : <p>Out of stock</p>}
      <p>{tags.join(", ")}</p>
      <button onClick={onAddToCart}>Add to Cart</button>
    </div>
  );
}

<ProductCard
  name="Wireless Mouse"       // string
  price={599}                    // number  -  recall Part 15.3, non-strings need {}
  inStock={true}                 // boolean
  tags={["electronics", "accessories"]} // array
  onAddToCart={() => console.log("Added!")} // FUNCTION as a prop  -  extremely common, see 17.5
/>
```

17.4 Default Prop Values

```
function Avatar({ src, size = 40 }) { // default value, exactly like a default function parameter (Part 3.3)
  return <img src={src} width={size} height={size} />;
}

<Avatar src="/photo.jpg" />          {/* size defaults to 40 */}
<Avatar src="/photo.jpg" size={80} /> {/* explicitly overridden */}
```

Why this is identical to Part 3.3's default parameters: because it literally is the same JavaScript feature - `props` is just a regular object parameter, and destructuring it with defaults uses the exact same syntax and rules you already learned (including the `undefined`-only-triggers-the-default rule from Part 1.5/3.3 - passing `size={null}` would NOT trigger the default, for the same reasons explained back then).

17.5 Callbacks as Props - How Children Communicate Back to Parents

The core rule of React data flow (fully explained in 17.6): data flows DOWN via props. So how does a child tell its parent "something happened"?

Answer: the parent passes a function down as a prop, and the child calls it.

```
function LikeButton({ onLike }) {
  return <button onClick={onLike}>□ Like</button>;
}

function Post() {
  function handleLike() {
    console.log("Post was liked!");
  }

  return (
    <div>
      <p>Some post content...</p>
      <LikeButton onLike={handleLike} />
    </div>
  );
}
```

Step-by-step: `Post` defines `handleLike`, a regular function. It passes that function DOWN to `LikeButton` as the `onLike` prop. `LikeButton` doesn't know or care what `onLike` actually does internally - it just knows to call it (`onClick={onLike}`) when its button is clicked. When the user clicks, `LikeButton`'s button fires its `onClick`, which calls `onLike` - which is actually `Post`'s `handleLike` function, now running in response to an event that happened inside a child component. This is how information effectively travels back "up" the component tree, despite props only flowing down: the child doesn't send data up directly - it invokes a function the parent gave it, and that function (defined in the parent) can do whatever the parent needs, including updating the parent's own state (Part 18).

Naming convention: props that are callback functions are conventionally named `onSomething` (`onClick`, `onLike`, `onSubmit`, `onDelete`) - mirroring the naming of native DOM events (Part 6.6/19) - and the corresponding handler function passed in is conventionally named `handleSomething` (`handleClick`, `handleLike`). This isn't enforced by React itself, but it's such a strong, near-universal convention that deviating from it makes code noticeably harder for other developers (and future you) to read quickly.

17.6 One-Way Data Flow

What it is

Data in React flows in one direction: from parent components DOWN to child components, via props. A child component can never directly modify a prop it receives, and can never directly reach "up" into its parent's data.

```
function Child({ count }) {
  count = count + 1; // [x] Don't do this  -  mutating a prop directly is a mistake and won't even work as intended
  return <p>{count}</p>;
}
```

Why props should be treated as read-only, precisely: props are passed by the parent, and the parent is the sole "owner" of that data. If a child could freely reassign or mutate its props, the parent would have no reliable way to know its own data had changed - the entire predictability of "UI = a function of State" (Part 14.3) would break down, because you could no longer trust that a component's output depends only on the props/state it was explicitly given. (Note: reassigning a destructured local variable like `count = count + 1` above doesn't even actually mutate the parent's data - it just reassigns a local copy - but doing this is still a strong anti-pattern signal that indicates confused thinking about data flow, and mutating an object or array prop directly, e.g. `props.user.name = "New"`, is a real, dangerous bug per Part 1.6/4.5's reference-type rules.)

Why one-way data flow is considered a feature, not a limitation: it makes applications dramatically easier to debug and reason about. If something is wrong with the data displayed deep in a component tree, you know it can only have come from props passed down from above - you never have to wonder whether some distant, unrelated child component silently reached in and changed it. This predictability is one of React's core design philosophies, and it directly motivates why state updates work the way they do (Part 18) and why "lifting state up" (Part 18) is the standard solution when sibling components need to share data.

Knowledge Check - Part 17

1. What is `props`, conceptually, in terms of plain JavaScript function calls you already understand?
2. Why is destructuring props in the function signature preferred over accessing `props.xyz` everywhere?
3. How does a child component "send information back up" to its parent, given that props only flow downward?
4. What naming convention is used for callback props, and for the handler functions passed into them?
5. Why is directly mutating a prop considered a serious anti-pattern, even beyond just "it's not allowed"?

Build a `Counter` display component that receives `count` as a prop and an `onIncrement` callback prop, rendering the count and a button that calls `onIncrement` when clicked. Then write a parent `App` component that defines a plain (non-reactive, just for this exercise) `count` variable and an increment function, and renders `Counter`, passing both down. (Don't worry about the button actually updating the display yet - that requires `useState`, Part 18. Focus purely on correctly wiring the props.)

View solution

```
function Counter({ count, onIncrement }) {
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={onIncrement}>Increment</button>
    </div>
  );
}

function App() {
  let count = 0; // NOTE: this won't actually cause a re-render when changed  -  that's exactly
                 // the limitation Part 18's useState exists to solve. This exercise is purely
                 // about correct prop-wiring, not about making the UI actually update.
```

```
  function handleIncrement() {
    count = count + 1;
    console.log("Count is now:", count); // you'll SEE this update in the console,
                                         // but the DISPLAYED count won't change yet  -  why? (Part 18 answers
  }

  return <Counter count={count} onIncrement={handleIncrement} />;
}
```

Why the displayed count won't visually update: changing a plain JavaScript variable doesn't tell React anything changed - React has no way to know it needs to re-run App's function body and re-render. This is precisely the gap `useState` (Part 18) fills: it gives React a way to know when data has changed, so it knows to re-render.

Next: Part 18 - State

PART 18 - STATE [Beginner][Intermediate][Advanced]

> *State is the single most important concept in practical React development. This section is deliberately exhaustive - nearly every bug you'll encounter as a beginning React developer traces back to a misunderstanding of something covered here.*

18.1 What State Is, and Why Props Aren't Enough

We saw at the end of Part 17 that changing a plain variable doesn't cause React to re-render - React has no way to "see" that a normal JavaScript variable changed. State is React's mechanism for data that:

1. Belongs to a specific component (unlike props, which are handed down from outside).
2. Can change over time, in response to user actions or other events.
3. When changed, triggers React to re-render that component (and its children) automatically.

18.2 useState - First Principles

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}
```

Explaining every piece, precisely:

- `useState(0)` - calling this Hook (a special kind of function, more on the naming/rules in Part 23) tells React: "this component needs a piece of state, and its initial value is `0`."
- The return value is an array of exactly two things (recall Part 4.7's array destructuring - this is exactly why `useState` returns an array rather than an object: array destructuring lets you name both values however you like, `[count, setCount]`, without being tied to fixed property names):

1. The current value of the state (`count`) - for this specific render.
2. A setter function (`setCount`) - the only correct way to change this state.

- `setCount(count + 1)` - calling the setter with a new value tells React: "update this piece of state to this new value, and re-render this component (and its children) to reflect it."

1. `setCount(count + 1)` is called (e.g., from the button's `onClick`).
2. React schedules a re-render of the `Counter` component.
3. React calls the `Counter` function again, from the top, as if it were a brand new function call.
4. This time, `useState(0)` - despite looking identical in the source code - returns the updated value (React tracks this internally, associated with this specific component instance, not with the literal `useState(0)` call itself; the `0` argument is only ever used for the very first render).
5. The function body re-runs, and the JSX it returns now reflects the new `count` value.
6. React compares this new output to the previous render's output (reconciliation, Part 14.4) and updates only the actual DOM nodes that changed.

⚠ Critical misconception to eliminate immediately: `setCount` does not immediately change the `count` variable in the currently-executing function call. It schedules a future re-render, during which a new call to the component function will receive the updated value.

```
function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
    console.log(count); // [x] still logs the OLD value! Not the new one!
  }

  return <button onClick={handleClick}>Count: {count}</button>;
}
```

Why `console.log(count)` still shows the old value: `count` is a `const` - a plain, regular JavaScript variable, holding whatever value it was assigned at the start of this specific render. Calling `setCount` doesn't reach back in time and mutate that already-existing `const count`; it tells React to schedule a re-render, during which a new invocation of `Counter()` will create a new `count` binding with the updated value. Within the current, already-running function call, `count` will never change - it's a fresh, frozen snapshot for that render.

18.3 Re-renders and Batching

What "batching" means

```
function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
    setCount(count + 1);
    setCount(count + 1);
  }

  return <button onClick={handleClick}>Count: {count}</button>;
}
```

⚠ You might expect clicking this button once to increase `count` by 3. It doesn't - it only increases by 1.

Why: React batches multiple state updates that happen within the same event handler (and, in modern React, within most other contexts too) into a single re-render, for performance - re-rendering three separate times for three state updates that all happen essentially simultaneously would be wasteful. But because all three `setCount(count + 1)` calls read the same `count` value (the snapshot from this render, per 18.2's rule - none of the three calls sees an "updated" value from a previous call in this same batch), all three are functionally `setCount(0 + 1)`, i.e., `setCount(1)` - three times. The last one "wins," but they were all setting the same value.

Mental model: think of each render as a photograph. `count` inside `handleClick` is frozen at whatever the photograph showed - calling `setCount` three times based on that one frozen photograph is like looking at the same old photo three times and saying "add one to what I see" three times - you get the same "add one" instruction three times, not a cumulative "add three."

```
function handleClick() {
  setCount(prevCount => prevCount + 1);
  setCount(prevCount => prevCount + 1);
  setCount(prevCount => prevCount + 1);
}
// Now count correctly increases by 3!
```

Why this works: instead of passing a value computed from the stale `count` snapshot, you pass a function that receives whatever the most up-to-date state value is at the moment React actually applies that specific update - React guarantees each functional update in a batch sees the result of the previous one in that same batch, chaining correctly: `0 -> 1 -> 2 -> 3`.

The precise rule for when to use the functional form: whenever your new state value depends on the previous state value, use the functional form `setCount(prev => ...)` rather than `setCount(count + 1)`. This isn't just about batching multiple calls in one handler - it's also important for correctness in async code (e.g., inside a `setTimeout` or after an `await`, where "the current `count` variable" from that render might be stale by the time the callback actually runs, even for a single update).

```
// ⚠ Subtly risky  -  if this fires after other updates have happened, `count` here could be stale:
setTimeout(() => setCount(count + 1), 1000);

// [x] Always correct, regardless of timing  -  always operates on whatever the LATEST state actually is:
setTimeout(() => setCount(prev => prev + 1), 1000);
```

 Practical rule of thumb: if your update doesn't depend on the previous value at all (e.g., `setName("Ammaar")` - setting to a completely new, independent value), the plain form is fine and

clearer. If it depends on the previous value in any way (+1, toggling a boolean, appending to an array), always use the functional form.

18.4 State with Different Data Types

```
const [name, setName] = useState("");
const [isOpen, setIsOpen] = useState(false);

setName("Ammaar");
setIsOpen(prev => !prev); // toggling  -  a very common pattern, always safe with the functional form
```

⚠ This is where the mutation-vs-immutability rules from Part 1.6 and Part 4.5 become absolutely critical, not just "good practice."

```
const [items, setItems] = useState(["Apple", "Banana"]);

// [x] WRONG  -  mutating the array directly
function addItemWrong(newItem) {
  items.push(newItem); // mutates the array in place
  setItems(items);       // passes the SAME array reference React already has!
}
```

Why this fails, precisely, connecting back to Part 14.4's reconciliation: React needs an efficient way to detect "did the state actually change?" For performance, it does this with a cheap reference comparison (`Object.is`, essentially `===`) between the old state value and the new one passed to the setter - NOT by deeply inspecting every element for changes (which would be slow for large data). When you `.push()` onto the array and then pass that same array reference back to `setItems`, React compares the old reference to the new one, finds they're identical (because it's literally the same array object in memory, recall Part 1.6), concludes "nothing changed," and may skip re-rendering entirely - even though the array's contents did technically change. This produces a confusing, hard-to-diagnose bug: the data is "correct" in memory, but the screen doesn't update.

```
// [x] CORRECT  -  create a NEW array
function addItem(newItem) {
  setItems([...items, newItem]); // spread syntax (Part 4.8) creates a brand new array reference
}

function removeItem(indexToRemove) {
  setItems(items.filter((_, index) => index !== indexToRemove)); // .filter() is non-mutating (Part 4.3)
}

function updateItem(indexToUpdate, newValue) {
  setItems(items.map((item, index) => index === indexToUpdate ? newValue : item)); // .map() is non-mutating
}
```

Why this works: `[...items, newItem]`, `.filter()`, and `.map()` all produce a brand new array, with a new reference in memory - even if the contents happen to be identical to before, React's reference comparison correctly detects "this is a different array object than last time" and proceeds to re-render.

The exact same principle applies to object state:

```
const [user, setUser] = useState({ name: "Ammaar", age: 17 });

// [x] WRONG
function updateAgeWrong() {
  user.age = 18;        // mutates the existing object
  setUser(user);           // same reference  -  React may not detect a change
}
```

```
// [x] CORRECT
function updateAge() {
  setUser({ ...user, age: 18 }); // spread creates a NEW object with the updated field
}
```

Recall this exact pattern from Part 4.6's `Object.assign`/spread discussion - it's not a coincidence. The spread merge pattern you learned for plain JavaScript objects is literally the professional standard technique for immutable React state updates. This connection is deliberate: everything from Part 4.5-4.8 was building directly toward this moment.

```
const [user, setUser] = useState({
  name: "Ammaar",
  address: { city: "Mumbai", pincode: "400001" }
});

// [x] WRONG  -  shallow spread only copies the TOP level; address is still the SAME nested object reference
function updateCityWrong() {
  const newUser = { ...user };
  newUser.address.city = "Delhi"; // this MUTATES the original nested address object!
  setUser(newUser);
}

// [x] CORRECT  -  spread at EVERY level that changes
function updateCity() {
  setUser({
    ...user,
    address: {
      ...user.address,
      city: "Delhi"
    }
  });
}
```

Why the "wrong" version is a real, subtle bug: `{ ...user }` only creates a shallow copy (recall Part 4.5 explicitly) - the top-level keys (`name`, `address`) are copied into a new object, but `address`'s value is still a reference to the exact same nested object as before. Mutating `newUser.address.city` therefore also mutates `user.address.city`, because they're the same object in memory. Even though `setUser(newUser)` passes a genuinely new top-level object reference (so React will re-render), the underlying data has been mutated in a way that can cause bugs elsewhere (anything else still holding a reference to the old `user.address` object would also see the "already changed" value, potentially before you intended it to).

 Practical implication for real projects: deeply nested state like this is a common source of bugs and awkward code precisely because of this pattern - it's a major reason experienced React developers try to keep state as flat as possible wherever practical, and why, for genuinely complex nested state, teams often reach for a state-management library that handles immutable updates more ergonomically (previewed conceptually in Part 30).

18.5 State Should Be Treated as Immutable - The Rule, Stated Directly

Never modify state directly. Always create a new value (new array, new object) and pass it to the setter function. This single rule, consistently applied, prevents the large majority of "my UI isn't updating" and "my UI updated somewhere I didn't expect" bugs you will otherwise encounter throughout your React career.

18.6 Multiple State Variables vs One State Object

```
// Option A  -  separate state variables
function Form() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [age, setAge] = useState(0);
  // ...
}

// Option B  -  one combined state object
function Form() {
  const [formData, setFormData] = useState({ name: "", email: "", age: 0 });

  function updateField(field, value) {
    setFormData(prev => ({ ...prev, [field]: value })); // recall Part 4.6's computed property syntax!
  }
}
```

When to prefer separate variables: when pieces of state are genuinely independent and don't need to update together - it's simpler to read and update each one directly.

When to prefer a combined object: when several pieces of data are logically related and tend to update together (like all the fields of one form), especially as the number of related fields grows - it avoids an ever-growing list of individual `useState` calls, and you'll build exactly this pattern in Part 20's forms section.

Knowledge Check - Part 18

1. Precisely, what does calling a `useState` setter function actually do? What does it NOT do?
2. Why does `console.log(count)` immediately after `setCount(count + 1)` still show the old value?
3. Explain React's batching behavior with an example where calling a setter three times in one handler doesn't produce three separate increments.
4. When should you use the functional update form `setCount(prev => prev + 1)` instead of `setCount(count + 1)`? Give two distinct reasons.
5. Why does mutating an array with `.push()` and then calling `setItems(items)` with that same array often fail to trigger a re-render, even though the array's contents did change?
6. Why is `{ ...user, address: { ...user.address, city: "Delhi" } }` necessary instead of just `{ ...user, address: { city: "Delhi" } }`? What would the shorter version get wrong?

1. Build a `LikeButton` component with a boolean `liked` state, toggling between "🤍 Like" and "❤ Liked" on click, using the functional update form.

2. Build a `ShoppingList` component with an array of string items in state, an input field, and an "Add" button that appends the input's value to the list - using correct immutable update patterns.

3. Given state `const [settings, setSettings] = useState({ theme: "light", notifications: { email: true, sms: false } });`, write a function `toggleSmsNotifications()` that correctly, immutably toggles just the nested `notifications.sms` value.

4. Debug this code - clicking the button is supposed to add a random number to an array, but the list never visually updates:`function NumberList() { const [numbers, setNumbers] = useState([]); function addNumber() { numbers.push(Math.floor(Math.random() * 100)); setNumbers(numbers); } return ( <div> <button onClick={addNumber}>Add Number</button> <ul>{numbers.map((n, i) => <li key={i}>{n}</li>)}</ul> </div> ); }`

> View solutions

Knowledge Check

1. It schedules a re-render of the component, during which the Hook will return the new value on the NEXT function call. It does NOT immediately change the current, already-running function call's local variable - that variable remains a fixed snapshot for the duration of that specific render.

2. Because `count` is a `const` holding a fixed value captured at the start of the current render. `setCount` schedules a future re-render where a NEW call to the component function will have a new `count` binding with the updated value - it doesn't retroactively change the value already assigned to the current call's `count`.

3. React batches multiple state updates within the same event handler into a single re-render for performance. If all three setter calls compute their new value from the SAME stale `count` snapshot (e.g., `setCount(count + 1)` three times, where `count` is, say, `0` throughout all three calls), all three effectively request the same result (`1`), so the final state ends up as `1`, not `3`.

4. (1) When the new value depends on the previous value - the functional form guarantees you're always building on the most up-to-date state, correctly chaining multiple updates within the same batch. (2) When state might be updated from inside an async callback (like `setTimeout` or after an `await`) where the closed-over state variable could be stale by the time the callback actually executes.

5. React detects state changes using a cheap reference comparison, not a deep content comparison. `.push()` mutates the array in place, so `items` is still literally the same array object (same reference) both before and after the mutation - passing that same reference to `setItems` means React's comparison finds no difference and may skip re-rendering, even though the array's actual contents changed.

6. `{ ...user, address: { city: "Delhi" } }` REPLACES the entire `address` object with a brand new one containing ONLY `city` - silently discarding any other fields that were on the original `address` object (like `pincode`). Spreading `...user.address` first (`{ ...user.address, city: "Delhi" }`) preserves all the existing nested fields, only overriding `city`.

Coding Challenges

1.

```
function LikeButton() {
  const [liked, setLiked] = useState(false);

  return (
    <button onClick={() => setLiked(prev => !prev)}>
      {liked ? "❤ Liked" : "♡ Like"}
    </button>
  );
}
```

1

```
function ShoppingList() {
  const [items, setItems] = useState([]);
  const [inputValue, setInputValue] = useState("");

  function handleAdd() {
    if (!inputValue.trim()) return;
    setItems(prev => [...prev, inputValue.trim()]);
    setInputValue("");
  }

  return (
    <div>
      <input value={inputValue} onChange={(e) => setInputValue(e.target.value)} />
      <button onClick={handleAdd}>Add</button>
      <ul>
        {items.map((item, index) => <li key={index}>{item}</li>)}
      </ul>
    </div>
  );
}
```

1

```
function toggleSmsNotifications() {
  setSettings(prev => ({
    ...prev,
    notifications: {
      ...prev.notifications,
      sms: !prev.notifications.sms
    }
  }));
}
```

1

```
function NumberList() {
  const [numbers, setNumbers] = useState([]);

  function addNumber() {
    setNumbers(prev => [...prev, Math.floor(Math.random() * 100)]); // new array, and uses functional form defensive
  }

  return (
    <div>
      <button onClick={addNumber}>Add Number</button>
      <ul>{numbers.map((n, i) => <li key={i}>{n}</li>)}</ul>
    </div>
  );
}
```

The bug was mutating `numbers` directly with `.push()` and passing the same array reference back to `setNumbers` - React's reference comparison found no change and skipped re-rendering. The fix creates a new array with spread syntax.

Interview Questions - React State

Q: What is React state, and how does it differ from props?
State is data owned and managed internally by a component, which can change over time and triggers a re-render when updated. Props are data passed into a component from its parent, and are read-only from the receiving component's perspective. State is "private and changeable"; props are "external and (to the receiver) read-only."

Q: What causes a component to re-render?
Primarily: its own state changing (via a `useState` setter), its parent re-rendering (causing it to re-render too, by default, covered further in Part 31), or a Context value it consumes changing (Part 25).

Q: Why must state updates be immutable?
Because React detects changes using a cheap reference comparison rather than deep inspection, for performance. Mutating existing state in place keeps the same reference, which React may interpret as "no change occurred," causing the UI to not update even though the underlying data did change. It also risks subtle bugs from shared mutable references elsewhere in the app.

Next: Part 19 - Events in React

PART 19 - EVENTS IN REACT [Beginner]

19.1 React Event Handlers vs Vanilla DOM Events

Recall Part 6.6's `addEventListener` pattern. React uses a similar but syntactically different approach, directly embedded in JSX:

```
// Vanilla DOM (Part 6.6):
button.addEventListener("click", handleClick);

// React:
<button onClick={handleClick}>Click me</button>
```

	Vanilla DOM	React
Attaching	`element.addEventListener("click", fn)`	`onClick={fn}` prop in JSX
Naming	lowercase event type as a string (`"click"`)	camelCase prop name (`onClick`)

	Vanilla DOM	React
Removing	`element.removeEventListener(...)`	Automatic - handled when the element unmounts
Multiple handlers on one element/event	Requires multiple `addEventListener` calls	Only one `onClick` prop per element - combine logic inside one function if needed

⚠ Common mistake - calling the function instead of passing a reference:

```
// [x] WRONG  -  this CALLS handleClick immediately during render, not on click!
<button onClick={handleClick()}>Click me</button>

// [x] CORRECT  -  this passes a REFERENCE to the function, to be called LATER, on click
<button onClick={handleClick}>Click me</button>
```

Why this happens, connecting back to Part 15.2: `{ }` in JSX evaluates a JavaScript expression. `handleClick()` (with parentheses) is an expression that calls the function immediately and uses its return value as the prop - not what you want for an event handler. `handleClick` (no parentheses) is an expression that evaluates to the function itself (recall Part 3's "functions are first-class values" - Part 3.4), which is what `onClick` needs: a function to call later, when the event actually occurs.

19.2 Passing Parameters to Event Handlers

Since you can't write `onClick={handleDelete(id)}` (that would call it immediately, per 19.1), how do you pass extra information a handler needs?

```
function TodoItem({ id, text, onDelete }) {
  return (
    <li>
      {text}
      <button onClick={() => onDelete(id)}>Delete</button>
    </li>
  );
}
```

Why this works: `() => onDelete(id)` is an inline arrow function (recall Part 3.2) - it's a new function, created fresh on this render, that itself takes no arguments but, when eventually called (by the click), calls `onDelete(id)` with the specific `id` value captured in its closure (recall Part 3.8 - this is a closure capturing `id` from `TodoItem`'s scope). React correctly calls this wrapper function when the click happens, and the wrapper function then calls the real handler with the right argument.

19.3 The Event Object in React (SyntheticEvent)

```
function Input() {
  function handleChange(event) {
    console.log(event.target.value); // exactly like vanilla DOM (Part 6.6)!
  }

  return <input onChange={handleChange} />;
}
```

React wraps the browser's native event object in its own cross-browser-consistent wrapper (historically called a "SyntheticEvent," though modern React versions have simplified this considerably to closely

mirror native events). In practice, for everything you'll do in this course, it behaves exactly like the native event object you learned in Part 6.6 - `event.target`, `event.preventDefault()`, `event.target.value` all work identically. The main practical thing to know: React automatically handles attaching/detaching these listeners efficiently (often using a form of event delegation internally, recall Part 6.7 - the concept you already learned is directly relevant to understanding what React does for you under the hood), so you don't manage `addEventListener`/`removeEventListener` yourself at all.

19.4 Common Event Props

```
<button onClick={handleClick}>Click</button>
<input onChange={handleChange} onFocus={handleFocus} onBlur={handleBlur} />
<form onSubmit={handleSubmit}>...</form>
<input onKeyDown={handleKeyDown} />
```

Same event types as Part 6.6 (`click`, `change`, `submit`, `keydown`, etc.), just camelCased and passed as JSX props instead of registered via `addEventListener`.

19.5 preventDefault in React - Form Submission

```
function LoginForm() {
  function handleSubmit(event) {
    event.preventDefault(); // recall Part 6.6  -  still exactly as necessary here!
    console.log("Submitting...");
  }

  return (
    <form onSubmit={handleSubmit}>
      <input type="email" />
      <button type="submit">Log In</button>
    </form>
  );
}
```

The exact same reasoning from Part 6.6 applies unchanged: without `preventDefault()`, the browser's default full-page-reload form submission behavior would still occur, even in a React app.

Knowledge Check - Part 19

1. Why does `onClick={handleClick()}` cause a bug, while `onClick={handleClick}` works correctly?
2. How do you pass a specific argument (like an item's `id`) to an event handler in JSX, given you can't call the function directly in the prop?
3. What does React's synthetic/wrapped event object provide that's different in practice from the raw native browser event, for the purposes of this course?
4. Why is `event.preventDefault()` still necessary inside a React form's `onSubmit` handler?

> View solutions

1. `handleClick()` (with parentheses) is a function CALL expression - it executes immediately, during render, and its return value (not the function itself) becomes the `onClick` prop's value. `handleClick` (no parentheses) evaluates to a reference to the function itself, which is what `onClick` needs - a function to be invoked later, when the actual click event occurs.
2. Wrap the call in an inline arrow function: `onClick={() => handleDelete(id)}`. This creates a new function that takes no arguments itself but calls the real handler with the specific argument, closing over `id` from the surrounding scope.
3. In practice, for this course's purposes, nothing meaningfully different - React's event object behaves the same as the native one for common properties/methods like `.target`, `.preventDefault()`. The wrapping mainly provides cross-browser consistency and integrates with React's internal event handling system, largely invisible to how you write handler code.
4. Because the browser's DEFAULT behavior for a form submission - regardless of whether React is involved - is still to perform a full page navigation/reload. `preventDefault()` stops that default browser behavior so your JavaScript (React's `onSubmit` handler) can handle the submission instead, exactly as in vanilla JS (Part 6.6).

Next: Part 20 - React Forms

PART 20 - REACT FORMS [Beginner][Intermediate]

20.1 Controlled Components

What it is

A controlled component is a form input whose value is entirely driven by React state - the input's displayed value always exactly mirrors a piece of state, and every keystroke updates that state, which then updates the input.

```
function NameInput() {
  const [name, setName] = useState("");

  return (
    <input
      value={name}                                  // the input ALWAYS shows exactly what's in state
      onChange={(event) => setName(event.target.value)} // every keystroke updates that state
    />
  );
}
```

Step-by-step, the full cycle:

1. Initial render: `name` is `""`, so the input displays empty.
2. User types "A". This fires the `onChange` event (recall Part 6.6 - fires on every keystroke, unlike `change`).

3. `setName(event.target.value)` is called with `"A"`, updating state.
4. This triggers a re-render (Part 18.2). The component function runs again, and this time `name` is `"A"`.
5. The `<input value={name} />` now renders with `value="A"` - matching what state says it should be.
6. Visually, this all happens instantaneously (within a single frame), so it looks like the input is just normally accepting typed input - but architecturally, React is fully in control of the displayed value at every moment, not the browser's default input behavior.

   Why "controlled" is the standard, recommended approach: because the input's value lives in React state, that value is trivially available anywhere else in your component (for live validation, for submitting the form, for showing a live character count, Part 7.1's pattern now done the React way) - you never need to reach into the DOM to "ask" the input what its current value is (which would be the "uncontrolled" approach, covered next), because you already have it, in state, at all times.

20.2 Uncontrolled Components (Conceptual)

What it is

An input whose value is managed by the DOM itself (the browser's native input behavior), not by React state - you read its current value only when you need it (typically via a `ref`, formally covered in Part 24), rather than tracking every keystroke in state.

```
function UncontrolledInput() {
  const inputRef = useRef(null); // preview  -  full detail in Part 24

  function handleSubmit(event) {
    event.preventDefault();
    console.log(inputRef.current.value); // read the value directly from the DOM, only when needed
  }

  return (
    <form onSubmit={handleSubmit}>
      <input ref={inputRef} defaultValue="" />
      <button type="submit">Submit</button>
    </form>
  );
}
```

When uncontrolled might be reasonable: very simple forms where you genuinely only need the value once, at submission time, and don't need live validation or character counts - or for certain input types (like file inputs, Part 7.5, which are always effectively uncontrolled, since you can't programmatically set a file input's value for security reasons). For this course, and for the overwhelming majority of real-world React forms, controlled components are the standard, recommended default. We mention uncontrolled components primarily so you recognize the pattern and understand the tradeoff, not as an equally-weighted alternative.

20.3 Textarea, Select, Checkbox, Radio

```
function FormExample() {
  const [bio, setBio] = useState("");
  const [country, setCountry] = useState("india");
  const [subscribed, setSubscribed] = useState(false);
  const [plan, setPlan] = useState("free");

  return (
    <form>
      {/* Textarea  -  controlled exactly like a text input */}
      <textarea value={bio} onChange={(e) => setBio(e.target.value)} />

      {/* Select  -  controlled via `value` on the <select> itself, not on each <option> */}
      <select value={country} onChange={(e) => setCountry(e.target.value)}>
        <option value="india">India</option>
        <option value="usa">USA</option>
      </select>

      {/* Checkbox  -  controlled via `checked`, not `value` (recall Part 7.1!) */}
      <label>
        <input
          type="checkbox"
          checked={subscribed}
          onChange={(e) => setSubscribed(e.target.checked)}
        />
        Subscribe to newsletter
      </label>

      {/* Radio buttons  -  same `name`, controlled by comparing `value` to shared state */}
      <label>
        <input
          type="radio"
          name="plan"
          value="free"
          checked={plan === "free"}
          onChange={(e) => setPlan(e.target.value)}
        />
        Free
      </label>
      <label>
        <input
          type="radio"
          name="plan"
          value="pro"
          checked={plan === "pro"}
          onChange={(e) => setPlan(e.target.value)}
        />
        Pro
      </label>
    </form>
  );
}
```

⚠ Recall from Part 7.1 - this rule carries over unchanged into React: checkboxes and radio buttons are controlled via `checked` (a boolean), never `value` for the purpose of tracking their toggled state.

20.4 Handling a Complete Form with One State Object

Recall Part 18.6's "combined state object" pattern - this is precisely where it shines:

```
function SignupForm() {
  const [formData, setFormData] = useState({
    username: "",
    email: "",
    password: ""
  });

  function handleChange(event) {
    const { name, value } = event.target; // recall Part 4.7 destructuring, and Part 7's <input name="..."> pattern
    setFormData(prev => ({ ...prev, [name]: value })); // computed property, Part 4.6; immutable update, Part 18.4
  }

  function handleSubmit(event) {
    event.preventDefault();
    console.log("Submitting:", formData);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input name="username" value={formData.username} onChange={handleChange} />
      <input name="email" type="email" value={formData.email} onChange={handleChange} />
      <input name="password" type="password" value={formData.password} onChange={handleChange} />
      <button type="submit">Sign Up</button>
    </form>
  );
}
```

Why this pattern is so powerful: a single `handleChange` function correctly handles every input in the form, no matter how many fields you add - because it reads the specific field to update from the input's own `name` attribute (`event.target.name`), rather than needing a separate, hand-written handler function per field. This is the standard professional pattern for any form with more than a couple of fields.

20.5 Validation and Error Messages

```
function SignupForm() {
  const [formData, setFormData] = useState({ username: "", email: "" });
  const [errors, setErrors] = useState({});

  function validate(data) {
    const newErrors = {};
    if (data.username.trim().length < 3) {
      newErrors.username = "Username must be at least 3 characters.";
    }
    if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(data.email)) { // recall Part 7.3's regex
      newErrors.email = "Enter a valid email.";
    }
    return newErrors;
  }

  function handleChange(event) {
    const { name, value } = event.target;
    const updated = { ...formData, [name]: value };
    setFormData(updated);
    setErrors(validate(updated)); // live re-validation on every change
  }

  function handleSubmit(event) {
    event.preventDefault();
    const validationErrors = validate(formData);
    setErrors(validationErrors);
    if (Object.keys(validationErrors).length === 0) {
      console.log("Form is valid, submitting:", formData);
    }
  }

  return (
    <form onSubmit={handleSubmit}>
      <div>
        <input name="username" value={formData.username} onChange={handleChange} />
        {errors.username && <p className="error">{errors.username}</p>}
      </div>
      <div>
        <input name="email" value={formData.email} onChange={handleChange} />
        {errors.email && <p className="error">{errors.email}</p>}
      </div>
      <button type="submit">Sign Up</button>
    </form>
  );
}
```

This is directly the React-ified version of the vanilla JS validation project from Part 7.6 - notice how the logic (the `validate` function itself) is nearly identical; what changed is how the UI reflects that logic - declaratively, via state and conditional rendering (Part 15.7), rather than imperatively querying and updating specific DOM error-message elements.

Knowledge Check - Part 20

1. What makes a form input "controlled" versus "uncontrolled"?
2. Why must checkboxes use `checked` rather than `value` to reflect their state, in both vanilla JS and React?

3 Why does using the input's `name` attribute inside a single shared `handleChange` function scale better than writing one handler per field?

4 Why is `[name]: value` (a computed property) essential in the shared `handleChange` pattern?

Build a controlled `<select>` for choosing a shipping method (`"standard"`, `"express"`, `"overnight"`), displaying the corresponding price (`₹50`, `₹150`, `₹400`) next to it, updating live as the selection changes.

View solution

```
function ShippingSelector() {
  const [method, setMethod] = useState("standard");

  const prices = { standard: 50, express: 150, overnight: 400 };

  return (
    <div>
      <select value={method} onChange={(e) => setMethod(e.target.value)}>
        <option value="standard">Standard</option>
        <option value="express">Express</option>
        <option value="overnight">Overnight</option>
      </select>
      <p>Price: ₹{prices[method]}</p>
    </div>
  );
}
```

Next: Part 21 - Effects

PART 21 - EFFECTS [Intermediate][Advanced]

`useEffect` *confuses more beginners than almost any other React concept, largely because it's usually taught as "the thing you use to fetch data" without first explaining what an Effect conceptually is and why it exists. We build that understanding carefully before writing a single line of* `useEffect` *code.*

21.1 What an Effect Is, and Why It Exists

Recall Part 3.5 and Part 14.3: a React component should ideally be a pure function of its props and state - given the same inputs, it produces the same JSX output, with no side effects during rendering itself.

But real applications need side effects somewhere - fetching data from a server, manually interacting with something outside React (like the browser's `document.title`, a third-party non-React library, a WebSocket connection), setting up a timer, logging analytics. These are all things that reach outside of "just calculating what the UI should look like."

What an Effect is, precisely: a way to synchronize your component with an external system, running after React has rendered and updated the actual DOM - deliberately separate from the pure rendering

calculation itself. This is precisely why the Hook is named `useEffect` - it lets a component perform a side effect, in a controlled, well-defined place, rather than smuggling one into the middle of otherwise-pure render logic.

21.2 Basic Syntax

```
import { useEffect } from "react";

function Component() {
  useEffect(() => {
    console.log("This runs AFTER the component renders and the DOM is updated.");
  }, []);

  return <div>Hello</div>;
}
```

The two arguments:

1. A function containing the side-effect logic (called the "effect function" or "setup function").
2. An optional dependency array.

21.3 The Dependency Array - Three Cases

```
useEffect(() => {
  console.log("Runs after EVERY render, no exceptions.");
});
```

Runs after every single render - the initial one, and every re-render thereafter. This is rarely what you actually want - it's easy to accidentally create infinite loops or unnecessary repeated work with this form (more on this below).

```
useEffect(() => {
  console.log("Runs ONLY ONCE, after the initial render.");
}, []);
```

Why an empty array means "only once": React compares the dependency array from the current render to the dependency array from the previous render, and only re-runs the effect if something in that array has changed. An empty array has nothing in it to ever change, so the effect runs after the first render and never again (for the lifetime of this specific component instance - see cleanup/unmounting, 21.5). This is the correct pattern for "run this once when the component first appears" logic - like an initial data fetch.

```
function UserProfile({ userId }) {
  useEffect(() => {
    console.log(`Fetching data for user ${userId}...`);
    // (real fetch logic goes here  -  fully covered in Part 22)
  }, [userId]); // re-runs whenever userId changes
}
```

Step-by-step: after every render, React compares each value in `[userId]` to its value from the previous render (using the same reference-comparison logic from Part 18.4). If `userId` is different from last time, the effect re-runs. If it's the same, the effect is skipped for this render.

The rule for what belongs in the dependency array: every reactive value used inside the effect function that could change between renders - props, state, and any other function/value derived from them. This isn't an arbitrary rule to memorize; it directly follows from the effect's whole purpose: synchronizing with something. If the effect reads `userId` to decide what to fetch, and `userId` changes but the effect doesn't re-run, your component is now out of sync - displaying data for the old `userId` even though the prop says something different. Modern tooling (an ESLint plugin specifically for Hooks, `eslint-plugin-react-hooks`, standard in virtually every React project) automatically warns you if you're missing a dependency your effect actually uses - treat this warning as a near-certain sign of a real bug, not noise to silence.

21.4 A Concrete, Fully Worked Example - Synchronizing document.title

```
function PageTitleUpdater({ unreadCount }) {
  useEffect(() => {
    document.title = unreadCount > 0 ? `(${unreadCount}) Inbox` : "Inbox";
  }, [unreadCount]);

  return <div>You have {unreadCount} unread messages.</div>;
}
```

Why this genuinely needs to be an Effect, rather than just being computed directly in the render body: `document.title` is outside React's control entirely - it's a property of the actual browser tab, not part of the JSX/DOM tree React manages. Directly setting `document.title = ...` in the middle of the component's render logic would be a side effect happening during rendering, which React explicitly does not want (render should be a pure calculation of JSX - recall 21.1). Wrapping it in `useEffect` ensures this side effect happens after rendering completes, in its own clearly-delineated place, and only when `unreadCount` actually changes.

21.5 Cleanup Functions

What it is

An Effect's function can optionally return another function - the cleanup function - which React calls before running the effect again (if dependencies changed) and when the component is removed from the screen ("unmounting").

```
useEffect(() => {
  const intervalId = setInterval(() => {
    console.log("Tick");
  }, 1000);

  return () => {
    clearInterval(intervalId); // cleanup
  };
}, []);
```

Why cleanup is essential - connecting directly back to Part 5.3 and Part 6.7: without this cleanup function, the `setInterval` would keep running forever, even after this component is removed from the screen - because nothing ever called `clearInterval`. This is precisely the memory-leak pattern warned about in Part 5.3 (an unreachable-in-theory resource kept alive by a forgotten reference) and Part 6.7 (an event listener that's never removed) - `useEffect`'s cleanup function is React's formal, built-in mechanism for preventing exactly this class of bug.

When cleanup runs, precisely:

1. Before the effect re-runs (if a dependency changed between renders) - cleaning up the previous effect's setup before setting up the new one.
2. When the component unmounts (is removed from the screen entirely) - final cleanup.

A second concrete example - event listeners:

```
function WindowWidthTracker() {
  const [width, setWidth] = useState(window.innerWidth);

  useEffect(() => {
    function handleResize() {
      setWidth(window.innerWidth);
    }

    window.addEventListener("resize", handleResize); // recall Part 6.6

    return () => {
      window.removeEventListener("resize", handleResize); // recall Part 6.7's warning about forgetting this!
    };
  }, []);

  return <p>Window width: {width}px</p>;
}
```

Why this specific cleanup matters so much here: if this component is rendered, then later removed from the screen (e.g., the user navigates to a different page/view), and the listener was never cleaned up, `handleResize` would still fire on every window resize, attempting to call `setWidth` on a component that no longer exists on screen - this doesn't just waste resources, it can cause actual React warnings/errors in some circumstances and is a textbook memory leak. We revisit this exact "component might unmount while something's still in flight" concern with full force for async operations specifically in Part 22.

21.6 Common Incorrect useEffect Patterns

⚠ Mistake 1 - making the effect function itself `async`:

```
// [x] WRONG
useEffect(async () => {
  const data = await fetchData();
  setData(data);
}, []);
```

Why this fails: recall from Part 10, Level 8 - an `async` function always returns a Promise. `useEffect` expects its function to return either nothing, or a cleanup function - it does not know what to do with a Promise, and this violates that contract (React will actually warn about this in development). Correct pattern - define an async function separately and call it inside the effect:

```
// [x] CORRECT
useEffect(() => {
  async function loadData() {
    const data = await fetchData();
    setData(data);
  }
  loadData(); // call the async function, but don't return its promise from the effect itself
}, []);
```

We build this exact pattern out fully, with proper loading/error states, in Part 22 - it's one of the single most common pieces of code you'll write in real React applications.

⚠ Mistake 2 - missing dependencies, causing stale/out-of-sync behavior:

```
function SearchResults({ query }) {
  const [results, setResults] = useState([]);

  useEffect(() => {
    fetchResults(query).then(setResults);
  }, []); // [x] missing `query`! Effect never re-runs when the search term changes!
}
```

Why this is a real, meaningful bug, not just a lint nitpick: the effect reads `query`, but the dependency array says "only run once." If `query` changes (the user types a new search), the effect does not re-run, and `results` stays stuck showing data for the original `query` forever - the component is now silently out of sync with its own props, exactly the failure mode described in 21.3.

⚠ Mistake 3 - creating an infinite loop:

```
function BuggyComponent() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    setCount(count + 1); // sets state...
  }); // ...with NO dependency array, so this runs after EVERY render, including the one this causes!
}
```

Why this loops forever: no dependency array means "run after every render." But this effect itself calls a state setter, which triggers another render, which runs the effect again, which calls the setter again... an infinite cycle, which will crash the browser tab or at minimum make your application completely unusable. The fix depends on intent: if this should run once, add `[]`. If it should run in response to something specific changing, add that specific dependency (and ensure the effect isn't just naively re-triggering the exact same state change every time).

21.7 When NOT to Use an Effect - Derived Values

This is one of the most valuable lessons in this entire Part, and one experienced React developers wish more beginners learned earlier: many things that "feel like" they should be an Effect are actually much simpler as a plain calculation during render.

```
// [Advanced] Unnecessary Effect  -  using an Effect to compute a value derived from existing state
function ShoppingCart({ items }) {
  const [total, setTotal] = useState(0);

  useEffect(() => {
    setTotal(items.reduce((sum, item) => sum + item.price, 0));
  }, [items]);

  return <p>Total: ₹{total}</p>;
}
```

Why this is unnecessary, and actually worse: `total` can be computed directly from `items` during render - there's no external system to synchronize with here; it's a pure calculation. Using an Effect here means the component renders ONCE with the stale/default `total`, THEN the Effect runs, calls `setTotal`, and triggers a SECOND render with the correct value - an unnecessary extra render, and

unnecessary complexity, for something that didn't need any of this machinery.

Correct version - just calculate it directly during render:

```
// [x] Correct  -  no Effect needed at all
function ShoppingCart({ items }) {
  const total = items.reduce((sum, item) => sum + item.price, 0); // plain calculation, recalculated every render

  return <p>Total: ₹{total}</p>;
}
```

The precise rule for deciding "does this need an Effect?": ask yourself - "is this side effect synchronizing my component with something OUTSIDE of React's rendering system" (a server via `fetch`, `document.title`, a timer, a subscription to an external event source, direct DOM APIs a library needs)? If yes, it likely belongs in an Effect. If you can compute the value directly from existing props/state using plain JavaScript, do that instead - during render, with no Effect at all. (For genuinely expensive calculations you don't want to redo on every single render, Part 23/31 introduces `useMemo` as the correct tool - still not `useEffect`.)

21.8 Synchronization with External Systems - The Unifying Mental Model

To tie this whole Part together: every single legitimate use of `useEffect` you'll encounter fits the same shape - "whenever these specific reactive values change, make this outside-of-React thing match."

- "Whenever `userId` changes, make the fetched data match the new user." (Part 22)
- "Whenever the component first appears, make the browser tab's title match the unread count." (21.4)
- "Whenever the component is on screen, make the window resize listener exist; whenever it's not, make sure it doesn't." (21.5)

Holding onto this "synchronization," rather than "a place to put miscellaneous logic," is what makes `useEffect` click - and is exactly why 21.7's "unnecessary Effect" example felt wrong the moment you looked at it that way: there was nothing external to synchronize with at all.

Knowledge Check - Part 21

1. Precisely, what does an empty dependency array `[]` tell React about when to re-run an effect, and why?
2. Why must you name every reactive value your effect uses inside its dependency array?
3. What is a cleanup function, and name the two situations in which React calls it.
4. Why can't the function passed to `useEffect` be `async` directly? What's the correct workaround?
5. Give the precise test for deciding whether something needs to be a `useEffect`, versus being computed directly during render.
6. Explain, step by step, why the `setCount(count + 1)` example with no dependency array causes an infinite loop.

1. Write a component `Clock` that displays the current time, updating every second, using `useEffect` with `setInterval` and proper cleanup.
2. Write a component `DocumentTitleSync({ title })` that keeps `document.title` in sync with a `title` prop, with the correct dependency array.
3. Identify and fix the bug - this effect is supposed to log a greeting whenever `name` changes, but it only ever logs once:`function Greeter({ name }) { useEffect(() => { console.log(`Hello, ${name}!`); }, []); }`
4. Rewrite this unnecessary-Effect code to compute `isEligible` directly during render instead:`function EligibilityChecker({ age }) { const [isEligible, setIsEligible] = useState(false); useEffect(() => { setIsEligible(age >= 18); }, [age]); return <p>{isEligible ? "Eligible" : "Not eligible"}</p>; }`

> View solutions

Knowledge Check

1. It tells React there are no reactive values this effect depends on, so it should compare the (always-empty, always-identical) array between renders, find no difference, and therefore only run the effect after the very first render - never again for the lifetime of that component instance (aside from re-running after a remount).
2. Because the effect's entire purpose is to keep something in sync with those values - if a value the effect reads isn't in the dependency array, the effect won't re-run when that value changes, silently leaving whatever the effect manages out of sync with the component's actual current props/state.
3. A cleanup function is an optional function returned from the effect function, used to undo/tear down whatever the effect set up. React calls it (1) right before re-running the effect due to a changed dependency, cleaning up the previous run's setup first, and (2) when the component unmounts (is removed from the screen), as final cleanup.
4. Because `async` functions always return a Promise, and `useEffect` expects its function to return either nothing or a plain cleanup function - it has no defined behavior for a returned Promise. The workaround is to define a separate (optionally async) function inside the effect and call it, without returning its promise from the effect function itself.
5. Ask whether the logic is synchronizing the component with something OUTSIDE React's own rendering/state system (a network request, a timer, a DOM API, a subscription). If the value can instead be directly computed from existing props/state using plain JavaScript, it should be calculated directly during render, not via `useEffect`.
6. The effect (with no dependency array) runs after every render. Inside, it calls `setCount`, which schedules a new render. That new render causes the effect to run again (since it has no dependency array restricting it), which calls `setCount` again, scheduling yet another render - this cycle has no natural stopping point and repeats indefinitely.

Coding Challenges

1.

```
function Clock() {
  const [time, setTime] = useState(new Date());

  useEffect(() => {
    const intervalId = setInterval(() => {
      setTime(new Date());
    }, 1000);

    return () => clearInterval(intervalId);
  }, []);

  return <p>{time.toLocaleTimeString()}</p>;
}
```

1

```
function DocumentTitleSync({ title }) {
  useEffect(() => {
    document.title = title;
  }, [title]);

  return null; // this component's only job is the side effect  -  it renders nothing itself
}
```

1

```
function Greeter({ name }) {
  useEffect(() => {
    console.log(`Hello, ${name}!`);
  }, [name]); // FIX: name must be in the dependency array so the effect re-runs when it changes
}
```

1

```
function EligibilityChecker({ age }) {
  const isEligible = age >= 18; // plain calculation during render  -  no Effect, no extra state, no extra render ne

  return <p>{isEligible ? "Eligible" : "Not eligible"}</p>;
}
```

Next: Part 22 - API Calls in React

PART 22 - API CALLS IN REACT [Advanced][Advanced]

> *This section directly combines Part 10 (Async JavaScript), Part 11 (APIs), Part 18 (State), and Part 21 (Effects) - everything you've learned converges here. This is, per the course's design, one of the most detailed React sections in the entire document, because it's also one of the most common things you'll actually do as a working React developer.*

22.1 The Complete Pattern, Explained Line by Line

```
function UserProfile({ userId }) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function loadData() {
      try {
        const response = await fetch(`https://api.example.com/users/${userId}`);

        if (!response.ok) {
          throw new Error(`HTTP ${response.status}`);
        }

        const result = await response.json();

        setData(result);
      } catch (error) {
        setError(error.message);
      } finally {
        setLoading(false);
      }
    }

    loadData();
  }, [userId]);

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;
  return <p>{data.name}</p>;
}
```

Explaining every single line, in order:

- `const [data, setData] = useState(null);` - starts as `null` because we have no data yet. `null` is chosen deliberately (rather than, say, `{}`) to clearly and unambiguously represent "no data exists yet," distinct from "data exists but happens to be an empty object."
- `const [loading, setLoading] = useState(true);` - starts as `true`, because as soon as this component mounts, a fetch is about to begin. Recall Part 18.2: initial state should reflect the actual initial situation.
- `const [error, setError] = useState(null);` - starts as `null`, meaning "no error has occurred (yet)."
- `useEffect(() => { ... }, [userId]);` - recall Part 21.3/21.8: this synchronizes the component's data with the server, specifically for the current `userId`. If `userId` changes (e.g., navigating to a different user's profile), the effect re-runs, re-fetching for the new ID - keeping the component correctly in sync.
- `async function loadData() { ... } loadData();` - recall Part 21.6, Mistake 1: the effect function itself cannot be `async`, so we define an inner async function and call it.
- The `try/catch/finally` - this is exactly Part 10, Level 10's pattern, applied with real React state setters instead of `console.log`.

- `if (!response.ok) throw new Error(...)` - recall Part 10/11's central lesson: `fetch()` doesn't reject for HTTP errors, so we must check and throw ourselves.
- `setError(error.message)` in the `catch` block - captures whatever went wrong (a network failure, an HTTP failure we manually threw, or a JSON-parsing failure) into state, so the UI can react to it.
- `setLoading(false)` in `finally` - recall Part 9.4/10, Level 10: `finally` runs regardless of success or failure, which is exactly the guarantee needed to correctly turn off the loading state in either case.
- The render logic (`if (loading) return ...`) - recall Part 15.7's "early return" pattern, now deciding between three distinct UI states based on the three pieces of state we're tracking.

22.2 The UI State Model: Idle, Loading, Success, Empty, Error

A complete, production-quality data-fetching component typically accounts for five distinct states, not just three:

```
function ProductList({ categoryId }) {
  const [products, setProducts] = useState(null); // null = haven't fetched yet ("idle"); [] = fetched, but empty
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  useEffect(() => {
    if (!categoryId) return; // "idle"  -  nothing to fetch yet, e.g., no category selected

    async function loadProducts() {
      setLoading(true);
      setError(null);
      try {
        const response = await fetch(`/api/products?category=${categoryId}`);
        if (!response.ok) throw new Error(`HTTP ${response.status}`);
        const data = await response.json();
        setProducts(data);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    }

    loadProducts();
  }, [categoryId]);

  if (!categoryId) return <p>Select a category to see products.</p>;          // IDLE
  if (loading) return <p>Loading products...</p>;                                   // LOADING
  if (error) return <p>Failed to load products: {error}</p>;                     // ERROR
  if (products.length === 0) return <p>No products found in this category.</p>; // EMPTY
  return (                                                                                  // SUCCESS
    <ul>
      {products.map(p => <li key={p.id}>{p.name}</li>)}
    </ul>
  );
}
```

State	Meaning	Common beginner mistake
Idle	Nothing has been requested yet (waiting on some precondition)	Forgetting this exists, and showing a spinner or empty list when really nothing should be happening yet

State	Meaning	Common beginner mistake
Loading	A request is currently in flight	Forgetting to reset `error`/handle stale data when a NEW load starts (see 22.2's `setError(null)` above)
Success (with data)	Request succeeded, data exists	-
Empty	Request succeeded, but the result set is genuinely empty	Confusing this with the error state, or the loading state - showing "no results" when actually still loading
Error	Request failed	Not resetting the error before a subsequent retry, so an old error message lingers even after a new attempt starts

⚠ A subtle but important detail in the code above: `setError(null)` is explicitly called at the start of `loadProducts`, before the request even begins. Why this matters: without it, if a previous fetch had failed (setting `error` to some message) and the user then triggers a new fetch attempt (e.g., by changing `categoryId` again), the OLD error message would continue showing on screen throughout the new attempt's loading phase, incorrectly suggesting the new attempt has already failed too.

22.3 Why useEffect(async () => {...}, []) Is Wrong - Full Recap

Covered fully in Part 21.6, Mistake 1 - the reasoning transfers directly: `useEffect`'s function must return nothing or a cleanup function, never a Promise, so the `async` keyword can never go directly on the effect function itself.

22.4 Request Cancellation with AbortController in React

The problem, specifically in a component context: what happens if a component unmounts (Part 21.5) while a `fetch` it started is still in flight? When that fetch eventually resolves, the `.then()`/`await` continuation will try to call `setData(...)` on a component that no longer exists on screen.

```
function UserProfile({ userId }) {
  const [data, setData] = useState(null);

  useEffect(() => {
    const controller = new AbortController(); // recall Part 10, Level 15

    async function loadData() {
      try {
        const response = await fetch(`/api/users/${userId}`, {
          signal: controller.signal
        });
        if (!response.ok) throw new Error(`HTTP ${response.status}`);
        const result = await response.json();
        setData(result); // only runs if we get here BEFORE any cleanup aborts the request
      } catch (error) {
        if (error.name === "AbortError") return; // expected  -  component unmounted or userId changed again; not a
        console.error(error);
      }
    }

    loadData();
```

```
    return () => {
      controller.abort(); // CLEANUP: cancel the in-flight request if the effect re-runs or the component unmounts
    };
  }, [userId]);

  return <div>{data?.name}</div>;
}
```

Why the cleanup function calling `controller.abort()` is exactly the right tool here, connecting Part 21.5 and Part 10, Level 15 directly: recall from 21.5 that React calls an effect's cleanup function (a) right before re-running the effect (if `userId` changes) and (b) on unmount. In both cases, whatever fetch was in flight for the previous `userId` (or for a component that's going away entirely) is no longer relevant - aborting it is exactly correct in both situations, using the exact same cleanup mechanism.

22.5 Race Conditions in React - Concretely Solved

Recall Part 10, Level 18's search-as-you-type race condition. Here's the complete, correct React solution, combining `AbortController` (22.4) with debouncing:

```
function SearchBox() {
  const [query, setQuery] = useState("");
  const [results, setResults] = useState([]);
  const [loading, setLoading] = useState(false);

  useEffect(() => {
    if (!query.trim()) {
      setResults([]);
      return;
    }

    const controller = new AbortController();

    // Debounce: wait 400ms after the user stops typing before firing the request
    const timeoutId = setTimeout(async () => {
      setLoading(true);
      try {
        const response = await fetch(`/api/search?q=${encodeURIComponent(query)}`, {
          signal: controller.signal
        });
        const data = await response.json();
        setResults(data);
      } catch (error) {
        if (error.name !== "AbortError") console.error(error);
      } finally {
        setLoading(false);
      }
    }, 400);

    return () => {
      clearTimeout(timeoutId); // cancel the PENDING debounce timer if query changes again quickly
      controller.abort();       // cancel any IN-FLIGHT request from a previous, now-stale query
    };
  }, [query]);

  return (
    <div>
      <input value={query} onChange={(e) => setQuery(e.target.value)} placeholder="Search..." />
      {loading && <p>Searching...</p>}
      <ul>{results.map(r => <li key={r.id}>{r.name}</li>)}</ul>
    </div>
```

```
  );
}
```

Why this single effect correctly solves BOTH the debouncing problem AND the race condition problem, using nothing but `useEffect`'s own dependency/cleanup mechanics:

1. Every keystroke updates `query` state, which re-runs the effect.
2. Before the effect body runs again, its cleanup from the previous run fires first (recall 21.5's precise cleanup-timing rule) - cancelling both the previous debounce timer (`clearTimeout`) and any request that was already in flight (`controller.abort()`).
3. This means only the most recent keystroke's timer ever actually survives long enough to fire (since every earlier one gets cleared the instant a newer keystroke arrives), and only the most recent request is ever allowed to actually resolve and call `setResults`.

This is a genuinely elegant, idiomatic React solution - notice that we didn't need a manually-tracked "latest request ID" or any extra bookkeeping variable at all (recall Part 10, Level 18 mentioned this as one possible fix); `useEffect`'s built-in cleanup-before-next-run semantics handle the entire "cancel anything stale" responsibility automatically, precisely because that's exactly what cleanup functions are for.

22.6 Stale Data and Refetching

"Stale data" refers to data displayed on screen that no longer accurately reflects the current, correct state of the world (e.g., another user updated a record on the server, or the local data was fetched a while ago and might be outdated).

Basic manual refetch pattern:

```
function Dashboard() {
  const [data, setData] = useState(null);
  const [refetchTrigger, setRefetchTrigger] = useState(0);

  useEffect(() => {
    async function load() {
      const response = await fetch("/api/dashboard");
      const result = await response.json();
      setData(result);
    }
    load();
  }, [refetchTrigger]); // including this in the dependency array is what lets us trigger a manual re-fetch

  function handleRefresh() {
    setRefetchTrigger(prev => prev + 1); // changing this value re-runs the effect, causing a refetch
  }

  return (
    <div>
      <button onClick={handleRefresh}>Refresh</button>
      {/* ... render data ... */}
    </div>
  );
}
```

Why incrementing an otherwise-meaningless counter works as a "refetch trigger": recall Part 21.3 - the effect re-runs whenever any dependency-array value changes. `refetchTrigger` doesn't need to carry

any real meaning itself; its only job is to be a value that changes on demand, which is sufficient to force the effect to re-run. This is a common, pragmatic pattern - though for more complex applications with many components needing coordinated refetching, this is exactly the kind of repetitive bookkeeping that dedicated data-fetching libraries (previewed in Part 30) are built to handle more elegantly.

22.7 Pagination in React

```
function PaginatedList() {
  const [page, setPage] = useState(1);
  const [items, setItems] = useState([]);
  const [loading, setLoading] = useState(true);
  const [hasMore, setHasMore] = useState(true);

  useEffect(() => {
    const controller = new AbortController();

    async function loadPage() {
      setLoading(true);
      try {
        const response = await fetch(`/api/items?page=${page}&limit=10`, {
          signal: controller.signal
        });
        const data = await response.json();
        setItems(data.items);
        setHasMore(data.items.length === 10);
      } catch (error) {
        if (error.name !== "AbortError") console.error(error);
      } finally {
        setLoading(false);
      }
    }

    loadPage();
    return () => controller.abort();
  }, [page]);

  return (
    <div>
      {loading ? <p>Loading...</p> : (
        <ul>{items.map(item => <li key={item.id}>{item.name}</li>)}</ul>
      )}
      <button disabled={page === 1} onClick={() => setPage(p => p - 1)}>Previous</button>
      <button disabled={!hasMore} onClick={() => setPage(p => p + 1)}>Next</button>
    </div>
  );
}
```

This is the direct React translation of Project 5 from Part 11.13 - notice the underlying logic (page state, boundary-disabled buttons) is identical; only the mechanism for triggering a re-fetch (changing page state, which the effect depends on) and rendering (JSX instead of manual DOM manipulation) has changed.

22.8 Infinite Scrolling in React

```
function InfiniteFeed() {
  const [items, setItems] = useState([]);
  const [page, setPage] = useState(1);
  const [loading, setLoading] = useState(false);

  useEffect(() => {
    async function loadMore() {
      setLoading(true);
      try {
        const response = await fetch(`/api/feed?page=${page}`);
        const newItems = await response.json();
        setItems(prev => [...prev, ...newItems]); // APPEND  -  recall the immutable spread pattern (Part 18.4)
      } finally {
        setLoading(false);
      }
    }
    loadMore();
  }, [page]);

  useEffect(() => {
    function handleScroll() {
      const nearBottom = window.innerHeight + window.scrollY >= document.body.offsetHeight - 300;
      if (nearBottom && !loading) {
        setPage(prev => prev + 1); // triggers the OTHER effect above, via its `page` dependency
      }
    }

    window.addEventListener("scroll", handleScroll);
    return () => window.removeEventListener("scroll", handleScroll); // recall 21.5  -  always clean up listeners
  }, [loading]);

  return (
    <div>
      {items.map(item => <div key={item.id}>{item.title}</div>)}
      {loading && <p>Loading more...</p>}
    </div>
  );
}
```

Why TWO separate `useEffect` calls here, rather than one combined effect: each effect synchronizes a distinct concern with the outside world - one synchronizes fetched data with the current `page`; the other synchronizes a scroll-event listener's existence with the component's lifetime. Recall Part 21.8's "synchronization" mental model - keeping genuinely separate concerns in separate effects (rather than one large effect doing several unrelated things) is standard, recommended practice, and makes each effect's dependency array meaningfully correct and easy to reason about on its own.

22.9 API Mutations from React - POST/PUT/PATCH/DELETE

Fetching (`GET`) typically happens automatically via `useEffect` on render/mount. Mutations (`POST`/`PUT`/`PATCH`/`DELETE`) are different - they're deliberate actions, usually triggered by a user interaction (a button click, a form submit), not something that should happen automatically as a side effect of rendering.

```
function TodoItem({ todo, onDeleted }) {
  const [isDeleting, setIsDeleting] = useState(false);
  const [error, setError] = useState(null);

  async function handleDelete() {
```

```
    setIsDeleting(true);
    setError(null);
    try {
      const response = await fetch(`/api/todos/${todo.id}`, { method: "DELETE" });
      if (!response.ok) throw new Error(`HTTP ${response.status}`);
      onDeleted(todo.id); // notify the parent (recall Part 17.5's callback-prop pattern) to remove it from the list
    } catch (err) {
      setError("Failed to delete. Please try again.");
      setIsDeleting(false); // only reset here  -  on SUCCESS the component is about to be removed anyway
    }
  }

  return (
    <li>
      {todo.text}
      <button onClick={handleDelete} disabled={isDeleting}>
        {isDeleting ? "Deleting..." : "Delete"}
      </button>
      {error && <p className="error">{error}</p>}
    </li>
  );
}
```

Why this mutation is triggered directly from an event handler, NOT from a `useEffect`: recall Part 21.7/21.8 - `useEffect` is for synchronizing a component with an external system automatically, in response to render-driven changes (props/state changing). A delete action is a deliberate, one-time user-initiated command, not something that should re-run automatically whenever the component re-renders for unrelated reasons - it belongs directly in the event handler that responds to the actual user action, exactly like the mutation-triggering code in Part 11.13's Project 7 (CRUD application), just now managing its own `isDeleting`/`error` state via `useState` instead of plain DOM manipulation.

⚠ Why `disabled={isDeleting}` on the button matters, recalling Part 10, Level 17 and Part 11.12: without disabling the button while the delete request is in flight, an impatient user could click "Delete" multiple times before the first request completes, potentially sending multiple `DELETE` requests for the same item - usually harmless for a `DELETE` (idempotent, per Part 11.2) but demonstrates the same general UI discipline needed for non-idempotent mutations like `POST`, where duplicate submissions can cause real problems (recall the "double-charge" warning from Part 10, Level 17).

Knowledge Check - Part 22

1. Why does the data-fetching state pattern typically use THREE separate `useState` calls (`data`, `loading`, `error`) rather than one combined object, in the simplest version? (Consider: is this a hard rule, recalling Part 18.6?)
2. Why must `setError(null)` be called at the START of a new fetch attempt, not just in the `catch` block?
3. Walk through exactly how `AbortController` combined with `useEffect`'s cleanup mechanism solves the race condition in the debounced search example, referencing when cleanup actually runs.
4. Why are two separate `useEffect` calls preferred over one large combined effect for the infinite-scroll example?
5. Why should a mutation like a DELETE request be triggered from an event handler rather than from inside a `useEffect`?

Build a `PostViewer` component that:

- Fetches a list of post titles from `https://jsonplaceholder.typicode.com/posts?_limit=10` on mount.
- Shows loading/error/empty/success states correctly.
- Has a "Retry" button visible only in the error state, which re-triggers the fetch.
- Cancels any in-flight request if the component unmounts.

View solution

```
function PostViewer() {
  const [posts, setPosts] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [retryCount, setRetryCount] = useState(0);

  useEffect(() => {
    const controller = new AbortController();

    async function loadPosts() {
      setLoading(true);
      setError(null);
      try {
        const response = await fetch(
          "https://jsonplaceholder.typicode.com/posts?_limit=10",
          { signal: controller.signal }
        );
        if (!response.ok) throw new Error(`HTTP ${response.status}`);
        const data = await response.json();
        setPosts(data);
      } catch (err) {
        if (err.name === "AbortError") return;
        setError(err.message);
      } finally {
        setLoading(false);
      }
    }

    loadPosts();
    return () => controller.abort();
  }, [retryCount]);

  if (loading) return <p>Loading posts...</p>;

  if (error) {
    return (
      <div>
        <p>Failed to load posts: {error}</p>
        <button onClick={() => setRetryCount(prev => prev + 1)}>Retry</button>
      </div>
    );
  }

  if (posts.length === 0) return <p>No posts found.</p>;

  return (
    <ul>
      {posts.map(post => <li key={post.id}>{post.title}</li>)}
    </ul>
```

```
  );
}
```

Design notes: `retryCount` serves the exact "refetch trigger" role from 22.6. Aborting on unmount (22.4) prevents a `setPosts`/`setError` call on an unmounted component if the user navigates away mid-request. The four UI states from 22.2 are all explicitly and separately handled.

Next: Part 23 - React Hooks

PART 23 - REACT HOOKS [Intermediate][Advanced]

23.1 What a Hook Is

What it is

A Hook is a special function (always starting with the word `use`) that lets a function component "hook into" React features - state, effects, and other capabilities - that would otherwise only be available in more complex, older-style class components (a pattern this course doesn't teach directly, as function components with Hooks are the modern, recommended standard - but you may encounter class components in existing/legacy codebases, and understanding Hooks gives you the conceptual foundation to recognize their class-based equivalents if needed).

You've already used two: `useState` (Part 18) and `useEffect` (Part 21).

23.2 The Rules of Hooks

There are exactly two rules, and they're not arbitrary - they follow directly from how React is able to correctly associate each `useState`/`useEffect` call with the right piece of state/effect across re-renders.

Rule 1: Only call Hooks at the top level. Never inside loops, conditions, or nested functions.

```
// [x] WRONG
function Component({ shouldTrack }) {
  if (shouldTrack) {
    const [count, setCount] = useState(0); // Hook called conditionally!
  }
}
```

Why this breaks React, precisely: React tracks each component's Hooks internally using a simple mechanism roughly analogous to an ordered list, matched up by the order the Hooks are called in, render after render - not by name or any other identifier. If `useState` is called conditionally, the order/count of Hook calls can differ from one render to the next, and React can no longer reliably match "the third Hook call this render" to "the third Hook call last render" - corrupting the state association entirely, often causing deeply confusing bugs (state values suddenly appearing to belong to the wrong piece of UI).

Rule 2: Only call Hooks from React function components (or from custom Hooks - Part 27). Never from regular JavaScript functions, class components, or outside of React's rendering process entirely.

23.3 useState - Recap

Fully covered in Part 18. Quick summary for this reference section: `const [value, setValue] = useState(initialValue);` - provides a piece of state and a way to update it, triggering a re-render.

23.4 useEffect - Recap

Fully covered in Part 21. Quick summary: `useEffect(() => { ...sideEffect...; return cleanup; }, [dependencies]);` - synchronizes a component with something outside React's rendering system.

23.5 useRef

Fully covered in depth in Part 24. Quick preview: `const ref = useRef(initialValue);` - provides a mutable container (`ref.current`) that persists across renders without causing a re-render when changed, commonly used for direct DOM element access.

23.6 useContext

Fully covered in depth in Part 25. Quick preview: `const value = useContext(MyContext);` - reads a value from the nearest matching Context Provider above this component in the tree, without needing it passed down explicitly through every intermediate component's props.

23.7 useReducer

Fully covered in depth in Part 26. Quick preview: `const [state, dispatch] = useReducer(reducerFunction, initialState);` - an alternative to `useState` for more complex state logic, especially when the next state depends on the previous state in complex, multi-part ways.

23.8 useMemo

What it is

```
const expensiveValue = useMemo(() => computeExpensiveValue(a, b), [a, b]);
```

Why it exists

Normally, a component's entire function body re-runs on every render (recall Part 18.2). If a particular calculation is genuinely expensive (e.g., processing a very large array, complex math), redoing it on every render - even when its inputs haven't changed - wastes performance. `useMemo` memoizes (caches) the result of a calculation, only recomputing it when one of the listed dependencies actually changes.

```
function ProductList({ products, searchTerm }) {
  const filteredProducts = useMemo(() => {
    console.log("Filtering..."); // only logs when `products` or `searchTerm` actually change
    return products.filter(p => p.name.toLowerCase().includes(searchTerm.toLowerCase()));
```

```
  }, [products, searchTerm]);

  return <ul>{filteredProducts.map(p => <li key={p.id}>{p.name}</li>)}</ul>;
}
```

Step-by-step: on the first render, the function inside `useMemo` runs, and its result is cached alongside the current values of `products`/`searchTerm`. On subsequent renders, React compares the new dependency values to the cached ones (Part 18.4's reference-comparison logic again) - if unchanged, it skips re-running the function and simply returns the cached result instantly; if changed, it recomputes and re-caches.

⚠ Critical, frequently-misunderstood caveat: `useMemo` is a performance optimization, not a correctness guarantee. Don't use it to "make state update correctly" or as a substitute for understanding when re-renders happen - its only job is to skip redundant expensive recalculations. We cover precisely when it's actually worth using (and, just as important, when it's premature/unnecessary overhead) in Part 31.

23.9 useCallback

What it is

```
const memoizedFunction = useCallback(() => {
  doSomething(a, b);
}, [a, b]);
```

Why it exists

Every time a component re-renders, any function defined inside it (like an inline arrow function passed as a prop) is a brand new function, with a new reference, even if its code is identical to the previous render's version (recall Part 1.6 - functions are reference types too). This usually doesn't matter - but it can, specifically when that function is passed as a prop to a child component wrapped in `React.memo` (Part 31), because `React.memo` uses the exact same reference-comparison logic (Part 18.4) to decide whether to skip re-rendering the child - a "new" function reference every time defeats that optimization, causing the child to re-render unnecessarily anyway.

```
function Parent() {
  const [count, setCount] = useState(0);

  const handleClick = useCallback(() => {
    console.log("Clicked!");
  }, []); // same function reference across renders, since it has no dependencies

  return <MemoizedChild onClick={handleClick} />;
}
```

`useMemo` vs `useCallback` - the precise relationship: `useCallback(fn, deps)` is functionally equivalent to `useMemo(() => fn, deps)` - it memoizes a function specifically, whereas `useMemo` memoizes the result of calling a function. Both exist for the same underlying reason (avoiding unnecessary recalculation/new references across renders) but target different kinds of values.

⚠ Like `useMemo`, `useCallback` is a performance tool, not something to reach for by default on every function in every component. We give this the full, honest treatment (including why overusing it can sometimes make performance worse) in Part 31.

23.10 Newer/Additional Hooks (Briefly, for Awareness)

React has introduced several additional Hooks beyond this "essential seven" as the library has evolved - for example, `useId` (for generating stable unique IDs, useful for accessibility attributes, Part 33), `useTransition` and `useDeferredValue` (for marking certain state updates as lower-priority, to keep the UI responsive during expensive re-renders), and `useSyncExternalStore` (for safely subscribing to external, non-React state sources). These are genuinely advanced/optional for this course's scope - you should know they exist and roughly what problem class they address, but the seven Hooks covered above (`useState`, `useEffect`, `useRef`, `useContext`, `useReducer`, `useMemo`, `useCallback`) plus custom Hooks (Part 27) cover the overwhelming majority of real-world React application code, including everything in this course's capstone project.

Knowledge Check - Part 23

1. Why can't Hooks be called conditionally or inside loops? Explain the mechanism, not just "it's the rule."
2. What's the precise relationship between `useMemo` and `useCallback`?
3. Why is `useMemo` described as a performance optimization rather than a correctness tool?

> View solutions

1. React associates each Hook call with its corresponding state/effect based on the ORDER Hooks are called in across renders, not by name. Calling a Hook conditionally can change how many Hooks (and in what order) are called from one render to the next, breaking React's ability to correctly match up Hook calls between renders, corrupting state.
2. `useCallback(fn, deps)` is functionally equivalent to `useMemo(() => fn, deps)` - `useCallback` memoizes a function reference itself, while `useMemo` memoizes the return value of calling a function; both use the same dependency-comparison mechanism to decide whether to recompute/return a new reference.
3. Because using it or not using it doesn't change what the correct final rendered output should be - a component works correctly with or without `useMemo`; its only effect is on how much redundant computation happens along the way to producing that (identical) correct output.

Cheat Sheet - React Hooks

Hook	Purpose	Returns
`useState(initial)`	Component-local state that triggers re-renders	`[value, setValue]`
`useEffect(fn, deps)`	Synchronize with an external system	nothing
`useRef(initial)`	Persistent mutable value, no re-render on change	`{ current: value }`

Hook	Purpose	Returns
`useContext(Context)`	Read a value from a Context Provider	the context value
`useReducer(reducer, initial)`	Complex state logic via actions	`[state, dispatch]`
`useMemo(fn, deps)`	Cache an expensive calculated value	the memoized value
`useCallback(fn, deps)`	Cache a function reference	the memoized function

Next: Part 24 - Refs

PART 24 - REFS [Intermediate]

24.1 What useRef Is

What it is

```
const myRef = useRef(initialValue);
```

`useRef` returns a plain, mutable JavaScript object with a single property: `{ current: initialValue }`. This object is created once and persists, unchanged as an object reference, across every re-render of the component.

Why it exists - the core distinction from `useState`

Recall Part 18.2: calling a `useState` setter schedules a re-render. `useRef` does the exact opposite by design: changing `ref.current` does NOT trigger a re-render, and does NOT wait for the next render to reflect the new value - it updates immediately, synchronously, like any normal object mutation.

```
function Example() {
  const renderCount = useRef(0);
  renderCount.current = renderCount.current + 1; // mutated directly  -  no setter function needed, unlike state!

  console.log(`This component has rendered ${renderCount.current} times`);
  // NOTE: this specific console.log pattern is illustrative of ref mechanics, not a recommended real pattern
}
```

Mental model: think of `useState` as a labeled, official whiteboard that the whole audience (the rendered UI) is watching - writing on it triggers an announcement ("re-render!") so everyone can see the update. Think of `useRef` as a private sticky note in your own pocket - you can scribble on it anytime, instantly, but nobody else (i.e., the rendered UI) is automatically notified or updated when you do.

24.2 When to Use a Ref Instead of State - The Precise Rule

Use `useState` when a value's change should be reflected in the UI.
Use `useRef` when you need to persist a value across renders, but changing it should NOT cause a re-render - because the value isn't something that's directly displayed, or because you specifically want to read/write it without triggering React's rendering cycle at all.

Common legitimate use cases for `useRef`:

- Holding a reference to an actual DOM element (24.3).
- Storing a mutable value that needs to survive across renders but isn't part of what's rendered (a timer ID to clear later, a previous value for comparison, a flag like "has this effect already run once").
- Storing values you need inside event handlers/effects without wanting them to trigger effect re-runs if included in a dependency array (an advanced pattern, used carefully).

24.3 DOM References - The Most Common Use of useRef

The problem: how do you get direct access to an actual DOM element in React, when you're not supposed to manually query the DOM yourself (recall Part 14.1's declarative philosophy - you describe the UI, you don't reach in and grab elements the way you did with `querySelector` in Part 6.1)?

```
function TextInputWithFocusButton() {
  const inputRef = useRef(null); // starts as null  -  no element yet

  function handleFocusClick() {
    inputRef.current.focus(); // .focus() is a real, native DOM method (recall Part 6, generally)!
  }

  return (
    <div>
      <input ref={inputRef} type="text" />
      <button onClick={handleFocusClick}>Focus the input</button>
    </div>
  );
}
```

Step-by-step: passing `ref={inputRef}` to a JSX element is special, built-in React behavior - after React actually creates the real DOM node for this `<input>`, it automatically sets `inputRef.current` to point directly at that real DOM element. From that point on, `inputRef.current` is the actual `<input>` DOM element, with every native DOM property/method you learned in Part 6 available (`.focus()`, `.value`, `.scrollIntoView()`, etc.).

Why it starts as `null`: before the component has actually rendered for the first time, there's no real DOM element yet for the ref to point to - `useRef(null)` correctly represents "no element yet" until React attaches it after the initial render.

```
function ChatWindow({ messages }) {
  const bottomRef = useRef(null);

  useEffect(() => {
    bottomRef.current?.scrollIntoView({ behavior: "smooth" }); // recall Part 1.8's optional chaining  -  defensive,
  }, [messages]); // re-run (and re-scroll) whenever the messages list changes

  return (
```

```
    <div>
      {messages.map(msg => <p key={msg.id}>{msg.text}</p>)}
      <div ref={bottomRef} /> {/* an invisible marker element at the very bottom */}
    </div>
  );
}
```

Why this combination of `useRef` and `useEffect` is the correct, idiomatic pattern: the DOM scroll position is something outside React's own state/rendering model (recall Part 21.1's definition of what belongs in an Effect) - we're reaching out to directly command the browser to scroll, which is a side effect, correctly placed inside `useEffect`, using a ref to get a handle on the actual DOM element to scroll to.

24.4 Persistent Values Across Renders - Beyond the DOM

```
function StopwatchLogger() {
  const intervalRef = useRef(null); // will hold the interval ID, NOT a DOM element this time

  function startTimer() {
    intervalRef.current = setInterval(() => console.log("tick"), 1000);
  }

  function stopTimer() {
    clearInterval(intervalRef.current); // reading the SAME persisted value later, in a different function call
  }

  return (
    <div>
      <button onClick={startTimer}>Start</button>
      <button onClick={stopTimer}>Stop</button>
    </div>
  );
}
```

Why a plain variable wouldn't work here: a regular `let intervalId` declared inside the component function body would be recreated from scratch on every render (recall Part 18.2 - the entire function body re-runs each render) - it couldn't reliably "remember" its value from a previous render/event handler call the way `useRef`'s persistent `.current` object can. This is the second core reason `useRef` exists beyond DOM access: a mutable value that survives across renders, without being tied to the rendering cycle at all.

⚠ Common mistake - using a ref for something that SHOULD be state:

```
// [Advanced] WRONG  -  this WON'T update the displayed count when clicked!
function BrokenCounter() {
  const countRef = useRef(0);

  function handleClick() {
    countRef.current = countRef.current + 1; // updates the ref...
  }

  return (
    <div>
      <p>Count: {countRef.current}</p> {/* ...but this text NEVER re-renders to show it! */}
      <button onClick={handleClick}>Increment</button>
    </div>
  );
}
```

Why this fails, precisely: changing `ref.current` does not trigger a re-render (24.1's core rule). The `<p>` displaying `countRef.current` was rendered once, showing `0`, and since nothing ever tells React to re-render this component, it never updates on screen - even though `countRef.current` is genuinely changing internally, with each click, if you were to log it. The fix is simply to use `useState` instead - anything you want reflected visually in the UI must be state, never a ref.

Knowledge Check - Part 24

1. What's the single most important behavioral difference between `useState` and `useRef`?
2. Why does `ref.current` start as `null` for a DOM ref, before becoming the actual element?
3. Why can't a plain local variable (`let x = 0` inside the component function) serve the same "persist across renders" purpose that `useRef` provides?
4. Explain, precisely, why the `BrokenCounter` example fails to visually update, even though the underlying ref value is genuinely changing.

   Build a component with a text input and a "Clear and Focus" button that clears the input's value and returns keyboard focus to it, using `useRef` (not `useState`) to manage the DOM interaction directly.

View solution

```
function ClearableInput() {
  const inputRef = useRef(null);

  function handleClear() {
    inputRef.current.value = "";
    inputRef.current.focus();
  }

  return (
    <div>
      <input ref={inputRef} type="text" />
      <button onClick={handleClear}>Clear and Focus</button>
    </div>
  );
}
```

Note: this deliberately uses an uncontrolled pattern (Part 20.2) via the ref directly setting `.value` - appropriate here since we don't need the input's value reflected elsewhere in the UI or tracked in state; we're simply issuing direct, imperative DOM commands, exactly the legitimate use case refs exist for.

Next: Part 25 - Context

PART 25 - CONTEXT [Intermediate][Advanced]

25.1 The Problem: Prop Drilling

Recall Part 17.6: data flows down through props, one level at a time. What happens when a deeply nested component needs data that lives several levels up the tree?

```
function App() {
  const [theme, setTheme] = useState("dark");
  return <Layout theme={theme} />;
}

function Layout({ theme }) {
  return <Sidebar theme={theme} />; // Layout doesn't actually USE theme itself  -  just passing it through
}

function Sidebar({ theme }) {
  return <UserMenu theme={theme} />; // Sidebar doesn't use it either  -  just passing it through again
}

function UserMenu({ theme }) {
  return <div className={theme}>Menu</div>; // FINALLY actually used, 3 levels down
}
```

What "prop drilling" means, and why it's a problem: `theme` has to be manually threaded through `Layout` and `Sidebar`, even though neither of those components actually cares about or uses `theme` themselves - they're just forced to accept and re-pass it along, purely because something further down the tree needs it. In a real application with deep component trees, this becomes genuinely painful: adding a new piece of shared data means touching every intermediate component along the path, even ones with no logical connection to that data, just to relay it through.

25.2 Context - The Solution

What it is

Context provides a way to pass data through the component tree without manually passing props down through every intermediate level - any descendant component, no matter how deeply nested, can directly access the value.

Mental model

If props are like passing a note hand-to-hand down a line of people (everyone in the chain has to physically handle it, even if it's not for them), Context is like a public announcement broadcast to everyone in a room at once - anyone who wants to listen can tune in directly, without anyone in between needing to relay it.

```
// ThemeContext.js
import { createContext } from "react";

export const ThemeContext = createContext(null); // the default value, used only if no Provider is found above
```

```
// App.jsx
import { ThemeContext } from "./ThemeContext.js";

function App() {
  const [theme, setTheme] = useState("dark");

  return (
    <ThemeContext.Provider value={theme}>
      <Layout />
```

```
    </ThemeContext.Provider>
  );
}

// Layout.jsx  -  no longer needs to know or care about theme AT ALL
function Layout() {
  return <Sidebar />;
}

// Sidebar.jsx  -  same, completely untouched
function Sidebar() {
  return <UserMenu />;
}

// UserMenu.jsx  -  reads the value DIRECTLY, no matter how deep it is
import { useContext } from "react";
import { ThemeContext } from "./ThemeContext.js";

function UserMenu() {
  const theme = useContext(ThemeContext); // gets "dark" directly, skipping Layout and Sidebar entirely
  return <div className={theme}>Menu</div>;
}
```

Step-by-step, precisely:

1. `createContext(defaultValue)` creates a Context "channel" - a way to broadcast a value.
2. `<ThemeContext.Provider value={theme}>` wraps a portion of the component tree, making `theme`'s current value available to every descendant inside it, at any depth.
3. `useContext(ThemeContext)`, called from any descendant component inside that Provider (regardless of how many intermediate components exist between them), retrieves the current value directly.
4. `Layout` and `Sidebar` are restored to their natural, simple form - they don't need to know this data exists at all.

Critical detail: if `theme` (the Provider's `value`) changes, every component consuming that context via `useContext` automatically re-renders to reflect the new value - Context is fully reactive, just like state, not a one-time snapshot.

25.3 A Complete, Realistic Example - Authenticated User Context

```
// AuthContext.jsx
import { createContext, useContext, useState } from "react";

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);

  function login(userData) {
    setUser(userData);
  }

  function logout() {
    setUser(null);
  }

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

// A custom hook (full treatment in Part 27) wrapping useContext  -  a very common, recommended convention:
export function useAuth() {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error("useAuth must be used within an AuthProvider"); // guards against misuse outside the Provider
  }
  return context;
}

// App.jsx
function App() {
  return (
    <AuthProvider>
      <Header />
      <MainContent />
    </AuthProvider>
  );
}

// Header.jsx  -  anywhere inside AuthProvider, at any depth:
function Header() {
  const { user, logout } = useAuth();
  return (
    <header>
      {user ? (
        <>
          <span>Welcome, {user.name}</span>
          <button onClick={logout}>Log out</button>
        </>
      ) : (
        <span>Not logged in</span>
      )}
    </header>
  );
}
```

Why wrapping `useContext(AuthContext)` in a custom `useAuth()` hook is considered best practice: it (1) provides a friendlier, more meaningful API (`useAuth()` reads better than `useContext(AuthContext)` everywhere), (2) centralizes the "did you forget to wrap your app in the Provider?" safety check in one place rather than repeating it everywhere, and (3) hides the underlying Context object from consumers entirely - they only ever need to know about `useAuth`, not the implementation detail of how that data is being shared. This exact pattern (a custom Hook wrapping Context) is standard, idiomatic practice in real-world React codebases - you'll build several more custom Hooks with this same shape in Part 27.

25.4 More Realistic Examples - App Settings

```
const SettingsContext = createContext(null);

function SettingsProvider({ children }) {
  const [settings, setSettings] = useState({ fontSize: "medium", language: "en" });

  function updateSetting(key, value) {
    setSettings(prev => ({ ...prev, [key]: value })); // recall Part 18.4's immutable update pattern
  }

  return (
    <SettingsContext.Provider value={{ settings, updateSetting }}>
      {children}
    </SettingsContext.Provider>
  );
}
```

25.5 When Context Should - and Should NOT - Be Used

Good fits for Context: data that's genuinely global or widely shared across many, deeply nested, otherwise-unrelated parts of the application - the current authenticated user, a UI theme, app-wide settings/preferences, the current locale/language.

Poor fits for Context:

- Frequently-changing, localized data - e.g., the current value of a single text input, or state that's only relevant to one small section of the UI. Recall from 25.2 that every consumer of a Context re-renders whenever its value changes - for data that changes very often (like every keystroke) but is only needed in a small, nearby part of the tree, plain `useState` passed via normal props (or kept fully local to a small component) is simpler and avoids unnecessary re-renders across unrelated parts of the app that happen to also consume that same Context.
- As a default, reflexive replacement for props - if data is only needed by a component and its direct children (one or two levels, not many), plain props are simpler, more explicit, and easier to trace than introducing a Context. The rule of thumb: reach for Context specifically to solve prop drilling through several unrelated intermediate layers - not as a default alternative to props in general.

  We revisit this exact tradeoff, with more nuance around performance implications specifically, in Part 30 (State Management) and Part 31 (Performance).

Knowledge Check - Part 25

1. What specific problem does prop drilling create, beyond just "extra typing"?
2. Walk through what happens, step by step, when a Context's `value` changes, in terms of which components re-render.
3. Why is wrapping `useContext` in a custom hook like `useAuth()` considered better practice than calling `useContext(AuthContext)` directly everywhere?
4. Give a concrete example of data that's a POOR fit for Context, and explain why, referencing re-render behavior.

> View solutions

1. It forces every intermediate component along the path to a deeply nested consumer to accept and manually re-pass a prop it has no actual use for itself, coupling components that should otherwise be unrelated, and making the codebase more fragile to change (adding new shared data means touching every intermediate component in the chain).
2. Every component that calls `useContext(ThatContext)` anywhere within the Provider's subtree automatically re-renders to reflect the new value - Context is fully reactive, not a one-time static read; this happens regardless of how deeply nested each consuming component is.
3. It provides a cleaner, more meaningful API for consumers, centralizes the check for "was this called outside its Provider" in one place instead of repeating it at every call site, and hides the underlying Context implementation detail so consuming components only need to know about the custom hook, not the Context object itself.
4. A single text input's live value as a user types is a poor fit - because every keystroke would change the Context value, and Context changes trigger a re-render in every consuming component throughout the app, even ones with no visual relationship to that particular input, causing unnecessary, potentially widespread re-rendering for something that's actually very localized in scope.

Next: Part 26 - Reducers

PART 26 - REDUCERS [Intermediate][Advanced]

26.1 The Problem useReducer Solves

Recall Part 18.6's discussion of combined state objects. As a component's state logic grows more complex - many related pieces of state, updated by many different actions, often with interdependencies - managing it all with several separate `useState` calls (or one large object, manually spread-updated everywhere) can become hard to follow, and easy to update incorrectly/inconsistently.

`useReducer` is an alternative way to manage state, centered around one core idea: instead of directly calling setter functions scattered throughout your component, you `dispatch` descriptive "actions," and

a single, centralized function (the "reducer") decides exactly how the state should change in response to each one.

26.2 The Vocabulary - Reducer, State, Action, Dispatch

- State: the current data (same concept as `useState`'s state - just potentially more complex/structured).
- Action: a plain object describing what happened (not how to update state - just a description of an event, e.g., `{ type: "increment" }`).
- Reducer: a pure function (recall Part 3.5's exact definition of purity) that takes the current state and an action, and returns the new state. `(state, action) => newState`.
- Dispatch: the function you call to send an action to the reducer, triggering a state update.

Why "reducer" is the correct name for this - connecting directly to Part 4.4: recall `.reduce()` from Part 4.4 - it takes an accumulator and a current element, and returns a new accumulator, one step at a time. `useReducer` works on the exact same fundamental shape: `(accumulator/state, action/element) => newAccumulator/newState`. If you deeply understood `.reduce()` in Part 4.4, `useReducer`'s core mechanic should feel immediately familiar - it's the identical pattern, applied specifically to managing component state over time instead of transforming an array.

26.3 Basic Syntax

```
function reducer(state, action) {
  switch (action.type) {
    case "increment":
      return { ...state, count: state.count + 1 }; // recall Part 18.4's immutable update pattern  -  still applies
    case "decrement":
      return { ...state, count: state.count - 1 };
    case "reset":
      return { ...state, count: 0 };
    default:
      return state; // ALWAYS include a default case that returns the unchanged state
  }
}

function Counter() {
  const [state, dispatch] = useReducer(reducer, { count: 0 });

  return (
    <div>
      <p>Count: {state.count}</p>
      <button onClick={() => dispatch({ type: "increment" })}>+</button>
      <button onClick={() => dispatch({ type: "decrement" })}>-</button>
      <button onClick={() => dispatch({ type: "reset" })}>Reset</button>
    </div>
  );
}
```

Step-by-step, when a button is clicked:

1. `dispatch({ type: "increment" })` is called.
2. React calls `reducer(currentState, { type: "increment" })`.

3 The `switch` statement (recall Part 2.2) matches `case "increment"`, and returns a new state object with `count` incremented.

4 React updates the component's state to this new returned value and schedules a re-render - the exact same underlying re-render mechanism as `useState` (Part 18.2), just with the logic for deciding the new value centralized in the `reducer` function instead of scattered across multiple inline setter calls.

⚠ Why the `default: return state;` case is essential, not just defensive boilerplate: a reducer must always return some valid state, for every possible action it might receive - if an unrecognized `action.type` fell through with no matching case and no default, the function would implicitly return `undefined` (recall Part 3.3's warning about missing `return` statements), silently wiping out the component's entire state. The default case guards against this.

26.4 Actions with a Payload

Actions often need to carry additional data beyond just their `type`:

```
function reducer(state, action) {
  switch (action.type) {
    case "setName":
      return { ...state, name: action.payload };
    case "addItem":
      return { ...state, items: [...state.items, action.payload] }; // recall Part 18.4's array immutability rule
    case "removeItem":
      return { ...state, items: state.items.filter(item => item.id !== action.payload) };
    default:
      return state;
  }
}

// Dispatching with a payload:
dispatch({ type: "setName", payload: "Ammaar" });
dispatch({ type: "addItem", payload: { id: 3, text: "New item" } });
dispatch({ type: "removeItem", payload: 3 }); // payload here is just the id to remove
```

Naming convention: `type` and `payload` are the near-universal, standard field names for actions in the React ecosystem (this convention originates from Redux, a popular external state management library that heavily popularized this exact pattern - previewed in Part 30) - while not enforced by `useReducer` itself (you could name these fields anything), following this convention makes your code instantly recognizable to any other React developer.

26.5 A More Complex, Realistic Example - Shopping Cart

```
function cartReducer(state, action) {
  switch (action.type) {
    case "addToCart": {
      const existingItem = state.items.find(item => item.id === action.payload.id);
      if (existingItem) {
        // item already in cart  -  increment its quantity, immutably
        return {
          ...state,
          items: state.items.map(item =>
            item.id === action.payload.id
              ? { ...item, quantity: item.quantity + 1 }
              : item
          )
        };
      }
      // new item  -  add it with quantity 1
      return {
        ...state,
        items: [...state.items, { ...action.payload, quantity: 1 }]
      };
    }
    case "removeFromCart":
      return {
        ...state,
        items: state.items.filter(item => item.id !== action.payload)
      };
    case "updateQuantity":
      return {
        ...state,
        items: state.items.map(item =>
          item.id === action.payload.id
            ? { ...item, quantity: action.payload.quantity }
            : item
        )
      };
    case "clearCart":
      return { ...state, items: [] };
    default:
      return state;
  }
}

function ShoppingCart() {
  const [cart, dispatch] = useReducer(cartReducer, { items: [] });

  const total = cart.items.reduce((sum, item) => sum + item.price * item.quantity, 0); // recall Part 4.4!

  return (
    <div>
      {cart.items.map(item => (
        <div key={item.id}>
          {item.name} × {item.quantity}
          <button onClick={() => dispatch({ type: "removeFromCart", payload: item.id })}>Remove</button>
        </div>
      ))}
      <p>Total: ₹{total}</p>
    </div>
  );
}
```

Why `useReducer` genuinely helps here, compared to several separate `useState` calls or manually-written inline update logic: the "add to cart, checking if the item already exists and either

incrementing or inserting" logic is meaningfully complex - encapsulating it entirely inside the reducer function keeps `ShoppingCart`'s component body clean, focused purely on rendering and dispatching simple, descriptive actions (`dispatch({ type: "addToCart", payload: product })`), rather than containing this branching logic inline, mixed in with JSX and rendering concerns.

26.6 useReducer vs useState - When to Use Which

Signal	Prefer `useState`	Prefer `useReducer`
State shape	Simple (a string, number, boolean, or small independent object)	Complex, with multiple related sub-values that often update together
Update logic	Straightforward, computed directly at the call site	Involves multiple cases/branches, or the next state depends on several parts of the current state in complex ways
Number of ways state can change	Few	Many distinct "kinds" of updates (naturally expressible as different action types)
Testability desired	-	Reducers are plain, pure functions - trivially testable in isolation, separate from any component (recall Part 3.5's purity benefits)

 Practical guidance: most components are perfectly well-served by `useState`, including several separate `useState` calls for genuinely independent pieces of data. Reach for `useReducer` specifically when you notice your `useState`-based update logic becoming tangled, repetitive across multiple handlers, or when several pieces of state need to be updated together, consistently, in response to the same events.

Knowledge Check - Part 26

1. Explain the connection between `.reduce()` (Part 4.4) and `useReducer` - why is the naming not a coincidence?
2. What MUST a reducer function always do, even for an action type it doesn't recognize, and why?
3. Why must reducer functions remain pure (recall Part 3.5), just like state updates in general must be immutable (Part 18.4)?
4. Give two concrete signals that suggest switching from `useState` to `useReducer` for a piece of state.

   Build a `useReducer`-based form reducer for a login form with `email` and `password` fields, supporting actions `{ type: "setField", field, value }` and `{ type: "reset" }`.

> View solution

```
const initialState = { email: "", password: "" };
```

```
function formReducer(state, action) {
  switch (action.type) {
    case "setField":
      return { ...state, [action.field]: action.value }; // recall Part 4.6's computed property syntax
    case "reset":
      return initialState;
    default:
      return state;
  }
}

function LoginForm() {
  const [formState, dispatch] = useReducer(formReducer, initialState);

  function handleChange(event) {
    dispatch({ type: "setField", field: event.target.name, value: event.target.value });
  }

  return (
    <form>
      <input name="email" value={formState.email} onChange={handleChange} />
      <input name="password" type="password" value={formState.password} onChange={handleChange} />
      <button type="button" onClick={() => dispatch({ type: "reset" })}>Reset</button>
    </form>
  );
}
```

Next: Part 27 - Custom Hooks

PART 27 - CUSTOM HOOKS [Advanced]

27.1 Why Custom Hooks Exist

The problem

Across Parts 21-22, you've now written the "fetch data with loading/error states" pattern (Part 22.1) multiple times. Across different components that all need this same behavior, you'd otherwise be copy-pasting the same `useState`/`useEffect` logic repeatedly - violating the DRY principle from Part 3.1, and creating a maintenance burden (a bug fix or improvement to the pattern has to be manually repeated everywhere it was copied).

What a custom Hook is

A custom Hook is simply a regular JavaScript function whose name starts with `use`, that itself calls one or more other Hooks internally, extracting and packaging up reusable stateful logic - not markup, not a specific piece of UI, just reusable behavior - that any component can plug into.

The naming convention (`use...`) is not just a style preference - it's functionally required: recall Part 23.2's Rules of Hooks - React's tooling (specifically the `eslint-plugin-react-hooks` linter mentioned in Part 21.3) identifies which functions are Hooks (and therefore need to follow the Rules of Hooks) specifically by checking whether their name starts with `use`. A function containing Hook calls that doesn't start with `use` won't be correctly recognized and checked by this tooling, risking Rule violations

going undetected.

27.2 useFetch - Extracting Part 22's Pattern

```
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const controller = new AbortController();

    async function load() {
      setLoading(true);
      setError(null);
      try {
        const response = await fetch(url, { signal: controller.signal });
        if (!response.ok) throw new Error(`HTTP ${response.status}`);
        const result = await response.json();
        setData(result);
      } catch (err) {
        if (err.name !== "AbortError") setError(err.message);
      } finally {
        setLoading(false);
      }
    }

    load();
    return () => controller.abort();
  }, [url]);

  return { data, loading, error };
}
```

Using it - this is the entire payoff:

```
function UserProfile({ userId }) {
  const { data: user, loading, error } = useFetch(`/api/users/${userId}`); // recall Part 4.7's rename-on-destructur

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;
  return <p>{user.name}</p>;
}

function ProductList({ categoryId }) {
  const { data: products, loading, error } = useFetch(`/api/products?category=${categoryId}`);
  // identical pattern, completely different component  -  ALL the fetch/loading/error/cancellation logic is reused,
}
```

Why this is a genuine, meaningful abstraction, not just moving code around: every single line of Part 22.1's careful, correct data-fetching logic (loading states, error handling, `response.ok` checking, `AbortController` cancellation) is now written once, tested once, and reused everywhere - any future improvement to this pattern (say, adding retry logic from Part 10, Level 17) only needs to happen in this one place.

27.3 useLocalStorage

```
function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    // Lazy initializer function  -  recall this runs ONLY on the very first render, not every render
    try {
      const stored = localStorage.getItem(key);
      return stored ? JSON.parse(stored) : initialValue; // recall Part 8.2's JSON.parse pattern
    } catch {
      return initialValue; // corrupted/inaccessible storage  -  fall back gracefully, recall Part 9.7's principled
    }
  });

  useEffect(() => {
    try {
      localStorage.setItem(key, JSON.stringify(value)); // recall Part 8.2's JSON.stringify pattern
    } catch (error) {
      console.error("Failed to save to localStorage:", error);
    }
  }, [key, value]); // re-sync to storage whenever the value (or key) changes

  return [value, setValue]; // same [value, setter] shape as useState  -  familiar, drop-in-feeling API
}
```

Why `useState(() => {...})` - a function, not a direct value - is used for the initializer here: this is called a lazy initializer. If we wrote `useState(JSON.parse(localStorage.getItem(key)))` directly, that `localStorage.getItem`/`JSON.parse` work would run on every single render (recall Part 18.2 - the whole function body re-runs each render), even though the initial value is only ever actually used on the very first render. Passing a function to `useState` tells React to only call it once, on that first render, avoiding this repeated, unnecessary work - a genuinely useful, if slightly obscure, `useState` feature worth knowing.

Using it:

```
function App() {
  const [theme, setTheme] = useLocalStorage("theme", "light");

  return (
    <div className={theme}>
      <button onClick={() => setTheme(theme === "light" ? "dark" : "light")}>
        Toggle Theme
      </button>
    </div>
  );
}
```

This directly, elegantly solves the exact "persist to localStorage" need from Part 8.5's to-do project - but now it's a generic, reusable one-liner for any piece of state you want automatically persisted, rather than hand-written save/load logic specific to one app.

27.4 useDebounce

```
function useDebounce(value, delayMs) {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    const timeoutId = setTimeout(() => {
      setDebouncedValue(value);
    }, delayMs);

    return () => clearTimeout(timeoutId); // recall 21.5/22.5  -  cancels the PENDING timer if `value` changes again
  }, [value, delayMs]);

  return debouncedValue;
}
```

Using it - a much cleaner version of Part 22.5's search example:

```
function SearchBox() {
  const [query, setQuery] = useState("");
  const debouncedQuery = useDebounce(query, 400);
  const { data: results, loading } = useFetch(
    debouncedQuery ? `/api/search?q=${encodeURIComponent(debouncedQuery)}` : null
  );

  return (
    <div>
      <input value={query} onChange={(e) => setQuery(e.target.value)} />
      {loading && <p>Searching...</p>}
      <ul>{results?.map(r => <li key={r.id}>{r.name}</li>)}</ul>
    </div>
  );
}
```

Why this composition (a `useDebounce` Hook feeding into a `useFetch` Hook) is powerful, and worth pausing on: each Hook does exactly one conceptual job - `useDebounce` only knows about delaying a value's updates; `useFetch` only knows about fetching with proper states/cancellation. Neither knows anything about the other, yet composing them together (`useFetch(debouncedQuery ? ... : null)`) reproduces the entire sophisticated race-condition-safe, debounced search behavior from Part 22.5 - but built from two small, independently reusable, independently understandable pieces, rather than one large, tangled effect. This is Hook composition, directly mirroring the component composition principle from Part 16.4, now applied to reusable logic instead of reusable markup.

27.5 usePrevious

```
function usePrevious(value) {
  const ref = useRef(); // starts as undefined

  useEffect(() => {
    ref.current = value; // updates AFTER the render that displays the CURRENT value, so it's always one step behind
  }, [value]);

  return ref.current; // returns whatever value was stored from the PREVIOUS render
}
```

Using it:

```
function ScoreTracker({ score }) {
  const previousScore = usePrevious(score);
```

```
  const trend = previousScore === undefined ? "" : score > previousScore ? "□" : score < previousScore ? "□" : "→□";

  return <p>Score: {score} {trend}</p>;
}
```

Why this genuinely needs a ref, connecting back to Part 24.2's precise rule: we want to remember a value across renders without that remembering itself causing another re-render (which would be redundant - we already have a render happening because `score` itself changed) - exactly the textbook `useRef` use case from Part 24.2. The subtlety worth understanding: because the ref update happens inside `useEffect` (which runs after rendering, per Part 21.1), during any given render, `ref.current` still holds the value from before this render's effect ran - i.e., the previous render's value - which is exactly the delayed-by-one-step behavior we want.

27.6 useWindowSize

```
function useWindowSize() {
  const [size, setSize] = useState({ width: window.innerWidth, height: window.innerHeight });

  useEffect(() => {
    function handleResize() {
      setSize({ width: window.innerWidth, height: window.innerHeight });
    }

    window.addEventListener("resize", handleResize); // recall Part 6.6/21.5
    return () => window.removeEventListener("resize", handleResize); // recall 21.5's cleanup necessity
  }, []); // empty array  -  this listener setup/teardown only needs to happen once, on mount/unmount

  return size;
}
```

Using it:

```
function ResponsiveLayout() {
  const { width } = useWindowSize();

  return width < 768 ? <MobileNav /> : <DesktopNav />;
}
```

This is a direct, generalized extraction of the `WindowWidthTracker` component from Part 21.5 - turning a one-off component into a reusable piece of logic any component can consume.

27.7 The General Pattern for Writing Your Own Custom Hooks

1. Notice you're writing the same combination of `useState`/`useEffect`/`useRef`/etc. more than once, for the same underlying purpose, across different components.
2. Extract that logic into a function named `use...`.
3. Have that function call whatever Hooks it needs internally, following the Rules of Hooks (Part 23.2) exactly as if it were a component.
4. Return whatever data/functions the consuming components will actually need - design this return value the way you'd design any function's return value or a component's props: as a clean, minimal, well-named interface, hiding the internal implementation details (the specific `useState`/`useEffect` calls) from whoever uses the Hook.

Knowledge Check - Part 27

1. Why must a custom Hook's name start with `use`? Is this purely stylistic?
2. What specific problem does `useFetch` solve that raw `useState`/`useEffect` in every component doesn't?
3. Why does `useLocalStorage` pass a FUNCTION to `useState`, rather than calling `localStorage.getItem` directly as the argument?
4. Explain how `useDebounce` and `useFetch` can be composed together to reproduce the debounced-search behavior from Part 22.5, and why each Hook only needs to know about its own single responsibility.
5. Why does `usePrevious` need `useRef` rather than `useState`?

> View solutions

1. React's tooling (the Hooks ESLint plugin) identifies which functions are Hooks - and therefore subject to the Rules of Hooks - specifically by checking for the `use` naming prefix. It's functionally significant, not purely a style choice: a Hook-calling function without this prefix won't be correctly analyzed for Rule violations by that tooling.
2. It eliminates the need to copy-paste the same loading/error/data state management and cancellation logic into every component that needs to fetch data - that logic is written and tested once, in one place, and reused as a simple function call everywhere it's needed.
3. Because `useState`'s argument (when not a function) is evaluated on EVERY render, even though an initial value is only actually needed on the very first render. Passing a function (a "lazy initializer") tells React to only invoke it once, avoiding unnecessary repeated `localStorage` reads/JSON parsing on every subsequent re-render.
4. `useDebounce` delays how often a rapidly-changing value (the search query) actually updates its returned value, only settling after a pause in changes. Feeding that debounced, settled value into `useFetch` means a new fetch only kicks off once the debounced value actually changes - and `useFetch`'s own effect-cleanup/AbortController logic (inherited from Part 22.4) correctly cancels any still-in-flight request from a previous value. Each Hook only handles its own concern (delaying a value; fetching for a given value) and neither needs any awareness of the other for the composition to work correctly.
5. Because we want to remember a value from the previous render without that remembering itself triggering an additional, redundant re-render - exactly the core `useRef` behavior from Part 24.1 (mutating `.current` doesn't cause a re-render), whereas using `useState` here would trigger an unnecessary extra render every time the previous value was updated.

Next: Part 28 - Routing

PART 28 - ROUTING [Intermediate][Advanced]

28.1 The Problem - Client-Side Navigation in an SPA

Recall Part 0.14: in a traditional multi-page site, clicking a link triggers a full HTTP request/response cycle (Part 0.4) - the browser discards the current page entirely and loads a new one from scratch. In a React SPA (Part 14.6), we deliberately want to avoid that full reload (it's slow, and would wipe out any in-memory JavaScript state) - but users still expect the browser's URL bar, back/forward buttons, and bookmarking/sharing links to work normally.

Client-side routing solves this: a library intercepts navigation, updates the URL (using browser APIs, without triggering an actual page reload), and swaps which React components are rendered - all while staying within the same, already-loaded SPA.

28.2 React Router - Setup and Basic Routes

We use React Router, the current standard, widely-adopted routing library for React.

```
npm install react-router-dom

// main.jsx
import { BrowserRouter } from "react-router-dom";

createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

Why `<BrowserRouter>` wraps the whole app: it provides routing context (recall Part 25's Context concept - React Router's internals are built on Context) to every component inside it, enabling `Link`, `useNavigate`, and route-matching components to function anywhere in the tree.

```
// App.jsx
import { Routes, Route } from "react-router-dom";
import Home from "./pages/Home.jsx";
import About from "./pages/About.jsx";
import ProductDetail from "./pages/ProductDetail.jsx";
import NotFound from "./pages/NotFound.jsx";

function App() {
  return (
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/about" element={<About />} />
      <Route path="/products/:id" element={<ProductDetail />} />
      <Route path="*" element={<NotFound />} />
    </Routes>
  );
}
```

Step-by-step: `<Routes>` looks at the current URL and renders the first `<Route>` whose `path` matches, in place - it functions somewhat like the `switch` statement from Part 2.2, matching one case (one route) based on the current URL "value."

28.3 Route Parameters

```
<Route path="/products/:id" element={<ProductDetail />} />

import { useParams } from "react-router-dom";

function ProductDetail() {
  const { id } = useParams(); // recall Part 4.7's destructuring  -  extracts the ":id" segment from the current URL

  const { data: product, loading, error } = useFetch(`/api/products/${id}`); // recall Part 27.2's custom Hook!

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;
  return <h1>{product.name}</h1>;
}
```

Step-by-step: if the URL is `/products/42`, React Router matches this against the `path="/products/:id"` pattern, extracting `id: "42"` (note: always a string, exactly like `dataset` values in Part 6.3 - you'll need `Number(id)` if you need it numerically) and makes it available via `useParams()`.

28.4 Link and Navigation

⚠ Why you can't just use a plain `<a href="...">` for internal navigation:

```
// [x] Triggers a FULL PAGE RELOAD  -  defeats the entire purpose of client-side routing!
<a href="/about">About</a>
```

Correct version:

```
import { Link } from "react-router-dom";

<Link to="/about">About</Link>
```

Why `Link` avoids a full reload, connecting back to Part 6.6's `preventDefault`: internally, `Link` renders an actual `<a>` tag (for accessibility and correct browser behavior like "open in new tab"), but attaches a click handler that calls `event.preventDefault()` (exactly the mechanism you learned in Part 6.6/19.5) to stop the browser's default full-navigation behavior, and instead uses browser APIs (the History API) to update the URL and let React Router re-render the matching component - all without leaving the currently-loaded page.

```
import { useNavigate } from "react-router-dom";

function LoginForm() {
  const navigate = useNavigate();

  async function handleSubmit(event) {
    event.preventDefault();
    await login(/* ... */); // recall Part 11.13's Project 10 pattern
    navigate("/dashboard"); // navigate AFTER an action completes, not via a user clicking a Link
  }

  return <form onSubmit={handleSubmit}>{/* ... */}</form>;
```

```
}
```

When to use `useNavigate` instead of `Link`: `Link` is for navigation the user directly clicks. `useNavigate` is for navigation that happens programmatically, as a consequence of some other action completing (a successful login, a form submission, a timeout redirect) - situations where there's no literal link for the user to click.

28.5 Nested Routes and Layouts

```
function App() {
  return (
    <Routes>
      <Route path="/" element={<DashboardLayout />}>
        <Route index element={<DashboardHome />} />       {/* renders at exactly "/" */}
        <Route path="orders" element={<Orders />} />        {/* renders at "/orders" */}
        <Route path="products" element={<Products />} />  {/* renders at "/products" */}
      </Route>
    </Routes>
  );
}

import { Outlet } from "react-router-dom";

function DashboardLayout() {
  return (
    <div>
      <Sidebar />
      <main>
        <Outlet /> {/* the matching CHILD route renders exactly here */}
      </main>
    </div>
  );
}
```

Why nested routes matter: they let you share common layout (like a persistent sidebar/header, `DashboardLayout` here) across many routes, without duplicating that layout markup in every single page component - directly applying the composition principle from Part 16.4 to page-level structure. `<Outlet />` acts as a placeholder - recall Part 16.6's `children` concept - it's conceptually very similar, but specifically for React Router's nested route matching rather than JSX composition you control directly.

28.6 404 / "Not Found" Routes

```
<Route path="*" element={<NotFound />} />
```

The `*` path acts as a catch-all/wildcard - matching any URL that didn't match any of the more specific routes listed above it. Order matters: React Router checks routes in the order they're written (roughly - nested/more specific matches are generally prioritized, but a catch-all should still be placed last, as a deliberate final fallback) - so this should always be the last route in your list.

28.7 Protected Route Concepts

```
function ProtectedRoute({ children }) {
  const { user } = useAuth(); // recall Part 25.3's custom Auth Context Hook

  if (!user) {
    return <Navigate to="/login" replace />; // Navigate is a component version of useNavigate, for declarative redi
  }

  return children;
}

// Usage:
<Route
  path="/dashboard"
  element={
    <ProtectedRoute>
      <Dashboard />
    </ProtectedRoute>
  }
/>
```

Step-by-step: `ProtectedRoute` is a component wrapping another component (recall the `children` pattern from Part 16.6) - it checks authentication status (via Context, per Part 25.3) and either renders the protected content, or redirects to `/login` using `<Navigate>`. This directly combines Part 25 (Context), Part 17 (children/composition), and Part 28's routing concepts into one practical, realistic pattern used in virtually every real-world authenticated application.

28.8 Query Strings in React Router

```
import { useSearchParams } from "react-router-dom";

function ProductList() {
  const [searchParams, setSearchParams] = useSearchParams();
  const category = searchParams.get("category"); // recall Part 11.5's URLSearchParams!

  function handleCategoryChange(newCategory) {
    setSearchParams({ category: newCategory }); // updates the URL's query string, e.g. ?category=electronics
  }

  // ...
}
```

`useSearchParams` gives you a `URLSearchParams`-like interface (recall Part 11.5) directly tied to React state and the actual browser URL - reading and writing query parameters without manually parsing `window.location.search` yourself.

Knowledge Check - Part 28

1. Why does a plain `<a href="...">` break the benefits of client-side routing, and how does `Link` avoid that problem?
2. When should you use `useNavigate` instead of `Link`?
3. What problem do nested routes with `<Outlet />` solve?

4. Why must the catch-all `path="*"` route be placed last in your list of routes?
5. Walk through, step by step, how `ProtectedRoute` decides whether to render its children or redirect.

> View solutions

1. A plain `<a href>` triggers the browser's default full-page navigation - a real HTTP request and complete page reload, discarding all in-memory JavaScript state (recall Part 0.4/0.14). `Link` renders an anchor tag but intercepts the click with `preventDefault()`, then uses the History API to update the URL and let React Router swap the matching component in-place, without ever leaving the loaded SPA.
2. When navigation should happen as a consequence of some other action completing programmatically (e.g., after a successful form submission or login), rather than in direct response to the user clicking a specific link element.
3. They let multiple routes share common layout/structure (like a persistent sidebar or header) without duplicating that markup in every individual page component - the shared layout renders once, with `<Outlet />` marking where the specific matched child route's content should appear.
4. Because it matches ANY URL as a fallback - if placed earlier, it would incorrectly "catch" and match URLs that should have matched a more specific route listed after it, since routes are generally checked in order.
5. It reads the current authenticated user from Context (via a custom hook like `useAuth`). If there's no user (not logged in), it renders `<Navigate to="/login" />`, redirecting the browser to the login page. If a user exists, it renders its `children` prop - the actual protected content - normally.

Next: Part 29 - Application Architecture

PART 29 - APPLICATION ARCHITECTURE [Intermediate][Advanced]

29.1 Why Folder Structure Matters

As an application grows beyond a handful of components, where things live becomes as important as how any individual piece of code works. A consistent, logical structure means any developer (including future-you) can predict where to find - or where to add - something, without hunting through the entire codebase.

29.2 A Realistic, Standard Folder Structure

```
src/
├── components/     (small, reusable, presentation-focused UI pieces)
├── pages/            (route-level components  -  Part 28's page components)
├── hooks/            (custom Hooks  -  Part 27's useFetch, useDebounce, etc.)
├── services/         (API/data-fetching logic  -  Part 11's request helpers)
├── utils/             (pure helper functions with no React dependency)
├── context/         (Context providers  -  Part 25)
├── assets/           (images, fonts, static files)
├── styles/           (global CSS, theme files)
└── App.jsx
```

What belongs in each directory, and why the boundary matters:

- `components/` - small, focused, often reusable pieces (`Button`, `Card`, `Modal`, `ProductCard`) - recall Part 16.5's extraction criteria. These generally shouldn't know about routing or make their own API calls directly; they receive data via props and communicate via callback props (Part 17.5).
- `pages/` - components tied to a specific route (Part 28) - `HomePage`, `ProductDetailPage`, `DashboardPage`. These typically compose components from `components/`, fetch the data they need (often via hooks from `hooks/`), and represent a full "screen" of the application.
- `hooks/` - custom Hooks (Part 27) - reusable stateful logic, decoupled from any specific piece of UI.
- `services/` - the API/service layer: functions that talk to your backend, built around the `apiRequest` helper pattern from Part 11.10. Crucially, components should not call `fetch()` directly - they should call functions from this layer.
- `utils/` - pure, non-React-specific helper functions (date formatting, currency formatting, validation logic like Part 7.3's regex patterns) - functions that could, in principle, be used in any JavaScript project, not just this React one.
- `context/` - Context providers and their associated custom Hooks (Part 25.3's `AuthProvider`/`useAuth` pattern).

29.3 Separation of Concerns - The UI Layer vs the Service Layer

This is the single most important architectural principle in this Part, and directly determines how maintainable a real application ends up being.

```
// [Advanced] Poor separation  -  fetch logic hardcoded directly inside a component
function ProductList() {
  const [products, setProducts] = useState([]);

  useEffect(() => {
    fetch("https://api.example.com/products")
      .then(res => res.json())
      .then(setProducts);
  }, []);
  // ...
}
```

Why this is a problem at scale: if the API's base URL changes, if you need to add authentication headers to every request, if you need consistent error handling (recall Part 11.10's carefully-built

`apiRequest` helper) - you'd have to find and update every single component that calls `fetch` directly. The API-calling logic and the UI-rendering logic are tangled together, making each harder to change, test, or reuse independently.

Better - a dedicated service layer:

```
// services/productService.js
import { apiRequest } from "./apiClient.js"; // the Part 11.10 helper, defined once

const BASE_URL = "https://api.example.com/products";

export function getProducts() {
  return apiRequest(BASE_URL);
}

export function getProduct(id) {
  return apiRequest(`${BASE_URL}/${id}`);
}

export function createProduct(data) {
  return apiRequest(BASE_URL, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(data)
  });
}

// hooks/useProducts.js
import { useState, useEffect } from "react";
import { getProducts } from "../services/productService.js";

export function useProducts() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    getProducts()
      .then(setProducts)
      .catch(err => setError(err.message))
      .finally(() => setLoading(false));
  }, []);

  return { products, loading, error };
}

// pages/ProductListPage.jsx
import { useProducts } from "../hooks/useProducts.js";
import ProductCard from "../components/ProductCard.jsx";

function ProductListPage() {
  const { products, loading, error } = useProducts();

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;

  return (
    <div>
      {products.map(product => <ProductCard key={product.id} product={product} />)}
    </div>
  );
}
```

Why this layering is worth the extra files, precisely: each layer has exactly one job and can change independently:

- Changing the API's base URL or auth strategy -> only touch `services/`.
- Changing how loading/error states are managed -> only touch `hooks/`.
- Changing how a product visually looks -> only touch `components/ProductCard.jsx`.
- None of these changes require touching the others. This is the same "separation of concerns" principle from Part 0.7 (HTML/CSS/JS) and Part 0.3 (frontend/backend) - now applied within the frontend itself, splitting UI, stateful logic, and data-fetching into their own well-defined layers.

29.4 Summary - The Layers, Stated Plainly

Layer	Responsibility	Should NOT do
UI layer (`components/`, `pages/`)	Render JSX based on props/state	Directly call `fetch`, contain complex business logic
Hooks layer (`hooks/`)	Manage stateful logic, bridge UI and services	Contain JSX, know about specific API URLs directly (delegate to services)
Service layer (`services/`)	Talk to APIs, handle request/response shape	Know about React at all, contain JSX or Hooks
Utils (`utils/`)	Pure, reusable, non-React helper logic	Depend on component state, Hooks, or React itself

---

Knowledge Check - Part 29

1. Why is it a problem to call `fetch()` directly inside many different components, rather than through a shared service layer?
2. What specifically changes (and what doesn't need to change) if your API's base URL moves, in a properly layered architecture?
3. What's the difference in responsibility between `components/` and `pages/`?
4. Why should `utils/` functions avoid any dependency on React itself?

> View solutions

1. Any change to how requests are made - a new base URL, added authentication headers, consistent error handling - would require finding and updating every component that independently calls `fetch`, rather than changing it in one centralized place. It also tightly couples UI-rendering code with data-fetching mechanics, making both harder to test, reuse, and reason about independently.

2 Only the `services/` layer needs to change (the base URL constant/configuration). Components, pages, and hooks that consume the service functions remain completely untouched, because they only interact with the service layer's stable function interface (`getProducts()`, etc.), not the underlying URL details.

3 `components/` holds small, typically reusable, presentation-focused pieces that receive data via props. `pages/` holds route-level components that represent a full screen, typically composing several `components/` pieces together and fetching whatever data that specific screen needs.

4 Because `utils/` is meant to hold pure, general-purpose helper logic (formatting, validation, calculations) that could, in principle, be reused in any JavaScript context - not just this specific React application. Depending on React-specific features (Hooks, JSX) would tie these otherwise-general utilities unnecessarily to the React environment, reducing their reusability and testability in isolation.

Next: Part 30 - State Management

PART 30 - STATE MANAGEMENT [Advanced]

30.1 The Spectrum of State You Already Know

This Part doesn't introduce fundamentally new mechanisms - it's about choosing correctly among tools you've already fully learned, at the right scale:

Tool	Scope	Covered in
Local state (`useState`)	One component	Part 18
Lifted state	Shared between a parent and its direct children, via props	Part 17.6 (conceptually)
Context	Shared broadly across a subtree, avoiding prop drilling	Part 25
Reducers (`useReducer`)	Complex update logic, often combined with the above	Part 26

30.2 Lifting State Up - Fully Explained

The scenario: two sibling components need to share and stay in sync with the same piece of data, but neither is an ancestor of the other.

```
// [Advanced] Doesn't work  -  each has its OWN independent state, they can never see each other's value
function TemperatureInput() {
  const [value, setValue] = useState("");
  return <input value={value} onChange={(e) => setValue(e.target.value)} />;
}

function App() {
```

```
  return (
    <div>
      <TemperatureInput /> {/* Celsius */}
      <TemperatureInput /> {/* meant to mirror as Fahrenheit, but can't see the other's state */}
    </div>
  );
}
```

The fix - "lift" the shared state UP to their nearest common ancestor, and pass it down as props to both:

```
function TemperatureInput({ value, onChange }) {
  return <input value={value} onChange={(e) => onChange(e.target.value)} />;
}

function App() {
  const [celsius, setCelsius] = useState("");

  const fahrenheit = celsius === "" ? "" : (Number(celsius) * 9) / 5 + 32; // derived value, recall Part 21.7  -  no

  return (
    <div>
      <TemperatureInput value={celsius} onChange={setCelsius} />
      <p>Fahrenheit: {fahrenheit}</p>
    </div>
  );
}
```

Why this works, connecting directly to Part 17.6's one-way data flow: by moving the actual state to their shared parent (`App`), both children can now be kept in sync - because they're both just reflections (via props) of one single source of truth. This isn't a new concept - it's the direct, practical consequence of one-way data flow: when siblings need to share data, that data's "home" must move up to wherever their paths first cross in the component tree.

When lifting state up is enough, versus when you need Context: lifting state works cleanly when the shared ancestor is close - one or two levels up. When the natural common ancestor is very far up the tree (recall Part 25.1's prop-drilling problem), lifting state that high and threading it back down through many unrelated intermediate components is exactly the scenario Context (Part 25) exists to solve instead.

30.3 Server State vs Client State - A Crucial Distinction

This is a genuinely important, often under-taught distinction.

- Client state: data that only exists in the browser, owned entirely by your UI - is a modal open? What's currently typed in a form field? Is a dropdown expanded? This is what `useState`/`useReducer`/Context are naturally designed for.
- Server state: data that actually lives on a server, and your component is just displaying a cached, potentially-stale copy of it - a list of products, a user's profile, order history (recall Part 22's entire pattern - everything fetched via `useFetch` is server state).

  Why treating server state exactly like client state causes real problems: server state has fundamentally different concerns that plain `useState`/`useEffect` don't naturally address:

- Caching: if two different components need the same data (e.g., the current user's name, shown in both a header and a profile page), should they each independently re-fetch it? That's wasteful.
- Staleness: the data on the server can change at any time, independent of your component's lifecycle - recall Part 22.6's manual refetch pattern; this is exactly the kind of concern that's tedious to hand-roll correctly and consistently everywhere.
- Background revalidation: automatically re-fetching data when the user refocuses the browser tab, or on a schedule, to keep displayed data reasonably fresh.
- Deduplication: if five components on the same page all call `useFetch` for the exact same URL at nearly the same time, should that really trigger five separate, redundant network requests?

30.4 TanStack Query (React Query) - Conceptual Introduction

What it is (conceptually - not a hard dependency for this course, but essential to understand as an industry-standard tool)

TanStack Query is a widely-used library specifically built to manage server state, solving the caching/staleness/deduplication/background-refetching concerns from 30.3 - concerns that your own hand-built `useFetch` (Part 27.2), while genuinely useful for learning and smaller projects, doesn't address.

```
// Conceptual illustration  -  NOT required for this course's capstone, but important to recognize:
import { useQuery } from "@tanstack/react-query";

function ProductList() {
  const { data: products, isLoading, error } = useQuery({
    queryKey: ["products"],                 // a cache key  -  identical keys share cached data automatically
    queryFn: () => getProducts()                // your existing service-layer function from Part 29.3!
  });

  if (isLoading) return <p>Loading...</p>;
  if (error) return <p>Error: {error.message}</p>;
  return <ul>{products.map(p => <li key={p.id}>{p.name}</li>)}</ul>;
}
```

Notice how strikingly similar this looks to `useFetch` (Part 27.2) - same `data`/`loading`/`error` shape, same general idea. What TanStack Query adds on top, automatically, without you writing any of it by hand: caching by `queryKey` (so multiple components requesting `["products"]` share one cached result and one request, rather than each fetching independently), automatic background revalidation, automatic retry-with-backoff (recall Part 10, Level 17 - built in, not hand-rolled), and a rich set of tools specifically for mutations (`useMutation`) with automatic cache invalidation after a successful write.

Why this course teaches `useFetch` from scratch FIRST, rather than starting with a library like this: understanding exactly what a data-fetching library is doing for you - because you've built a simpler version yourself in Part 22 and Part 27 - makes you a dramatically more effective user of that library, and lets you correctly debug issues when the library's abstractions inevitably leak or behave unexpectedly. This course's capstone project (Part on the capstone) uses the hand-built `useFetch`-style pattern throughout, precisely because it's built entirely from concepts this course has already taught end-to-end - but you should recognize TanStack Query (or similar libraries) as the natural, standard next step for real production applications with more demanding server-state needs.

30.5 A Practical Decision Framework

Question	Answer
Is this data purely local UI state (modal open, form input, toggle)?	`useState` (Part 18)
Does a sibling component need this same state?	Lift it to their common parent (30.2)
Is this data needed by many, deeply-nested, unrelated components?	Context (Part 25)
Is the update logic for this state genuinely complex, with many related cases?	`useReducer` (Part 26)
Does this data actually live on a server, and could multiple components need the same fetched data?	A dedicated server-state solution (a custom `useFetch`-style hook for learning/smaller apps, or a library like TanStack Query for production-scale needs)

Knowledge Check - Part 30

1. What does "lifting state up" mean, and when is it the appropriate solution versus reaching for Context?
2. Give three concerns that come with SERVER state that don't naturally apply to plain client UI state.
3. Why does `useQuery`'s `queryKey` matter for avoiding redundant network requests across multiple components?
4. Why does this course build `useFetch` by hand in Part 27 before mentioning TanStack Query here?

> View solutions

1. Lifting state up means moving a piece of state to the nearest common ancestor of the components that need to share it, then passing it down to each as props. It's appropriate when that common ancestor is close (a level or two up); when the natural common ancestor is far away and passing props through many unrelated intermediate components would recreate the prop-drilling problem, Context is the more appropriate tool instead.
2. Any three of: caching shared data across multiple components without redundant fetches, handling staleness (the server's data can change independent of the component's lifecycle), background revalidation (refetching to stay reasonably up to date), deduplication of near-simultaneous identical requests, and built-in retry logic.
3. Components using the same `queryKey` share the SAME cached data and, typically, the same in-flight request - so if multiple components request data with an identical key at nearly the same time, the library can serve them all from one shared request/cache entry instead of firing off separate, redundant network calls for identical data.

4 Because understanding exactly what problem a data-fetching library like TanStack Query is solving - and how - is dramatically clearer once you've built a simpler version of the same core mechanics yourself; it also means you're not dependent on a specific external library to understand or complete the course's core concepts, and can more effectively debug and reason about such a library once you do adopt one.

Next: Part 31 - Performance

PART 31 - PERFORMANCE [Advanced][Expert]

31.1 Why Components Re-render - The Precise Rules

Recall Part 18.2: a component re-renders when its own state changes. There's one more crucial rule, not yet stated explicitly: by default, when a component re-renders, EVERY one of its child components re-renders too - even if that child's own props haven't actually changed.

```
function Parent() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <button onClick={() => setCount(count + 1)}>Count: {count}</button>
      <ExpensiveChild /> {/* re-renders every time Parent does, even though it takes NO props from count at all! */}
    </div>
  );
}
```

Why this default behavior exists, and why it's usually fine: re-rendering (recall Part 14.4) is not the same as an actual DOM update - it's React re-running the component function to get a new JSX description, then diffing it against the previous one, and only touching the real DOM where something actually changed. For most components, re-rendering (recalculating JSX, then finding "nothing changed") is extremely cheap - modern React and the underlying DOM diffing are highly optimized. This is why the course's advice throughout Parts 14-30 has been to write natural, straightforward code without worrying about this - premature optimization here is a genuine anti-pattern, not a virtue.

31.2 When Re-renders Actually Become a Problem

Re-rendering becomes a measurable performance concern specifically when:

- A component's render logic is itself genuinely expensive (heavy computation, rendering thousands of DOM nodes).
- Re-renders are happening far more often than necessary, cascading through a large subtree, at a frequency that's visibly janky to the user (e.g., during rapid typing, scrolling, or animation).

The correct first step is always to measure, not guess. React DevTools (a browser extension) includes a Profiler tab that records exactly which components re-rendered, how long each took, and why - use this to identify genuine bottlenecks before reaching for any of the optimization tools below. Optimizing code that isn't actually slow adds complexity for zero real benefit - and, as you'll see below, can sometimes make things worse.

31.3 React.memo

What it is

```
const ExpensiveChild = React.memo(function ExpensiveChild({ data }) {
  console.log("Rendering ExpensiveChild");
  return <div>{/* expensive rendering here */}</div>;
});
```

What it does: wraps a component so that React skips re-rendering it if its props are the same (via the same reference-comparison logic from Part 18.4) as the previous render - even if its parent re-rendered.

⚠ The critical gotcha, connecting directly back to Part 23.9's `useCallback` motivation:

```
function Parent() {
  const [count, setCount] = useState(0);

  // [x] This defeats React.memo entirely!
  return <ExpensiveChild onSave={() => console.log("saved")} />;
  // A NEW function is created on EVERY render of Parent (recall Part 1.6  -  functions are reference types),
  // so ExpensiveChild's `onSave` prop is NEVER equal to its previous value, and React.memo's
  // comparison always finds "something changed," causing it to re-render every single time anyway.
}
```

The fix - `useCallback` (recall Part 23.9), used for EXACTLY this scenario:

```
function Parent() {
  const [count, setCount] = useState(0);
  const handleSave = useCallback(() => console.log("saved"), []); // same reference across renders

  return <ExpensiveChild onSave={handleSave} />; // now React.memo's comparison correctly finds no change
}
```

This is precisely the connection promised back in Part 23.9 - `useCallback` exists almost entirely to support this exact pattern: memoizing a function passed as a prop to a `React.memo`-wrapped child, so the memoization actually has a chance to work.

31.4 useMemo and useCallback - When They're Actually Worth Using

Recall Part 23.8/23.9's warnings: these are performance tools, not correctness requirements. The honest, precise guidance:

- Use `useMemo` when a calculation is measurably expensive (verified via profiling, 31.2) AND its dependencies don't change on every render.

- Use `useCallback` specifically when passing a function as a prop to a `React.memo`-wrapped child (31.3), or as a dependency to another Hook (like `useEffect`) where a new reference every render would cause that Hook to re-run unnecessarily.

  ⚠ Why reflexively wrapping EVERY value/function in `useMemo`/`useCallback` can make things WORSE, not better: `useMemo`/`useCallback` themselves have a small cost - they require React to store the previous dependencies and compare them on every render. For a cheap calculation or a component with no `React.memo`-wrapped children depending on stable references, this comparison overhead can exceed the cost of just... recalculating the simple thing normally. The honest, professional rule: default to NOT using them. Add them deliberately, backed by profiling data showing an actual measured problem, not as a defensive habit applied everywhere "just in case."

31.5 Lazy Loading and Code Splitting

Recall Part 12.1's dynamic `import()`. React provides `lazy` and `Suspense` specifically to apply this to components:

```
import { lazy, Suspense } from "react";

const AdminDashboard = lazy(() => import("./AdminDashboard.jsx")); // only downloaded when actually needed

function App() {
  return (
    <Suspense fallback={<p>Loading dashboard...</p>}>
      <AdminDashboard />
    </Suspense>
  );
}
```

Why this matters for performance, specifically: without this, `AdminDashboard`'s entire code (and everything it imports) would be bundled into your app's initial JavaScript download - even for users who never visit that admin page. `lazy()` defers loading that code until the component is actually about to be rendered, keeping the initial bundle smaller and the initial page load faster. `<Suspense>` provides a `fallback` UI to show while that code is being fetched - directly analogous to the loading states from Part 22.2, but for loading code rather than data.

When to reach for this: routes/pages that aren't needed immediately (an admin panel most users never see, a settings page, a rarely-used modal with a heavy dependency) are ideal candidates - recall Part 28's routing: it's common practice to `lazy()`-load entire page components, so each route's code only downloads when a user actually navigates there.

31.6 Why Premature Optimization Is Genuinely Bad - Stated Directly

Every tool in this Part adds real complexity: `React.memo` requires reasoning carefully about prop reference stability; `useMemo`/`useCallback` add visual noise and their own overhead; code-splitting adds loading states and slightly more complex bundling. None of this complexity is free. Applying it everywhere, reflexively, without evidence of an actual problem:

- Makes code harder to read and reason about.
- Can, as shown in 31.4, sometimes make performance worse, not better.

- Spends engineering time solving a problem that may not exist, instead of building features or fixing real bugs.

  The correct workflow, always: build the feature naturally and correctly first (per everything taught in Parts 14-30) -> measure with the Profiler if performance actually feels or measures poorly -> optimize the specific, identified bottleneck with the narrowest appropriate tool.

Knowledge Check - Part 31

1. Why does a child component re-render when its parent re-renders, even if the child's own props haven't changed?
2. Why is measuring with the Profiler the correct first step, rather than guessing which components are "probably slow"?
3. Explain precisely why wrapping a callback in `useCallback` is necessary for `React.memo` to actually be effective on a child receiving that callback as a prop.
4. Give a concrete scenario where using `useMemo` would likely make performance WORSE, not better.
5. What problem does `lazy()` + `Suspense` solve, and how does it relate to dynamic `import()` from Part 12.1?

> View solutions

1. This is React's default rendering behavior - recall Part 14.4, rendering is a cheap recalculation-then-diff process, not a direct DOM mutation, so re-rendering children by default (rather than requiring explicit opt-in tracking of exactly what changed) is a reasonable, usually-inexpensive default that keeps the mental model simple. Optimizing away unnecessary child re-renders is an explicit opt-in (`React.memo`), not the default.
2. Because performance intuition is frequently wrong - a component that "feels" like it should be slow might not meaningfully impact actual performance, while a genuinely slow bottleneck elsewhere goes unnoticed. Profiling gives concrete, measured evidence of which specific components are actually expensive and how often they're really re-rendering, ensuring optimization effort targets real problems.
3. `React.memo` skips re-rendering a component only if its props are reference-equal to the previous render. A new inline function created on every parent render has a new reference every time (functions are reference types, Part 1.6), so the memoized child's prop is never seen as "the same" - defeating the memoization. `useCallback` gives that function a stable reference across renders (when its dependencies haven't changed), allowing `React.memo`'s comparison to actually succeed.
4. Wrapping a cheap, simple calculation (e.g., a basic string concatenation or a small arithmetic expression) in `useMemo` - the overhead of storing and comparing dependencies on every render can exceed the cost of simply recalculating the cheap value directly every time.
5. `lazy()` + `Suspense` defers downloading a component's code until it's actually about to be rendered, rather than including it in the initial bundle - directly built on the same dynamic `import()` mechanism from Part 12.1 (which returns a Promise, loading a module on demand at runtime), specifically adapted for lazily loading React components with a declarative loading fallback via `Suspense`.

Next: Part 32 - Error Handling in React

PART 32 - ERROR HANDLING IN REACT [Intermediate][Advanced]

32.1 The Categories of Errors in a React App

Category	Example	Where it's handled
API errors	A `fetch` request fails or returns a bad status	`try/catch` inside async functions - recall Part 22 fully
Validation errors	A form field doesn't meet requirements	Local component state, recall Part 20.5
Runtime errors during rendering	A component throws while trying to render (e.g., accessing a property of `undefined`)	Error Boundaries - new to this Part
Component errors	A bug in a component's logic that throws unexpectedly	Error Boundaries

32.2 Synchronous vs Asynchronous Errors - Why React Treats Them Differently

Recall Part 9.1-9.2: synchronous errors happen immediately, during a function's direct execution, and are caught by a `try/catch` wrapped around that exact call. React components render synchronously - if a component's function body throws while React is calling it to get JSX, that's a synchronous rendering error.

Why a normal `try/catch` inside a component doesn't handle THIS specific class of error well:

```
function ProductPrice({ product }) {
  try {
    return <p>₹{product.price.toFixed(2)}</p>; // throws if product is undefined/null
  } catch (error) {
    return <p>Error displaying price</p>;
  }
}
```

This actually does work for this one component - but it doesn't scale. You'd need to wrap every single component individually in its own `try/catch`, and - more importantly - a `try/catch` cannot catch an error that occurs in a child component's render, only errors in the exact code directly inside the `try` block. If `ProductPrice` is one of many children rendered by a parent, and a different child throws during rendering, this `try/catch` does nothing for that.

32.3 Error Boundaries

What it is

A special kind of component that can catch rendering errors anywhere in its child component tree, log them, and display a fallback UI instead of letting the error crash the entire application (which is React's default behavior for an uncaught rendering error - the whole app can go blank).

⚠ Important, honest technical note: Error Boundaries currently must be written as class components (the pre-Hooks component style this course has otherwise avoided teaching directly, per Part 23.1's note) - there is, as of this course's writing, no Hook-based equivalent for this specific capability. In practice, most teams either write this one small class component themselves (shown below) or use a small, widely-trusted community package that provides one, and treat it as a "just works" utility rather than something to deeply customize often.

```
import { Component } from "react";

class ErrorBoundary extends Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }

  static getDerivedStateFromError(error) {
    return { hasError: true }; // update state so the next render shows the fallback UI
  }

  componentDidCatch(error, errorInfo) {
    console.error("Caught by Error Boundary:", error, errorInfo); // logging (Part 32.5)
  }

  render() {
    if (this.state.hasError) {
      return <h2>Something went wrong. Please refresh the page.</h2>; // fallback UI
    }
    return this.props.children; // recall Part 16.6  -  renders children normally when there's no error
  }
}
```

Using it:

```
function App() {
  return (
    <ErrorBoundary>
      <Header />
      <MainContent /> {/* if THIS throws during render, ErrorBoundary catches it */}
      <Footer />
    </ErrorBoundary>
  );
}
```

Step-by-step: if any component inside `<ErrorBoundary>` throws during rendering, React "catches" it at the nearest enclosing Error Boundary, calls `getDerivedStateFromError` (updating the boundary's own state to remember an error occurred) and `componentDidCatch` (for logging), then re-renders the boundary showing its fallback UI instead of the crashed subtree - the rest of the application outside this boundary continues working normally.

32.4 Strategic Placement of Error Boundaries

You don't need (and generally shouldn't want) just one Error Boundary wrapping your entire app - that makes any single component's rendering error take down the entire UI, showing one generic fallback for everything.

```
function App() {
  return (
    <div>
      <ErrorBoundary fallback={<p>Header failed to load</p>}>
        <Header />
      </ErrorBoundary>

      <ErrorBoundary fallback={<p>Something went wrong with the product list</p>}>
        <ProductList />
      </ErrorBoundary>

      <ErrorBoundary fallback={<p>Footer failed to load</p>}>
        <Footer />
      </ErrorBoundary>
    </div>
  );
}
```

Why placing multiple, smaller Error Boundaries around independent sections is better practice: if `ProductList` throws due to a bug, the `Header` and `Footer` remain completely unaffected and functional - the user experiences a contained, partial failure rather than a complete application crash. This is a direct application of the same "isolate failures to limit their blast radius" principle you've seen elsewhere (recall Part 9.7's careful, scoped error handling).

32.5 What Error Boundaries Do NOT Catch

⚠ Critical limitation to understand precisely: Error Boundaries only catch errors during rendering, in lifecycle methods, and in constructors of the component tree below them. They do NOT catch:

- Errors inside event handlers (use regular `try/catch` there, exactly as covered throughout Part 9 and Part 22.9).
- Errors inside asynchronous code (`setTimeout` callbacks, `.then()` callbacks, `await`ed code) - again, use `try/catch`/`.catch()` (Part 9, Part 10).
- Errors during server-side rendering (not covered in this course's scope).
- Errors thrown inside the Error Boundary component itself.

```
function BuggyButton() {
  function handleClick() {
    throw new Error("Oops!"); // [x] an Error Boundary does NOT catch this  -  it's inside an event handler
  }
  return <button onClick={handleClick}>Click me</button>;
}
```

Correct handling for this case - a normal `try/catch`, directly in the handler:

```
function BuggyButton() {
  function handleClick() {
    try {
      riskyOperation();
```

```
    } catch (error) {
      console.error(error);
      // show some local error state/message, recall Part 22.9's pattern
    }
  }
  return <button onClick={handleClick}>Click me</button>;
}
```

The precise mental division, stated directly: Error Boundaries handle unexpected errors during rendering that you didn't anticipate. `try/catch` (synchronous) and `try/catch`+`async/await` (asynchronous) handle errors in event handlers, effects, and any async operations - exactly the tools you've already mastered throughout Part 9, Part 10, and Part 22. Error Boundaries are a supplement to that existing knowledge, covering the one specific gap (render-time errors propagating through the component tree) that plain `try/catch` structurally cannot reach.

32.6 Retry Mechanisms

```
class ErrorBoundary extends Component {
  state = { hasError: false };

  static getDerivedStateFromError() {
    return { hasError: true };
  }

  handleRetry = () => {
    this.setState({ hasError: false }); // reset  -  re-attempts rendering children on the next render
  };

  render() {
    if (this.state.hasError) {
      return (
        <div>
          <p>Something went wrong.</p>
          <button onClick={this.handleRetry}>Try Again</button>
        </div>
      );
    }
    return this.props.children;
  }
}
```

Why setting `hasError` back to `false` and re-rendering can actually help, in some real cases: if the original error was caused by something transient (e.g., a genuinely temporary bad state, or the user retrying after fixing whatever input caused it), simply attempting to render the children again can succeed. This won't help for a persistent, code-level bug (it'll just throw again immediately), but it's a reasonable, low-cost first line of defense for the user, directly echoing the retry-UI recommendation from Part 11.12.

Knowledge Check - Part 32

1. Why can't a plain `try/catch` written inside one component protect against a rendering error thrown by a different, sibling component?

2. Precisely, what kinds of errors do Error Boundaries NOT catch? How should those be handled instead?

3 Why is placing several small Error Boundaries around independent UI sections generally better than one large Error Boundary around the whole app?

4 Why must Error Boundaries currently be written as class components, and how does this course otherwise expect you to work with React?

> View solutions

1 A `try/catch` only catches errors thrown directly within the synchronous execution of the code inside its `try` block - it has no reach into a completely separate component's own render execution, which happens as its own distinct function call, entirely outside that `try` block's scope.

2 Errors in event handlers, errors in asynchronous code (timers, promises, awaited code), errors during server-side rendering, and errors thrown inside the Error Boundary itself. These should be handled with ordinary `try/catch` (synchronous) or `try/catch` combined with `async/await` (asynchronous), exactly as taught throughout Part 9, Part 10, and Part 22.

3 It contains the "blast radius" of any single component's failure - if one section throws, only that section's Error Boundary fallback is shown, while the rest of the application (other sections, wrapped in their own separate boundaries) continues functioning normally, rather than the entire app going blank from one localized bug.

4 There is currently no Hook-based equivalent providing this specific capability (catching rendering errors in a subtree) - it's a React platform limitation, not a stylistic choice. This course otherwise exclusively teaches function components with Hooks (per Part 23.1's note); the Error Boundary class component is treated as a small, largely "install and reuse" utility rather than a template for how to write components generally.

Next: Part 33 - Accessibility

PART 33 - ACCESSIBILITY [Intermediate]

33.1 Why Accessibility Matters

Accessibility (often abbreviated "a11y" - "a," 11 letters, "y") means building interfaces that people with disabilities - visual, motor, auditory, cognitive - can actually use. This includes people using screen readers, people who navigate entirely by keyboard (unable to use a mouse), people with low vision relying on high contrast or zoom, and more. Beyond being the ethically correct default, it's frequently a legal requirement for many organizations, and - practically - accessible markup tends to be more robust, better-structured markup overall.

33.2 Semantic HTML - The Foundation

The single highest-leverage accessibility practice: use the HTML element that actually matches your content's meaning, rather than generic `<div>`s for everything.

```
// [Advanced] Poor  -  a screen reader has no idea this is a button
<div onClick={handleSubmit}>Submit</div>

// [x] Good  -  a real <button> is automatically keyboard-focusable, announced correctly by screen readers, and trig
<button onClick={handleSubmit}>Submit</button>
```

Why this matters, precisely: native HTML elements (`<button>`, `<nav>`, `<main>`, `<header>`, `<input>`) come with built-in accessibility behavior - keyboard focusability, correct screen-reader announcements, expected keyboard interactions - for free, built by browser vendors over decades of refinement. A `<div>` has none of this by default; recreating it correctly by hand (with `tabIndex`, ARIA attributes, and manual keyboard event handling) is possible but is genuinely difficult to get exactly right, and is almost always unnecessary - just use the semantic element that already does it correctly.

33.3 Buttons vs Clickable s

Directly extending 33.2: never build a "button" out of a `<div>` or `<span>` with an `onClick` handler. A real `<button>` gets keyboard focus via Tab, activates on both Enter and Space, and is correctly announced as "button" by screen readers - none of which a `<div onClick>` provides without substantial extra manual work.

33.4 Labels and Forms

Recall Part 20's form patterns - every input needs an associated, programmatically-linked label:

```
// [x] Correct  -  <label htmlFor> is linked to <input id> (recall Part 15.3's camelCase JSX attributes  -  htmlFor
<label htmlFor="email">Email</label>
<input id="email" type="email" value={email} onChange={handleChange} />
```

Why this linkage matters beyond visual proximity: a screen reader announces the label's text when the associated input receives focus, so a visually-impaired user knows what they're being asked to fill in. It also means clicking the label text itself focuses the input - a small but genuinely helpful usability improvement for everyone, not just screen reader users.

33.5 Keyboard Navigation and Focus

Every interactive element (buttons, links, inputs) should be reachable and operable using only the keyboard - Tab to move between elements, Enter/Space to activate. Recall 33.2/33.3: using real semantic elements gets you this for free.

Managing focus programmatically - recall Part 24.3's `useRef` pattern:

```
function Modal({ isOpen, onClose }) {
  const closeButtonRef = useRef(null);

  useEffect(() => {
    if (isOpen) {
      closeButtonRef.current?.focus(); // move focus INTO the modal when it opens
    }
  }, [isOpen]);

  return isOpen ? (
    <div role="dialog" aria-modal="true">
      <button ref={closeButtonRef} onClick={onClose}>Close</button>
      {/* modal content */}
```

```
    </div>
  ) : null;
}
```

Why this matters: a sighted mouse user visually sees a modal appear. A keyboard/screen-reader user needs focus to be programmatically moved into the new modal content - otherwise, they'd remain "stuck" tabbing through whatever was behind the modal, with no indication anything new has appeared.

33.6 ARIA Basics

ARIA (Accessible Rich Internet Applications) attributes supplement HTML when semantic elements alone aren't sufficient to convey meaning - most commonly needed for custom, JavaScript-driven UI patterns that don't have a perfect native HTML equivalent (a custom dropdown, a tab interface, a toast notification).

```
<button aria-label="Close menu">x</button> {/* provides an accessible name when visible text isn't descriptive enoug

<div role="alert">Your changes have been saved.</div> {/* announces dynamically-appearing content to screen readers

<button aria-expanded={isOpen} onClick={toggle}>Menu</button> {/* communicates a toggle's current state */}
```

⚠ The first rule of ARIA (a genuinely well-known accessibility community principle): "No ARIA is better than bad ARIA." Prefer a correct native semantic element (33.2) whenever one exists - only reach for ARIA attributes to fill gaps that native HTML genuinely cannot express on its own.

33.7 Alt Text for Images

```
<img src="/product.jpg" alt="Wireless mouse in black, top view" /> {/* describes the CONTENT/purpose, not just "imag

<img src="/decorative-swirl.png" alt="" /> {/* empty alt for PURELY decorative images  -  tells screen readers to sk
```

Why an empty `alt=""` (not a missing `alt` attribute) is correct for decorative images: a missing `alt` attribute causes some screen readers to announce the image's file name instead - genuinely unhelpful noise. An explicit, empty `alt=""` correctly signals "this image carries no meaningful content - skip it."

33.8 Color Contrast (Conceptual)

Text needs sufficient contrast against its background to be readable by users with low vision or color blindness. The Web Content Accessibility Guidelines (WCAG) define specific minimum contrast ratios (commonly cited: 4.5:1 for normal text). You don't need to memorize exact ratios for this course, but should know this is a checkable, measurable requirement - browser DevTools' Elements/Accessibility panels typically include contrast-ratio checking tools directly.

Knowledge Check - Part 33

1. Why does using a real `<button>` element provide accessibility benefits "for free" compared to a `<div onClick>`?
2. What does linking a `<label>` to an `<input>` via `htmlFor`/`id` provide beyond visual layout?

3 State the "first rule of ARIA," and explain why it makes sense.

4 When is an empty `alt=""` the correct choice, versus a descriptive `alt` value?

> View solutions

1 Native `<button>` elements come with built-in keyboard focusability (reachable via Tab), standard activation behavior (Enter and Space both trigger it), and correct screen-reader announcement as a "button" - all implemented natively by the browser. A `<div>` has none of this by default and would require substantial manual work (tabIndex, keyboard event handlers, ARIA roles) to replicate correctly.

2 It creates a programmatic association a screen reader can use to announce the label's text when the input receives focus, and it makes clicking the label text itself focus the associated input - neither of which happens automatically from visual proximity/CSS positioning alone.

3 "No ARIA is better than bad ARIA" - prefer correct native semantic HTML elements whenever one exists, and reach for ARIA attributes only to fill genuine gaps native HTML can't express. This makes sense because incorrectly applied ARIA can override or conflict with an element's natural semantics, actively making the experience worse for assistive technology users than having no ARIA at all.

4 Use a descriptive `alt` value for images that convey meaningful content the user needs to understand. Use an empty `alt=""` specifically for purely decorative images that carry no informational content - this explicitly tells screen readers to skip announcing it entirely, rather than announcing an unhelpful file name (which happens if the `alt` attribute is missing entirely).

Next: Part 34 - Security

PART 34 - SECURITY [Intermediate][Advanced]

This Part consolidates and extends security warnings that have appeared throughout the course (Part 6.2's XSS warning, Part 11.9's API key warning, Part 11.13 Project 10's token storage caveat) into one dedicated, complete treatment.

34.1 Cross-Site Scripting (XSS) - Full Recap and React Context

Recall Part 6.2 in full: XSS occurs when untrusted content is rendered in a way that lets it execute as actual code/markup rather than being displayed as inert text.

The good news: JSX protects you from XSS automatically, by default.

```
function Comment({ text }) {
  return <p>{text}</p>; // even if text is "<script>stealCookies()</script>", this renders as LITERAL TEXT, safely
}
```

Why this is safe, by default: recall Part 15.2 - `{text}` in JSX is treated as a value to display, and React automatically escapes it before inserting it into the DOM (conceptually similar to always using

`textContent`, per Part 6.2's safe pattern) - it never interprets that string as markup to be parsed and executed.

```
function BlogPost({ htmlContent }) {
  return <div dangerouslySetInnerHTML={{ __html: htmlContent }} />; // deliberately, explicitly UNSAFE
}
```

Why the name is deliberately alarming: React's team intentionally named this API `dangerouslySetInnerHTML` (not simply `innerHTML`) specifically so that anyone using it has to type the word "dangerous," forcing a moment of deliberate consideration. It's the direct equivalent of Part 6.2's `element.innerHTML = ...` warning - if `htmlContent` ever contains attacker-controlled input, this is a full XSS vulnerability, exactly as covered in Part 6.2.

When it's legitimately needed: rendering genuinely trusted HTML content - e.g., content from a trusted CMS/rich-text editor that your own backend has already sanitized, or static content you wrote yourself. Never use it to render arbitrary user-submitted or third-party content without passing it through a dedicated, well-tested sanitization library first (which strips dangerous markup like `<script>` tags and event-handler attributes while preserving safe formatting).

34.2 API Keys and Environment Variables - Full Recap

Fully covered in Part 11.9 - the core rule carries over unchanged into React: never embed a secret API key in frontend code. One React/Vite-specific addition worth stating precisely:

```
// .env file
VITE_API_URL=https://api.example.com   // [x] fine  -  not actually secret, just configuration
VITE_SECRET_KEY=sk_live_abc123           // [x] STILL WRONG, even prefixed with VITE_!
```

Why even a Vite environment variable is not a place for real secrets: Vite (and similar frontend build tools) embeds any environment variable prefixed with `VITE_` directly into your built JavaScript bundle, at build time - recall Part 0.3/11.9, everything in that bundle ships to every visitor's browser and is fully readable. Frontend environment variables are appropriate for non-secret configuration (like a public API base URL) - they provide zero additional protection for anything that's genuinely sensitive. Real secrets belong exclusively on a server you control, as established fully in Part 11.9's architecture diagram.

34.3 Authentication Tokens and localStorage - The Honest Tradeoff

Recall Part 11.13, Project 10's flagged caveat. Stated fully here:

Storage location	Readable by JavaScript (vulnerable to XSS)?	Automatically sent with requests?
`localStorage` (Part 8.1)	[x] Yes - any script running on the page can read it	[x] No - must be manually attached (recall Part 11.6's Bearer token pattern)
`HttpOnly` cookie (Part 8.3)	[x] No - inaccessible to JavaScript entirely	[x] Yes - automatically included by the browser

The tradeoff, stated honestly: `localStorage` is simple and gives you full manual control, but if your application ever has an XSS vulnerability (34.1) anywhere, an attacker's injected script can directly read and exfiltrate the token. `HttpOnly` cookies are immune to this specific attack vector (JavaScript literally cannot read them), but introduce their own separate concern - CSRF (Cross-Site Request Forgery), where a malicious site can trick a browser into sending an authenticated request using the automatically-attached cookie, without needing to read the token's value at all (CSRF has its own well-established mitigation techniques, like CSRF tokens, beyond this course's scope, but worth knowing the concern exists). There's no single universally "correct" answer - it's a genuine security tradeoff that real engineering teams weigh deliberately, and this course presents both honestly rather than as a simple right-vs-wrong choice.

34.4 CORS Misconceptions - Recap

Recall Part 11.8's full, precise treatment. The single most important misconception worth restating directly: CORS is not a security feature that protects your API/server from unauthorized access in general - it specifically protects browser users from malicious websites making unauthorized cross-origin requests using their browser session. A non-browser client (a script, `curl`, another server) is completely unaffected by CORS restrictions. If you need to actually restrict who can access your API, that requires real server-side authentication/authorization (checking credentials, tokens, permissions) - CORS headers alone do not provide this.

34.5 HTTPS

Why it matters: HTTP traffic (without the "S" - recall Part 0.4/0.6) is sent in plain text across the network - any intermediary (a malicious actor on the same public WiFi, an ISP, anyone positioned between the client and server) can read or even modify it in transit. HTTPS encrypts this traffic, so even if intercepted, it can't be read or tampered with. Every production website handling any sensitive data (which is nearly all of them, given login forms, personal data, etc.) must use HTTPS. Modern hosting platforms (previewed in Part 37) typically provide free, automatic HTTPS certificates, making this close to a zero-effort requirement today - but understanding why it's non-negotiable matters.

34.6 Input Validation - Frontend vs Backend

Recall Part 7.6's honest caveat, restated as a firm principle: frontend validation (Part 7, Part 20.5) exists for user experience - immediate, helpful feedback. It provides zero actual security, because (recall Part 0.3) any client-side check can be trivially bypassed by anyone sending requests directly to your API, without going through your frontend UI at all.

The rule, stated without qualification: every piece of validation that matters for security or data integrity must ALSO be enforced on the server, regardless of what the frontend already checks. Frontend validation is a convenience layer on top of mandatory backend validation - never a substitute for it.

Knowledge Check - Part 34

1 Why does JSX's `{value}` syntax protect against XSS by default, and what specifically makes `dangerouslySetInnerHTML` an exception to that protection?

2. Why doesn't prefixing an environment variable with `VITE_` make it safe to store a real secret in?
3. State the security tradeoff between storing an auth token in `localStorage` versus an `HttpOnly` cookie, honestly, without declaring one universally "correct."
4. Why is frontend form validation alone never sufficient for actual security?

> View solutions

1. JSX's `{value}` syntax automatically escapes interpolated values before inserting them into the DOM, treating them as literal text rather than parsing them as markup - conceptually equivalent to always using `textContent` (Part 6.2). `dangerouslySetInnerHTML` deliberately bypasses this protection, directly inserting raw HTML that WILL be parsed and can execute embedded scripts if the content is attacker-controlled.
2. Vite embeds any `VITE_`-prefixed environment variable directly into the built JavaScript bundle at build time, and that entire bundle is downloaded to and fully readable by every visitor's browser - the `VITE_` prefix only controls which variables get exposed to the frontend build; it provides no actual secrecy or access control.
3. `localStorage` is simple and fully under your manual control, but is readable by any JavaScript running on the page - so an XSS vulnerability anywhere in the app could let an attacker steal the token directly. `HttpOnly` cookies are inaccessible to JavaScript entirely (immune to that specific theft vector), but are automatically attached to requests by the browser, introducing CSRF as a separate concern to defend against instead. Neither option eliminates all risk; each trades one class of vulnerability for a different one.
4. Because any client-side check can be trivially bypassed by an attacker (or just a technically capable user) sending requests directly to the API/server, entirely skipping the frontend UI and its validation logic - the server has no way to know or enforce that a request "went through" the frontend's checks, so any validation that actually matters for security or data integrity must be independently enforced there.

Next: Part 35 - Testing

PART 35 - TESTING [Intermediate][Advanced]

35.1 The Three Levels of Testing

Type	Tests...	Speed	Example
Unit tests	A single function/component in isolation	Very fast	Does `formatPrice(1500)` return `"₹1,500"`?

Type	Tests...	Speed	Example
Integration tests	Several pieces working together	Fast-moderate	Does submitting a form correctly update the displayed list?
End-to-end (E2E) tests	The entire application, in a real (or real-like) browser, simulating actual user behavior	Slow	Does a full signup -> login -> checkout flow work correctly?

Why you need all three, at different proportions (a widely-cited mental model is a "testing pyramid" - many fast unit tests, fewer integration tests, very few E2E tests): unit tests are fast and pinpoint exactly what broke, but can't catch issues in how pieces interact. E2E tests catch real, holistic problems but are slow, more brittle (small unrelated UI changes can break them), and expensive to run frequently. Integration tests sit in the middle, testing realistic combinations without the full cost of true E2E testing.

35.2 JavaScript Unit Tests with Vitest

Vitest is the current standard testing tool for Vite-based projects (recall Part 13.7/14.6 - it's built by the same ecosystem, designed to integrate seamlessly).

```
npm install -D vitest

// mathUtils.js
export function add(a, b) {
  return a + b;
}

// mathUtils.test.js
import { describe, it, expect } from "vitest";
import { add } from "./mathUtils.js";

describe("add", () => {
  it("adds two positive numbers", () => {
    expect(add(2, 3)).toBe(5);
  });

  it("handles negative numbers", () => {
    expect(add(-1, -1)).toBe(-2);
  });
});
```

Reading this, piece by piece:

- `describe(name, fn)` - groups related tests together, purely for organization/readable output.
- `it(name, fn)` (or its alias `test`) - defines one individual test case, with a description of what it verifies.
- `expect(value).toBe(expected)` - an assertion: states what you expect to be true, and fails the test loudly if it isn't. `.toBe()` checks strict equality (recall Part 1.8's `===`); other matchers exist for different needs (`.toEqual()` for deep object/array equality, `.toBeTruthy()`, `.toThrow()` for expecting a function to throw, etc.).

```
npx vitest run   # run all tests once
npx vitest         # run in watch mode, re-running automatically as files change
```

35.3 Testing Pure Functions (Utils)

Recall Part 3.5 and Part 29.3's `utils/` layer - pure functions are, by design, the easiest things to test, precisely because they have no side effects and no dependencies to mock:

```
// utils/validators.js
export function isValidEmail(email) {
  return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email); // recall Part 7.3
}

// utils/validators.test.js
import { describe, it, expect } from "vitest";
import { isValidEmail } from "./validators.js";

describe("isValidEmail", () => {
  it("accepts a valid email", () => {
    expect(isValidEmail("test@example.com")).toBe(true);
  });

  it("rejects a string with no @", () => {
    expect(isValidEmail("not-an-email")).toBe(false);
  });

  it("rejects an empty string", () => {
    expect(isValidEmail("")).toBe(false);
  });
});
```

35.4 React Component Testing with React Testing Library

React Testing Library (RTL) is the standard tool for testing React components, built around a deliberate philosophy: test components the way a real user would interact with them (finding elements by visible text/role, clicking, typing) - not by reaching into a component's internal implementation details (state variable names, internal function names), which are irrelevant to whether the component actually works for a user.

```
npm install -D @testing-library/react @testing-library/jest-dom @testing-library/user-event

// Counter.jsx
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}

// Counter.test.jsx
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import { describe, it, expect } from "vitest";
import Counter from "./Counter.jsx";

describe("Counter", () => {
  it("starts at 0", () => {
    render(<Counter />);
    expect(screen.getByText("Count: 0")).toBeInTheDocument();
  });
```

```
  it("increments when the button is clicked", async () => {
    const user = userEvent.setup();
    render(<Counter />);

    await user.click(screen.getByRole("button", { name: "Increment" }));

    expect(screen.getByText("Count: 1")).toBeInTheDocument();
  });
});
```

Explaining the key pieces:

- `render(<Counter />)` - renders the component into a virtual DOM environment for testing.
- `screen.getByText(...)` / `screen.getByRole(...)` - find elements the way a real user would perceive them (by visible text, or by accessible role - recall Part 33's accessibility material; a `<button>` has the implicit role `"button"`, directly connecting accessible markup to testable markup - this is not a coincidence: accessible code is, by nature, easier to correctly locate and test).
- `userEvent.setup()` + `user.click(...)` - simulates a real user interaction, more realistically than lower-level event-firing utilities.
- `expect(...).toBeInTheDocument()` - an assertion (from the `jest-dom` matcher library) confirming an element is actually present and rendered.

  Why querying by role/text (not by implementation details like a CSS class or internal variable name) is the deliberate, recommended philosophy: a test that queries `screen.getByRole("button", { name: "Increment" })` will keep passing even if you completely refactor the component's internals (switch from `useState` to `useReducer`, rename internal variables) - as long as the user-facing behavior stays the same. A test that reached into internal implementation details would break on any refactor, even ones that don't actually change behavior - testing implementation details produces brittle tests that discourage healthy refactoring.

35.5 Mocking API Calls (Conceptual)

What "mocking" means: replacing a real dependency (like an actual network request) with a fake, controlled substitute during a test - so tests run fast, deterministically, and without depending on a real, possibly-unreliable external server.

```
// Conceptual illustration
import { vi } from "vitest";

vi.stubGlobal("fetch", vi.fn(() =>
  Promise.resolve({
    ok: true,
    json: () => Promise.resolve([{ id: 1, name: "Test Product" }])
  })
));

// Now any component under test that calls fetch() receives this fake, controlled response
// instead of making a real network request  -  recall Part 10/11's fetch mechanics, now faked deliberately.
```

Why mocking matters for testing components that fetch data (recall Part 22's entire pattern): a real test shouldn't depend on an actual external API being online, fast, and returning consistent data - that would make tests slow, flaky (sometimes passing, sometimes failing, for reasons unrelated to your

code), and dependent on network access. Mocking lets you test "does my component correctly show a loading state, then the data, then handle an error" deterministically, by controlling exactly what the fake `fetch` returns (including deliberately simulating failures, to test your error-handling paths from Part 22.2).

Knowledge Check - Part 35

1. What's the practical tradeoff between unit tests, integration tests, and E2E tests, in terms of speed versus what they actually verify?
2. Why does React Testing Library encourage querying by role/text rather than by CSS class names or internal implementation details?
3. Why does testing a pure function (recall Part 3.5) tend to be simpler than testing a component with side effects?
4. Why is mocking `fetch` important for testing a component's loading/error/success states reliably?

> View solutions

1. Unit tests are fast and pinpoint failures precisely but only verify isolated pieces, missing integration issues. E2E tests verify the whole real application working together but are slow and more brittle to unrelated changes. Integration tests are a middle ground - testing realistic combinations of pieces without the full cost/slowness of true E2E testing.
2. Because it tests components the way an actual user experiences and interacts with them, producing tests that remain valid across internal refactors that don't change real behavior - while tests coupled to implementation details (like state variable names) break on any refactor, even ones that preserve correct behavior, discouraging healthy code changes.
3. A pure function's output depends only on its inputs, with no side effects or external dependencies to set up, fake, or clean up - you can call it directly with test inputs and assert on its return value with no additional infrastructure, unlike testing a component with rendering, DOM interaction, or network side effects involved.
4. Because a real network request is slow, depends on an external server actually being available and returning specific data, and can't easily be made to deliberately fail on demand - mocking lets a test control exactly what `fetch` returns (success with specific data, or a simulated failure), making it possible to reliably and deterministically test every state (loading, success, error) a component can be in.

Next: Part 36 - TypeScript Introduction

PART 36 - TYPESCRIPT INTRODUCTION [Intermediate][Advanced]

JavaScript remains this course's primary language throughout - this Part is a genuinely useful on-ramp to TypeScript, not a replacement for anything taught earlier. Everything here builds on JavaScript concepts you already know deeply.

36.1 Why TypeScript Exists

The problem it solves

Recall every `TypeError: Cannot read properties of undefined` you've encountered throughout this course (Part 9, Part 22). Many of these bugs come from JavaScript's dynamic typing (recall Part 1.6) - a variable's type isn't checked until the exact moment the problematic line actually runs. If that line only runs for a rare, untested code path, the bug can hide undetected until a real user hits it in production.

TypeScript is a superset of JavaScript that adds an optional static type system - you describe what shape your data should be, and a separate tool (the TypeScript compiler) checks your entire codebase for type mismatches before you ever run it, catching an entire category of bugs at write-time/build-time instead of at runtime.

```
function add(a: number, b: number): number {
  return a + b;
}

add(2, "3"); // [x] TypeScript catches this IMMEDIATELY, in your editor, before you even save/run the file
```

Mental model: JavaScript catches type errors the way a driving test catches mistakes - by watching you actually drive and pointing out problems as they happen. TypeScript catches them the way a car's pre-trip inspection does - checking everything is correctly connected before you ever turn the key, preventing certain classes of failure from ever happening on the road at all.

36.2 Basic Types

```
let age: number = 17;
let name: string = "Ammaar";
let isStudent: boolean = true;
let tags: string[] = ["react", "javascript"]; // array of strings
let scores: number[] = [95, 88, 91];
```

TypeScript can often infer types automatically, without you writing them explicitly:

```
let age = 17; // TypeScript infers this is `number` automatically  -  no annotation needed
age = "seventeen"; // [x] Error: Type 'string' is not assignable to type 'number'
```

36.3 Function Types

```
function greet(name: string): string { // parameter type, and a RETURN type annotation
  return `Hello, ${name}`;
}

function logMessage(message: string): void { // void = this function returns nothing meaningful
  console.log(message);
}
```

Recall Part 3.3's default parameters - they work identically, with types:

```
function greet(name: string = "Guest"): string {
  return `Hello, ${name}`;
}
```

36.4 Object Types and Interfaces

```
interface User {
  id: number;
  name: string;
  email: string;
  isAdmin?: boolean; // the ? marks an OPTIONAL property  -  recall Part 17.4's default prop concept, now type-enfor
}

function printUser(user: User): void {
  console.log(user.name);
}

printUser({ id: 1, name: "Ammaar", email: "ammaar@example.com" }); // [x] valid  -  isAdmin is optional
printUser({ id: 1, name: "Ammaar" }); // [x] Error: missing required property 'email'
```

Why this directly recalls Part 4.7's object destructuring: an `interface` is essentially a formal, enforced description of exactly the kind of object shape you were already informally relying on throughout Part 4 and Part 17 - TypeScript just makes that shape explicit and checks every usage against it.

```
type ID = number | string; // a UNION type  -  can be EITHER a number OR a string

function findUser(id: ID) {
  // ...
}

findUser(42);      // [x] valid
findUser("abc123"); // [x] also valid
findUser(true);      // [x] Error
```

`interface` vs `type` - practical guidance: both can describe object shapes, and the distinction has some genuine technical nuance beyond this course's scope, but a common practical convention: use `interface` for object shapes (especially ones that might be extended later), and `type` for unions, primitives, or more complex composed types. Many teams use them close to interchangeably for simple object shapes.

36.5 Union Types

```
type Status = "idle" | "loading" | "success" | "error"; // recall Part 22.2's UI state model  -  now literally type-

function renderStatus(status: Status) {
  // TypeScript KNOWS status can only ever be one of these 4 exact strings
}

renderStatus("done"); // [x] Error: Argument of type '"done"' is not assignable to type 'Status'
```

Why this is a genuinely powerful connection back to Part 22.2: recall the five-state UI model (idle/loading/success/empty/error) from Part 22.2 - in plain JavaScript, nothing stops you from accidentally typing `"loadingg"` somewhere and silently breaking your conditional rendering logic. A TypeScript union type makes that exact category of typo impossible to compile, catching it immediately rather than as a confusing runtime UI bug.

36.6 Generics (Basics)

What they are

A way to write a function or type that works with multiple types, while still preserving type-safety, rather than either committing to one specific type or giving up type-checking entirely.

```
function firstElement<T>(array: T[]): T { // T is a placeholder  -  "whatever type this array actually contains"
  return array[0];
}

const firstNumber = firstElement([1, 2, 3]);       // TypeScript infers T = number, so firstNumber is typed as `numb
const firstString = firstElement(["a", "b", "c"]); // T = string here instead
```

Mental model, connecting directly to your existing deep understanding of `.map()`, `.filter()`, `.reduce()` from Part 4.4: generics let TypeScript correctly track "whatever type went in, that same (or a related) type comes back out" - precisely the relationship you already intuitively understand from `array.map(callback)` always preserving useful type information about its input and output, just now made explicit and enforced by the type system itself.

36.7 React Props with TypeScript

```
interface ButtonProps {
  label: string;
  onClick: () => void;      // a function type: takes no arguments, returns nothing
  variant?: "primary" | "secondary"; // optional, and constrained to exactly these two strings
}

function Button({ label, onClick, variant = "primary" }: ButtonProps) {
  return <button className={variant} onClick={onClick}>{label}</button>;
}

<Button label="Submit" onClick={() => console.log("clicked")} />          {/* [x] valid */}
<Button label="Submit" onClick={() => console.log("clicked")} variant="danger" /> {/* [x] Error: "danger" isn't a va
```

Why this is a direct, powerful extension of everything you learned in Part 17: recall Part 17.6's entire discussion of props as a contract between parent and child. TypeScript makes that contract enforced and checked, not just a convention you have to remember and get right by careful reading - a typo in a prop name, a missing required prop, or an invalid value for a constrained prop (like `variant` above) is caught immediately, at write-time, before you ever click around in the browser to discover it broke.

36.8 State with TypeScript

```
const [count, setCount] = useState<number>(0); // explicit type parameter, though often inferable automatically:
const [count2, setCount2] = useState(0); // TypeScript infers `number` automatically from the initial value

// When the initial value doesn't fully convey the eventual type (recall Part 22.1's `data` starting as `null`):
interface UserData {
  id: number;
  name: string;
}

const [user, setUser] = useState<UserData | null>(null); // explicitly: "either a UserData object, or null"
```

Why the explicit `<UserData | null>` is necessary here specifically, connecting to Part 22.1: if you just wrote `useState(null)`, TypeScript would infer the type as simply `null` - permanently, forever - and

would then error the moment you tried to later call `setUser({ id: 1, name: "Ammaar" })`, since that doesn't match the inferred `null` type. The explicit union type tells TypeScript the full range of what this state will actually hold over the component's lifetime, exactly mirroring the conceptual reasoning you already did in Part 22.1 when choosing `null` specifically to represent "no data yet" - TypeScript just requires you to state that reasoning formally.

Knowledge Check - Part 36

1. What core problem does TypeScript solve, in terms of WHEN type-related bugs are caught, compared to plain JavaScript?
2. Why does `interface User { isAdmin?: boolean }`'s `?` directly recall a concept from Part 17.4?
3. Explain, using the UI state model from Part 22.2, why a union type like `"idle" | "loading" | "success" | "error"` is more useful than a plain `string` type for that same purpose.
4. Why must `useState<UserData | null>(null)` be given an explicit type parameter, rather than relying on inference from just `null`?

> View solutions

1. TypeScript catches type-related mismatches at write-time/compile-time - before the code is ever run - via a separate type-checking tool, rather than JavaScript's default of only discovering such errors at runtime, when the specific problematic line of code actually executes (which might be a rarely-hit code path that goes undetected until a real user encounters it).
2. Part 17.4 covered default prop values, where a prop might not be provided by the caller and falls back to a default. TypeScript's `?` on an interface property marks that property as similarly optional - an object satisfying this interface doesn't need to include it - directly mirroring the same "this might not be provided" concept, now formally type-checked.
3. A plain `string` type would allow ANY string value, including typos like `"loadingg"`, with no compile-time warning - such a typo would only be discovered as a confusing runtime bug in the UI's conditional rendering logic. A union type restricts the value to EXACTLY the listed set of valid strings, so any typo or invalid value is caught immediately at compile-time, before the code ever runs.
4. Because TypeScript would otherwise infer the state's type purely from the single initial value passed to `useState` - inferring just `null` - and would then reject any later attempt to set the state to an actual `UserData` object, since that wouldn't match the inferred `null`-only type. The explicit `<UserData | null>` tells TypeScript the complete range of values this state will hold over the component's full lifetime, not just its initial value.

Next: Part 37 - Production Concepts

PART 37 - PRODUCTION CONCEPTS [Intermediate][Advanced]

37.1 Development Builds vs Production Builds

Recall Part 14.6's `npm run dev` - this starts a development server: fast rebuilds, helpful error overlays, unminified code with readable variable names, and extra runtime checks (like `<StrictMode>`'s double-invocation behavior, Part 14.6). None of this is appropriate to actually ship to real users - it's slower and larger than necessary, purely to optimize for a smooth development experience.

```
npm run build
```

What this command does: runs Vite's production build process - it bundles all your JavaScript/CSS into a small number of optimized files, minifies them (stripping whitespace, shortening variable names, removing comments - all things that make code human-readable but add unnecessary size for a machine that just needs to execute it), removes development-only warnings/checks, and produces a `dist/` folder ready to be deployed.

37.2 Environment Variables in Production

Recall Part 34.2's security treatment. Practically, a project typically has multiple environment configurations:

```
.env.development    # used during `npm run dev`
.env.production       # used during `npm run build`

# .env.production
VITE_API_URL=https://api.myrealapp.com

# .env.development
VITE_API_URL=http://localhost:3000
```

```
const apiUrl = import.meta.env.VITE_API_URL; // Vite's way of reading these  -  automatically picks the right file
```

Why this matters practically: your frontend code can be written once, referencing `apiUrl`, while automatically pointing at your local development backend during development and your real production backend once built for production - without ever hardcoding either URL directly into your component code.

37.3 SPA Routing Issues in Production - A Real, Common Gotcha

The problem: recall Part 28's client-side routing - routes like `/products/42` are handled entirely by JavaScript, after the initial page loads. But what happens if a user directly navigates to `https://myapp.com/products/42` (types it directly, refreshes the page, or a bookmarked link) - before any of your JavaScript has had a chance to run?

The web server receives a request for the literal path `/products/42`. If it's a naive static file server, it looks for an actual file at that path, finds nothing (recall Part 0.14 - an SPA only has one real HTML file), and returns a `404 Not Found` - even though this is a perfectly valid route within your React application.

The standard fix: configure your hosting/server to perform what's often called a "SPA fallback" or "rewrite" - any request that doesn't match a real static file (like an actual image or CSS file) should be served your single `index.html` file instead of a 404. Once that `index.html` loads, your JavaScript (including React Router, Part 28) takes over, reads the current URL (`/products/42`), and correctly renders the matching route - client-side, exactly as it would have if the user had navigated there by clicking a `Link` from within the already-loaded app. Virtually every modern static-hosting platform (see 37.5) provides a simple, well-documented way to configure this - it's a near-universal requirement for deploying any SPA, not something specific to React.

37.4 API URLs Across Environments

Directly connecting 37.2 and Part 29.3's service layer: your `services/` layer functions should reference the environment-provided `apiUrl`, never a hardcoded string:

```
// services/apiClient.js
const BASE_URL = import.meta.env.VITE_API_URL;

export async function apiRequest(path, options = {}) {
  // ... the full helper from Part 11.10, using BASE_URL + path
}
```

This ensures the exact same, unmodified application code correctly targets a local backend during development and the real production backend once deployed - without ever needing to manually edit source code between environments.

37.5 Deployment (Conceptual, Provider-Agnostic)

The general shape of deploying a React SPA:

1. Run `npm run build` (37.1), producing a `dist/` folder of static files (HTML, JS, CSS, images).
2. Upload/deploy that `dist/` folder to a static hosting service, or serve it from your own server (a VPS running something like Nginx).
3. Configure SPA fallback routing (37.3).
4. Configure production environment variables (37.2) at build time (since environment variables are baked into the bundle at build time - recall Part 34.2's exact mechanism - they must be set correctly before running `npm run build`, not after).

Why this course doesn't lock you into one specific provider: the underlying concepts (build, deploy static files, configure SPA fallback, set environment variables) are consistent across virtually every modern static-hosting platform and VPS setup - once you understand the concepts in this Part, applying them to whichever specific platform you or your team chooses is a matter of following that platform's documentation for these same, universal steps.

37.6 Debugging Production Issues

Why production bugs can be genuinely harder to diagnose than development ones: minified code (37.1) means stack traces (Part 9.2) show cryptic, shortened variable/function names instead of your original, readable source - and `<StrictMode>`'s extra development-only warnings (Part 14.6) are absent in production, so certain classes of bugs that would have been caught loudly during development might only surface subtly in production.

Source maps are the standard solution: a special file (generated automatically by your build tool) that maps minified production code back to your original, readable source code - letting browser DevTools display a real, useful stack trace even for a production error, without shipping the actual unminified source to users.

37.7 Logging in Production

Recall Part 9.3's `console.error`. In production, you generally don't have direct access to users' browser consoles - real applications typically integrate an error-tracking/logging service that automatically captures uncaught errors (including ones caught by Error Boundaries, Part 32.3-32.5) from real users' browsers and reports them to a dashboard developers can review - turning "a user silently experienced a bug and never told us" into "we got an automatic, detailed report the moment it happened." This is beyond this course's scope to implement, but important to know exists as the standard, expected practice for any real production application.

37.8 Frontend/Backend Separation - Full-Circle Recap

This entire course has consistently maintained the frontend/backend boundary from Part 0.3. In production, this typically manifests as: your React SPA is deployed as static files (37.5) on one hosting service, while your actual backend API (which this course intentionally didn't build, per Part 0.3's scope) runs as a separate deployed service, and your frontend's `apiUrl` (37.2/37.4) simply points at wherever that backend actually lives. This separation - established conceptually in the very first Part of this course - is exactly what makes independent deployment, scaling, and development of the frontend and backend possible.

Knowledge Check - Part 37

1. Why is a development build inappropriate to actually ship to real users?
2. Explain the SPA routing 404 problem in your own words, and why the "serve `index.html` for any unmatched path" fix works.
3. Why must production environment variables be set correctly BEFORE running `npm run build`, not after?
4. What do source maps provide, and why are they necessary specifically because of minification?

> View solutions

1. Development builds prioritize fast rebuilds, readable unminified code, and helpful debugging tools/warnings - none of which real end users need, and all of which make the resulting files larger and slower than necessary. Production builds optimize instead for what users actually need: small, fast-loading, minified files.

2. A web server receiving a direct request for a client-side route (like `/products/42`) looks for an actual file at that path and finds none (since an SPA has only one real HTML file), returning a 404 - even though that's a valid route within the React application once its JavaScript has loaded and taken over routing. Serving `index.html` for any unmatched path works because it lets the SPA's JavaScript load first, at which point client-side routing (React Router) reads the current URL and correctly renders the matching route itself.

3. Because environment variables prefixed for frontend use (recall Part 34.2) are embedded directly into the built JavaScript bundle AT BUILD TIME - once `npm run build` has run, those values are baked into the static output files; changing an environment variable afterward has no effect on already-built files, requiring a fresh build with the correct values in place beforehand.

4. Source maps map minified, production-optimized code back to the original, readable source code, allowing browser DevTools to display useful stack traces (with real function/variable names and file locations) for production errors - necessary because minification (which strips/renames things for size) would otherwise make production stack traces cryptic and very difficult to debug.

Next: Project-Based Learning: The Full Project Catalogue

PROJECT-BASED LEARNING - THE FULL PROJECT CATALOGUE [Beginner][Intermediate][Advanced][Expert]

How to Use This Catalogue

Everything up to this point has taught you individual concepts in isolation, each with its own small examples. Projects are where those concepts get combined - a real task rarely calls for "just closures" or "just `useState`"; it calls for reading a spec, deciding which of everything you know applies, and assembling those pieces into something that actually works end-to-end.

The format for every project below is deliberately the same, every time:

1. Requirements - what the finished project must actually do, stated precisely (not vague: "make a todo app" is not a requirement; "add a todo, mark it complete, delete it, and persist the list across a page reload" is).

2. Concepts this exercises - an explicit list of which earlier Parts this project draws on, so you can go back and review anything rusty before attempting it, not after getting stuck.

3. Hints - a small, deliberately incomplete nudge in the right direction - enough to unstick you without handing you the answer.
4. Solution - inside a `<details>` block, so it stays hidden until you choose to look. Attempt the project yourself first. Reading a solution before attempting the project yourself teaches you to recognize code, not to produce it - a meaningfully different (and much weaker) skill.
5. Explanation - after the solution, walking through the non-obvious decisions in it (why this particular approach, what alternative would have gone wrong, which earlier Part's concept each piece leans on).

   A note on difficulty progression across tiers: the six tiers below (Beginner, DOM, Async/API, React Beginner, Intermediate React, Advanced React) are ordered to match the course's own teaching order - attempt them in order the first time through. Projects 17-24 (Async/API) deliberately overlap in spirit with the 10 projects already built inside Part 11.13's API Mastery Challenge - if you already built thorough versions of similar projects there, treat these as a chance to rebuild the same kind of project faster and cleaner, now that the patterns are second nature, rather than entirely unfamiliar territory.

TIER 1 - BEGINNER PROJECTS (1-8) [Beginner]

Focus: pure JavaScript fundamentals - variables, functions, control flow, basic DOM output (usually just `console.log`, or a single simple DOM update). No async, no complex state.

Requirements:

- Track a numeric count starting at 0.
- Functions to increment, decrement, and reset the count.
- Print the current count to the console after every change.
- The count must never be allowed to go below 0 (decrementing at 0 does nothing, and logs a message explaining why).

Concepts this exercises: Part 1 (variables), Part 2 (conditionals), Part 3 (functions, especially closures if you choose the closure-based approach below).

Hints:

- You have two structurally different ways to hold the `count` variable: as a plain top-level `let`, mutated directly inside each function; or enclosed inside a single outer function that returns an object of methods sharing access to a private `count` via closure (recall Part 3.8's closure counter example - this project is that example, extended).
- Think about why you might prefer the closure version even though the plain top-level version is shorter: what does the plain version fail to protect against, that closure encapsulation prevents entirely?

> View solution

```
function createCounter() {
  let count = 0; // private  -  recall Part 3.8, inaccessible from outside this function except via the returned met
```

```
  return {
    increment() {
      count++;
      console.log(`Count: ${count}`);
    },
    decrement() {
      if (count === 0) {
        console.log("Cannot decrement below 0.");
        return;
      }
      count--;
      console.log(`Count: ${count}`);
    },
    reset() {
      count = 0;
      console.log(`Count: ${count}`);
    }
  };
}

const counter = createCounter();
counter.increment(); // Count: 1
counter.increment(); // Count: 2
counter.decrement(); // Count: 1
counter.reset();       // Count: 0
counter.decrement(); // Cannot decrement below 0.
```

Explanation: the closure-based version (recall Part 3.8 in full) makes `count` genuinely private - there is no `counter.count` property anyone could reach in from outside and set to an invalid value like `-500`, bypassing your validation logic entirely. A plain top-level `let count = 0` with separate `increment()`/`decrement()` functions would work too, but leaves `count` sitting in the enclosing scope, freely reassignable by any other code sharing that scope - for a single small script this barely matters, but the habit of reaching for encapsulation here is exactly the same habit that matters enormously once you're building real applications with many interacting pieces (and is precisely the instinct that later motivates React's component-local state, Part 18).

Requirements:

- Generate a random target number between 1 and 100 (recall `Math.random()`, Part 4.2).
- Accept guesses (via a hardcoded array of guesses to simulate input, since this is a pure-JS project with no DOM/prompt dependency yet).
- After each guess, report "too high," "too low," or "correct!"
- Track and report the number of guesses taken once correct.

Concepts this exercises: Part 2 (loops, conditionals), Part 4.2 (`Math.random()`, `Math.floor()`).

Hints:

- `Math.floor(Math.random() * 100) + 1` produces an integer from 1 to 100 inclusive - walk through why, precisely, recalling Part 4.2's explanation of `Math.random()`'s `[0, 1)` range.
- Use a `for...of` loop (Part 2.6) over your simulated guesses array, with a way to stop early once correct - recall Part 2.7's `break`.

View solution

```
function playGuessingGame(guesses) {
  const target = Math.floor(Math.random() * 100) + 1;
  console.log(`(Target was ${target}, for verification)`);

  for (let i = 0; i < guesses.length; i++) {
    const guess = guesses[i];
    const attemptNumber = i + 1;

    if (guess === target) {
      console.log(`Correct! You found it in ${attemptNumber} attempt(s).`);
      return;
    } else if (guess < target) {
      console.log(`Guess ${attemptNumber} (${guess}): too low.`);
    } else {
      console.log(`Guess ${attemptNumber} (${guess}): too high.`);
    }
  }

  console.log(`Out of guesses! The number was ${target}.`);
}

playGuessingGame([50, 75, 62, 68, 65]);
```

Explanation: this project is a direct, deliberate rehearsal of Part 2's control-flow fundamentals with nothing else mixed in - no async, no DOM - specifically so those patterns (loop + early-exit conditional + accumulator-style tracking of `attemptNumber`) become automatic before Part 6 adds DOM complexity and Part 10 adds async complexity on top.

Requirements:

- A function `calculate(a, operator, b)` supporting `+`, `-`, `*`, `/`.
- Division by zero must be handled explicitly - not allowed to silently return `Infinity` or crash; it should throw a clear, custom error (recall Part 9.6).
- An unrecognized operator should also throw a clear error, rather than silently returning `undefined`.

Concepts this exercises: Part 3 (functions), Part 9.6 (custom Error classes, `throw`).

Hints:

- Recall Part 2.4's `switch` statement - a calculator's operator dispatch is one of the clearest real motivating cases for `switch` over a long `if/else if` chain.
- Recall Part 9.6's custom `Error` subclass pattern for the division-by-zero and unknown-operator cases specifically (rather than a generic `throw new Error(...)`) - what does a custom error class buy you that a generic one doesn't, when a caller later needs to `catch` and handle these two cases differently?

View solution

```
class CalculatorError extends Error {
  constructor(message) {
    super(message);
    this.name = "CalculatorError";
  }
```

```
}

function calculate(a, operator, b) {
  switch (operator) {
    case "+":
      return a + b;
    case "-":
      return a - b;
    case "*":
      return a * b;
    case "/":
      if (b === 0) {
        throw new CalculatorError("Division by zero is not allowed.");
      }
      return a / b;
    default:
      throw new CalculatorError(`Unknown operator: "${operator}"`);
  }
}

try {
  console.log(calculate(10, "/", 2)); // 5
  console.log(calculate(10, "/", 0)); // throws
} catch (error) {
  if (error instanceof CalculatorError) {
    console.error(`Calculator error: ${error.message}`);
  } else {
    throw error; // recall Part 9.6  -  re-throw anything you don't specifically know how to handle
  }
}
```

Explanation: the custom `CalculatorError` class (recall Part 9.6 in full) lets a caller use `error instanceof CalculatorError` to distinguish "this is a known, expected calculator problem I can show the user a friendly message for" from "this is some unrelated, unexpected bug" - and the `default: throw` case ensures an unrecognized operator fails loudly and immediately, rather than the function silently returning `undefined` and the bug only surfacing confusingly somewhere much later, when that `undefined` gets used.

Requirements:

- Functions `celsiusToFahrenheit(c)` and `fahrenheitToCelsius(f)`.
- Round results to 1 decimal place.
- Reject non-numeric input with a clear thrown error.

Concepts this exercises: Part 1.6 (`typeof`), Part 4.2 (`Math` methods, `toFixed`).

Hints:

- The formulas: `F = C * 9/5 + 32`, and the inverse `C = (F - 32) * 5/9`.
- Recall Part 4.2's `Number.prototype.toFixed(1)` - note it returns a string, not a number (a small but real gotcha worth explicitly re-confirming here).
- Recall Part 1.6's `typeof value !== "number"` check for input validation.

> View solution

```
function celsiusToFahrenheit(celsius) {
  if (typeof celsius !== "number" || Number.isNaN(celsius)) {
    throw new Error("celsiusToFahrenheit expects a valid number.");
  }
  const fahrenheit = celsius * 9 / 5 + 32;
  return Number(fahrenheit.toFixed(1)); // convert back to a number  -  recall the toFixed() gotcha above
}

function fahrenheitToCelsius(fahrenheit) {
  if (typeof fahrenheit !== "number" || Number.isNaN(fahrenheit)) {
    throw new Error("fahrenheitToCelsius expects a valid number.");
  }
  const celsius = (fahrenheit - 32) * 5 / 9;
  return Number(celsius.toFixed(1));
}

console.log(celsiusToFahrenheit(100)); // 212
console.log(fahrenheitToCelsius(32));    // 0
console.log(celsiusToFahrenheit(37));    // 98.6
```

Explanation: the `Number.isNaN(celsius)` check specifically (rather than just `typeof celsius !== "number"`) matters because `NaN`'s `typeof` is, confusingly, `"number"` (recall Part 1.6's full treatment of this exact gotcha) - so `celsiusToFahrenheit(NaN)` would otherwise silently pass the type check and produce a useless `NaN` result rather than a clear, immediate error.

Requirements:

- Given a string, report: total character count, word count, and count of each individual vowel (a/e/i/o/u, case-insensitive).

Concepts this exercises: Part 4.1 (string methods), Part 4.4 (`reduce`).

Hints:

- Recall Part 4.1's `.trim().split(/\s+/)` pattern for robust word-splitting (handling multiple consecutive spaces correctly - recall exactly why a plain `.split(" ")` is fragile here).
- A vowel count is a natural fit for `.reduce()` (Part 4.4) accumulating into an object, one key per vowel.

View solution

```
function analyzeText(text) {
  const trimmed = text.trim();
  const characterCount = trimmed.length;
  const wordCount = trimmed === "" ? 0 : trimmed.split(/\s+/).length;

  const vowelCounts = trimmed.toLowerCase().split("").reduce((counts, char) => {
    if ("aeiou".includes(char)) {
      counts[char] = (counts[char] || 0) + 1; // recall Part 4.4's "counting occurrences" reduce pattern
    }
    return counts;
  }, {});

  return { characterCount, wordCount, vowelCounts };
}

console.log(analyzeText("The quick brown fox jumps"));
// { characterCount: 25, wordCount: 5, vowelCounts: { e: 2, u: 2, i: 1, o: 2 } }
```

Explanation: the `trimmed === "" ? 0 : ...` guard for `wordCount` matters because `"".split(/\s+/)` actually returns `[""]` (an array containing one empty string, length 1) rather than `[]` - a genuinely easy-to-miss edge case that would otherwise report "1 word" for empty input.

Requirements:

- Generate a random valid hex color code (e.g., `"#a3f29c"`).
- Generate a random valid `rgb(r, g, b)` string.

Concepts this exercises: Part 4.2 (`Math.random()`), Part 4.1 (template literals, `padStart`).

Hints:

- Each hex color channel is 0-255, converted to a 2-digit hex string - recall `Number.prototype.toString(16)` for base conversion, and `String.prototype.padStart(2, "0")` (Part 4.1) for ensuring exactly 2 digits (e.g., `5` -> `"5"` -> needs to become `"05"`).

View solution

```
function randomChannel() {
  return Math.floor(Math.random() * 256); // 0-255 inclusive
}

function randomHexColor() {
  const channels = [randomChannel(), randomChannel(), randomChannel()];
  const hex = channels
    .map(value => value.toString(16).padStart(2, "0")) // e.g. 5 -> "5" -> "05"
    .join("");
  return `#${hex}`;
}

function randomRgbColor() {
  const [r, g, b] = [randomChannel(), randomChannel(), randomChannel()];
  return `rgb(${r}, ${g}, ${b})`;
}

console.log(randomHexColor());  // e.g. "#a3f29c"
console.log(randomRgbColor()); // e.g. "rgb(163, 242, 156)"
```

Explanation: `padStart(2, "0")` (recall Part 4.1) is what prevents a channel value like `5` (hex `"5"`, just one character) from producing a malformed 5-character color code instead of the required 6 - a bug that would be easy to miss during casual testing (since most random values happen to produce 2-digit hex already) but would fail unpredictably roughly 1-in-16 times per channel.

Requirements:

- A function that returns the current time formatted as `HH:MM:SS` (24-hour format, zero-padded).
- (This project's real DOM-updating, ticking version belongs to Tier 2 below, Project 9's sibling concepts - here, focus purely on the correct time-formatting logic in isolation.)

Concepts this exercises: Part 4.2 (`Date` object), Part 4.1 (`padStart`).

Hints:

- Recall `new Date()`, and its `.getHours()`, `.getMinutes()`, `.getSeconds()` methods (each returning an unpadded number).

> View solution

```
function getFormattedTime(date = new Date()) { // recall Part 3.3's default parameters  -  defaults to "now" but sta
  const hours = String(date.getHours()).padStart(2, "0");
  const minutes = String(date.getMinutes()).padStart(2, "0");
  const seconds = String(date.getSeconds()).padStart(2, "0");
  return `${hours}:${minutes}:${seconds}`;
}

console.log(getFormattedTime()); // e.g. "14:07:03"
console.log(getFormattedTime(new Date(2024, 0, 1, 9, 5, 2))); // "09:05:02"
```

Explanation: accepting `date = new Date()` as a default parameter (recall Part 3.3), rather than hardcoding `new Date()` inside the function body, is what makes this function testable - the second example call above can pass a specific, known date and assert on an exact expected output, something impossible if the function always, unconditionally used the live current time internally. This exact "accept a default but allow override for testing" pattern recurs constantly in real code.

Requirements:

- Track elapsed time via `start()`, `stop()`, and `reset()` methods.
- `getElapsedSeconds()` returns total elapsed time across possibly multiple start/stop cycles (pause and resume).
- (Again: the DOM-driven, visually-ticking version belongs in Tier 2 - this is the underlying timing logic, tested in isolation first.)

Concepts this exercises: Part 3.8 (closures), Part 4.2 (`Date.now()`).

Hints:

- `Date.now()` returns milliseconds since a fixed reference point - the difference between two calls to it gives you elapsed time.
- You need to track: is it currently running, when did the current running period start, and how much total time had already accumulated before that.

> View solution

```
function createStopwatch() {
  let isRunning = false;
  let startTimestamp = null;
  let accumulatedMs = 0;

  return {
    start() {
      if (isRunning) return; // recall Part 3.8  -  guard against redundant start() calls corrupting the timing
      isRunning = true;
      startTimestamp = Date.now();
    },
    stop() {
      if (!isRunning) return;
```

```
      accumulatedMs += Date.now() - startTimestamp;
      isRunning = false;
    },
    reset() {
      isRunning = false;
      startTimestamp = null;
      accumulatedMs = 0;
    },
    getElapsedSeconds() {
      const currentSegment = isRunning ? Date.now() - startTimestamp : 0;
      return (accumulatedMs + currentSegment) / 1000;
    }
  };
}

const sw = createStopwatch();
sw.start();
// ... time passes ...
sw.stop();
console.log(sw.getElapsedSeconds());
```

Explanation: the `isRunning` guard inside `start()` prevents a subtle bug: calling `start()` twice in a row without an intervening `stop()` would otherwise overwrite `startTimestamp` with a later timestamp, silently losing track of time that had already elapsed since the first `start()` call. `getElapsedSeconds()`'s handling of `isRunning ? Date.now() - startTimestamp : 0` is what lets you query elapsed time while still running (not just after `stop()`) - directly foreshadowing the real ticking-display version in Tier 2, which will call this exact method repeatedly, e.g. once per second, via `setInterval` (Part 10).

Next: Tier 2 - DOM Projects (9-16)

TIER 2 - DOM PROJECTS (9-16) [Beginner][Intermediate]

Focus: real DOM manipulation and event handling (Part 6), forms (Part 7), and browser storage (Part 8). These are the first projects that actually render into and read from a real page.

Requirements:

- Add a new todo via a text input + "Add" button (Enter key also works - recall Part 7.2's form submission).
- Each todo displays with a checkbox to mark complete (strikes through the text) and a delete button.
- Empty input must not add an empty todo.
- The full list persists across page reloads via `localStorage` (Part 8.1).

Concepts this exercises: Part 6 (DOM creation/events), Part 7.2 (form submission, preventing default), Part 8.1 (`localStorage`, JSON serialization).

Hints:

- Recall Part 6.9's event delegation pattern: rather than attaching a separate click listener to every single todo's delete button (which would also need re-attaching every time you add a new todo), attach one listener to the parent list container and check `event.target` to determine which specific todo was clicked.

- Store todos as an array of `{ id, text, completed }` objects - recall Part 8.1's requirement to `JSON.stringify`/`JSON.parse` since `localStorage` only stores strings.
- Re-render the entire list from the array after every change, rather than trying to surgically patch individual DOM nodes - simpler to reason about correctly, at the cost of some (here, negligible) performance.

> View solution

```
<input type="text" id="todo-input" placeholder="Add a todo..." />
<button id="add-button">Add</button>
<ul id="todo-list"></ul>
```

```
const STORAGE_KEY = "todos";

function loadTodos() {
  const raw = localStorage.getItem(STORAGE_KEY);
  return raw ? JSON.parse(raw) : []; // recall Part 8.1  -  getItem returns null if the key doesn't exist yet
}

function saveTodos(todos) {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(todos));
}

let todos = loadTodos();

function render() {
  const list = document.getElementById("todo-list");
  list.innerHTML = ""; // clear and rebuild  -  recall the "re-render the whole list" tradeoff discussed above

  todos.forEach(todo => {
    const li = document.createElement("li");
    li.dataset.id = todo.id; // recall Part 6.5's dataset attributes  -  used below for event delegation

    const checkbox = document.createElement("input");
    checkbox.type = "checkbox";
    checkbox.checked = todo.completed;
    checkbox.className = "toggle-checkbox";

    const span = document.createElement("span");
    span.textContent = todo.text;
    if (todo.completed) span.style.textDecoration = "line-through";

    const deleteButton = document.createElement("button");
    deleteButton.textContent = "Delete";
    deleteButton.className = "delete-button";

    li.append(checkbox, span, deleteButton);
    list.appendChild(li);
  });
}

function addTodo(text) {
  const trimmed = text.trim();
  if (trimmed === "") return; // reject empty todos
  todos.push({ id: Date.now(), text: trimmed, completed: false });
  saveTodos(todos);
  render();
}

document.getElementById("add-button").addEventListener("click", () => {
  const input = document.getElementById("todo-input");
```

```
  addTodo(input.value);
  input.value = "";
});

document.getElementById("todo-input").addEventListener("keydown", (event) => {
  if (event.key === "Enter") { // recall Part 6.7's keyboard events
    document.getElementById("add-button").click();
  }
});

// Event delegation (recall Part 6.9)  -  ONE listener handles every current AND future todo item
document.getElementById("todo-list").addEventListener("click", (event) => {
  const li = event.target.closest("li"); // recall Part 6.9's .closest() for finding the right ancestor
  if (!li) return;
  const id = Number(li.dataset.id);

  if (event.target.classList.contains("delete-button")) {
    todos = todos.filter(todo => todo.id !== id);
    saveTodos(todos);
    render();
  } else if (event.target.classList.contains("toggle-checkbox")) {
    todos = todos.map(todo =>
      todo.id === id ? { ...todo, completed: !todo.completed } : todo // recall Part 4.7's immutable update pattern
    );
    saveTodos(todos);
    render();
  }
});

render(); // initial render on page load, from whatever was loaded from localStorage
```

Explanation: two patterns here directly foreshadow React, deliberately: the "re-render the whole list from the array, every time, rather than surgically patching individual nodes" approach is exactly the mental model Part 14-18 will formalize as React's own rendering model (with React handling the "figure out the minimal actual DOM changes" step for you, instead of your own `innerHTML = ""` + rebuild). And the immutable `todos.map(todo => todo.id === id ? {...todo, completed: !todo.completed} : todo)` update (rather than mutating an existing todo object directly) is the exact same immutability discipline Part 18.4 will require for React state updates - practicing it here, where it's optional, builds the habit before it becomes mandatory.

Requirements:

- Create, edit (in place), and delete text notes.
- Each note has a title and body.
- Notes persist via `localStorage`.
- A "last edited" timestamp displayed per note, updated on every edit.

Concepts this exercises: Part 6 (DOM manipulation), Part 7 (form inputs, `contenteditable` or paired inputs), Part 8.1 (`localStorage`), Part 4.2 (`Date` formatting, recall Project 7 above).

Hints:

- Store notes as `{ id, title, body, lastEdited }`. Recall Project 9's exact `localStorage` save/load pattern - reuse it directly, just with a different shape of object.

- For "editable in place," the simplest robust approach: clicking a note swaps its display `<span>` for an `<input>`/`<textarea>` pre-filled with the current value; blurring or pressing Enter saves and swaps back (recall Part 6.7's `blur` event).

> View solution

```
const STORAGE_KEY = "notes";
let notes = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

function save() {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(notes));
}

function updateNote(id, changes) {
  notes = notes.map(note =>
    note.id === id ? { ...note, ...changes, lastEdited: new Date().toISOString() } : note
  );
  save();
  render();
}

function render() {
  const container = document.getElementById("notes-container");
  container.innerHTML = "";

  notes.forEach(note => {
    const card = document.createElement("div");
    card.className = "note-card";

    const titleInput = document.createElement("input");
    titleInput.value = note.title;
    titleInput.addEventListener("blur", () => updateNote(note.id, { title: titleInput.value }));

    const bodyTextarea = document.createElement("textarea");
    bodyTextarea.value = note.body;
    bodyTextarea.addEventListener("blur", () => updateNote(note.id, { body: bodyTextarea.value }));

    const timestamp = document.createElement("small");
    timestamp.textContent = `Last edited: ${new Date(note.lastEdited).toLocaleString()}`;

    const deleteButton = document.createElement("button");
    deleteButton.textContent = "Delete note";
    deleteButton.addEventListener("click", () => {
      notes = notes.filter(n => n.id !== note.id);
      save();
      render();
    });

    card.append(titleInput, bodyTextarea, timestamp, deleteButton);
    container.appendChild(card);
  });
}

document.getElementById("new-note-button").addEventListener("click", () => {
  notes.push({ id: Date.now(), title: "Untitled", body: "", lastEdited: new Date().toISOString() });
  save();
  render();
});

render();
```

Explanation: attaching each note's `blur` listener individually (rather than delegating, unlike Project 9) is a deliberate choice here, not an oversight - event delegation (Part 6.9) works cleanly for a single shared event type on a stable set of triggering elements (all delete buttons), but `blur` doesn't bubble by default in older browsers and mixing multiple distinct save-triggers per note-card makes direct per-element listeners the clearer, more maintainable choice at this project's scale. Recognizing when delegation is the right tool versus when direct listeners are clearer is itself part of the skill Part 6.9 is building.

Requirements:

- A reusable `openModal(content)` / `closeModal()` pair.
- Modal closes on: clicking a close button, clicking the background overlay, or pressing Escape.
- Only one modal open at a time.
- Body scroll is disabled while a modal is open (prevents the page behind it scrolling).

Concepts this exercises: Part 6 (DOM creation, event handling), Part 6.7 (keyboard events), Part 33.5's focus-management concepts (previewed early, informally, here).

Hints:

- Recall Part 6.9 - clicking the overlay itself (not its contents) to close is a great use case for checking `event.target === event.currentTarget` (i.e., "the click landed directly on the element the listener is attached to, not a descendant that bubbled up to it").
- `document.body.style.overflow = "hidden"` / `""` is the simplest way to toggle body scroll.

View solution

```
function openModal(contentHtml) {
  closeModal(); // enforce "only one at a time" by always closing any existing modal first

  const overlay = document.createElement("div");
  overlay.className = "modal-overlay";
  overlay.id = "active-modal-overlay";

  const modal = document.createElement("div");
  modal.className = "modal";
  modal.innerHTML = `
    <button class="modal-close">×</button>
    <div class="modal-content">${contentHtml}</div>
  `;

  overlay.appendChild(modal);
  document.body.appendChild(overlay);
  document.body.style.overflow = "hidden";

  overlay.addEventListener("click", (event) => {
    if (event.target === overlay) closeModal(); // clicked the overlay itself, not modal content  -  recall the targ
  });

  modal.querySelector(".modal-close").addEventListener("click", closeModal);

  document.addEventListener("keydown", handleEscapeKey);
}
```

```
function handleEscapeKey(event) {
  if (event.key === "Escape") closeModal();
}

function closeModal() {
  const existing = document.getElementById("active-modal-overlay");
  if (existing) existing.remove();
  document.body.style.overflow = "";
  document.removeEventListener("keydown", handleEscapeKey); // recall Part 6.8  -  always clean up listeners you no
}
```

Explanation: `document.removeEventListener("keydown", handleEscapeKey)` inside `closeModal()` is not optional cleanup here - without it, every single `openModal()` call would add another `keydown` listener to `document` that's never removed, so after opening and closing a modal 5 times, pressing Escape would trigger `closeModal()` 5 times redundantly (harmless here since `closeModal` is idempotent, but this exact unbounded-listener-accumulation pattern is a genuine, common source of real memory leaks and increasingly-slow pages - recall Part 6.8's full treatment). This project is also a direct, hands-on preview of exactly the kind of cleanup Part 21.4's `useEffect` cleanup function formalizes for React.

Requirements:

- Cycle through an array of image URLs, one visible at a time.
- Previous/Next buttons, plus auto-advance every 3 seconds.
- Auto-advance pauses while the user hovers over the slideshow, resuming on mouse-leave.
- Loops back to the first image after the last.

Concepts this exercises: Part 10.3 (`setInterval`/`clearInterval`), Part 6.7 (mouse events).

Hints:

- Recall Part 10.3's `setInterval` returning an ID you must save to later `clearInterval` it - this project needs that ID stored in an outer-scope variable so both the "start on page load" code and the "pause on hover" code can reference the same running interval.
- The looping index logic: `(currentIndex + 1) % images.length` correctly wraps back to 0 after the last index - recall Part 1's modulo operator.

> View solution

```
const images = ["/img1.jpg", "/img2.jpg", "/img3.jpg"];
let currentIndex = 0;
let intervalId = null;

function showImage(index) {
  document.getElementById("slideshow-img").src = images[index];
}

function goToNext() {
  currentIndex = (currentIndex + 1) % images.length; // wraps to 0 after the last image
  showImage(currentIndex);
}

function goToPrevious() {
```

```
  currentIndex = (currentIndex - 1 + images.length) % images.length; // the `+ images.length` avoids a negative resu
  showImage(currentIndex);
}

function startAutoAdvance() {
  intervalId = setInterval(goToNext, 3000);
}

function stopAutoAdvance() {
  clearInterval(intervalId); // recall Part 10.3  -  always pair setInterval with clearInterval
}

document.getElementById("next-button").addEventListener("click", goToNext);
document.getElementById("prev-button").addEventListener("click", goToPrevious);

const slideshow = document.getElementById("slideshow");
slideshow.addEventListener("mouseenter", stopAutoAdvance);
slideshow.addEventListener("mouseleave", startAutoAdvance);

showImage(currentIndex);
startAutoAdvance();
```

Explanation: `goToPrevious`'s `(currentIndex - 1 + images.length) % images.length` (rather than the simpler-looking `(currentIndex - 1) % images.length`) exists specifically because JavaScript's `%` operator, unlike some languages, can return a negative result for a negative left-hand operand (e.g., `-1 % 3` is `-1` in JS, not `2`) - adding `images.length` first guarantees the value going into `%` is non-negative, producing the correct wraparound. This is a genuinely easy mistake to make and a good one to have hit once, deliberately, here.

Requirements:

- A text input filters a displayed list of items in real time as the user types.
- Filtering is case-insensitive and matches anywhere in the item's text (not just the start).
- Displays a "No results found" message when the filter matches nothing.
- Filtering is debounced (recall Part 10.16) so it doesn't re-filter on every single keystroke for a large list.

Concepts this exercises: Part 6.7 (`input` event), Part 4.1 (string methods), Part 10.16 (debouncing).

Hints:

- Recall Part 10.16's debounce implementation directly - wrap your filter-and-render function in it.
- `.toLowerCase().includes(query.toLowerCase())` is the standard case-insensitive substring match.

> View solution

```
const allItems = ["Apple", "Banana", "Cherry", "Date", "Elderberry", "Fig", "Grape"];

function debounce(fn, delayMs) { // recall Part 10.16's full implementation and explanation
  let timeoutId;
  return function (...args) {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => fn.apply(this, args), delayMs);
  };
}
```

```
function renderFilteredList(query) {
  const list = document.getElementById("results-list");
  list.innerHTML = "";

  const matches = allItems.filter(item =>
    item.toLowerCase().includes(query.toLowerCase())
  );

  if (matches.length === 0) {
    list.innerHTML = "<li>No results found.</li>";
    return;
  }

  matches.forEach(item => {
    const li = document.createElement("li");
    li.textContent = item;
    list.appendChild(li);
  });
}

const debouncedRender = debounce(renderFilteredList, 300);

document.getElementById("search-input").addEventListener("input", (event) => {
  debouncedRender(event.target.value);
});

renderFilteredList(""); // show everything initially
```

Explanation: without debouncing (recall Part 10.16's full "why" in the async curriculum), typing a 5-character word would trigger `renderFilteredList` 5 separate times, each doing a full array filter and DOM rebuild - wasteful for this tiny 7-item array, but the exact same un-debounced pattern against a real search API call (rather than a local array) is precisely what Part 22.6 identifies as a direct cause of request race conditions, making the habit worth building here on a low-stakes local example first.

Requirements:

- Present multiple-choice questions one at a time from a hardcoded array.
- Track score across all questions.
- Show correct/incorrect feedback immediately after each answer, before advancing.
- Display a final score summary after the last question.

Concepts this exercises: Part 4.4 (array methods), Part 6 (dynamic rendering), Part 3.8 (closures for encapsulated state, recall Project 1).

Hints:

- Structure each question as `{ question, options, correctAnswerIndex }`.
- Track `currentQuestionIndex` and `score` as encapsulated state (recall Project 1's closure-counter pattern, or simply careful top-level variables for a project this size).

> View solution

```
const questions = [
  { question: "What does DOM stand for?", options: ["Document Object Model", "Data Object Model", "Document Order Mo
  { question: "Which keyword declares a block-scoped variable?", options: ["var", "let", "function"], correctAnswerI
  { question: "What does `===` check that `==` doesn't?", options: ["Nothing different", "Type, in addition to value
];

let currentQuestionIndex = 0;
let score = 0;

function renderQuestion() {
  const container = document.getElementById("quiz-container");

  if (currentQuestionIndex >= questions.length) {
    container.innerHTML = `<h2>Quiz complete! Score: ${score} / ${questions.length}</h2>`;
    return;
  }

  const q = questions[currentQuestionIndex];
  container.innerHTML = `
    <h3>${q.question}</h3>
    ${q.options.map((option, i) => `<button class="option-button" data-index="${i}">${option}</button>`).join("")}
    <p id="feedback"></p>
  `;

  container.querySelectorAll(".option-button").forEach(button => {
    button.addEventListener("click", () => handleAnswer(Number(button.dataset.index)));
  });
}

function handleAnswer(selectedIndex) {
  const q = questions[currentQuestionIndex];
  const feedback = document.getElementById("feedback");
  const isCorrect = selectedIndex === q.correctAnswerIndex;

  if (isCorrect) {
    score++;
    feedback.textContent = "Correct!";
  } else {
    feedback.textContent = `Incorrect. The correct answer was: ${q.options[q.correctAnswerIndex]}`;
  }

  document.querySelectorAll(".option-button").forEach(button => button.disabled = true); // prevent double-answering

  setTimeout(() => {
    currentQuestionIndex++;
    renderQuestion();
  }, 1500); // recall Part 10.3  -  pause briefly so feedback is actually readable before advancing
}

renderQuestion();
```

Explanation: `data-index="${i}"` (recall Part 6.5's `dataset`) combined with reading it back via `Number(button.dataset.index)` is what lets `handleAnswer` know which specific option was clicked without needing separate named variables per option - a pattern that scales cleanly to any number of options per question, unlike hardcoding `option1Button`, `option2Button`, etc. individually.

Requirements:

- Add expenses with a description and amount.
- Display running total.

- Delete individual expenses (total updates accordingly).
- Categorize expenses (a dropdown of categories); show a per-category subtotal breakdown.
- Persists via `localStorage`.

Concepts this exercises: Part 4.4 (`reduce` for totals/groupings), Part 7 (form + `<select>`), Part 8.1 (`localStorage`).

Hints:

- A per-category breakdown is a textbook `.reduce()` use case (recall Part 4.4's grouping pattern) - accumulate into an object keyed by category.
- Reuse Project 9's exact `localStorage` load/save pattern directly.

View solution

```
const STORAGE_KEY = "expenses";
let expenses = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

function save() {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(expenses));
}

function getTotal() {
  return expenses.reduce((sum, expense) => sum + expense.amount, 0); // recall Part 4.4's summing reduce
}

function getCategoryBreakdown() {
  return expenses.reduce((breakdown, expense) => {
    breakdown[expense.category] = (breakdown[expense.category] || 0) + expense.amount;
    return breakdown;
  }, {});
}

function render() {
  const list = document.getElementById("expense-list");
  list.innerHTML = "";

  expenses.forEach(expense => {
    const li = document.createElement("li");
    li.textContent = `${expense.description}: ₹${expense.amount.toFixed(2)} (${expense.category})`;

    const deleteButton = document.createElement("button");
    deleteButton.textContent = "×";
    deleteButton.addEventListener("click", () => {
      expenses = expenses.filter(e => e.id !== expense.id);
      save();
      render();
    });

    li.appendChild(deleteButton);
    list.appendChild(li);
  });

  document.getElementById("total-display").textContent = `Total: ₹${getTotal().toFixed(2)}`;

  const breakdown = getCategoryBreakdown();
  document.getElementById("breakdown-display").innerHTML = Object.entries(breakdown)
    .map(([category, amount]) => `<li>${category}: ₹${amount.toFixed(2)}</li>`)
    .join("");
```

```
}

document.getElementById("expense-form").addEventListener("submit", (event) => {
  event.preventDefault(); // recall Part 7.2
  const description = document.getElementById("description-input").value.trim();
  const amount = Number(document.getElementById("amount-input").value);
  const category = document.getElementById("category-select").value;

  if (description === "" || Number.isNaN(amount) || amount <= 0) return; // basic validation, recall Part 7.6

  expenses.push({ id: Date.now(), description, amount, category });
  save();
  render();
  event.target.reset();
});

render();
```

Explanation: `Object.entries(breakdown)` (converting the accumulated `{category: total}` object back into an iterable array of `[key, value]` pairs - recall Part 4.7) is what makes the breakdown renderable via `.map()`, since `.map()` is an Array method and objects don't have it directly - a small but very frequently recurring pattern any time an object built via `reduce()` needs to be displayed as a list.

Requirements:

- Display a grid of products (name, price, image) from a hardcoded array.
- Filter by category (dropdown) and sort by price (ascending/descending).
- Filter and sort combine correctly (e.g., "Electronics, sorted by price descending" works simultaneously, not just each in isolation).

Concepts this exercises: Part 4.4 (`.filter()`, `.sort()` - recall the Part 4 `.sort()` correction on numeric comparators specifically), Part 6 (dynamic grid rendering), Part 7 (`<select>` inputs).

Hints:

- Recall Part 4.4's critical `.sort()` gotcha (and the corrected Part 4 example) - sorting numbers requires an explicit comparator function, `(a, b) => a.price - b.price`, never the default string-based sort.
- Apply filter then sort, as two separate, composable steps on the same array - not intertwined into one complex loop.

View solution

```
const products = [
  { id: 1, name: "Wireless Mouse", price: 799, category: "Electronics" },
  { id: 2, name: "Notebook", price: 60, category: "Stationery" },
  { id: 3, name: "Mechanical Keyboard", price: 3499, category: "Electronics" },
  { id: 4, name: "Desk Lamp", price: 899, category: "Home" },
  { id: 5, name: "Pen Set", price: 150, category: "Stationery" }
];

function getFilteredAndSortedProducts(category, sortDirection) {
  let result = category === "all"
    ? products
    : products.filter(p => p.category === category); // recall Part 4.4's .filter()
```

```
  result = [...result].sort((a, b) => // recall Part 4's corrected .sort() example  -  [...result] copies before sor
    sortDirection === "asc" ? a.price - b.price : b.price - a.price
  );

  return result;
}

function render() {
  const category = document.getElementById("category-filter").value;
  const sortDirection = document.getElementById("sort-select").value;
  const filtered = getFilteredAndSortedProducts(category, sortDirection);

  const grid = document.getElementById("product-grid");
  grid.innerHTML = filtered.map(p => `
    <div class="product-card">
      <h3>${p.name}</h3>
      <p>₹${p.price.toFixed(2)}</p>
      <small>${p.category}</small>
    </div>
  `).join("");
}

document.getElementById("category-filter").addEventListener("change", render);
document.getElementById("sort-select").addEventListener("change", render);

render();
```

Explanation: `[...result].sort(...)` (spreading into a new array before sorting, rather than `result.sort(...)` directly) matters specifically because `.sort()` mutates the array it's called on in place and returns that same mutated array (recall Part 4.4) - sorting `products` (or the `result` reference pointing at it, when `category === "all"`) directly would silently and permanently reorder the original `products` source array itself, corrupting it for every future render, not just the current one. This exact mutation trap is one of the single most common real-world array bugs, and is precisely why Part 18.4 will insist on immutable update patterns for React state.

Next: Tier 3 - Async/API Projects (17-24)

TIER 3 - ASYNC/API PROJECTS (17-24) [Intermediate][Advanced]

Focus: real network requests, using the `apiRequest` helper built across Part 11. These projects deliberately echo the spirit of Part 11.13's own 10 projects - if you built thorough versions there already, treat these as reinforcement at higher speed, not entirely new territory. Each one below assumes access to your Part 11.10 `apiRequest` helper.

Requirements:

- Fetch a random user profile from a public API (e.g., `randomuser.me`) and display name, photo, email.
- A "New User" button fetches another one.
- Full loading/error/success state handling (recall Part 11.13's required state model for every project in that section).

Concepts this exercises: Part 10 (Promises/async-await), Part 11.5-11.10 (fetch, `apiRequest`), Part 11.4 (loading/error/success states).

Hints:

- Recall Part 11.10's final `apiRequest` version - use it directly rather than a raw `fetch` call, to get consistent error handling for free.
- Disable the "New User" button while a request is in flight (recall Part 22.6's double-submission concern, applied here in plain JS).

View solution

```
async function fetchRandomUser() {
  const button = document.getElementById("new-user-button");
  const display = document.getElementById("user-display");

  button.disabled = true;
  display.innerHTML = "<p>Loading...</p>";

  try {
    const data = await apiRequest("https://randomuser.me/api/"); // recall Part 11.10's apiRequest helper
    const user = data.results[0];

    display.innerHTML = `
      <img src="${user.picture.large}" alt="${user.name.first} ${user.name.last}" />
      <h3>${user.name.first} ${user.name.last}</h3>
      <p>${user.email}</p>
    `;
  } catch (error) {
    display.innerHTML = `<p class="error">Failed to load user: ${error.message}</p>`; // recall Part 11.10's normali
  } finally {
    button.disabled = false; // recall Part 9.5's finally  -  runs whether the request succeeded or failed
  }
}

document.getElementById("new-user-button").addEventListener("click", fetchRandomUser);
fetchRandomUser(); // load one immediately on page load
```

Explanation: `finally` (recall Part 9.5 in full) is what guarantees `button.disabled = false` runs on both the success path and the error path, without duplicating that line inside both the `try` block's end and the `catch` block - a small thing here, but the exact same `finally`-for-cleanup pattern is precisely what Part 22's React version generalizes into `setLoading(false)` always running regardless of outcome.

Requirements:

- Search a city name, fetch and display its current weather (temperature, condition, humidity).
- Handle a city-not-found case distinctly from a network failure (recall Part 11.7's distinction between "request succeeded but returned an error status" and "request itself failed").
- Debounced search-as-you-type (recall Project 13/Part 10.16) is a bonus extension, not a strict requirement.

Concepts this exercises: Part 11.7 (distinguishing error types), Part 11.10 (`apiRequest`), Part 7.2 (form submission).

Hints:

- Recall Part 11.7's precise distinction: a 404 "city not found" response is a successful HTTP round-trip carrying an error status - your `apiRequest` helper should already normalize this into a thrown error your `catch` block can display, exactly as designed in Part 11.10.

View solution

```
document.getElementById("weather-form").addEventListener("submit", async (event) => {
  event.preventDefault();
  const city = document.getElementById("city-input").value.trim();
  if (city === "") return;

  const resultDiv = document.getElementById("weather-result");
  resultDiv.innerHTML = "<p>Loading...</p>";

  try {
    // Example uses a hypothetical weather API requiring an API key server-side in real use (recall Part 11.9/34.2)
    // conceptually, imagine this calls YOUR OWN backend, which holds the real key, per Part 11.9's architecture.
    const data = await apiRequest(`/api/weather?city=${encodeURIComponent(city)}`); // recall Part 11.2's encodeURIC

    resultDiv.innerHTML = `
      <h3>${data.cityName}</h3>
      <p>${data.temperatureCelsius}°C, ${data.condition}</p>
      <p>Humidity: ${data.humidityPercent}%</p>
    `;
  } catch (error) {
    if (error.status === 404) {
      resultDiv.innerHTML = `<p class="error">City "${city}" not found. Check the spelling.</p>`;
    } else {
      resultDiv.innerHTML = `<p class="error">Something went wrong: ${error.message}</p>`; // recall Part 11.10's ge
    }
  }
});
```

Explanation: checking `error.status === 404` specifically (rather than showing the same generic error message for every failure) directly applies Part 11.7's distinction to give the user a precise, actionable message - "check your spelling" is genuinely different guidance than "our server/network is having a problem," and conflating them (as a naive single generic `catch` block would) leaves the user unsure whether retrying will help at all.

Requirements:

- Search a GitHub username, display their avatar, bio, repo count, follower count.
- Handle the "user not found" case (GitHub's real public API returns 404 for this) distinctly.
- Debounce the search input (recall Part 10.16) so a request doesn't fire on every keystroke.

Concepts this exercises: Part 11 in full (real public API, real error codes), Part 10.16 (debouncing), Part 22.6 (race-condition awareness, applied in plain JS here).

Hints:

- GitHub's real public API endpoint: `https://api.github.com/users/{username}` - no key required for basic profile lookups, genuinely usable directly from the frontend (recall Part 11.9 - this is an example of a public, non-secret endpoint, unlike a key-gated one).

- Combine debouncing with a simple "ignore stale responses" check (recall Part 22.6's race-condition discussion) using a request-counter or the input's current value at the time the response arrives.

> View solution

```
function debounce(fn, delayMs) {
  let timeoutId;
  return function (...args) {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => fn.apply(this, args), delayMs);
  };
}

async function searchGithubUser(username) {
  const resultDiv = document.getElementById("profile-result");

  if (username.trim() === "") {
    resultDiv.innerHTML = "";
    return;
  }

  resultDiv.innerHTML = "<p>Searching...</p>";

  try {
    const user = await apiRequest(`https://api.github.com/users/${encodeURIComponent(username)}`);

    // Stale-response guard (recall Part 22.6): only render if the input STILL matches what we searched for
    if (document.getElementById("username-input").value.trim() !== username) return;

    resultDiv.innerHTML = `
      <img src="${user.avatar_url}" alt="${user.login}" width="80" />
      <h3>${user.name || user.login}</h3>
      <p>${user.bio || "No bio."}</p>
      <p>${user.public_repos} repos · ${user.followers} followers</p>
    `;
  } catch (error) {
    if (document.getElementById("username-input").value.trim() !== username) return; // still guard against stale er
    resultDiv.innerHTML = error.status === 404
      ? `<p class="error">No GitHub user named "${username}".</p>`
      : `<p class="error">Search failed: ${error.message}</p>`;
  }
}

const debouncedSearch = debounce(searchGithubUser, 400);

document.getElementById("username-input").addEventListener("input", (event) => {
  debouncedSearch(event.target.value.trim());
});
```

Explanation: the stale-response guard (`if (...input.value !== username) return`) is a simplified, plain-JS version of exactly the race condition Part 22.6 covers in full depth with `AbortController` - without it, typing "octo" then quickly correcting to "octocat" could produce two in-flight requests, and if the first ("octo") happens to resolve after the second ("octocat") due to unpredictable network timing, the stale "octo" result would incorrectly overwrite the correct "octocat" result on screen. This guard is a legitimate, simpler alternative to `AbortController` for cases where you don't need to cancel the stale request's actual network activity - only prevent its result from being displayed.

Requirements:

- Convert an amount from one currency to another using live exchange rates from an API.
- Dropdown selectors for "from" and "to" currencies.
- Handle the case where the API itself is temporarily unavailable, with a clear retry option (recall Part 10.18's retry pattern).

Concepts this exercises: Part 11 (fetch, `apiRequest`), Part 10.18 (retry with backoff).

Hints:

- Recall Part 10.18's `fetchWithRetry` pattern directly - wrap this project's API call in it, rather than a single unretried attempt.

View solution

```
async function fetchWithRetry(url, options = {}, maxRetries = 2) { // recall Part 10.18's full implementation
  for (let attempt = 0; attempt <= maxRetries; attempt++) {
    try {
      return await apiRequest(url, options);
    } catch (error) {
      const isLastAttempt = attempt === maxRetries;
      if (isLastAttempt) throw error;
      const delayMs = 500 * 2 ** attempt; // exponential backoff, recall Part 10.18
      await new Promise(resolve => setTimeout(resolve, delayMs));
    }
  }
}

document.getElementById("convert-form").addEventListener("submit", async (event) => {
  event.preventDefault();
  const amount = Number(document.getElementById("amount-input").value);
  const from = document.getElementById("from-select").value;
  const to = document.getElementById("to-select").value;
  const resultDiv = document.getElementById("conversion-result");

  resultDiv.textContent = "Converting...";

  try {
    const data = await fetchWithRetry(`/api/exchange-rate?from=${from}&to=${to}`);
    const converted = (amount * data.rate).toFixed(2);
    resultDiv.textContent = `${amount} ${from} = ${converted} ${to}`;
  } catch (error) {
    resultDiv.innerHTML = `
      <p class="error">Conversion failed after several attempts: ${error.message}</p>
      <button id="retry-button">Try again</button>
    `;
    document.getElementById("retry-button").addEventListener("click", () => {
      document.getElementById("convert-form").requestSubmit(); // re-triggers this same submit handler
    });
  }
});
```

Explanation: `document.getElementById("convert-form").requestSubmit()` (rather than manually re-calling the handler function) is used specifically so the retry button goes through the exact same `submit` event path as a real user submission - keeping a single source of truth for "how a conversion request happens" rather than maintaining two separate, potentially-diverging code paths for "normal submit" versus "retry."

Requirements:

- Search movies by title via a public movie API, displaying poster, title, year.
- Paginate results (recall Part 11.11) with Next/Previous controls.
- Empty search input shows an instructional message rather than an error.

Concepts this exercises: Part 11.11 (pagination), Part 11.4 (5-state UI model, including the distinct "empty" state).

Hints:

- Recall Part 11.4's explicit distinction between "empty" (a valid search that legitimately returned zero results) and "idle" (no search has been attempted yet) - this project needs both, distinctly.

View solution

```
let currentPage = 1;
let currentQuery = "";

async function searchMovies(query, page) {
  const resultsDiv = document.getElementById("movie-results");

  if (query.trim() === "") {
    resultsDiv.innerHTML = "<p>Enter a movie title to search.</p>"; // "idle" state, recall Part 11.4
    return;
  }

  resultsDiv.innerHTML = "<p>Searching...</p>";

  try {
    const data = await apiRequest(`/api/movies?query=${encodeURIComponent(query)}&page=${page}`);

    if (data.results.length === 0) {
      resultsDiv.innerHTML = `<p>No movies found for "${query}".</p>`; // "empty" state  -  distinct from "idle" abo
      return;
    }

    resultsDiv.innerHTML = `
      <div class="movie-grid">
        ${data.results.map(movie => `
          <div class="movie-card">
            <img src="${movie.posterUrl}" alt="${movie.title}" />
            <h4>${movie.title} (${movie.year})</h4>
          </div>
        `).join("")}
      </div>
      <button id="prev-page" ${page === 1 ? "disabled" : ""}>Previous</button>
      <span>Page ${page} of ${data.totalPages}</span>
      <button id="next-page" ${page >= data.totalPages ? "disabled" : ""}>Next</button>
    `;

    document.getElementById("prev-page")?.addEventListener("click", () => {
      currentPage--;
      searchMovies(currentQuery, currentPage);
    });
    document.getElementById("next-page")?.addEventListener("click", () => {
      currentPage++;
      searchMovies(currentQuery, currentPage);
    });
  } catch (error) {
```

```
    resultsDiv.innerHTML = `<p class="error">Search failed: ${error.message}</p>`;
  }
}

document.getElementById("search-form").addEventListener("submit", (event) => {
  event.preventDefault();
  currentQuery = document.getElementById("search-input").value;
  currentPage = 1; // always reset to page 1 on a NEW search  -  recall this exact requirement from Part 11.11
  searchMovies(currentQuery, currentPage);
});
```

Explanation: resetting `currentPage = 1` specifically on a new search submission (not on pagination clicks, which intentionally change it) directly applies Part 11.11's stated requirement - without this reset, searching "Batman" while sitting on page 3 of a previous "Superman" search would confusingly request page 3 of Batman results, which might not exist or might skip its most relevant early matches entirely.

Requirements:

- Browse a product catalogue from an API, paginated, with a page-size selector (e.g., 10/25/50 per page).
- Changing page size resets to page 1 and re-fetches.
- URL reflects current page (bonus: using the History API, or simply a query-string read on load) so a reload preserves position.

Concepts this exercises: Part 11.11 (pagination), state synchronization concepts (foreshadowing Part 21's synchronization mental model, applied in plain JS).

Hints:

- Treat "current page" and "page size" as the two pieces of state driving every fetch - any change to either should trigger exactly one re-fetch, never a missed one and never a duplicate.

> View solution

```
let state = { page: 1, pageSize: 10 };

async function fetchAndRenderProducts() {
  const grid = document.getElementById("product-grid");
  grid.innerHTML = "<p>Loading...</p>";

  try {
    const data = await apiRequest(`/api/products?page=${state.page}&pageSize=${state.pageSize}`);

    grid.innerHTML = data.products.map(p => `
      <div class="product-card"><h4>${p.name}</h4><p>₹${p.price}</p></div>
    `).join("");

    document.getElementById("page-info").textContent = `Page ${state.page} of ${Math.ceil(data.totalCount / state.pa
  } catch (error) {
    grid.innerHTML = `<p class="error">${error.message}</p>`;
  }
}

document.getElementById("page-size-select").addEventListener("change", (event) => {
  state = { page: 1, pageSize: Number(event.target.value) }; // changing page size ALWAYS resets to page 1
```

```
  fetchAndRenderProducts();
});

document.getElementById("next-page-button").addEventListener("click", () => {
  state = { ...state, page: state.page + 1 };
  fetchAndRenderProducts();
});

document.getElementById("prev-page-button").addEventListener("click", () => {
  if (state.page === 1) return;
  state = { ...state, page: state.page - 1 };
  fetchAndRenderProducts();
});

fetchAndRenderProducts();
```

Explanation: replacing `state` entirely with a new object (`state = { ...state, page: state.page + 1 }`) on every change, rather than mutating individual properties (`state.page++`), is the exact same immutability discipline as Project 9's todo-toggle and Project 16's sort - a deliberate, repeated habit-building choice, since Part 18.4 will require precisely this pattern for React state and inconsistent practice now makes that transition harder later.

Requirements:

- Load an initial batch of items; automatically load more as the user scrolls near the bottom (rather than explicit Next/Previous buttons).
- Prevent duplicate/overlapping fetches if the user scrolls quickly (only one "load more" request in flight at a time).
- A visible loading indicator at the bottom while fetching the next batch.

Concepts this exercises: Part 6.7 (scroll events), Part 11.11 (pagination, adapted to an append-only model), Part 10.16-style guarding against redundant triggers.

Hints:

- Recall Part 10.16's debounce concept, applied slightly differently here: rather than debouncing the scroll event itself, guard the fetch with an `isLoading` flag checked at the very top of the fetch function - an even more direct way to prevent overlapping requests than time-based debouncing.
- `window.scrollY + window.innerHeight >= document.body.offsetHeight - threshold` is a standard "near the bottom" check.

> View solution

```
let items = [];
let page = 1;
let isLoading = false;
let hasMore = true;

async function loadMoreItems() {
  if (isLoading || !hasMore) return; // the core duplicate-request guard
  isLoading = true;
  document.getElementById("loading-indicator").style.display = "block";

  try {
```

```
    const data = await apiRequest(`/api/feed?page=${page}`);
    items = [...items, ...data.items]; // APPEND, not replace  -  the key difference from Project 22's page-replace
    hasMore = data.items.length > 0;
    page++;
    renderItems();
  } catch (error) {
    console.error("Failed to load more items:", error.message);
  } finally {
    isLoading = false;
    document.getElementById("loading-indicator").style.display = "none";
  }
}

function renderItems() {
  document.getElementById("feed-container").innerHTML = items.map(item => `
    <div class="feed-item"><h4>${item.title}</h4><p>${item.body}</p></div>
  `).join("");
}

window.addEventListener("scroll", () => {
  const nearBottom = window.scrollY + window.innerHeight >= document.body.offsetHeight - 300;
  if (nearBottom) loadMoreItems();
});

loadMoreItems(); // load the first batch on page load
```

Explanation: the `if (isLoading || !hasMore) return` guard at the very top of `loadMoreItems` is what makes this safe against rapid scrolling - a `scroll` event fires many times per second during active scrolling (recall the sheer frequency discussed conceptually in Part 6.7), so without this guard, scrolling quickly near the bottom would fire `loadMoreItems()` dozens of times before the first request even resolves, launching dozens of duplicate, overlapping, wasteful requests - a much more direct and reliable fix here than trying to debounce the scroll event itself, since the real constraint isn't "don't check too often," it's "never start a second fetch while one is already running."

Requirements:

- Full Create, Read, Update, Delete against a REST API for a single resource (e.g., a "tasks" or "contacts" API).
- List view, an add form, inline or modal editing, and delete with a confirmation step.
- All four operations correctly update the displayed list without requiring a manual page reload.
- Proper loading/error state handling on every one of the four operations independently (an error creating a new item shouldn't corrupt or clear the existing list).

Concepts this exercises: Part 11.6 (all HTTP methods - GET/POST/PUT/DELETE), Part 11.10 (`apiRequest` for every operation), Project 11's modal pattern (for edit/confirm-delete).

Hints:

- Recall Part 11.6's full method-by-method treatment - this project is the first one requiring literally all four methods against the same resource, in one cohesive interface.

- After a successful create/update/delete, the simplest robust approach is to just re-fetch the full list from the server (recall Part 11.13, Project 8's "Simple CRUD Interface" for precisely this same design decision) - trading a small amount of extra network traffic for a strong guarantee that the displayed list is never out of sync with the server's actual state.

> View solution

```
async function loadTasks() {
  const list = document.getElementById("task-list");
  list.innerHTML = "<p>Loading...</p>";
  try {
    const tasks = await apiRequest("/api/tasks"); // GET, recall Part 11.6
    renderTasks(tasks);
  } catch (error) {
    list.innerHTML = `<p class="error">Failed to load tasks: ${error.message}</p>`;
  }
}

function renderTasks(tasks) {
  document.getElementById("task-list").innerHTML = tasks.map(task => `
    <li data-id="${task.id}">
      <span>${task.title}</span>
      <button class="edit-button">Edit</button>
      <button class="delete-button">Delete</button>
    </li>
  `).join("");
}

document.getElementById("add-task-form").addEventListener("submit", async (event) => {
  event.preventDefault();
  const title = document.getElementById("new-task-input").value.trim();
  if (title === "") return;

  try {
    await apiRequest("/api/tasks", { method: "POST", body: JSON.stringify({ title }) }); // recall Part 11.6's POST
    event.target.reset();
    await loadTasks(); // re-fetch the full list  -  recall Part 11.13 Project 8's reasoning for this exact approach
  } catch (error) {
    alert(`Failed to add task: ${error.message}`); // isolated to THIS operation  -  the existing list stays untouch
  }
});

document.getElementById("task-list").addEventListener("click", async (event) => {
  const li = event.target.closest("li");
  if (!li) return;
  const id = li.dataset.id;

  if (event.target.classList.contains("delete-button")) {
    if (!confirm("Delete this task?")) return; // simple native confirmation step
    try {
      await apiRequest(`/api/tasks/${id}`, { method: "DELETE" }); // recall Part 11.6's DELETE
      await loadTasks();
    } catch (error) {
      alert(`Failed to delete: ${error.message}`);
    }
  } else if (event.target.classList.contains("edit-button")) {
    const newTitle = prompt("New title:");
    if (!newTitle) return;
    try {
      await apiRequest(`/api/tasks/${id}`, { method: "PUT", body: JSON.stringify({ title: newTitle }) }); // recall
      await loadTasks();
    } catch (error) {
      alert(`Failed to update: ${error.message}`);
```

```
    }
  }
});

loadTasks();
```

Explanation: each of the three mutating operations (`POST`/`PUT`/`DELETE`) has its own, independent `try/catch` - a failed "add task" `alert`s an error but never touches `task-list`'s existing, already-successfully-loaded content, and the same isolation applies to edit and delete. This is precisely the requirement stated above ("an error creating a new item shouldn't corrupt or clear the existing list") and is a direct, practical application of Part 9's principle that error handling should be scoped as narrowly as the specific operation that might fail, not wrapped around unrelated code that was already working correctly.

Next: Tier 4 - React Beginner Projects (25-30)

TIER 4 - REACT BEGINNER PROJECTS (25-30) [Intermediate]

Focus: rebuilding several of Tier 1/2's plain-JS projects in React, to feel directly, concretely, exactly what React changes (and what it doesn't) - components, `useState`, props, controlled forms, but no `useEffect`/API calls yet (that's Tier 5).

Requirements: the exact same behavior as Project 1, rebuilt as a React component: increment, decrement (never below 0), reset.

Concepts this exercises: Part 16 (components), Part 18 (`useState`).

Hints:

- Compare this directly against Project 1's closure-based solution - where did the "private state" concept go? (Answer to think through before checking: `useState`'s local variable is the closure-based encapsulation, just provided by React instead of hand-written.)

> View solution

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0); // recall Part 18.1  -  this replaces Project 1's hand-written closure enti

  function handleDecrement() {
    if (count === 0) return; // same validation logic as Project 1, now inside the component
    setCount(count - 1);
  }

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      <button onClick={handleDecrement}>Decrement</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}

export default Counter;
```

Explanation: this is a direct, concrete answer to the question posed in the hint - `useState(0)`'s `count` variable is private to this specific component instance, in exactly the same sense Project 1's closure-enclosed `count` was private to that specific `createCounter()` call, just without you having to hand-write the enclosing function yourself. Recall Part 18.1's full explanation of why `setCount` (not direct reassignment) is required for React to know to re-render - that requirement has no equivalent in Project 1's plain-JS version, since nothing there needed to trigger a DOM re-render automatically.

Requirements: the same functional requirements as Project 9 (add/complete/delete, persisted to `localStorage`), rebuilt in React.

Concepts this exercises: Part 18 (`useState` with arrays), Part 21 (`useEffect` for `localStorage` sync - a light first use, ahead of Tier 5's deeper API-focused `useEffect` work).

Hints:

- Recall Part 18.4's array immutability rules directly - every add/toggle/delete must produce a new array via spread/`.filter()`/`.map()`, never `.push()`/direct mutation.
- Two `useEffect` calls: one to load from `localStorage` on mount (`[]` dependency array), one to save to `localStorage` whenever `todos` changes (`[todos]` dependency array) - recall Part 21.3's dependency array semantics.

View solution

```
import { useState, useEffect } from "react";

function TodoApp() {
  const [todos, setTodos] = useState(() => { // recall Part 18.7's lazy initializer  -  reads localStorage only once
    const saved = localStorage.getItem("todos");
    return saved ? JSON.parse(saved) : [];
  });
  const [inputValue, setInputValue] = useState("");

  useEffect(() => { // recall Part 21.3  -  this effect SYNCHRONIZES todos state to localStorage, running after ever
    localStorage.setItem("todos", JSON.stringify(todos));
  }, [todos]);

  function handleAdd(event) {
    event.preventDefault(); // recall Part 20's controlled form submission
    const trimmed = inputValue.trim();
    if (trimmed === "") return;
    setTodos([...todos, { id: Date.now(), text: trimmed, completed: false }]); // recall Part 18.4's spread-to-add p
    setInputValue("");
  }

  function handleToggle(id) {
    setTodos(todos.map(todo => // recall Part 18.4's .map()-to-update pattern  -  SAME logic as Project 9's plain-JS
      todo.id === id ? { ...todo, completed: !todo.completed } : todo
    ));
  }

  function handleDelete(id) {
    setTodos(todos.filter(todo => todo.id !== id)); // recall Part 18.4's .filter()-to-remove pattern
  }

  return (
    <div>
      <form onSubmit={handleAdd}>
```

```
        <input value={inputValue} onChange={(e) => setInputValue(e.target.value)} placeholder="Add a todo..." />
        <button type="submit">Add</button>
      </form>
      <ul>
        {todos.map(todo => (
          <li key={todo.id}> {/* recall Part 18.5's key requirement for rendered lists */}
            <input type="checkbox" checked={todo.completed} onChange={() => handleToggle(todo.id)} />
            <span style={{ textDecoration: todo.completed ? "line-through" : "none" }}>{todo.text}</span>
            <button onClick={() => handleDelete(todo.id)}>Delete</button>
          </li>
        ))}
      </ul>
    </div>
  );
}

export default TodoApp;
```

Explanation: comparing this directly against Project 9's plain-JS version is genuinely instructive: the immutable update logic (`.map()`/`.filter()`/spread) is identical in both - that discipline was never a React-specific requirement, just a good habit Project 9 built early. What React actually replaces is the manual `render()` function that rebuilt `innerHTML` from scratch on every change - here, returning new JSX from the component function IS the render step, and React's diffing (recall Part 14.6's virtual DOM discussion) handles turning that into the minimal actual DOM changes, automatically.

Requirements: the same requirements as Project 10, rebuilt in React - create/edit-in-place/delete, `localStorage` persistence, "last edited" timestamp.

Concepts this exercises: Part 18 (state for a list of objects, per-item editing state), Part 19 (controlled inputs), Part 21 (persistence effect).

Hints:

- "Editable in place" in React means each note needs its own small piece of "am I currently in edit mode" state - recall Part 18.6's discussion of colocating state at the right level (this state belongs to the individual `NoteCard`, not lifted to the parent, since no other component needs to know about it).

> View solution

```
import { useState, useEffect } from "react";

function NoteCard({ note, onUpdate, onDelete }) {
  const [isEditing, setIsEditing] = useState(false); // colocated LOCAL state  -  recall Part 18.6, nothing outside
  const [draftTitle, setDraftTitle] = useState(note.title);
  const [draftBody, setDraftBody] = useState(note.body);

  function handleSave() {
    onUpdate(note.id, { title: draftTitle, body: draftBody });
    setIsEditing(false);
  }

  if (!isEditing) {
    return (
      <div className="note-card">
        <h3>{note.title}</h3>
        <p>{note.body}</p>
        <small>Last edited: {new Date(note.lastEdited).toLocaleString()}</small>
        <button onClick={() => setIsEditing(true)}>Edit</button>
        <button onClick={() => onDelete(note.id)}>Delete</button>
```

```
      </div>
    );
  }

  return (
    <div className="note-card">
      <input value={draftTitle} onChange={(e) => setDraftTitle(e.target.value)} /> {/* recall Part 19's controlled i
      <textarea value={draftBody} onChange={(e) => setDraftBody(e.target.value)} />
      <button onClick={handleSave}>Save</button>
      <button onClick={() => setIsEditing(false)}>Cancel</button>
    </div>
  );
}

function NotesApp() {
  const [notes, setNotes] = useState(() => {
    const saved = localStorage.getItem("notes");
    return saved ? JSON.parse(saved) : [];
  });

  useEffect(() => {
    localStorage.setItem("notes", JSON.stringify(notes));
  }, [notes]);

  function handleAddNote() {
    setNotes([...notes, { id: Date.now(), title: "Untitled", body: "", lastEdited: new Date().toISOString() }]);
  }

  function handleUpdateNote(id, changes) {
    setNotes(notes.map(note =>
      note.id === id ? { ...note, ...changes, lastEdited: new Date().toISOString() } : note
    ));
  }

  function handleDeleteNote(id) {
    setNotes(notes.filter(note => note.id !== id));
  }

  return (
    <div>
      <button onClick={handleAddNote}>New Note</button>
      {notes.map(note => (
        <NoteCard key={note.id} note={note} onUpdate={handleUpdateNote} onDelete={handleDeleteNote} />
      ))}
    </div>
  );
}

export default NotesApp;
```

Explanation: `NoteCard`'s `isEditing`/`draftTitle`/`draftBody` state deliberately lives inside `NoteCard`, not in the parent `NotesApp` - this is Part 18.6's "colocate state as close to where it's used as possible" principle in direct action: if editing-state lived in `NotesApp` instead (e.g., as `editingNoteId` plus separate draft fields), every `NoteCard` would need to check "is it MY id currently being edited" and the parent would need to manage draft values for whichever note happens to be mid-edit - meaningfully more complex, for zero actual benefit, since no other part of the app needs to know or react to any single note's edit-in-progress state.

Requirements:

- Display a grid of product cards from a static array of data (passed as props from a parent), each showing image, name, price, and an "Add to Cart" button.
- Clicking "Add to Cart" increments a cart-count badge displayed elsewhere on the page (shared state, lifted to a common ancestor).

  Concepts this exercises: Part 17 (props, especially passing data down through multiple component layers), Part 18.6 (lifting state up).

  Hints:

- Recall Part 18.6's full "lifting state up" example precisely - the cart count needs to be visible to both the badge (wherever it's rendered) and every individual product card's button, so it must live in their nearest common ancestor, passed down as props to both.

View solution

```
import { useState } from "react";

const products = [
  { id: 1, name: "Wireless Mouse", price: 799, image: "/mouse.jpg" },
  { id: 2, name: "Keyboard", price: 3499, image: "/keyboard.jpg" }
];

function ProductCard({ product, onAddToCart }) {
  return (
    <div className="product-card">
      <img src={product.image} alt={product.name} />
      <h4>{product.name}</h4>
      <p>₹{product.price}</p>
      <button onClick={() => onAddToCart(product)}>Add to Cart</button>
    </div>
  );
}

function CartBadge({ count }) {
  return <span className="cart-badge">Cart: {count}</span>;
}

function ProductPage() {
  const [cartCount, setCartCount] = useState(0); // lifted up  -  recall Part 18.6's full reasoning for exactly this

  function handleAddToCart(product) {
    setCartCount(cartCount + 1);
    console.log(`Added ${product.name} to cart.`);
  }

  return (
    <div>
      <header>
        <CartBadge count={cartCount} /> {/* receives cartCount as a prop  -  doesn't own it */}
      </header>
      <div className="product-grid">
        {products.map(product => (
          <ProductCard key={product.id} product={product} onAddToCart={handleAddToCart} /> // receives the SETTER fu
        ))}
      </div>
    </div>
  );
}
```

```
export default ProductPage;
```

Explanation: neither `ProductCard` nor `CartBadge` holds `cartCount` in its own `useState` - both are given it (or a way to change it) purely via props, from `ProductPage`, their nearest common ancestor. This is the textbook shape of Part 18.6's "lifting state up": recognizing that two sibling-ish components (or a component and a header far above it in the tree) both need access to the same piece of state means that state cannot correctly live inside either of them individually - it must move up to whichever component contains both, exactly as demonstrated here.

Requirements: the same functional requirements as Project 13, rebuilt in React - real-time, debounced, case-insensitive filtering, with a distinct "no results" state.

Concepts this exercises: Part 18 (state for both the query and the source list), Part 21 (a light custom-hook-style debounce, foreshadowing Part 27), Part 19 (controlled input).

Hints:

- A debounced value in React is commonly implemented as its own small piece of state, updated by a `useEffect` with a `setTimeout`/cleanup pattern - recall Part 21.4's cleanup function, used here specifically to cancel a pending debounce timer if the user types again before it fires.

View solution

```
import { useState, useEffect } from "react";

const allItems = ["Apple", "Banana", "Cherry", "Date", "Elderberry", "Fig", "Grape"];

function useDebouncedValue(value, delayMs) { // a small custom hook  -  directly foreshadows Part 27's useDebounce
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    const timeoutId = setTimeout(() => setDebouncedValue(value), delayMs);
    return () => clearTimeout(timeoutId); // recall Part 21.4  -  cancels the PREVIOUS pending timeout if value chan
  }, [value, delayMs]);

  return debouncedValue;
}

function SearchApp() {
  const [query, setQuery] = useState("");
  const debouncedQuery = useDebouncedValue(query, 300);

  const matches = allItems.filter(item =>
    item.toLowerCase().includes(debouncedQuery.toLowerCase())
  );

  return (
    <div>
      <input value={query} onChange={(e) => setQuery(e.target.value)} placeholder="Search fruits..." />
      {matches.length === 0 ? (
        <p>No results found.</p>
      ) : (
        <ul>{matches.map(item => <li key={item}>{item}</li>)}</ul>
      )}
    </div>
  );
}

export default SearchApp;
```

Explanation: notice `query` (updated on every keystroke, for the input's own responsiveness) and `debouncedQuery` (updated only after the user pauses typing, used for the actual filtering) are two separate pieces of state - this separation is exactly why the input never feels laggy (it's always bound to the instantly-updating `query`) while the potentially-expensive filtering work only happens at the debounced rate. The cleanup function's `clearTimeout` (recall Part 21.4) is what makes this correct rather than merely "usually working" - without it, rapid typing would schedule many overlapping timeouts, and while only the last one would matter for the final displayed value, the wasted timers are unnecessary and, in a real API-call version of this same pattern (Part 22.6), would translate directly into wasted, overlapping network requests.

Requirements:

- A multi-field form (name, email, age, a dropdown, a checkbox) with real-time validation feedback per field (recall Part 20.5).
- Submit is disabled until all fields are valid.
- On successful submit, display a summary of the submitted data and reset the form.

Concepts this exercises: Part 20 (controlled forms in full, multi-field state, validation), Part 19 (event handling for varied input types).

Hints:

- Recall Part 20.4's single-object-state pattern for multi-field forms (`const [formData, setFormData] = useState({...})`) rather than one `useState` call per field - and Part 20.6's generic `handleChange` using the input's `name` attribute to update the correct field dynamically.

View solution

```
import { useState } from "react";

const initialFormData = { name: "", email: "", age: "", role: "developer", agreedToTerms: false };

function validate(formData) { // recall Part 20.5's validation-function-returns-errors-object pattern
  const errors = {};
  if (formData.name.trim() === "") errors.name = "Name is required.";
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(formData.email)) errors.email = "Enter a valid email.";
  if (!formData.age || Number(formData.age) < 13) errors.age = "Age must be 13 or older.";
  if (!formData.agreedToTerms) errors.agreedToTerms = "You must agree to the terms.";
  return errors;
}

function FormApp() {
  const [formData, setFormData] = useState(initialFormData);
  const [submittedData, setSubmittedData] = useState(null);

  const errors = validate(formData); // re-computed on every render  -  recall Part 20.5's "validate on every render
  const isValid = Object.keys(errors).length === 0;

  function handleChange(event) {
    const { name, value, type, checked } = event.target; // recall Part 20.6's generic handler, destructuring type/c
    setFormData({ ...formData, [name]: type === "checkbox" ? checked : value }); // recall Part 4.7's computed prope
  }

  function handleSubmit(event) {
    event.preventDefault();
```

```
    if (!isValid) return; // defense in depth  -  the disabled button should already prevent this, recall Part 20.5
    setSubmittedData(formData);
    setFormData(initialFormData);
  }

  return (
    <div>
      <form onSubmit={handleSubmit}>
        <input name="name" value={formData.name} onChange={handleChange} placeholder="Name" />
        {errors.name && <p className="error">{errors.name}</p>}

        <input name="email" value={formData.email} onChange={handleChange} placeholder="Email" />
        {errors.email && <p className="error">{errors.email}</p>}

        <input name="age" type="number" value={formData.age} onChange={handleChange} placeholder="Age" />
        {errors.age && <p className="error">{errors.age}</p>}

        <select name="role" value={formData.role} onChange={handleChange}>
          <option value="developer">Developer</option>
          <option value="designer">Designer</option>
        </select>

        <label>
          <input name="agreedToTerms" type="checkbox" checked={formData.agreedToTerms} onChange={handleChange} />
          I agree to the terms
        </label>
        {errors.agreedToTerms && <p className="error">{errors.agreedToTerms}</p>}

        <button type="submit" disabled={!isValid}>Submit</button>
      </form>

      {submittedData && (
        <div className="summary">
          <h3>Submitted:</h3>
          <p>{submittedData.name} ({submittedData.email}), age {submittedData.age}, role: {submittedData.role}</p>
        </div>
      )}
    </div>
  );
}

export default FormApp;
```

Explanation: the single generic `handleChange` (rather than a separate handler function per field) works because every input consistently uses its `name` attribute to match a key in `formData`, and `[name]: value` (recall Part 4.7's computed property syntax) dynamically targets the correct field - this is precisely Part 20.6's stated payoff: adding a sixth form field later requires zero new handler code, only a new input element with the matching `name` attribute and a corresponding validation rule.

Next: Tier 5 - Intermediate React Projects (31-36)

TIER 5 - INTERMEDIATE REACT PROJECTS (31-36) [Advanced]

Focus: real API calls inside React (Part 22's full loading/error/success model), custom hooks (Part 27), and the service-layer architecture (Part 29). These projects are meaningfully larger and closer to real applications.

Requirements:

- Product list (static data is fine) with Add to Cart.
- A cart view: quantity per item (adjustable +/-), per-line subtotal, cart total.
- Removing an item, and correctly handling quantity reaching 0 (auto-remove, or a minimum of 1 - your choice, stated clearly).
- Cart persists via `localStorage`.

Concepts this exercises: Part 18.4 (nested/complex state updates), Part 21 (persistence effect), Part 29 (a small `utils/` layer for cart-total calculations, kept separate from component code).

Hints:

- Model the cart as an array of `{ productId, quantity }`, not full duplicated product objects - look up product details from the static `products` array by `productId` when rendering, avoiding data duplication/drift.
- Recall Part 29.3's `utils/` layer concept: a pure `calculateCartTotal(cartItems, products)` function, tested in isolation, is cleaner than inlining that math directly inside JSX.

View solution

```
// utils/cartUtils.js  -  recall Part 29.3's utils layer: pure, framework-independent functions
export function calculateSubtotal(cartItem, products) {
  const product = products.find(p => p.id === cartItem.productId);
  return product ? product.price * cartItem.quantity : 0;
}

export function calculateCartTotal(cartItems, products) {
  return cartItems.reduce((total, item) => total + calculateSubtotal(item, products), 0); // recall Part 4.4's reduc
}

// ShoppingCart.jsx
import { useState, useEffect } from "react";
import { calculateSubtotal, calculateCartTotal } from "./utils/cartUtils.js";

const products = [
  { id: 1, name: "Wireless Mouse", price: 799 },
  { id: 2, name: "Keyboard", price: 3499 }
];

function ShoppingCart() {
  const [cartItems, setCartItems] = useState(() => {
    const saved = localStorage.getItem("cart");
    return saved ? JSON.parse(saved) : [];
  });

  useEffect(() => {
    localStorage.setItem("cart", JSON.stringify(cartItems));
  }, [cartItems]);

  function handleAddToCart(productId) {
    const existing = cartItems.find(item => item.productId === productId);
    if (existing) {
      setCartItems(cartItems.map(item => // recall Part 18.4's .map()-to-update pattern
        item.productId === productId ? { ...item, quantity: item.quantity + 1 } : item
      ));
    } else {
      setCartItems([...cartItems, { productId, quantity: 1 }]);
```

```
    }
  }

  function handleQuantityChange(productId, delta) {
    setCartItems(
      cartItems
        .map(item => item.productId === productId ? { ...item, quantity: item.quantity + delta } : item)
        .filter(item => item.quantity > 0) // auto-remove at 0  -  stated design choice above
    );
  }

  function handleRemove(productId) {
    setCartItems(cartItems.filter(item => item.productId !== productId));
  }

  return (
    <div>
      <div className="product-list">
        {products.map(product => (
          <div key={product.id}>
            <span>{product.name}  -  ₹{product.price}</span>
            <button onClick={() => handleAddToCart(product.id)}>Add to Cart</button>
          </div>
        ))}
      </div>

      <div className="cart">
        <h3>Cart</h3>
        {cartItems.map(item => {
          const product = products.find(p => p.id === item.productId);
          return (
            <div key={item.productId}>
              <span>{product.name}</span>
              <button onClick={() => handleQuantityChange(item.productId, -1)}>-</button>
              <span>{item.quantity}</span>
              <button onClick={() => handleQuantityChange(item.productId, 1)}>+</button>
              <span>₹{calculateSubtotal(item, products).toFixed(2)}</span>
              <button onClick={() => handleRemove(item.productId)}>Remove</button>
            </div>
          );
        })}
        <p><strong>Total: ₹{calculateCartTotal(cartItems, products).toFixed(2)}</strong></p>
      </div>
    </div>
  );
}

export default ShoppingCart;
```

Explanation: storing `{ productId, quantity }` rather than full copies of product objects inside the cart is a data-normalization choice with a real, concrete payoff: if a product's price ever changed (in a real app, from a fresh API fetch), a cart storing duplicated product data would show a stale price until the cart itself was somehow refreshed, while this normalized version always looks up the current `products` array - a direct, practical illustration of why Part 29's separation-of-concerns thinking about where data lives matters even in a fairly small feature like a cart.

Requirements: rebuild Project 21 (Movie Search) in React, using Part 22's full 5-state UI model (idle/loading/success/empty/error), `AbortController` to cancel stale requests (Part 22.6), and debounced search-as-you-type.

Concepts this exercises: Part 22 in full (this is the project Part 22 was building toward), Part 27 (extracting the logic into a custom `useMovieSearch` hook).

Hints:

- Recall Part 22.6's complete `AbortController` + debounce solution to the race-condition problem - implement it exactly as taught, not a simplified version, since this project exists specifically to cement that exact pattern.
- Extracting the entire search-and-fetch logic into a custom hook (recall Part 27.2's `useFetch` pattern, adapted here) keeps the component itself purely about rendering.

View solution

```
// hooks/useMovieSearch.js
import { useState, useEffect } from "react";

function useMovieSearch(query) {
  const [status, setStatus] = useState("idle"); // recall Part 22.2's 5-state model
  const [movies, setMovies] = useState([]);
  const [error, setError] = useState(null);

  useEffect(() => {
    if (query.trim() === "") {
      setStatus("idle");
      setMovies([]);
      return;
    }

    const controller = new AbortController(); // recall Part 22.6 in full
    const timeoutId = setTimeout(async () => { // the debounce delay
      setStatus("loading");
      try {
        const response = await fetch(`/api/movies?query=${encodeURIComponent(query)}`, {
          signal: controller.signal
        });
        if (!response.ok) throw new Error(`Request failed: ${response.status}`);
        const data = await response.json();

        setMovies(data.results);
        setStatus(data.results.length === 0 ? "empty" : "success"); // recall Part 22.2's distinct empty state
      } catch (err) {
        if (err.name === "AbortError") return; // recall Part 22.6  -  a cancelled request is NOT a real error to di
        setError(err.message);
        setStatus("error");
      }
    }, 400);

    return () => { // recall Part 21.4's cleanup  -  runs before the NEXT effect execution, or on unmount
      clearTimeout(timeoutId);
      controller.abort(); // cancels an in-flight request if the query changes again before it resolves
    };
  }, [query]);

  return { status, movies, error };
}

export default useMovieSearch;

// MovieSearch.jsx
import { useState } from "react";
import useMovieSearch from "./hooks/useMovieSearch.js";
```

```
function MovieSearch() {
  const [query, setQuery] = useState("");
  const { status, movies, error } = useMovieSearch(query);

  return (
    <div>
      <input value={query} onChange={(e) => setQuery(e.target.value)} placeholder="Search movies..." />

      {status === "idle" && <p>Enter a movie title to search.</p>}
      {status === "loading" && <p>Searching...</p>}
      {status === "empty" && <p>No movies found for "{query}".</p>}
      {status === "error" && <p className="error">{error}</p>}
      {status === "success" && (
        <div className="movie-grid">
          {movies.map(movie => (
            <div key={movie.id}>
              <img src={movie.posterUrl} alt={movie.title} />
              <h4>{movie.title} ({movie.year})</h4>
            </div>
          ))}
        </div>
      )}
    </div>
  );
}

export default MovieSearch;
```

Explanation: notice the debounce here is implemented via the `setTimeout` inside the effect (rather than Project 29's separate `useDebouncedValue` hook) - both are valid, but combining the debounce delay directly with the abort-on-cleanup logic in one effect means a single `return` cleanup function handles both concerns (cancel the pending timeout AND abort any in-flight request) consistently, which is precisely the complete, correct version of the race-condition fix Part 22.6 builds up to, rather than a partial fix that only solves debouncing OR only solves cancellation.

Requirements:

- Search a city, display current weather plus a short forecast (e.g., next 3 days).
- Save a list of "favorite cities" (persisted), each showing a compact live weather summary, refreshed on demand.
- Switching between Celsius/Fahrenheit display (a client-side unit conversion, not a re-fetch).

Concepts this exercises: Part 22 (multiple independent API calls, one per favorite city), Part 25 (Context for a shared unit-preference setting, avoiding prop-drilling it through every weather card), Part 27 (a reusable `useWeather(city)` hook).

Hints:

- Recall Part 25's exact prop-drilling problem and Context's solution - the Celsius/Fahrenheit preference is needed by potentially many deeply-nested weather-card components; Context avoids threading it through every intermediate layer as a prop.
- Each favorite city's weather fetch is independent - a custom `useWeather(city)` hook, called once per `FavoriteCityCard` instance, naturally gives each its own independent loading/error/data state (recall Part 23.6's "each hook call gets its own state" principle).

View solution

```
// contexts/UnitContext.jsx  -  recall Part 25 in full
import { createContext, useContext, useState } from "react";

const UnitContext = createContext();

export function UnitProvider({ children }) {
  const [unit, setUnit] = useState("celsius");
  return (
    <UnitContext.Provider value={{ unit, toggleUnit: () => setUnit(u => u === "celsius" ? "fahrenheit" : "celsius") }}>
      {children}
    </UnitContext.Provider>
  );
}

export function useUnit() {
  return useContext(UnitContext); // recall Part 25.4's custom-hook-wrapper convention for consuming Context
}

// hooks/useWeather.js  -  recall Part 27's custom hook pattern
import { useState, useEffect } from "react";

function useWeather(city) {
  const [status, setStatus] = useState("idle");
  const [data, setData] = useState(null);

  useEffect(() => {
    const controller = new AbortController();
    setStatus("loading");

    fetch(`/api/weather?city=${encodeURIComponent(city)}`, { signal: controller.signal })
      .then(res => res.json())
      .then(result => { setData(result); setStatus("success"); })
      .catch(err => { if (err.name !== "AbortError") setStatus("error"); });

    return () => controller.abort();
  }, [city]);

  return { status, data };
}

export default useWeather;

// FavoriteCityCard.jsx
import useWeather from "./hooks/useWeather.js";
import { useUnit } from "./contexts/UnitContext.jsx";

function toFahrenheit(celsius) {
  return (celsius * 9) / 5 + 32; // recall Project 4's exact conversion formula
}

function FavoriteCityCard({ city }) {
  const { status, data } = useWeather(city); // this instance's OWN independent state  -  recall Part 23.6
  const { unit } = useUnit(); // recall Part 25  -  read from Context, no prop needed from any parent

  if (status === "loading") return <div className="city-card">Loading {city}...</div>;
  if (status === "error") return <div className="city-card">Couldn't load {city}.</div>;
  if (status !== "success") return null;

  const displayTemp = unit === "celsius" ? data.temperatureCelsius : toFahrenheit(data.temperatureCelsius);

  return (
    <div className="city-card">
```

```
      <h4>{city}</h4>
      <p>{displayTemp.toFixed(1)}°{unit === "celsius" ? "C" : "F"}</p>
    </div>
  );
}

export default FavoriteCityCard;
```

Explanation: each `FavoriteCityCard` calling `useWeather(city)` independently is what gives every card its own loading/error/data state, entirely isolated from every other card - this directly applies Part 23.6's principle that each call to a Hook gets its own private state, not each component definition; rendering five `FavoriteCityCard`s produces five completely independent `useWeather` states running in parallel, exactly as needed for five cities to load (and potentially fail) independently of one another. Reading `unit` from Context (rather than as a prop passed down from whatever renders the list of favorite cities) avoids needing every intermediate component between the app root and `FavoriteCityCard` to pass that prop through unchanged, purely to reach this one deeply-nested consumer - precisely Part 25's stated payoff.

Requirements: the same functional scope as Project 15 (add/delete/categorize/persist/per-category breakdown), rebuilt in React with a `useReducer` (rather than multiple `useState` calls) managing the expense list, given the several related update operations involved.

Concepts this exercises: Part 26 (`useReducer`, action-based state updates), Part 29 (`utils/` layer for the breakdown calculation).

Hints:

- Recall Part 26.2's guidance on when `useReducer` is the better choice over `useState`: multiple related state-transition types (add/delete/edit) acting on the same piece of state is exactly the scenario Part 26.2 identifies as reducer-shaped.
- Actions: `{ type: "ADD_EXPENSE", payload: {...} }` and `{ type: "DELETE_EXPENSE", payload: id }`, following Part 26.3's action-object convention.

View solution

```
// expenseReducer.js  -  recall Part 26.3's reducer function convention
function expenseReducer(state, action) {
  switch (action.type) {
    case "ADD_EXPENSE":
      return [...state, action.payload]; // recall Part 26.4  -  reducers MUST return new state, never mutate
    case "DELETE_EXPENSE":
      return state.filter(expense => expense.id !== action.payload);
    default:
      throw new Error(`Unknown action type: ${action.type}`); // recall Part 26.3's default-case safety net
  }
}

export default expenseReducer;
```

```
// utils/expenseUtils.js  -  recall Part 29.3
export function getCategoryBreakdown(expenses) {
  return expenses.reduce((breakdown, expense) => {
    breakdown[expense.category] = (breakdown[expense.category] || 0) + expense.amount;
    return breakdown;
  }, {});
```

```
}

// ExpenseManager.jsx
import { useReducer, useEffect } from "react";
import expenseReducer from "./expenseReducer.js";
import { getCategoryBreakdown } from "./utils/expenseUtils.js";

function ExpenseManager() {
  const [expenses, dispatch] = useReducer(expenseReducer, [], () => { // recall Part 26.5's lazy init third argument
    const saved = localStorage.getItem("expenses");
    return saved ? JSON.parse(saved) : [];
  });

  useEffect(() => {
    localStorage.setItem("expenses", JSON.stringify(expenses));
  }, [expenses]);

  function handleAdd(event) {
    event.preventDefault();
    const form = new FormData(event.target); // an alternative to per-field useState  -  reading directly from the f
    dispatch({
      type: "ADD_EXPENSE",
      payload: {
        id: Date.now(),
        description: form.get("description"),
        amount: Number(form.get("amount")),
        category: form.get("category")
      }
    });
    event.target.reset();
  }

  const total = expenses.reduce((sum, e) => sum + e.amount, 0);
  const breakdown = getCategoryBreakdown(expenses);

  return (
    <div>
      <form onSubmit={handleAdd}>
        <input name="description" placeholder="Description" required />
        <input name="amount" type="number" placeholder="Amount" required />
        <select name="category">
          <option>Food</option><option>Transport</option><option>Other</option>
        </select>
        <button type="submit">Add</button>
      </form>

      <ul>
        {expenses.map(expense => (
          <li key={expense.id}>
            {expense.description}: ₹{expense.amount.toFixed(2)}
            <button onClick={() => dispatch({ type: "DELETE_EXPENSE", payload: expense.id })}>×</button>
          </li>
        ))}
      </ul>

      <p>Total: ₹{total.toFixed(2)}</p>
      <ul>{Object.entries(breakdown).map(([cat, amt]) => <li key={cat}>{cat}: ₹{amt.toFixed(2)}</li>)}</ul>
    </div>
  );
}

export default ExpenseManager;
```

Explanation: `useReducer`'s `dispatch({ type: "ADD_EXPENSE", payload: {...} })` calls read, at the call site, exactly what kind of change is happening - compare this to an equivalent `useState`-based version, which would need a bespoke `setExpenses(...)` call with inline array-manipulation logic duplicated at every single place an expense gets added or removed. Centralizing all the state-transition logic inside the single `expenseReducer` function (rather than scattered across every event handler) is precisely Part 26.2's stated benefit - one place to look for "how does this state change," easier to test in isolation (recall Part 35.3's pure-function testing ease), and harder to accidentally get subtly wrong differently in two different handlers.

Requirements:

- Protected route (recall Part 28.5) requiring a logged-in state to access.
- A sidebar with navigation between at least 3 sub-views (e.g., Users, Reports, Settings), using nested routes (recall Part 28.4).
- A shared layout (header/sidebar) persisting across sub-view navigation, not re-rendered per navigation.
- Each sub-view independently fetches its own data with full loading/error handling.

Concepts this exercises: Part 28 (nested routes, protected routes, layout routes via `<Outlet>`), Part 22 (per-view independent data fetching), Part 25 (auth state via Context).

Hints:

- Recall Part 28.4's `<Outlet>` pattern precisely - the shared layout component renders `<Outlet>` where the currently-active nested route's content should appear, keeping the header/sidebar itself outside any individual sub-view's re-render cycle.
- Recall Part 28.5's protected-route wrapper pattern for gating access to the entire `/admin/*` route tree behind a login check.

> View solution

```
// AdminLayout.jsx  -  recall Part 28.4's shared-layout-with-Outlet pattern
import { Outlet, Link } from "react-router-dom";

function AdminLayout() {
  return (
    <div className="admin-layout">
      <header>Admin Dashboard</header>
      <nav>
        <Link to="/admin/users">Users</Link>
        <Link to="/admin/reports">Reports</Link>
        <Link to="/admin/settings">Settings</Link>
      </nav>
      <main>
        <Outlet /> {/* the currently-matched nested route renders HERE  -  header/nav above are NOT re-created on na
      </main>
    </div>
  );
}

export default AdminLayout;
```

```
// ProtectedRoute.jsx  -  recall Part 28.5
import { Navigate } from "react-router-dom";
import { useAuth } from "./contexts/AuthContext.jsx";

function ProtectedRoute({ children }) {
  const { isLoggedIn } = useAuth();
  return isLoggedIn ? children : <Navigate to="/login" replace />;
}

export default ProtectedRoute;

// App.jsx routes (excerpt)  -  recall Part 28.3's nested route syntax
<Routes>
  <Route path="/login" element={<LoginPage />} />
  <Route
    path="/admin"
    element={<ProtectedRoute><AdminLayout /></ProtectedRoute>}
  >
    <Route path="users" element={<UsersView />} />
    <Route path="reports" element={<ReportsView />} />
    <Route path="settings" element={<SettingsView />} />
  </Route>
</Routes>

// UsersView.jsx  -  an example sub-view with its OWN independent fetch (each sub-view follows this same shape)
import { useState, useEffect } from "react";

function UsersView() {
  const [status, setStatus] = useState("loading");
  const [users, setUsers] = useState([]);

  useEffect(() => {
    const controller = new AbortController();
    fetch("/api/users", { signal: controller.signal })
      .then(res => res.json())
      .then(data => { setUsers(data); setStatus("success"); })
      .catch(err => { if (err.name !== "AbortError") setStatus("error"); });
    return () => controller.abort();
  }, []);

  if (status === "loading") return <p>Loading users...</p>;
  if (status === "error") return <p className="error">Failed to load users.</p>;
  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}

export default UsersView;
```

Explanation: because `<Outlet>` lives inside `AdminLayout`, and `AdminLayout` itself only renders once when you first navigate into `/admin/*` (not once per sub-view navigation - recall Part 28.4's precise explanation of nested route matching), the header and sidebar never unmount/remount as you click between Users/Reports/Settings - only the `<Outlet>`'s content swaps. This is precisely why each sub-view's `useEffect` (with its own `[]` dependency array) re-fetches correctly on navigating into that view, while the layout around it stays untouched and doesn't re-fetch anything of its own.

Requirements:

- List, search, and filter products by category and stock status (in stock / low stock / out of stock, derived from a `quantity` field with defined thresholds).

- Edit a product's quantity inline, with an optimistic UI update (recall Part 22's data-flow model) that rolls back on API failure.
- A low-stock warning banner listing all currently low-stock items.

  Concepts this exercises: Part 22 (API calls, error rollback), Part 29 (derived-state calculations kept in `utils/`, not recomputed ad hoc inline), Part 31 (avoiding unnecessary re-renders on a potentially large product list, using `React.memo` where justified).

  Hints:

- "Optimistic update": update the displayed quantity immediately on the user's edit (before the API call resolves), then if the API call fails, revert it back to the previous value and show an error - recall Part 22's full request lifecycle, applied here in the opposite direction of the usual "wait for the response before showing anything" flow.
- Recall Part 31.3's honest guidance on `React.memo`: only reach for it here if you can articulate specifically which unnecessary re-render it prevents (e.g., editing one product's quantity re-rendering every other unrelated product row) - not reflexively on every component.

View solution

```
// utils/inventoryUtils.js  -  recall Part 29.3
export function getStockStatus(quantity) {
  if (quantity === 0) return "out-of-stock";
  if (quantity <= 5) return "low-stock";
  return "in-stock";
}

export function getLowStockItems(products) {
  return products.filter(p => getStockStatus(p.quantity) === "low-stock");
}

// ProductRow.jsx
import { memo, useState } from "react"; // recall Part 31.3's React.memo
import { getStockStatus } from "./utils/inventoryUtils.js";

const ProductRow = memo(function ProductRow({ product, onQuantityChange }) {
  const [inputValue, setInputValue] = useState(product.quantity);

  async function handleBlur() {
    const newQuantity = Number(inputValue);
    if (newQuantity === product.quantity) return; // no actual change  -  skip the API call entirely
    onQuantityChange(product.id, newQuantity, product.quantity); // pass the OLD value too, for rollback
  }

  return (
    <tr>
      <td>{product.name}</td>
      <td className={getStockStatus(product.quantity)}>{getStockStatus(product.quantity)}</td>
      <td>
        <input
          type="number"
          value={inputValue}
          onChange={(e) => setInputValue(e.target.value)}
          onBlur={handleBlur}
        />
      </td>
    </tr>
  );
```

```
}); // recall Part 31.3  -  this row only re-renders if ITS OWN product/onQuantityChange props actually change

export default ProductRow;

// InventorySystem.jsx
import { useState, useCallback } from "react";
import ProductRow from "./ProductRow.jsx";
import { getLowStockItems } from "./utils/inventoryUtils.js";

function InventorySystem({ initialProducts }) {
  const [products, setProducts] = useState(initialProducts);
  const [error, setError] = useState(null);

  const handleQuantityChange = useCallback(async (id, newQuantity, oldQuantity) => { // recall Part 31.4's useCallba
    setProducts(prev => prev.map(p => p.id === id ? { ...p, quantity: newQuantity } : p)); // OPTIMISTIC update  -

    try {
      const response = await fetch(`/api/products/${id}`, {
        method: "PUT",
        body: JSON.stringify({ quantity: newQuantity })
      });
      if (!response.ok) throw new Error("Update failed");
    } catch (err) {
      setProducts(prev => prev.map(p => p.id === id ? { ...p, quantity: oldQuantity } : p)); // ROLLBACK on failure
      setError(`Failed to update ${id}: reverted to previous value.`);
    }
  }, []); // stable across renders  -  recall Part 31.4's exact reasoning for why this pairs correctly with memo abo

  const lowStockItems = getLowStockItems(products);

  return (
    <div>
      {lowStockItems.length > 0 && (
        <div className="warning-banner">
          Low stock: {lowStockItems.map(p => p.name).join(", ")}
        </div>
      )}
      {error && <p className="error">{error}</p>}
      <table>
        <tbody>
          {products.map(product => (
            <ProductRow key={product.id} product={product} onQuantityChange={handleQuantityChange} />
          ))}
        </tbody>
      </table>
    </div>
  );
}

export default InventorySystem;
```

Explanation: `useCallback` wrapping `handleQuantityChange` (recall Part 31.4's full, honest treatment) is what makes `React.memo` on `ProductRow` actually effective here: without it, `InventorySystem` re-rendering (e.g., because `error` changed) would create a brand-new `handleQuantityChange` function reference every time, which would count as a changed prop for every single `ProductRow`, defeating `memo`'s comparison and causing every row to re-render anyway - precisely the pitfall Part 31.4 calls out explicitly. The optimistic-update-then-rollback pattern is a genuinely important, realistic technique: it makes the UI feel instantaneous for the common case (successful update) at the cost of a brief visual "flicker back" in the rare failure case - a tradeoff real production inventory/admin tools make constantly, and one worth having implemented once, deliberately, with your own hands.

Next: Tier 6 - Advanced React Projects (37-42)

TIER 6 - ADVANCED REACT PROJECTS (37-42) [Expert]

Focus: these six projects intentionally combine most of the entire course - routing, layered architecture, Context, reducers, custom hooks, error boundaries, accessibility, performance - into single, cohesive applications, each roughly capstone-adjacent in scope. Full solutions here are given as architecture outlines with the most important/non-obvious pieces implemented in detail, rather than exhaustive line-by-line code for every file - by this tier, you have every individual pattern already fully worked in earlier Parts and Tiers, and the actual skill being exercised is assembling known pieces into a coherent whole, which a complete pre-written solution would rob you of practicing. (The course's own Capstone project, immediately following this catalogue, gives you that full, complete, staged build treatment.)

Requirements: manage a resource (e.g., "employees") with full create/read/update/delete, search, sort, pagination, and a properly separated architecture (recall Part 29 in full: `services/` -> `hooks/` -> `pages/`).

Concepts this exercises: Parts 22, 26, 28, 29 combined.

Architecture outline:

```
src/
  services/employeeService.js    -  getEmployees(params), createEmployee(data), updateEmployee(id, data), deleteEmpl
                                    (recall Part 29.3  -  EVERY fetch call lives here, nowhere else)
  hooks/useEmployees.js          -  wraps employeeService with loading/error/data state (recall Part 22/27's useFetc
  hooks/useEmployeeMutations.js  -  wraps create/update/delete, each returning its own status, used by forms/row act
  pages/EmployeeListPage.jsx     -  search/sort/pagination UI, calls useEmployees
  pages/EmployeeFormPage.jsx     -  shared between "create" and "edit" (recall Part 28.3's :id param distinguishing
  components/EmployeeTable.jsx   -  pure presentational  -  recall Part 29.2's UI-layer-has-no-fetch-logic rule
```

The one piece worth showing in full - the shared create/edit form pattern (a genuinely common real-world question):

```
// pages/EmployeeFormPage.jsx
import { useParams, useNavigate } from "react-router-dom"; // recall Part 28.3's useParams
import { useState, useEffect } from "react";
import { getEmployee, createEmployee, updateEmployee } from "../services/employeeService.js";

function EmployeeFormPage() {
  const { id } = useParams(); // undefined on the "create" route, a real id on the "edit" route  -  recall Part 28.3
  const isEditMode = Boolean(id);
  const navigate = useNavigate(); // recall Part 28.2  -  for redirecting after a successful save
  const [formData, setFormData] = useState({ name: "", role: "", email: "" });
  const [status, setStatus] = useState(isEditMode ? "loading" : "idle"); // only need to LOAD existing data in edit

  useEffect(() => {
    if (!isEditMode) return;
    getEmployee(id).then(data => { setFormData(data); setStatus("idle"); });
  }, [id, isEditMode]);

  async function handleSubmit(event) {
    event.preventDefault();
    setStatus("saving");
    try {
      if (isEditMode) {
        await updateEmployee(id, formData);
      } else {
```

```
        await createEmployee(formData);
      }
      navigate("/employees"); // recall Part 28.2  -  programmatic navigation after a successful mutation
    } catch (error) {
      setStatus("error");
    }
  }

  if (status === "loading") return <p>Loading employee...</p>;

  return (
    <form onSubmit={handleSubmit}>
      {/* controlled inputs bound to formData, recall Part 20  -  omitted here for brevity, identical pattern to Pro
      <button type="submit" disabled={status === "saving"}>{isEditMode ? "Update" : "Create"}</button>
      {status === "error" && <p className="error">Save failed. Try again.</p>}
    </form>
  );
}

export default EmployeeFormPage;
```

Explanation: the single `EmployeeFormPage` handling both create and edit (branching on `isEditMode`, derived from whether `useParams()`'s `id` exists) avoids duplicating the entire form UI across two separate components that would otherwise need to stay in sync with every future field addition - this is a direct, larger-scale application of exactly the same "don't repeat yourself, branch on a condition instead" instinct from every earlier Part's mistake-avoidance sections.

Requirements: product listing/detail pages, cart (recall Project 31), checkout form, order confirmation - routed as a real multi-page flow.

Concepts this exercises: Part 28 (multi-page routing with params, e.g. `/products/:id`), Part 25 (cart state via Context, shared across the entire routed app), Part 20 (checkout form validation).

Architecture outline:

```
src/
  contexts/CartContext.jsx    -  recall Part 25: cart state + addItem/removeItem/updateQuantity, consumed via useCar
  pages/ProductListPage.jsx   -  /products
  pages/ProductDetailPage.jsx  -  /products/:id, recall Part 28.3's useParams to read the id, fetch that one product
  pages/CartPage.jsx           -  /cart, reads from CartContext  -  recall Project 31's cart-line-item rendering log
  pages/CheckoutPage.jsx      -  /checkout, a form (recall Project 30's validation pattern) that on submit calls an
  pages/OrderConfirmationPage.jsx  -  /order-confirmation/:orderId
```

The one piece worth showing in full - routing with a param, fetching the one matching item:

```
// pages/ProductDetailPage.jsx
import { useParams } from "react-router-dom";
import { useState, useEffect } from "react";
import { useCart } from "../contexts/CartContext.jsx";

function ProductDetailPage() {
  const { id } = useParams();
  const [product, setProduct] = useState(null);
  const [status, setStatus] = useState("loading");
  const { addItem } = useCart(); // recall Part 25  -  Context, no prop drilling needed from wherever CartProvider s

  useEffect(() => {
    setStatus("loading");
    fetch(`/api/products/${id}`)
      .then(res => res.json())
```

```
      .then(data => { setProduct(data); setStatus("success"); })
      .catch(() => setStatus("error"));
  }, [id]); // re-fetches correctly if the user navigates from one product detail page directly to another

  if (status === "loading") return <p>Loading...</p>;
  if (status === "error") return <p>Product not found.</p>;

  return (
    <div>
      <h2>{product.name}</h2>
      <p>₹{product.price}</p>
      <button onClick={() => addItem(product.id)}>Add to Cart</button>
    </div>
  );
}

export default ProductDetailPage;
```

Explanation: the `[id]` dependency array is what correctly handles a subtle routing gotcha (recall Part 28.3's own callout of this exact case): navigating from `/products/5` directly to `/products/9` via a `<Link>` does not unmount and remount `ProductDetailPage` - React Router reuses the same component instance, since it's the same route pattern matching - so without `id` in the dependency array, the effect wouldn't re-run and the page would keep showing product 5's data under product 9's URL.

Requirements: multiple chart/stat widgets on one dashboard, each independently loading its own data, a date-range filter affecting all widgets simultaneously, and graceful handling if one widget's data fails to load without breaking the others.

Concepts this exercises: Part 32 (Error Boundaries - scoped per-widget, recall Part 32.5's "strategic placement" guidance precisely), Part 25 (shared date-range filter via Context), Part 31 (avoiding an expensive recomputation on every render via `useMemo`).

Architecture outline:

```
src/
  contexts/DateRangeContext.jsx  -  shared { startDate, endDate } read by every widget
  components/ErrorBoundary.jsx   -  recall Part 32.3's class component implementation, reused as-is
  components/widgets/RevenueWidget.jsx, UsersWidget.jsx, ConversionWidget.jsx  -  each: useDateRange() + its own fet
  pages/DashboardPage.jsx        -  wraps EACH widget in its OWN <ErrorBoundary>, not one boundary around the whole
```

The one piece worth showing in full - per-widget error boundary placement, the key architectural decision:

```
// pages/DashboardPage.jsx
import ErrorBoundary from "../components/ErrorBoundary.jsx";
import RevenueWidget from "../components/widgets/RevenueWidget.jsx";
import UsersWidget from "../components/widgets/UsersWidget.jsx";
import ConversionWidget from "../components/widgets/ConversionWidget.jsx";

function DashboardPage() {
  return (
    <div className="dashboard-grid">
      {/* recall Part 32.5  -  EACH widget gets its OWN boundary, so one widget crashing doesn't take down the other
      <ErrorBoundary fallback={<p>Revenue widget failed to load.</p>}>
        <RevenueWidget />
      </ErrorBoundary>
      <ErrorBoundary fallback={<p>Users widget failed to load.</p>}>
        <UsersWidget />
```

```
      </ErrorBoundary>
      <ErrorBoundary fallback={<p>Conversion widget failed to load.</p>}>
        <ConversionWidget />
      </ErrorBoundary>
    </div>
  );
}

export default DashboardPage;
```

Explanation: this is precisely Part 32.5's "strategic placement" guidance made concrete at real application scale - a single `<ErrorBoundary>` wrapping the entire `DashboardPage` would mean one buggy widget (say, a `RevenueWidget` bug triggered only by a specific edge case in this week's data) takes down the entire dashboard, hiding two perfectly-functional widgets along with it. Per-widget boundaries mean a crash is contained to exactly the blast radius of the thing that actually broke - a real, meaningful reliability difference at the cost of a small amount of extra boilerplate, which is exactly the tradeoff Part 32.5 argues is almost always worth making for independent, self-contained sections of a real dashboard.

Requirements: projects containing tasks, tasks assignable to statuses (To Do/In Progress/Done) via drag-and-drop or simple status buttons, nested routing (`/projects/:projectId/tasks/:taskId`), and a `useReducer`-managed task-state model given the many related task-mutation actions (create/update-status/delete/reassign).

Concepts this exercises: Part 26 (a genuinely reducer-shaped domain - many related actions on one connected piece of state), Part 28 (deeply nested routes with multiple params).

Architecture outline:

```
src/
  reducers/taskReducer.js   -  actions: ADD_TASK, MOVE_TASK (status change), DELETE_TASK, ASSIGN_TASK
  pages/ProjectDetailPage.jsx     -  /projects/:projectId  -  renders task board, reads projectId via useParams
  pages/TaskDetailPage.jsx         -  /projects/:projectId/tasks/:taskId  -  recall Part 28.4's MULTI-param nested r
```

The one piece worth showing in full - the reducer, since this project's core justification IS its reducer-shape:

```
// reducers/taskReducer.js  -  recall Part 26.3's convention
function taskReducer(tasks, action) {
  switch (action.type) {
    case "ADD_TASK":
      return [...tasks, { ...action.payload, status: "todo" }];
    case "MOVE_TASK": // { taskId, newStatus }
      return tasks.map(task =>
        task.id === action.payload.taskId ? { ...task, status: action.payload.newStatus } : task
      );
    case "DELETE_TASK":
      return tasks.filter(task => task.id !== action.payload);
    case "ASSIGN_TASK": // { taskId, assigneeId }
      return tasks.map(task =>
        task.id === action.payload.taskId ? { ...task, assigneeId: action.payload.assigneeId } : task
      );
    default:
      throw new Error(`Unknown action: ${action.type}`);
  }
}

export default taskReducer;
```

Explanation: four distinct action types, all operating on the same `tasks` array, is precisely the shape Part 26.2 identifies as reducer-appropriate - imagine the `useState`-only alternative: four separate inline array-manipulation blocks scattered across wherever each respective button/drag-handler lives in the component tree, each one needing to independently get the immutability logic exactly right. Centralizing them here means there is exactly one place responsible for "how does a task list legally change," which is both easier to verify correct and, per Part 35.3, trivially unit-testable in complete isolation from any rendered UI at all.

Requirements: login/logout, protected routes, an auth token stored (with the Part 34.3 tradeoff explicitly acknowledged and chosen deliberately), and automatically attaching that token to authenticated API requests.

Concepts this exercises: Part 25 (auth state via Context, app-wide), Part 28.5 (protected routes), Part 34.2-34.3 (token storage tradeoff, applied for real), Part 11.6 (Bearer token header pattern).

Architecture outline:

```
src/
  contexts/AuthContext.jsx  -  { user, login(credentials), logout() }, recall Part 25 in full
  services/apiClient.js     -  recall Part 11.10's apiRequest, EXTENDED to automatically attach the auth token heade
  components/ProtectedRoute.jsx  -  recall Project 35's exact implementation, reused directly
```

The one piece worth showing in full - the token-attaching API client, the genuinely new piece here:

```
// services/apiClient.js  -  extends Part 11.10's apiRequest with automatic auth header attachment
const BASE_URL = import.meta.env.VITE_API_URL; // recall Part 37.4

export async function apiRequest(path, options = {}) {
  const token = localStorage.getItem("authToken"); // recall Part 34.3's EXPLICIT, ACKNOWLEDGED tradeoff  -  chosen

  const headers = {
    "Content-Type": "application/json",
    ...options.headers,
    ...(token ? { Authorization: `Bearer ${token}` } : {}) // recall Part 11.6's Bearer token pattern  -  only attac
  };

  const response = await fetch(`${BASE_URL}${path}`, { ...options, headers });

  if (response.status === 401) { // the token was rejected/expired  -  recall Part 11.7's status code meaning
    localStorage.removeItem("authToken");
    window.location.href = "/login"; // a hard redirect here is a deliberate, simple choice for an expired-session c
    return;
  }

  if (!response.ok) {
    throw new Error(`Request failed: ${response.status}`);
  }

  return response.json();
}
```

Explanation: this is Part 11.10's `apiRequest` helper, unchanged in its core shape, with exactly one new concern layered on: reading a stored token and attaching it as a `Bearer` header (recall Part 11.6) to every outgoing request, automatically. Centralizing this here means no individual component or service function ever needs to remember to manually attach the token itself - every call site simply calls `apiRequest(...)` as before, and authentication becomes an invisible, consistent property of the request layer rather than something that could be forgotten at any individual call site. The explicit

`localStorage` choice (rather than an `HttpOnly` cookie) is made here knowingly, with Part 34.3's tradeoff table in mind - a real project would make this exact same decision deliberately, not by default.

This project is not built independently here - it IS the course's full Capstone project, immediately following this catalogue, built across 15 explicit stages rather than presented as a single finished solution. Everything in Tiers 4-6 above - routing, layered architecture, Context, reducers, custom hooks, performance, error boundaries, accessibility, security, and now authentication - comes together there in one sustained, staged build. Continue directly into the Capstone below.

Next: The Capstone Project - Inventory + Order Management Dashboard

THE CAPSTONE PROJECT - INVENTORY + ORDER MANAGEMENT DASHBOARD [Expert]

This is the course's single largest sustained project - a genuinely real-shaped application, built the way real applications actually get built: requirements before code, architecture before components, and working software at every stage rather than one enormous final code dump. Every stage below builds directly on the previous one. Nothing here introduces a concept you haven't already learned in full depth earlier in this course - this project's entire challenge is combining everything correctly, which is a genuinely different skill from learning each piece in isolation.

Why This Particular Project

An inventory + order management dashboard was chosen deliberately, not arbitrarily, because it naturally requires nearly everything this course has taught, in a way that feels like one coherent application rather than a checklist of disconnected features:

- Two connected resources (products and orders, where orders reference products) - requiring real data relationships, not just one isolated CRUD resource (recall Project 24 and 37's single-resource CRUD, extended here).
- Multiple views needing routing (a product list, a product detail/edit view, an order list, an order detail view, a dashboard summary) - a genuine multi-page application (Part 28).
- State that's genuinely shared across distant parts of the tree (the current user, a "low stock" alert count visible in a header far from the inventory list that computes it) - motivating Context (Part 25) for real reasons, not contrived ones.
- A domain with several related state transitions (an order moving through statuses: pending -> processing -> shipped -> delivered/cancelled) - motivating `useReducer` (Part 26) for real reasons.

- Real asynchronous complexity (loading states across multiple independent views, the possibility of a request failing, a search-as-you-type product filter needing debouncing and race-condition safety) - exercising Part 10/11/22 in full, not as isolated exercises.
- A large enough surface area to make architecture (Part 29) actually matter - small projects can get away with sloppy organization; this one cannot, and building it will make you feel, directly, exactly why Part 29's layered structure earns its complexity.

STAGE 1 - Requirements

Before writing a single line of code, a real project starts with a precise, written specification of what "done" means. Vague requirements produce vague, inconsistent software - recall this exact principle from every project's "Requirements" section throughout the catalogue above, now applied at a larger scale where getting it wrong costs much more rework.

Products:

1. View a paginated, searchable, filterable (by category, by stock status) list of all products.
2. View a single product's full detail page.
3. Create a new product.
4. Edit an existing product's details (name, price, category, quantity).
5. Delete a product (with a confirmation step - recall Project 24's `confirm()` pattern, upgraded here to a proper modal, recall Project 11).
6. See a persistent, always-visible indicator of how many products are currently low-stock (quantity ≤ 5, recall Project 36's exact threshold convention).

Orders:
7. View a paginated, filterable (by status) list of all orders.
8. View a single order's full detail page, showing its line items (each referencing a product) and current status.
9. Create a new order: select one or more products, specify quantities, computing a running total as items are added.
10. Update an order's status, following a defined valid transition sequence: `pending` -> `processing` -> `shipped` -> `delivered`, or `pending`/`processing` -> `cancelled` (a `shipped` or `delivered` order can no longer be cancelled - a genuine business rule the UI must enforce, not just suggest).
11. Creating an order must correctly decrement the relevant products' stock quantities; cancelling an order must correctly restore them.

Dashboard/Overview:
12. A landing page summarizing: total products, total orders by status (a small breakdown), and the low-stock product list.

Cross-cutting:
13. Every view that fetches data handles loading/error/empty states explicitly (recall Part 22.2's 5-state model, applied consistently everywhere in this app - no view is allowed to silently show nothing while loading, or silently show nothing on error).

14. The application is usable via keyboard alone, with correct semantic markup throughout (recall Part 33 in full - this is a real requirement here, not an afterthought bolted on at the end).
15. No real backend is being built (recall Part 0.3's scope boundary, honored throughout this entire course) - this capstone targets a REST API assumed to already exist at the routes used throughout (documented precisely in Stage 7 below), exactly the same "frontend-only, backend-agnostic" boundary every earlier API project in this course has maintained.

- User authentication/authorization (Project 41 covers this pattern separately; adding it here is a natural, well-understood extension left as your own exercise once the capstone is complete, using exactly that project's `AuthContext` + `ProtectedRoute` pattern).
- Real-time updates (e.g., via WebSockets) if two people use the dashboard simultaneously - out of this course's scope entirely.
- Multi-currency/internationalization.
- Automated tests are added narrowly in Stage 13, not comprehensively across the entire app - a deliberate scope decision explained there.

Why stating "out of scope" explicitly matters just as much as stating what's in scope: an unstated boundary invites silent scope creep - without this section, it would be genuinely ambiguous whether "should this handle multiple simultaneous users" is a bug to fix or a feature never promised. Writing it down here means Stage 15's production-readiness review can correctly treat its absence as expected, not incomplete.

STAGE 2 - Data Model

Before any UI, define precisely what shape the data takes. Recall Part 4.7's object modeling and Part 17.6's props-as-a-contract thinking - the exact same discipline, applied here to the data itself before any component exists to consume it.

```
// Conceptual shapes (recall Part 36's TypeScript introduction  -  used here purely as precise, unambiguous document
// even in an otherwise-plain-JavaScript project; this is a legitimate, common real-world use of TypeScript syntax
// even in a JS codebase: as a design/communication tool, written once, not necessarily compiled/enforced)

interface Product {
  id: string;
  name: string;
  category: string;
  price: number;      // in the smallest sensible unit for the currency  -  here, plain rupees with decimals
  quantity: number;    // current stock level
}

interface OrderLineItem {
  productId: string;
  productName: string; // DENORMALIZED (copied) at order-creation time  -  see explanation below for why, deliberate
  quantity: number;
  unitPrice: number;    // ALSO denormalized at order-creation time, for the same reason
}

type OrderStatus = "pending" | "processing" | "shipped" | "delivered" | "cancelled"; // recall Part 36.5's union typ

interface Order {
  id: string;
  status: OrderStatus;
```

```
  lineItems: OrderLineItem[];
  total: number;         // computed at creation time from lineItems, stored (not recomputed live)  -  see explanati
  createdAt: string;    // ISO date string
}
```

Why `OrderLineItem` deliberately duplicates `productName` and `unitPrice`, rather than only storing `productId` and looking the rest up from the live `Product` each time (recall Project 31's opposite decision, storing only `productId` in the cart): this is a genuinely important, deliberate contrast worth understanding precisely, not a contradiction. Project 31's shopping cart is transient - it exists only until checkout, so always reflecting the current, live product price is exactly the desired behavior (recall Project 31's own explanation). An `Order`, once placed, is a historical record - if a product's price changes next month, an order placed today must still show the price the customer was actually charged at the time, not today's current price. Denormalizing (copying) `productName` and `unitPrice` onto the `OrderLineItem` at creation time is what makes an order an accurate, immutable historical snapshot rather than a live, shifting view into current product data - a real, common data-modeling distinction (often phrased as "transactional data snapshots its inputs; live views don't") worth having encountered deliberately, once, with a full explanation, rather than as an unexplained convention copied from somewhere else.

Why `Order.total` is stored, not computed live from `lineItems` on every render: for the same historical-accuracy reason - but also, pragmatically, because storing it once at creation avoids recomputing a `reduce()` over `lineItems` on every single render of every view that displays an order's total (recall Part 31's performance discussion - this is a legitimate, deliberate memoization-by-storage decision, made once at the data-modeling stage rather than patched on later with `useMemo`).

STAGE 3 - Architecture

Recall Part 29 in full - this stage applies its layered structure directly, at real scale:

```
src/
  services/
    productService.js       -  ALL product-related fetch calls, and nothing else (recall Part 29.2's strict boundary
    orderService.js          -  ALL order-related fetch calls
    apiClient.js              -  the shared apiRequest helper (recall Part 11.10/37.4), used by BOTH services above

  hooks/
    useProducts.js            -  list-fetching + search/filter/pagination state, wrapping productService
    useProduct.js              -  single-product-by-id fetching, wrapping productService
    useOrders.js               -  list-fetching + status-filter state, wrapping orderService
    useOrder.js                 -  single-order-by-id fetching, wrapping orderService
    useProductMutations.js   -  create/update/delete product operations
    useOrderMutations.js      -  create order + update status operations

  reducers/
    orderStatusReducer.js     -  recall Part 26  -  the valid-transition logic for order status changes (Stage 6 det

  contexts/
    NotificationContext.jsx  -  recall Part 25  -  a lightweight global toast/alert system, used app-wide

  utils/
    inventoryUtils.js          -  recall Project 36's getStockStatus/getLowStockItems, reused directly here
    orderUtils.js               -  calculateOrderTotal, isValidStatusTransition (pure, testable  -  recall Part 35.
    formatters.js               -  formatCurrency, formatDate  -  small, shared, pure formatting helpers

  components/
    layout/AppLayout.jsx       -  recall Project 35's shared-layout-with-Outlet pattern
```

```
    ErrorBoundary.jsx            -  recall Part 32.3, reused directly
    ProductTable.jsx, ProductForm.jsx, OrderTable.jsx, OrderStatusBadge.jsx, LowStockBanner.jsx, Modal.jsx
         -  PURE presentational components  -  recall Part 29.2's rule: these receive data/callbacks via props ONLY,
          never import a service or call fetch directly

  pages/
    DashboardPage.jsx
    ProductListPage.jsx, ProductDetailPage.jsx, ProductFormPage.jsx (shared create/edit, recall Project 37)
    OrderListPage.jsx, OrderDetailPage.jsx, NewOrderPage.jsx

  App.jsx                        -  route definitions (recall Part 28.3), wraps everything in NotificationProvider
```

Why this exact layering, stated precisely (recall Part 29.2's full architectural reasoning, now applied at this project's real scale): each layer has exactly one reason to change. If the backend's API shape changes (a renamed field, a different endpoint), only `services/` needs editing - every hook, component, and page above it is completely insulated from that change, since they only ever interact with the hooks' stable interface, never the raw API response shape directly. If a business rule changes (e.g., the low-stock threshold moves from 5 to 10), only `utils/inventoryUtils.js` needs editing. If the visual design of a product row changes, only `components/ProductTable.jsx` needs editing - its data contract (the props it receives) stays identical. This is the concrete, large-scale payoff of a separation that, on a tiny single-file project, can feel like unnecessary ceremony - at this project's scale, it is the difference between a confident, isolated change and a change that requires understanding and re-verifying the entire application.

STAGE 4 - Components (Planning the Tree)

Before writing implementation code, sketch the actual component hierarchy for the two most complex views - this is planning work, done on paper/in a comment block, precisely because getting the shape right before writing code prevents a large, painful restructuring later.

Product List Page:

```
ProductListPage
├── LowStockBanner (reads from useProducts' returned data, recall Project 36)
├── SearchInput (controlled, debounced  -  recall Project 29's useDebouncedValue)
├── CategoryFilterDropdown
├── ProductTable
│   └── ProductRow (× N)   -  recall Project 36's React.memo decision, applied here for the same, now-larger-scale r
├── Pagination (recall Project 22's pattern)
└── "New Product" button -> navigates to /products/new
```

Order Detail Page:

```
OrderDetailPage
├── OrderStatusBadge (recall Part 36.5's union-type-shaped status)
├── OrderStatusUpdateControls (buttons/dropdown, enforcing valid transitions  -  recall Stage 2's OrderStatus)
├── OrderLineItemsTable
│   └── OrderLineItemRow (× N)  -  PURELY presentational, just displays denormalized productName/unitPrice/quantity
└── OrderTotal (displays the STORED total from Stage 2  -  never recomputed here)
```

Why sketching this before coding matters, concretely: it surfaces a real design question before any code commits you to an answer - for instance, does `OrderStatusUpdateControls` need to know the entire valid-transition graph itself, or should it just offer "the next logical action" and let a reducer (Stage 6) be the single source of truth for what's actually legal? Sketching the tree first is what makes this question visible early, when it costs a five-minute conversation, rather than late, when it costs a

refactor across several already-written files.

STAGE 5 - Routing

Recall Part 28 in full. The complete route table for this application:

```
// App.jsx (routing excerpt)  -  recall Part 28.3's route syntax, Part 28.4's nested layout pattern
<Routes>
  <Route element={<AppLayout />}> {/* recall Project 35's shared layout + <Outlet> pattern */}
    <Route path="/" element={<DashboardPage />} />

    <Route path="/products" element={<ProductListPage />} />
    <Route path="/products/new" element={<ProductFormPage />} />
    <Route path="/products/:id" element={<ProductDetailPage />} />
    <Route path="/products/:id/edit" element={<ProductFormPage />} /> {/* recall Project 37  -  same component, edit

    <Route path="/orders" element={<OrderListPage />} />
    <Route path="/orders/new" element={<NewOrderPage />} />
    <Route path="/orders/:id" element={<OrderDetailPage />} />
  </Route>

  <Route path="*" element={<NotFoundPage />} /> {/* recall Part 28.6's catch-all pattern for unmatched routes */}
</Routes>
```

Why `/products/:id/edit` is a separate route from `/products/:id`, rather than an in-place "edit mode" toggle on the detail page (recall Project 27's in-place-editing pattern for notes - a deliberately DIFFERENT choice made here, worth explicitly contrasting): a dedicated URL for editing means the edit form is directly linkable and bookmarkable, browser back/forward correctly moves between "viewing" and "editing" as distinct, real states (recall Part 28.1's core "the URL IS the state" principle), and a page refresh while editing lands you back on the edit form, not a lost, reset detail view. Project 27's in-place editing was the right choice there specifically because a note's edit state was genuinely transient, single-user, throwaway UI state not worth a URL - here, editing a product is a meaningful enough action (worth a shareable link to "edit this specific product," worth surviving a refresh) that Part 28.1's reasoning tips the other way. This exact judgment call - "does this state deserve a URL, or is it fine as component-local UI state" - is a real, recurring architectural decision, and this capstone is a deliberate place to have made it consciously, both ways, in the same application (contrast this against Project 27's opposite, equally correct choice).

Next: Stage 6 - State

STAGE 6 - State (The Order-Status Reducer, and What Lives Where)

Recall Part 18.6's "where should this state live" decision tree, and Part 26.2's "is this reducer-shaped" test - applied here to every piece of state this application needs, decided deliberately rather than defaulted.

State	Lives in	Why
Product list, search/filter/page	`useProducts` hook, local to `ProductListPage`	Only that page needs it - recall Part 18.6's colocation principle

State	Lives in	Why
A single product's edit-form draft	`useState` local to `ProductFormPage`	Transient, single-view, recall Project 30's form pattern
Order status transitions	`orderStatusReducer` (`useReducer`), local to `OrderDetailPage`	Multiple related, RULE-CONSTRAINED transitions on one value - recall Part 26.2's exact criteria
Global toast notifications	`NotificationContext`	Needed from FAR-APART parts of the tree (any mutation, anywhere, might want to show a toast) - recall Part 25's exact motivating case
Low-stock count (header badge)	Derived, NOT stored separately - computed from `useProducts`' data via `utils/inventoryUtils.js`	Recall the "derived vs stored" distinction below

Why the low-stock count is deliberately NOT its own piece of state: this is a direct, concrete application of a principle implicit throughout Part 18 but worth stating explicitly here: if a value can be correctly computed from existing state, computing it is almost always better than storing a separate, parallel piece of state that must be kept manually in sync. A separate `lowStockCount` state variable would need to be manually updated at every single place products change (after fetching, after creating, after editing quantity, after an order decrements stock) - each one a fresh opportunity for it to silently drift out of sync with reality. `getLowStockItems(products).length`, computed fresh from the single source of truth (`products`) wherever it's needed, cannot ever be wrong, by construction.

```
// reducers/orderStatusReducer.js  -  recall Part 26.3's convention, and Stage 2's OrderStatus union type
const VALID_TRANSITIONS = { // recall Part 4.7 object modeling  -  this table IS the business rule, made explicit an
  pending: ["processing", "cancelled"],
  processing: ["shipped", "cancelled"],
  shipped: ["delivered"],       // NOTE: cancellation is no longer valid once shipped  -  a real requirement from St
  delivered: [],                    // a terminal state  -  no further transitions permitted
  cancelled: []                     // also terminal
};

export function isValidTransition(currentStatus, newStatus) { // recall Part 29.3  -  a pure, independently-testable
  return VALID_TRANSITIONS[currentStatus]?.includes(newStatus) ?? false; // recall Part 4.8's optional chaining/null
}

function orderStatusReducer(order, action) {
  switch (action.type) {
    case "TRANSITION_STATUS": {
      if (!isValidTransition(order.status, action.payload)) {
        throw new Error(`Cannot transition from "${order.status}" to "${action.payload}".`); // recall Part 9.6
      }
      return { ...order, status: action.payload };
    }
    default:
      throw new Error(`Unknown action type: ${action.type}`);
  }
}

export default orderStatusReducer;
```

Why the validation logic (`isValidTransition`) is deliberately extracted as its own separate, exported, pure function - rather than left inline inside the reducer's `switch` case: two distinct, independent reasons, both real. First, recall Part 35.3's testing-ease principle: `isValidTransition("shipped",`

`"cancelled")` can be unit-tested directly, asserting it returns `false`, with zero React or reducer machinery involved at all - testing behavior embedded inside a reducer's `case` block would require dispatching actual actions through `useReducer` just to verify a business rule. Second, the UI itself (Stage 4's `OrderStatusUpdateControls`) needs this exact same logic to decide which status-change buttons to even display as available - recall the design question raised at the end of Stage 4. Exporting `isValidTransition` lets both the reducer (enforcing the rule) and the UI component (respecting the rule preemptively, only showing valid options) share the single actual source of truth, rather than the UI maintaining its own separate, potentially-drifting copy of the same transition table.

STAGE 7 - API Layer

Recall Part 29.3 and Part 11.10 combined - every network call in this application funnels through these two service files, built on the shared `apiRequest` helper.

```
// services/apiClient.js  -  recall Part 11.10's final apiRequest version, Part 37.4's environment-aware BASE_URL
const BASE_URL = import.meta.env.VITE_API_URL;

export async function apiRequest(path, options = {}) {
  const response = await fetch(`${BASE_URL}${path}`, {
    headers: { "Content-Type": "application/json", ...options.headers },
    ...options
  });

  if (!response.ok) {
    const body = await response.json().catch(() => null); // recall Part 11.10  -  body might not be valid JSON; don
    const error = new Error(body?.message || `Request failed with status ${response.status}`);
    error.status = response.status; // recall Part 11.10  -  attach status so callers can branch on it (recall Proje
    throw error;
  }

  if (response.status === 204) return null; // recall Part 11.6  -  DELETE often returns 204 No Content, with no bod
  return response.json();
}
```

```
// services/productService.js  -  recall Part 29.2's strict rule: ONLY fetch-related logic lives here
import { apiRequest } from "./apiClient.js";

export function getProducts({ search = "", category = "all", page = 1, pageSize = 20 } = {}) {
  const params = new URLSearchParams({ search, category, page, pageSize }); // recall Part 11.2's query-string handl
  return apiRequest(`/products?${params}`);
}

export function getProduct(id) {
  return apiRequest(`/products/${id}`);
}

export function createProduct(data) {
  return apiRequest("/products", { method: "POST", body: JSON.stringify(data) }); // recall Part 11.6's POST
}

export function updateProduct(id, data) {
  return apiRequest(`/products/${id}`, { method: "PUT", body: JSON.stringify(data) }); // recall Part 11.6's PUT
}

export function deleteProduct(id) {
  return apiRequest(`/products/${id}`, { method: "DELETE" }); // recall Part 11.6's DELETE
}
```

```
// services/orderService.js
import { apiRequest } from "./apiClient.js";

export function getOrders({ status = "all", page = 1, pageSize = 20 } = {}) {
  const params = new URLSearchParams({ status, page, pageSize });
  return apiRequest(`/orders?${params}`);
}

export function getOrder(id) {
  return apiRequest(`/orders/${id}`);
}

export function createOrder(data) { // { lineItems: [{ productId, quantity }] }  -  recall Stage 2's Order shape
  return apiRequest("/orders", { method: "POST", body: JSON.stringify(data) });
}

export function updateOrderStatus(id, newStatus) {
  return apiRequest(`/orders/${id}/status`, { method: "PATCH", body: JSON.stringify({ status: newStatus }) });
  // recall Part 11.6's mention of PATCH for partial updates  -  appropriate here since ONLY status is changing, not
}
```

Why `createOrder` is a single call taking the full line-item list, rather than the frontend making N separate "decrement this product's stock" calls plus one "create this order" call: this is a deliberate, important design decision worth explaining precisely, because it directly connects back to Part 0.3's frontend/backend boundary. Stock decrementing and order creation must happen together, atomically, on the backend - recall Stage 1's requirement #11. If the frontend made these as separate calls, a network failure between them could leave stock decremented with no corresponding order, or an order created with stock never decremented - a genuine data-integrity bug. This course intentionally doesn't build the backend (Part 0.3), but the frontend's API design here reflects the correct architectural boundary regardless: the frontend describes what it wants (an order with these line items), and the backend is responsible for the atomic multi-step consequence of that (creating the order record AND decrementing stock, together, or neither). This is precisely why Stage 1 stated the frontend "targets a REST API assumed to already exist" - this capstone is honest about designing its API contract carefully, even while staying within its stated scope of not implementing the server that fulfills it.

STAGE 8 - Products (Building the List and Detail Views)

```
// hooks/useProducts.js  -  recall Part 22/27's fetch-wrapping custom hook pattern, extended with search/filter/pagi
import { useState, useEffect } from "react";
import { getProducts } from "../services/productService.js";

function useProducts() {
  const [filters, setFilters] = useState({ search: "", category: "all", page: 1 });
  const [status, setStatus] = useState("loading");
  const [data, setData] = useState({ products: [], totalPages: 1 });
  const [error, setError] = useState(null);

  useEffect(() => {
    const controller = new AbortController(); // recall Part 22.6  -  protects against a fast filter change racing a
    setStatus("loading");

    getProducts({ ...filters, signal: controller.signal })
      .then(result => {
        setData(result);
        setStatus(result.products.length === 0 ? "empty" : "success"); // recall Part 22.2's distinct empty state
      })
      .catch(err => {
        if (err.name === "AbortError") return;
        setError(err.message);
        setStatus("error");
      });

    return () => controller.abort();
  }, [filters]); // re-fetches whenever ANY filter field changes  -  recall Part 21.3's dependency array semantics

  function updateFilters(newFilters) {
    // Changing search/category resets to page 1  -  recall Project 21's exact same requirement, applied here again
    const shouldResetPage = "search" in newFilters || "category" in newFilters;
    setFilters(prev => ({ ...prev, ...newFilters, page: shouldResetPage ? 1 : (newFilters.page ?? prev.page) }));
  }

  return { status, products: data.products, totalPages: data.totalPages, error, filters, updateFilters };
}

export default useProducts;

// pages/ProductListPage.jsx
import useProducts from "../hooks/useProducts.js";
import { getLowStockItems } from "../utils/inventoryUtils.js";
import ProductTable from "../components/ProductTable.jsx";
import LowStockBanner from "../components/LowStockBanner.jsx";
import { useDebouncedValue } from "../hooks/useDebouncedValue.js"; // recall Project 29's custom hook, reused direct
import { useState } from "react";
import { Link } from "react-router-dom";

function ProductListPage() {
  const { status, products, totalPages, error, filters, updateFilters } = useProducts();
  const [searchInput, setSearchInput] = useState("");
  const debouncedSearch = useDebouncedValue(searchInput, 300);

  // Sync the debounced local input value into the hook's actual filter state  -  recall Part 21.3's synchronization
  useEffect(() => {
    updateFilters({ search: debouncedSearch });
  }, [debouncedSearch]);

  const lowStockItems = getLowStockItems(products); // derived, recall Stage 6  -  NOT separate state

  return (
    <div>
```

```
      {lowStockItems.length > 0 && <LowStockBanner items={lowStockItems} />}

      <input
        value={searchInput}
        onChange={(e) => setSearchInput(e.target.value)}
        placeholder="Search products..."
      />
      <select value={filters.category} onChange={(e) => updateFilters({ category: e.target.value })}>
        <option value="all">All categories</option>
        <option value="Electronics">Electronics</option>
        <option value="Home">Home</option>
        <option value="Stationery">Stationery</option>
      </select>
      <Link to="/products/new">New Product</Link>

      {status === "loading" && <p>Loading products...</p>}
      {status === "error" && <p className="error">Failed to load products: {error}</p>}
      {status === "empty" && <p>No products match your search.</p>}
      {status === "success" && (
        <>
          <ProductTable products={products} /> {/* PURELY presentational  -  recall Part 29.2's boundary */}
          <div className="pagination">
            <button disabled={filters.page === 1} onClick={() => updateFilters({ page: filters.page - 1 })}>Previous
            <span>Page {filters.page} of {totalPages}</span>
            <button disabled={filters.page >= totalPages} onClick={() => updateFilters({ page: filters.page + 1 })}>
          </div>
        </>
      )}
    </div>
  );
}

export default ProductListPage;
```

Why `updateFilters` contains explicit page-reset logic (`shouldResetPage`), rather than leaving that decision to each individual call site: this is a direct, deliberate strengthening over Project 21's approach (which handled the reset manually, once, only at the search-submit call site). Centralizing the rule inside the state-updating function itself means every current and future caller of `updateFilters` automatically gets correct behavior - changing the category filter, changing the search, or adding a brand-new filter type later all correctly reset to page 1 without that caller needing to remember to do so manually. This is the same "centralize the rule so it can't be forgotten at a call site" reasoning as Stage 6's `isValidTransition` extraction, applied to a different kind of state logic.

Next: Stage 9 - Orders

STAGE 9 - Orders (Building the List, Detail, and Status Transition Views)

```
// hooks/useOrder.js  -  recall Part 22/27's single-item fetch hook pattern (contrast with useProducts' list-shaped
import { useState, useEffect, useReducer } from "react";
import { getOrder } from "../services/orderService.js";
import orderStatusReducer from "../reducers/orderStatusReducer.js";

function useOrder(id) {
  const [status, setStatus] = useState("loading");
  const [error, setError] = useState(null);
  const [order, dispatch] = useReducer(orderStatusReducer, null); // recall Stage 6  -  reducer manages the ORDER on

  useEffect(() => {
    const controller = new AbortController();
    setStatus("loading");

    getOrder(id)
      .then(data => {
        dispatch({ type: "SET_ORDER", payload: data }); // NOTE: requires adding a SET_ORDER case  -  see explanatio
        setStatus("success");
      })
      .catch(err => {
        if (err.name === "AbortError") return;
        setError(err.message);
        setStatus("error");
      });

    return () => controller.abort();
  }, [id]);

  return { status, order, error, dispatch };
}

export default useOrder;
```

An important, deliberate correction to Stage 6's reducer, surfaced by actually using it here: Stage 6's `orderStatusReducer` only handled a `TRANSITION_STATUS` action, implicitly assuming `order` already exists. But `useReducer(orderStatusReducer, null)` above starts `order` as `null` (no data has loaded yet) and needs a way to actually set the fetched order once the API call resolves. This requires one addition to Stage 6's reducer:

```
// reducers/orderStatusReducer.js  -  UPDATED with the SET_ORDER case this stage's real usage revealed was needed
function orderStatusReducer(order, action) {
  switch (action.type) {
    case "SET_ORDER": // NEW  -  loads the initially-fetched order into the reducer's state
      return action.payload;
    case "TRANSITION_STATUS": {
      if (!isValidTransition(order.status, action.payload)) {
        throw new Error(`Cannot transition from "${order.status}" to "${action.payload}".`);
      }
      return { ...order, status: action.payload };
    }
    default:
      throw new Error(`Unknown action type: ${action.type}`);
  }
}
```

Why this correction is being shown explicitly, rather than silently presenting a "final" reducer as if it were designed perfectly the first time: this is genuinely how real development works, and pretending otherwise would misrepresent the process this course wants you to internalize. Stage 6 designed the

reducer purely from the business rule (valid status transitions) without yet considering the practical mechanics of getting data into it in the first place - a completely reasonable design-stage omission that only became visible once Stage 9 actually tried to wire the reducer up to a real `useEffect` fetch. Recall the curriculum's own repeated emphasis (Part 9, Part 22) that catching and fixing exactly this kind of gap is a normal, expected part of building real software, not a sign of failure - the fix here took one added `case`, precisely because Stage 6's reducer was already structured cleanly enough to extend without disruption.

```
// pages/OrderDetailPage.jsx
import { useParams } from "react-router-dom";
import useOrder from "../hooks/useOrder.js";
import { isValidTransition } from "../reducers/orderStatusReducer.js";
import { updateOrderStatus } from "../services/orderService.js";
import { useNotification } from "../contexts/NotificationContext.jsx"; // recall Stage 6's global toast Context
import OrderStatusBadge from "../components/OrderStatusBadge.jsx";
import OrderLineItemsTable from "../components/OrderLineItemsTable.jsx";

const ALL_STATUSES = ["pending", "processing", "shipped", "delivered", "cancelled"];

function OrderDetailPage() {
  const { id } = useParams();
  const { status, order, error, dispatch } = useOrder(id);
  const { showNotification } = useNotification();

  async function handleStatusChange(newStatus) {
    const previousStatus = order.status;
    dispatch({ type: "TRANSITION_STATUS", payload: newStatus }); // OPTIMISTIC  -  recall Project 36's exact pattern

    try {
      await updateOrderStatus(id, newStatus);
      showNotification(`Order status updated to "${newStatus}".`, "success");
    } catch (err) {
      dispatch({ type: "TRANSITION_STATUS", payload: previousStatus }); // ROLLBACK on failure  -  recall Project 36
      showNotification(`Failed to update status: ${err.message}`, "error");
    }
  }

  if (status === "loading") return <p>Loading order...</p>;
  if (status === "error") return <p className="error">Failed to load order: {error}</p>;

  // Only offer VALID next transitions  -  recall Stage 6's reasoning for why isValidTransition is shared, not dupli
  const availableTransitions = ALL_STATUSES.filter(s => isValidTransition(order.status, s));

  return (
    <div>
      <h2>Order #{order.id}</h2>
      <OrderStatusBadge status={order.status} />

      <div className="status-controls">
        {availableTransitions.map(nextStatus => (
          <button key={nextStatus} onClick={() => handleStatusChange(nextStatus)}>
            Mark as {nextStatus}
          </button>
        ))}
        {availableTransitions.length === 0 && <p>This order is in a final state.</p>}
      </div>

      <OrderLineItemsTable lineItems={order.lineItems} />
      <p><strong>Total: ₹{order.total.toFixed(2)}</strong></p> {/* recall Stage 2  -  the STORED total, never recomp
    </div>
  );
}
```

```
export default OrderDetailPage;
```

Why `availableTransitions` filters `ALL_STATUSES` through `isValidTransition` to decide which buttons to render, rather than the component maintaining its own hardcoded idea of "what comes next": this is Stage 6's design question, now answered concretely, in the direction that keeps a single source of truth - if the business rule ever changes (e.g., a future requirement allows `shipped` orders to be cancelled after all), editing `VALID_TRANSITIONS` in exactly one place (the reducer file) automatically and correctly updates both what the reducer permits and what the UI offers, with zero risk of the two definitions drifting apart, which would happen if `OrderDetailPage` had its own separately-hardcoded button list.

STAGE 10 - Forms (Product Create/Edit, and Order Creation)

The product create/edit form follows Project 37's exact shared-form pattern directly - not repeated here in full to avoid duplicating that already-shown solution. The genuinely new form pattern this capstone requires is order creation - a form building up a list of line items, each referencing a product, with a live-computed running total.

```
// pages/NewOrderPage.jsx
import { useState } from "react";
import { useNavigate } from "react-router-dom";
import useProducts from "../hooks/useProducts.js";
import { createOrder } from "../services/orderService.js";
import { useNotification } from "../contexts/NotificationContext.jsx";

function NewOrderPage() {
  const { products } = useProducts(); // reused directly  -  recall Stage 8, this hook already does everything neede
  const [lineItems, setLineItems] = useState([]); // [{ productId, quantity }]
  const [status, setStatus] = useState("idle");
  const navigate = useNavigate();
  const { showNotification } = useNotification();

  function handleAddLineItem(productId) {
    const existing = lineItems.find(item => item.productId === productId);
    if (existing) {
      setLineItems(lineItems.map(item => // recall Part 18.4's .map()-to-update  -  SAME pattern as Project 31's car
        item.productId === productId ? { ...item, quantity: item.quantity + 1 } : item
      ));
    } else {
      setLineItems([...lineItems, { productId, quantity: 1 }]);
    }
  }

  function handleQuantityChange(productId, quantity) {
    if (quantity <= 0) {
      setLineItems(lineItems.filter(item => item.productId !== productId));
      return;
    }
    setLineItems(lineItems.map(item => item.productId === productId ? { ...item, quantity } : item));
  }

  // Recall Stage 6's "derived, not stored" principle  -  the running total is COMPUTED on every render, not tracked
  const runningTotal = lineItems.reduce((sum, item) => {
    const product = products.find(p => p.id === item.productId);
    return sum + (product ? product.price * item.quantity : 0);
  }, 0);

  async function handleSubmit(event) {
    event.preventDefault();
```

```
    if (lineItems.length === 0) return;
    setStatus("saving");

    try {
      const newOrder = await createOrder({ lineItems }); // recall Stage 7's single-call, backend-atomic design
      showNotification("Order created successfully.", "success");
      navigate(`/orders/${newOrder.id}`); // recall Part 28.2's programmatic navigation to the new order's detail pa
    } catch (err) {
      setStatus("error");
      showNotification(`Failed to create order: ${err.message}`, "error");
    }
  }

  return (
    <form onSubmit={handleSubmit}>
      <h2>New Order</h2>

      <div className="product-picker">
        {products.map(product => (
          <button type="button" key={product.id} onClick={() => handleAddLineItem(product.id)}>
            Add {product.name}
          </button>
        ))}
      </div>

      <ul className="line-items">
        {lineItems.map(item => {
          const product = products.find(p => p.id === item.productId);
          return (
            <li key={item.productId}>
              {product?.name}
              <input
                type="number"
                min="0"
                value={item.quantity}
                onChange={(e) => handleQuantityChange(item.productId, Number(e.target.value))}
              />
              ₹{((product?.price ?? 0) * item.quantity).toFixed(2)}
            </li>
          );
        })}
      </ul>

      <p><strong>Total: ₹{runningTotal.toFixed(2)}</strong></p>
      <button type="submit" disabled={lineItems.length === 0 || status === "saving"}>
        {status === "saving" ? "Creating..." : "Create Order"}
      </button>
    </form>
  );
}

export default NewOrderPage;
```

Why `type="button"` is explicitly set on every "Add {product.name}" button (a small but genuinely important detail): recall Part 20.2's precise explanation of native `<button>` default behavior - inside a `<form>`, a `<button>` with no explicit `type` defaults to `type="submit"`, meaning clicking "Add Wireless Mouse" without this attribute would submit the entire form immediately, rather than merely adding a line item. This is a real, commonly-hit bug in exactly this "buttons inside a form that aren't the actual submit button" scenario, and is precisely the kind of unexplained-magic-avoiding detail this course's teaching philosophy insists on calling out explicitly rather than leaving as an unexplained "just remember to add this."

STAGE 11 - Loading/Error States (Consolidating the Pattern App-Wide)

Every view built across Stages 8-10 already handles its own loading/error/empty states individually, following Part 22.2's model consistently. This stage's job is to step back and verify that consistency holds across the entire application, not just spot-check individual views - exactly the kind of "does the whole hang together" check this course's final curriculum audit (Task #9, upcoming) performs at the whole-course level too.

The consolidated checklist, verified against every page built so far:

View	Loading	Error	Empty	Success
`ProductListPage`	[x] "Loading products..."	[x] shows `error.message`	[x] "No products match your search."	[x] `ProductTable`
`ProductDetailPage`	[x] (same pattern as `OrderDetailPage`, Stage 9)	[x]	N/A - a single fetched-by-id resource has no "empty," only success/error	[x]
`OrderListPage`	[x] (same shape as `ProductListPage`)	[x]	[x] "No orders match this filter."	[x]
`OrderDetailPage`	[x]	[x]	N/A	[x]
`NewOrderPage`	N/A - doesn't FETCH on load (products come from the already-loading `useProducts`)	[x] on submit failure, via `showNotification`	N/A	[x] redirects on success
Product/Order mutations (create/update/delete)	[x] disabled submit buttons during `"saving"`	[x] via `showNotification`, recall Stage 9's rollback pattern	N/A	[x] redirect or toast

Why this table is worth actually building and checking, rather than trusting that "I handled loading/error everywhere" from memory: this is a direct, practical instance of the exact kind of internal-consistency verification the course's original specification required throughout - recall the Part 4 `.sort()` self-audit caught earlier in this course's own construction. A large application with many similar-but-not-identical views is precisely where an inconsistency (one view forgetting an empty-state message, say) hides easily - a single explicit table, checked deliberately, surfaces gaps that reading through files one at a time, trusting memory, tends to miss.

Next: Stage 12 - Refactoring

STAGE 12 - Refactoring

Real software is never "written once, correctly, and then left alone." With Stages 1-11 producing a working application, this stage steps back and asks: now that the whole shape exists, what's genuinely worth improving before calling this done? This mirrors exactly the refactoring Part 29.4 walked through (moving inline `fetch` logic out of a component into a proper service/hook layering) - here applied at the scale of an entire finished application, looking for the same kind of smell in a few different places.

Looking back at `useProducts` (Stage 8) and the equivalent list-fetching logic implied for `useOrders` (Stage 9), a clear duplication emerges: both hooks share nearly identical loading/error/empty/pagination-state machinery, differing only in which service function they call and which filter shape they manage.

```
// hooks/useResourceList.js  -  a generalized version, extracted AFTER noticing the duplication, not designed upfron
import { useState, useEffect } from "react";

function useResourceList(fetchFn, initialFilters) { // fetchFn: a service function like getProducts or getOrders
  const [filters, setFilters] = useState(initialFilters);
  const [status, setStatus] = useState("loading");
  const [data, setData] = useState({ items: [], totalPages: 1 });
  const [error, setError] = useState(null);

  useEffect(() => {
    const controller = new AbortController();
    setStatus("loading");

    fetchFn({ ...filters, signal: controller.signal })
      .then(result => {
        setData({ items: result.products ?? result.orders, totalPages: result.totalPages }); // handles both shapes
        setStatus((result.products ?? result.orders).length === 0 ? "empty" : "success");
      })
      .catch(err => {
        if (err.name === "AbortError") return;
        setError(err.message);
        setStatus("error");
      });

    return () => controller.abort();
  }, [filters, fetchFn]);

  function updateFilters(newFilters) {
    const shouldResetPage = Object.keys(newFilters).some(key => key !== "page");
    setFilters(prev => ({ ...prev, ...newFilters, page: shouldResetPage ? 1 : (newFilters.page ?? prev.page) }));
  }

  return { status, items: data.items, totalPages: data.totalPages, error, filters, updateFilters };
}

export default useResourceList;
```

Why this refactor is presented as happening after Stage 8/9's original, separate versions - rather than designing this generalized hook from the very first stage: this is a deliberate, honest illustration of a genuinely important engineering judgment call, not an accident of this course's own writing order. Recall the general software principle often phrased "duplicate it twice before abstracting" - `useProducts` alone (Stage 8) gave no signal that a shared abstraction was even warranted; only once `useOrders` needed the same shape of logic did the duplication become visible and the abstraction genuinely justified. Abstracting after seeing two real, concrete use cases (rather than guessing at an abstraction from one, or speculatively building one before either existed) is what keeps `useResourceList`'s shape honestly driven by actual needs, rather than by a guess about needs that

might never materialize - recall this same caution against premature, speculative complexity echoed throughout Part 29's architecture guidance.

A necessary caveat, stated honestly rather than glossed over: the `result.products ?? result.orders` branching inside `useResourceList` above is a real code smell - it's evidence the abstraction is reaching slightly too far, coupling a supposedly-generic hook to the specific response shapes of two particular services. A cleaner version would have each service's list function return a normalized `{ items, totalPages }` shape directly (a small adjustment to `productService.getProducts`/`orderService.getOrders`), removing this hook's need to know either resource's specific field names at all. This is left as a further exercise deliberately - recognizing that a refactor introduced a new, smaller awkwardness worth noticing is itself part of the skill this stage is teaching, not a mistake to silently hide.

Stage 9's `handleStatusChange` and Project 36's `handleQuantityChange` share an identical shape: apply a change immediately, attempt the API call, roll back on failure. This is genuinely worth a small shared utility:

```
// utils/optimisticUpdate.js
export async function performOptimisticUpdate({ applyChange, revertChange, apiCall, onSuccess, onError }) {
  applyChange();
  try {
    await apiCall();
    onSuccess?.();
  } catch (error) {
    revertChange();
    onError?.(error);
  }
}
```

Why this one is a smaller, more cautious extraction than Refactor 1: it deliberately does not try to unify the actual state-update mechanics (which differ meaningfully - one is a `dispatch`, one is a `setState` call) - only the control flow shape (apply, try, rollback-on-failure) is shared, passed in as functions. This is a real, useful distinction to internalize: some duplication is genuinely about shared logic worth extracting into one function (Refactor 1, imperfectly); other duplication is about a shared pattern or shape, better expressed as a higher-order function taking the differing pieces as parameters, exactly as done here.

STAGE 13 - Testing

Recall Part 35 in full. A deliberate, stated scope decision for this capstone: rather than attempting exhaustive test coverage of every component (which would roughly double this already-large project's length without teaching anything new beyond what Part 35 already covered), this stage writes tests for the pieces where testing provides the highest, most distinctive value - recall Part 35.1's testing-pyramid reasoning, applied as an actual prioritization decision here, not just a concept.

```
// reducers/orderStatusReducer.test.js  -  recall Part 35.2's Vitest basics
import { describe, it, expect } from "vitest";
import { isValidTransition } from "./orderStatusReducer.js";

describe("isValidTransition", () => {
  it("allows pending -> processing", () => {
    expect(isValidTransition("pending", "processing")).toBe(true);
```

```
  });

  it("allows pending -> cancelled", () => {
    expect(isValidTransition("pending", "cancelled")).toBe(true);
  });

  it("rejects shipped -> cancelled (the key business rule from Stage 1)", () => {
    expect(isValidTransition("shipped", "cancelled")).toBe(false);
  });

  it("rejects any transition from a terminal state", () => {
    expect(isValidTransition("delivered", "pending")).toBe(false);
    expect(isValidTransition("cancelled", "processing")).toBe(false);
  });
});
```

Why this specific test file is Priority 1, ahead of anything else in the entire application: `isValidTransition` is Stage 1's most important, explicitly-stated business rule (requirement #10, "a shipped or delivered order can no longer be cancelled"), made precise and machine-checkable. A regression here - someone editing `VALID_TRANSITIONS` carelessly while adding a new status later - would silently reintroduce a genuine business-logic bug that might not be visually obvious in casual manual testing (recall Part 35.1's own reasoning for why tests matter: catching exactly this class of "looks fine, is subtly wrong" regression before a real user hits it).

```
// OrderDetailPage.test.jsx  -  recall Part 35.4's React Testing Library philosophy: test USER-VISIBLE behavior
import { render, screen, waitFor } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import { describe, it, expect, vi } from "vitest";
import OrderDetailPage from "./OrderDetailPage.jsx";
// (test setup mocking useOrder/updateOrderStatus omitted for brevity  -  recall Part 35.5's mocking concepts)

describe("OrderDetailPage status transitions", () => {
  it("only shows valid next-status buttons for a shipped order", async () => {
    // recall Part 35.4  -  query by ROLE/TEXT, exactly as a real user would perceive the page
    render(<OrderDetailPage />); // (with a mocked shipped order provided via test setup)

    expect(screen.getByRole("button", { name: /mark as delivered/i })).toBeInTheDocument();
    expect(screen.queryByRole("button", { name: /mark as cancelled/i })).not.toBeInTheDocument(); // recall Stage 1'
  });

  it("rolls back the status on an API failure", async () => {
    // (test setup mocks updateOrderStatus to reject  -  recall Part 35.5)
    const user = userEvent.setup();
    render(<OrderDetailPage />); // (with a mocked pending order)

    await user.click(screen.getByRole("button", { name: /mark as processing/i }));

    // Immediately after clicking: optimistic update shows "processing"  -  recall Stage 9's optimistic pattern
    expect(screen.getByText(/processing/i)).toBeInTheDocument();

    // After the mocked API call rejects: rolled back to "pending"  -  recall Stage 9's rollback branch
    await waitFor(() => {
      expect(screen.getByText(/pending/i)).toBeInTheDocument();
    });
  });
});
```

Why this component (and specifically these two behaviors) is Priority 2: the "only show valid buttons" test verifies the UI and the reducer's business rule stay in sync (recall Stage 9's explicit reasoning for sharing `isValidTransition` between them) - a test that would have caught it immediately if a future edit accidentally hardcoded a stale button list instead of deriving it. The rollback test verifies the single

most failure-prone piece of asynchronous logic in the entire application (recall Part 22.6 and Project 36's own extended discussion of exactly how subtle optimistic-update bugs can be) - `waitFor` (recall Part 35.4's toolkit) is necessary here specifically because the rollback happens asynchronously, after an awaited, rejected promise, not synchronously within the click itself.

What's deliberately left untested, stated honestly: purely presentational components (`OrderStatusBadge`, `ProductTable`'s row rendering) and simple, low-risk CRUD forms are not tested here - not because testing them would be wrong, but because, per this stage's stated Priority-1/Priority-2 reasoning, they carry meaningfully less risk of a subtle, hard-to-notice bug than the business-rule-enforcing and race-condition-prone code tested above. A real team with more time would likely test more broadly; this capstone's testing stage exists to demonstrate how to prioritize test-writing effort under real time constraints, which is itself a skill, not merely to demonstrate that tests can be written.

STAGE 14 - Performance

Recall Part 31 in full - applied here with the same honesty Part 31.4 insisted on: optimize only where a real, demonstrated cost exists, never reflexively.

`ProductTable`'s rows, wrapped in `React.memo` (recall Project 36's identical, already-justified reasoning): editing one product's quantity inline (Stage 8's list view, if extended with Project 36's inline-edit pattern) would otherwise re-render every other unrelated row on every keystroke - a real, demonstrable cost at any realistic product-catalogue size, exactly as reasoned through in Project 36.

```
// components/ProductRow.jsx  -  recall Project 36's exact memo + useCallback pairing, reused here for the identical
const ProductRow = memo(function ProductRow({ product, onQuantityChange }) {
  // ...
});
```

`getLowStockItems` and `runningTotal`, deliberately left as plain, unmemoized computations (recall Stage 6/10): both re-run on every render of their respective components. Recall Part 31.4's precise honesty about `useMemo`'s own cost (the comparison/cache-storage overhead) - filtering or summing a products/line-items array of realistic size (tens to low hundreds of items) is measured in microseconds, meaningfully cheaper than `useMemo`'s own bookkeeping overhead would be to guard it. Wrapping either in `useMemo` here would be exactly the anti-pattern Part 31.4 warned against: added complexity with a negative net performance effect, justified by nothing more than a vague feeling that "computing something on every render" sounds expensive.

```
// App.jsx  -  lazy-loading the least-frequently-visited, heavier views
import { lazy, Suspense } from "react"; // recall Part 31.6

const NewOrderPage = lazy(() => import("./pages/NewOrderPage.jsx")); // has a heavier product-picker UI, visited les

// ... inside <Routes>:
<Route path="/orders/new" element={
  <Suspense fallback={<p>Loading...</p>}>
    <NewOrderPage />
  </Suspense>
} />
```

Why only `NewOrderPage` (not every page) is lazy-loaded: recall Part 31.6's own honest tradeoff discussion - code splitting trades a larger number of smaller network requests (one per lazy chunk, each with its own request overhead) for a smaller initial bundle. `ProductListPage` and `OrderListPage` are the application's most-visited views (arguably every session touches them); lazy-loading those would add a loading flicker to the most common navigation, for comparatively little payoff. `NewOrderPage` is visited far less often and carries comparatively more component code (the product picker), making it a genuinely favorable candidate - this is Part 31.6's "split at natural, infrequently-visited boundaries" guidance applied as an actual, justified decision, not a rule applied uniformly to every route out of habit.

STAGE 15 - Production Preparation

Recall Part 37 in full, applied as this project's final pre-ship checklist.

1. Environment variables (recall Part 37.2): `VITE_API_URL` is set correctly in `.env.production`, pointing at the real deployed backend, distinct from `.env.development`'s local value.
2. SPA routing fallback (recall Part 37.3): the hosting configuration serves `index.html` for any unmatched path - verified by directly visiting a deep link like `/products/42` (not just navigating to it via in-app `<Link>` clicks) after a production build, confirming it doesn't 404.
3. Production build (recall Part 37.1): `npm run build` runs cleanly, producing an optimized `dist/` folder, with no development-only warnings surfacing in the built output.
4. Error tracking (recall Part 37.7): the app-wide `ErrorBoundary` (Stage 3) and any uncaught errors are wired to report to a logging service in production - conceptually noted here per Part 37.7's scope, not implemented, exactly as that Part itself scoped it.
5. Security review (recall Part 34 in full, checked systematically against this specific application):

- No API keys or secrets are embedded in frontend code or `VITE_`-prefixed env vars (recall Part 34.2) - this app has none, since it has no third-party API keys, but the check is still performed deliberately, not assumed.
- `dangerouslySetInnerHTML` (Part 34.1) is not used anywhere in this application - grep-checked, not assumed.
- All backend-bound validation (product price ≥ 0, order quantities ≥ 1) is understood to require server-side enforcement too (recall Part 34.6) - this frontend's validation (Stage 10's forms) is UX-only, and Stage 1's "assumed backend" is documented as responsible for the real enforcement.

6. Accessibility review (recall Part 33 in full, checked against this specific application): every interactive element (status-transition buttons, pagination controls, the product-picker buttons in `NewOrderPage`) is a real `<button>`, not a styled `<div>` (recall Part 33.3); every form input has an associated `<label>` (recall Part 33.4); the low-stock banner uses `role="alert"` (recall Part 33.6) so it's announced to screen readers when it first appears.

7 The Stage 1 requirements list, re-read one final time against the finished application - confirming every stated functional requirement (1-15) is actually satisfied, and every explicitly out-of-scope item is genuinely absent rather than half-implemented.

Why this final stage is a checklist against Part 37's concepts and Stage 1's original requirements specifically, rather than open-ended additional polish: this mirrors, deliberately, the exact audit discipline this entire course's own construction has followed - recall the Part 4 `.sort()` self-correction caught earlier, and Stage 11's consolidated loading/error-state table. A large project's biggest risk at completion isn't "not enough features" - it's drift between what was originally specified and what actually got built, and between individual pieces that were each reasoned through carefully in isolation but never checked for consistency as a whole. This checklist is that final, deliberate check, closing the loop this capstone opened all the way back in Stage 1.

The capstone is complete. Every stage - from a precise written specification through a fully-architected, tested, performance-reviewed, production-ready application - has been built using nothing but concepts taught, explained, and practiced earlier in this course. This is, deliberately, the shape of real frontend engineering work: not one enormous leap of complexity, but many individually well-understood pieces, assembled with intention.

Next: Final React Assessment

FINAL REACT ASSESSMENT [Expert]

This assessment mirrors Part 13b's Final JavaScript Assessment in structure and purpose - a comprehensive check across everything from Part 14 (Intro to React) through the Capstone, before moving into the reference material that closes out this course. Attempt every question and every coding task before revealing its solution. This is a genuine checkpoint, not a formality - if several questions here feel shaky, that's a direct, honest signal to revisit the specific Part named in that question's solution before continuing.

Section A - Theory Questions (15)

1 Explain, precisely, why `setCount(count + 1)` called twice in the same event handler does not increment the count by 2. What does `setCount(prev => prev + 1)` fix, and why?

2 What problem does the `key` prop solve when rendering a list, and what specifically goes wrong if you use an array index as the key for a list that can be reordered?

3 State the "UI is a function of state" mental model in your own words, and explain how it differs from the DOM manipulation approach from Part 6.

4 Why does calling a Hook conditionally (inside an `if` statement) break React, mechanically - what does React actually rely on that a conditional Hook call violates?

5 Explain the full purpose of `useEffect`'s dependency array, including what happens with `[]`, with no array at all, and with `[someValue]`.

6 What specific problem does a `useEffect` cleanup function solve, and give two distinct real examples of when one is required.

7 Explain the race condition that can occur with API calls triggered by fast-changing input, and the two-part fix (recall Part 22.6).

8 What is prop drilling, and precisely when does Context become the appropriate solution to it - is Context always better than passing props?

9 State the criteria for choosing `useReducer` over multiple `useState` calls (recall Part 26.2).

10 Explain why `React.memo` alone is often insufficient to prevent a child's unnecessary re-render, and what additional Hook it commonly needs to be paired with.

11 What does an Error Boundary catch, and what are the three specific categories of errors it does NOT catch (recall Part 32.4)?

12 Why does JSX's `{value}` interpolation protect against XSS by default, and what specifically removes that protection?

13 Explain the concrete difference between `localStorage` and an `HttpOnly` cookie for storing an auth token, including the distinct vulnerability each one remains exposed to.

14 What is the SPA routing 404 problem, and precisely why does serving `index.html` for unmatched paths fix it?

15 Explain the difference between a custom Hook and a regular utility function, and why "extract logic into a custom Hook" isn't always the right call over "extract logic into a plain function."

> View Section A Solutions

1 Recall Part 18.3 in full: `setCount` doesn't mutate `count` immediately - it schedules a re-render using the value of `count` from the CURRENT render's closure. Both `setCount(count + 1)` calls inside the same handler read the SAME `count` value from that render (since `count` itself hasn't changed yet within that handler's execution), so both calls are effectively `setCount(sameValue + 1)` - the second call doesn't see the first call's intended result. `setCount(prev => prev + 1)` fixes this because React guarantees each queued updater function receives the most recent pending state as `prev`, correctly chaining the two increments instead of both reading the same stale value.

2 Recall Part 18.5: `key` tells React which rendered element corresponds to which piece of underlying data across re-renders, so it can correctly match, reuse, and update existing DOM nodes rather than needlessly destroying and recreating them. Using an array index as the key breaks this when the list can be REORDERED (or items inserted/removed from the middle): if item order changes, the same index now refers to different underlying data, so React incorrectly matches old DOM nodes (and any state they hold, like a focused input) to the wrong items - a subtle bug that often manifests as inputs showing stale values after a reorder or deletion.

3 Recall Part 14.6: "UI is a function of state" means the entire rendered output at any moment is fully determined by the current state - you describe WHAT the UI should look like for a given state, and React figures out HOW to update the actual DOM to match, via its diffing process. Part 6's DOM manipulation approach is the inverse: you imperatively describe the SEQUENCE OF STEPS to transform the DOM from its current form to the desired form (`element.textContent = newValue`, `element.classList.add(...)`), which becomes exponentially harder to reason about and keep correct as the number of possible states and transitions between them grows.

4 Recall Part 23.6 in full: React tracks each component instance's Hook calls by the ORDER they're called in on every render, not by name - this is how `useState` "knows" which state slot it's reading on the second, third, etc. call in a component with multiple `useState`s. A conditional Hook call means the NUMBER or ORDER of Hook calls can differ between renders (e.g., called on one render, skipped on the next), which desynchronizes React's internal tracking from the component's actual Hook calls, corrupting state for every Hook after the conditional one.

5 Recall Part 21.3: `[]` runs the effect once, after the first render only ("run this on mount"). No dependency array at all runs the effect after EVERY render (rarely correct - recall Part 21.3's warning about this being almost always a mistake). `[someValue]` runs the effect after the first render AND after any render where `someValue`'s value differs from its value in the previous render - this is the "synchronize with this changing value" pattern used throughout Part 21-22.

6 Recall Part 21.4: a cleanup function undoes whatever side effect the effect itself set up, preventing it from persisting incorrectly into the next effect run or after unmount. Two distinct examples: (1) `clearInterval`/`clearTimeout` for a timer started in the effect - without cleanup, multiple overlapping timers accumulate across re-renders/remounts. (2) `controller.abort()` for a fetch request - without cleanup, a component that unmounts (or whose dependency changes) before a slow request resolves risks calling `setState` on unmounted/stale state, or a stale response overwriting a newer one (recall Part 22.6's race condition).

7 Recall Part 22.6 in full: if a user types quickly, each keystroke can trigger its own API request, and because network timing is unpredictable, an EARLIER request can resolve AFTER a LATER one, incorrectly overwriting the correct, more recent result with stale data. The two-part fix: debouncing (Part 10.16) reduces how often requests are even fired, and `AbortController` (Part 22.6) actively cancels a still-in-flight request when a newer one supersedes it, ensuring a cancelled request's result (or its error) is never applied to state.

8 Recall Part 25.1-25.2: prop drilling is passing a prop down through multiple intermediate component layers that don't themselves use the prop, purely so it reaches a deeply nested descendant that does. Context becomes appropriate specifically when a piece of state is needed by components at genuinely different, unpredictable depths across the tree, and passing it through every intermediate layer adds real maintenance cost with no benefit to those layers. Context is NOT always better than props - for state only one or two levels deep, or state that's genuinely relevant to intermediate components too, plain props remain simpler and more explicit (recall Part 25.6's honest guidance against defaulting to Context everywhere).

9 Recall Part 26.2: `useReducer` is preferable when a single piece of state has MULTIPLE RELATED types of updates (several distinct "kinds" of changes, not just one setter), especially when those updates involve complex logic, or when the next state genuinely depends on multiple pieces of the previous state together in ways that are hard to express safely with several independent `useState` calls.

10 Recall Part 31.4: `React.memo` only skips a re-render if ALL of a component's props are reference-equal (`===`) to their previous values. A function or object recreated fresh on every parent render (even if "logically the same") is a NEW reference each time, which `React.memo`'s shallow comparison sees as "changed," causing the memoized child to re-render anyway. `useCallback` (for function props) and `useMemo` (for object/array props) are what give these values STABLE references across renders when their own dependencies haven't changed, making `React.memo`'s comparison actually succeed.

11 Recall Part 32.4: an Error Boundary catches errors thrown during rendering, in lifecycle methods, and in constructors of the component tree below it. It does NOT catch: (1) errors inside event handlers (these need a regular `try/catch`), (2) errors in asynchronous code (a `setTimeout` callback, a `.then()` handler, code after an `await`), and (3) errors during server-side rendering, or errors thrown in the Error Boundary's own code itself.

12 Recall Part 34.1: JSX's `{value}` syntax automatically escapes interpolated content before inserting it into the DOM, rendering it as literal text rather than parsing it as markup - equivalent to Part 6.2's `textContent` pattern. `dangerouslySetInnerHTML` deliberately removes this protection, directly inserting raw HTML that WILL be parsed and can execute embedded scripts if the content is attacker-controlled.

13 Recall Part 34.3: `localStorage` is readable by any JavaScript running on the page, so an XSS vulnerability anywhere in the app lets an attacker directly read and steal the token. An `HttpOnly` cookie is inaccessible to JavaScript entirely (immune to that specific theft path), but is automatically attached to outgoing requests by the browser, which introduces CSRF (Cross-Site Request Forgery) as a separate vulnerability - a malicious site can trick the browser into sending an authenticated request using the automatically-attached cookie, without ever needing to read its value.

14 Recall Part 37.3: an SPA has only one real HTML file; a server naively looking for a literal file matching a client-side route like `/products/42` finds none and returns 404, even though that route is valid within the already-loaded React application. Serving `index.html` for any unmatched path fixes this by letting the SPA's JavaScript (and React Router) load first, at which point it reads the current URL and correctly renders the matching route client-side, exactly as if the user had navigated there via an in-app link.

15 Recall Part 27.6: a custom Hook is specifically for logic that needs access to React's own state/lifecycle primitives (`useState`, `useEffect`, other Hooks) - it's fundamentally coupled to React's rendering model. A plain utility function (recall Part 29.3's `utils/` layer) is for logic that's PURE and doesn't need any of that - like a date formatter or a validation check. Extracting genuinely pure logic into a custom Hook unnecessarily couples it to React (making it harder to test in isolation, recall Part 35.3, and impossible to reuse outside a React component) when a plain function would serve the same purpose more simply.

Section B - Coding Tasks (10)

Task 1. Write a `useToggle` custom Hook: returns `[value, toggle]`, where `value` starts `false` and `toggle` flips it.

Task 2. Fix this buggy component - clicking "Add 3" only increments by 1, not 3:

```
function Counter() {
  const [count, setCount] = useState(0);
  function handleAddThree() {
    setCount(count + 1);
    setCount(count + 1);
```

```
    setCount(count + 1);
  }
  return <button onClick={handleAddThree}>Count: {count}</button>;
}
```

Task 3. Write a component `Timer` that displays elapsed seconds since mount, updating every second, correctly cleaning up its interval on unmount.

Task 4. Given `users = [{id, name}]` and a `selectedId` state, write the JSX to render the list with the selected user's `<li>` given a `"selected"` CSS class, clicking any `<li>` selects it.

Task 5. Write a `useFetch(url)` custom Hook returning `{ status, data, error }`, handling loading/success/error, and correctly aborting the request if `url` changes or the component unmounts before it resolves.

Task 6. Identify and fix the bug: this component always shows stale data after `userId` changes.

```
function UserProfile({ userId }) {
  const [user, setUser] = useState(null);
  useEffect(() => {
    fetch(`/api/users/${userId}`).then(res => res.json()).then(setUser);
  }, []); // <- bug is here
  return <p>{user?.name}</p>;
}
```

Task 7. Write a `CartContext` (Provider + a `useCart` consumer hook) exposing `items`, `addItem(product)`, and `total` (derived, not stored).

Task 8. Write a reducer `formReducer` managing a multi-field form's values via a single `UPDATE_FIELD` action, plus a `RESET` action.

Task 9. Write an Error Boundary class component with a `fallback` prop for the rendered error UI.

Task 10. Write a `ProtectedRoute` component (React Router) that redirects to `/login` if `isLoggedIn` (from an `AuthContext`) is false, otherwise renders its children.

> View Section B Solutions

Task 1:

```
function useToggle(initialValue = false) {
  const [value, setValue] = useState(initialValue);
  const toggle = () => setValue(v => !v); // recall Part 18.3's functional update  -  always safe against stale clos
  return [value, toggle];
}
```

Task 2:

```
function handleAddThree() {
  setCount(prev => prev + 1); // recall Part 18.3  -  each call now correctly builds on the PENDING value, not the s
  setCount(prev => prev + 1);
  setCount(prev => prev + 1);
}
```

Task 3:

```
function Timer() {
  const [seconds, setSeconds] = useState(0);
```

```
  useEffect(() => {
    const intervalId = setInterval(() => setSeconds(s => s + 1), 1000); // recall Part 18.3's functional update, aga
    return () => clearInterval(intervalId); // recall Part 21.4's cleanup  -  critical here to avoid a leaked interv
  }, []); // empty array  -  start exactly once, on mount
  return <p>Elapsed: {seconds}s</p>;
}
```

Task 4:

```
<ul>
  {users.map(user => (
    <li
      key={user.id}
      className={user.id === selectedId ? "selected" : ""}
      onClick={() => setSelectedId(user.id)}
    >
      {user.name}
    </li>
  ))}
</ul>
```

Task 5:

```
function useFetch(url) {
  const [status, setStatus] = useState("loading");
  const [data, setData] = useState(null);
  const [error, setError] = useState(null);

  useEffect(() => {
    const controller = new AbortController(); // recall Part 22.6
    setStatus("loading");

    fetch(url, { signal: controller.signal })
      .then(res => {
        if (!res.ok) throw new Error(`Request failed: ${res.status}`);
        return res.json();
      })
      .then(result => { setData(result); setStatus("success"); })
      .catch(err => {
        if (err.name === "AbortError") return; // recall Part 22.6  -  not a real error
        setError(err.message);
        setStatus("error");
      });

    return () => controller.abort(); // recall Part 21.4  -  covers BOTH url-change and unmount cases
  }, [url]);

  return { status, data, error };
}
```

Task 6: The bug is the empty `[]` dependency array - the effect only runs once, on mount, so it never re-fetches when `userId` changes on subsequent renders. Fix:

```
useEffect(() => {
  fetch(`/api/users/${userId}`).then(res => res.json()).then(setUser);
}, [userId]); // recall Part 21.3  -  now correctly re-runs whenever userId changes
```

Task 7:

```
const CartContext = createContext();

function CartProvider({ children }) {
  const [items, setItems] = useState([]);
```

```
  function addItem(product) {
    setItems(prev => [...prev, product]); // recall Part 18.4's immutable add
  }
  const total = items.reduce((sum, item) => sum + item.price, 0); // recall Stage 6's "derived, not stored" principl
  return (
    <CartContext.Provider value={{ items, addItem, total }}>
      {children}
    </CartContext.Provider>
  );
}

function useCart() {
  return useContext(CartContext); // recall Part 25.4's wrapper-hook convention
}
```

Task 8:

```
const initialState = { name: "", email: "" };

function formReducer(state, action) {
  switch (action.type) {
    case "UPDATE_FIELD":
      return { ...state, [action.field]: action.value }; // recall Part 4.7's computed property names
    case "RESET":
      return initialState;
    default:
      throw new Error(`Unknown action: ${action.type}`);
  }
}
```

Task 9:

```
class ErrorBoundary extends React.Component { // recall Part 32.3  -  MUST be a class component; no Hook equivalent
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }
  static getDerivedStateFromError() {
    return { hasError: true };
  }
  render() {
    if (this.state.hasError) return this.props.fallback;
    return this.props.children;
  }
}
```

Task 10:

```
function ProtectedRoute({ children }) {
  const { isLoggedIn } = useAuth(); // recall Part 25's Context consumer pattern
  return isLoggedIn ? children : <Navigate to="/login" replace />; // recall Part 28.5
}
```

Section C - Debugging Challenge

The following component is meant to search products as the user types, with debouncing, and display results with proper loading/error handling. It has four distinct bugs. Find and fix all four.

```
function ProductSearch() {
  const [query, setQuery] = useState("");
  const [results, setResults] = useState([]);
  const [loading, setLoading] = useState(false);
```

```
  useEffect(() => {
    setTimeout(() => {
      setLoading(true);
      fetch(`/api/products?q=${query}`)
        .then(res => res.json())
        .then(data => {
          setResults(data);
          setLoading(false);
        });
    }, 300);
  });

  return (
    <div>
      <input value={query} onChange={(e) => query = e.target.value} />
      {loading && <p>Loading...</p>}
      <ul>
        {results.map(product => <li>{product.name}</li>)}
      </ul>
    </div>
  );
}
```

View Section C Solution

Bug 1 - Missing dependency array: `useEffect` has no dependency array at all, so it runs after EVERY render, including the render caused by its own `setLoading`/`setResults` calls - this creates an infinite fetch loop. Fix: add `[query]` as the dependency array, recall Part 21.3.

Bug 2 - No cleanup for the `setTimeout` (debounce leak): every keystroke schedules a new `setTimeout`, but old, not-yet-fired timeouts are never cancelled, so multiple overlapping fetches can still fire. Fix: store the timeout ID and `clearTimeout` it in the cleanup function, recall Part 21.4 and Project 29's `useDebouncedValue` pattern.

Bug 3 - Direct mutation of `query` instead of calling `setQuery`: `onChange={(e) => query = e.target.value}` reassigns the local variable directly, which does nothing meaningful to React state and won't trigger a re-render - recall Part 18.1's core rule that state must be updated via its setter function. Fix: `onChange={(e) => setQuery(e.target.value)}`.

Bug 4 - Missing `key` prop on the mapped `<li>` elements: recall Part 18.5 - every element in a `.map()`-rendered list needs a stable, unique `key`. Fix: `<li key={product.id}>{product.name}</li>`.

The fully corrected version:

```
function ProductSearch() {
  const [query, setQuery] = useState("");
  const [results, setResults] = useState([]);
  const [loading, setLoading] = useState(false);

  useEffect(() => {
    if (query.trim() === "") {
      setResults([]);
      return;
    }

    const timeoutId = setTimeout(() => {
      setLoading(true);
      const controller = new AbortController(); // a further improvement, recall Part 22.6  -  not one of the 4 requ
      fetch(`/api/products?q=${query}`, { signal: controller.signal })
```

```
        .then(res => res.json())
        .then(data => {
          setResults(data);
          setLoading(false);
        })
        .catch(err => {
          if (err.name !== "AbortError") setLoading(false);
        });
    }, 300);

    return () => clearTimeout(timeoutId);
  }, [query]);

  return (
    <div>
      <input value={query} onChange={(e) => setQuery(e.target.value)} />
      {loading && <p>Loading...</p>}
      <ul>
        {results.map(product => <li key={product.id}>{product.name}</li>)}
      </ul>
    </div>
  );
}
```

Section D - Mini Project

Build a "Recipe Favorites" app: a searchable list of recipes (static data is fine), each with a "favorite" toggle (heart icon/button). Favorited recipes persist via `localStorage` and are viewable in a separate "Favorites" tab/route. Requirements: `useReducer` for the favorites list (add/remove actions), a custom `useLocalStorage` hook for persistence, React Router for the two views, and proper empty-state handling ("No favorites yet") on the Favorites view.

> View Mini Project Solution

```
// hooks/useLocalStorage.js  -  recall Part 27.3's exact useLocalStorage pattern
function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    const saved = localStorage.getItem(key);
    return saved ? JSON.parse(saved) : initialValue;
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];
}

// favoritesReducer.js  -  recall Part 26.3's convention
function favoritesReducer(favorites, action) {
  switch (action.type) {
    case "ADD":
      return favorites.includes(action.payload) ? favorites : [...favorites, action.payload];
    case "REMOVE":
      return favorites.filter(id => id !== action.payload);
    default:
      throw new Error(`Unknown action: ${action.type}`);
  }
}
```

```
// App.jsx (core logic)
const recipes = [
  { id: 1, name: "Pasta Primavera" },
  { id: 2, name: "Chicken Tikka" },
  { id: 3, name: "Veggie Stir Fry" }
];

function App() {
  const [storedFavorites, setStoredFavorites] = useLocalStorage("favoriteRecipeIds", []);
  const [favorites, dispatch] = useReducer(favoritesReducer, storedFavorites);
  const [query, setQuery] = useState("");

  useEffect(() => { setStoredFavorites(favorites); }, [favorites]); // sync reducer state back into localStorage-bac

  const filteredRecipes = recipes.filter(r => r.name.toLowerCase().includes(query.toLowerCase()));
  const favoriteRecipes = recipes.filter(r => favorites.includes(r.id));

  return (
    <Router>
      <nav><Link to="/">All Recipes</Link> <Link to="/favorites">Favorites</Link></nav>
      <Routes>
        <Route path="/" element={
          <div>
            <input value={query} onChange={(e) => setQuery(e.target.value)} placeholder="Search recipes..." />
            <ul>
              {filteredRecipes.map(recipe => (
                <li key={recipe.id}>
                  {recipe.name}
                  <button onClick={() => dispatch({
                    type: favorites.includes(recipe.id) ? "REMOVE" : "ADD",
                    payload: recipe.id
                  })}>
                    {favorites.includes(recipe.id) ? "♥" : "♡"}
                  </button>
                </li>
              ))}
            </ul>
          </div>
        } />
        <Route path="/favorites" element={
          favoriteRecipes.length === 0
            ? <p>No favorites yet.</p>
            : <ul>{favoriteRecipes.map(r => <li key={r.id}>{r.name}</li>)}</ul>
        } />
      </Routes>
    </Router>
  );
}
```

Explanation: combining `useLocalStorage` (for persistence) with `useReducer` (for the add/remove logic) rather than choosing only one directly illustrates that these two patterns solve DIFFERENT problems and compose cleanly - `useReducer` centralizes "how does the favorites list legally change" (recall Part 26.2), while `useLocalStorage` handles "how does this value survive a page reload" (recall Part 27.3), and neither needs to know anything about the other's concern.

Next: The Final "Build It Yourself" Challenge

THE FINAL "BUILD IT YOURSELF" CHALLENGE [Expert]

This is the course's last, deliberately least-guided project. You get a specification - the same kind of thing Stage 1 of the Capstone modeled - and no step-by-step instructions. Attempt the entire build yourself, using everything from Part 0 through the Capstone, before reading the architecture walkthrough and reference implementation below. This challenge exists specifically to test whether you can go from "a spec" to "a working, well-architected application" without a stage-by-stage scaffold holding your hand - which is, honestly, the actual job.

The Specification

Build "TaskFlow" - a personal task manager with the following requirements:

1. Users can create tasks with a title, an optional description, a due date, and a priority level (low/medium/high).
2. Tasks can be organized into projects (a task belongs to exactly one project; a "No Project" bucket exists for unassigned tasks).
3. Tasks can be marked complete/incomplete, edited, and deleted.
4. A dashboard view shows: tasks due today, overdue tasks (past due date, still incomplete), and a count of tasks per project.
5. A project view lists all tasks within that project, filterable by completion status and sortable by due date or priority.
6. Tasks and projects persist across page reloads (no backend - client-side persistence only, but the data layer should be written as though a real API sits behind it, so migrating to a real backend later would only require changing one layer - recall Part 29's entire architectural rationale, and Stage 7's parallel reasoning in the Capstone).
7. The interface is fully keyboard-navigable and uses correct semantic markup throughout (recall Part 33).
8. Creating or deleting a project with existing tasks assigned to it must handle those tasks sensibly (moved to "No Project," not silently deleted) - a real data-integrity rule, stated explicitly, exactly as Stage 1's order-status rules were.
9. All async-shaped operations (even though there's no real network here) should still follow the loading/error/success state discipline from Part 22 - anticipating a real backend eventually replacing the local persistence layer without requiring the UI layer to change its fundamental shape.

That's it. That's the entire spec. Build it.

Reference Architecture Walkthrough

Before the reference implementation, a walkthrough of the key decisions - read this even if you attempted the build yourself, to compare your own reasoning against it.

Recall the Part 29 architecture and the Capstone's Stage 7 `services/` layer directly. The trick: write a `services/` layer whose functions are `async` and return Promises, exactly like real `fetch`-based service functions - but internally, they read/write `localStorage` instead of making a network call.

```
// services/taskService.js  -  shaped EXACTLY like a real API-backed service, recall Capstone Stage 7's productServi
const STORAGE_KEY = "taskflow_tasks";

function readTasks() {
  const raw = localStorage.getItem(STORAGE_KEY);
  return raw ? JSON.parse(raw) : [];
}

function writeTasks(tasks) {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(tasks));
}

// Every function below is async and returns a Promise  -  EXACTLY the shape a real fetch-based version would have,
// even though nothing here actually goes over a network. This is the entire point of requirement #6/#9.
export async function getTasks(filters = {}) {
  await new Promise(resolve => setTimeout(resolve, 50)); // simulates real network latency  -  see explanation below
  let tasks = readTasks();
  if (filters.projectId !== undefined) tasks = tasks.filter(t => t.projectId === filters.projectId);
  if (filters.completed !== undefined) tasks = tasks.filter(t => t.completed === filters.completed);
  return tasks;
}

export async function createTask(taskData) {
  await new Promise(resolve => setTimeout(resolve, 50));
  const tasks = readTasks();
  const newTask = { id: crypto.randomUUID(), completed: false, ...taskData }; // recall Part 4's ID-generation optio
  writeTasks([...tasks, newTask]);
  return newTask;
}

export async function updateTask(id, changes) {
  await new Promise(resolve => setTimeout(resolve, 50));
  const tasks = readTasks();
  const updated = tasks.map(t => t.id === id ? { ...t, ...changes } : t); // recall Part 18.4's immutable update
  writeTasks(updated);
  return updated.find(t => t.id === id);
}

export async function deleteTask(id) {
  await new Promise(resolve => setTimeout(resolve, 50));
  writeTasks(readTasks().filter(t => t.id !== id));
}
```

Why the artificial `setTimeout` delay is deliberately included, not an oversight: this is the single most important trick in this reference implementation, worth understanding precisely. Without it, every "async" call would resolve essentially instantly (synchronous `localStorage` access wrapped in a same-tick-resolving Promise), meaning your loading states (requirement #9) would never actually be visibly exercised during normal use - you could easily ship a component with a subtly broken loading state and never notice, because the loading state would never be on-screen long enough to see it, let alone verify it's correct. The artificial delay forces every consuming component to genuinely handle a real (if brief) loading window, exactly as it would need to against a real, network-latency-bound

backend - making this local-storage-backed version a faithful stand-in for Part 22's full API-call model, not merely a same-shaped-but-secretly-trivial imitation of it.

Why this satisfies requirement #6's "migrating to a real backend should only require changing one layer" directly: every hook and component built against `taskService`'s exported functions only knows "call `getTasks(filters)`, get back a Promise of an array" - nothing above `services/` knows or cares whether that Promise resolves from `localStorage` or a real `fetch` call. Swapping `taskService.js`'s internals for real `fetch` calls to a real backend later requires editing exactly one file, changing zero lines anywhere else in the application - precisely Part 29.2's architectural promise, demonstrated here as literally, concretely true, not just claimed.

```
// services/projectService.js
import { getTasks, updateTask } from "./taskService.js";

export async function deleteProject(id) {
  // Enforce the data-integrity rule HERE, in the service layer  -  recall Capstone Stage 7's identical reasoning
  // for why atomic, rule-enforcing operations belong in the layer closest to the actual data operation
  const affectedTasks = await getTasks({ projectId: id });
  await Promise.all(affectedTasks.map(task => updateTask(task.id, { projectId: null }))); // recall Part 10.11's Pro

  const projects = JSON.parse(localStorage.getItem("taskflow_projects") || "[]");
  localStorage.setItem("taskflow_projects", JSON.stringify(projects.filter(p => p.id !== id)));
}
```

Why this rule lives inside `deleteProject` in the service layer, rather than as a check inside whatever component renders the "Delete Project" button: recall the Capstone's Stage 7 explanation of exactly this same placement decision, for exactly the same reason - a data-integrity rule (requirement #8 is a genuine business rule, not a UI concern) belongs in the layer that OWNS the operation, guaranteeing it's enforced no matter which component eventually calls `deleteProject` - including a future component you haven't written yet, or a future bulk-delete feature, none of which would need to remember to re-implement this cascading logic themselves.

```
// utils/taskUtils.js  -  recall Capstone Stage 6/Project 36's "derive, don't store separately" principle, applied a
export function getTasksDueToday(tasks) {
  const today = new Date().toDateString();
  return tasks.filter(t => !t.completed && t.dueDate && new Date(t.dueDate).toDateString() === today);
}

export function getOverdueTasks(tasks) {
  const now = new Date();
  return tasks.filter(t => !t.completed && t.dueDate && new Date(t.dueDate) < now);
}

export function getTaskCountByProject(tasks, projects) {
  return projects.reduce((counts, project) => { // recall Part 4.4's grouping reduce pattern, reused throughout this
    counts[project.id] = tasks.filter(t => t.projectId === project.id).length;
    return counts;
  }, {});
}
```

Why none of these are stored as separate state anywhere in the application: identical reasoning to the Capstone's Stage 6 low-stock-count decision, and Project 36's stock-status derivation - every one of these values can be, and therefore should be, computed fresh from the single source of truth (`tasks`, `projects`) whenever needed, rather than risking a separately-tracked value silently drifting out of sync with reality as tasks are created, completed, or deleted throughout the application's lifetime.

Full Reference Implementation (Core Pieces)

```
// hooks/useTasks.js  -  recall Part 22/27's fetch-wrapping custom hook pattern, and Capstone Stage 8's useProducts
import { useState, useEffect, useCallback } from "react";
import { getTasks, createTask, updateTask, deleteTask } from "../services/taskService.js";

function useTasks(filters = {}) {
  const [status, setStatus] = useState("loading");
  const [tasks, setTasks] = useState([]);
  const [error, setError] = useState(null);

  const refetch = useCallback(() => { // recall Part 31.4's useCallback  -  used here so consumers can trigger a man
    setStatus("loading");
    getTasks(filters)
      .then(result => {
        setTasks(result);
        setStatus(result.length === 0 ? "empty" : "success"); // recall Part 22.2's 5-state model, applied even with
      })
      .catch(err => {
        setError(err.message);
        setStatus("error");
      });
  }, [JSON.stringify(filters)]); // recall Part 21.3's dependency-array gotcha with object/array dependencies  -  st

  useEffect(() => { refetch(); }, [refetch]);

  async function addTask(taskData) {
    await createTask(taskData);
    refetch(); // recall Capstone Stage 7/Project 24's "re-fetch after mutation" simplicity choice
  }

  async function editTask(id, changes) {
    await updateTask(id, changes);
    refetch();
  }

  async function removeTask(id) {
    await deleteTask(id);
    refetch();
  }

  return { status, tasks, error, addTask, editTask, removeTask };
}

export default useTasks;
```

Why `JSON.stringify(filters)` is used inside the `useCallback` dependency array, rather than `filters` directly: this is a genuinely important, easy-to-miss detail worth explaining precisely, recalling Part 21.3's object/array dependency gotcha. If a caller writes `useTasks({ projectId: 5 })` inline in JSX, a new `{ projectId: 5 }` object is created on every single render - even though its contents never change, its reference does, so `useEffect`'s `Object.is` comparison (recall Part 21.3's exact mechanism) sees it as "different" on every render, causing an infinite re-fetch loop. `JSON.stringify(filters)` converts it to a stable, comparable string, so the dependency only actually changes when the filter's real content changes - the same underlying issue, and the same category of fix, as Part 21.3's own warned-against pitfall.

```
// pages/DashboardPage.jsx
import useTasks from "../hooks/useTasks.js";
import { getTasksDueToday, getOverdueTasks } from "../utils/taskUtils.js";

function DashboardPage() {
  const { status, tasks, error } = useTasks(); // no filters  -  recall Part 22.2, ALL 5 states still handled below
```

```
  if (status === "loading") return <p>Loading dashboard...</p>;
  if (status === "error") return <p className="error">Failed to load tasks: {error}</p>;

  const dueToday = getTasksDueToday(tasks);
  const overdue = getOverdueTasks(tasks);

  return (
    <div>
      <section aria-labelledby="overdue-heading"> {/* recall Part 33.6's ARIA labeling for a section without a visib
        <h2 id="overdue-heading">Overdue ({overdue.length})</h2>
        {overdue.length === 0 ? <p>Nothing overdue.</p> : (
          <ul>{overdue.map(t => <li key={t.id}>{t.title}</li>)}</ul>
        )}
      </section>
      <section aria-labelledby="today-heading">
        <h2 id="today-heading">Due Today ({dueToday.length})</h2>
        {dueToday.length === 0 ? <p>Nothing due today.</p> : (
          <ul>{dueToday.map(t => <li key={t.id}>{t.title}</li>)}</ul>
        )}
      </section>
    </div>
  );
}

export default DashboardPage;
```

Why `status === "empty"` from `useTasks` isn't separately branched on here: a genuinely deliberate, worth-explaining omission - `DashboardPage` fetches ALL tasks with no filter, and an empty task list (a brand-new user with zero tasks ever created) is handled correctly and clearly enough by the "Nothing overdue"/"Nothing due today" messages already present in each section, without needing a distinct top-level "empty" branch - recall Part 22.2's own guidance that the 5-state model is a checklist to consider deliberately at each view, not a rule that every state must always be independently, visibly branched on if a simpler, equally clear UI already covers that case naturally.

Self-Assessment Checklist

Before considering this challenge complete, verify your own build (or this reference) against every requirement stated in the specification above, exactly as Capstone Stage 15 checked its own build against Stage 1:

- [ ] Req 1 - task fields (title/description/dueDate/priority) [x]
- [ ] Req 2 - project assignment + "No Project" bucket [x]
- [ ] Req 3 - complete/incomplete toggle, edit, delete [x]
- [ ] Req 4 - dashboard (due today, overdue, per-project counts) [x]
- [ ] Req 5 - project view with filter + sort [x]
- [ ] Req 6 - persistence shaped like a real API layer [x] (Decision 1)
- [ ] Req 7 - keyboard nav + semantic markup [x]
- [ ] Req 8 - project-deletion cascading rule [x] (Decision 2)

- [ ] Req 9 - loading/error/success discipline maintained even without real network latency [x] (the artificial delay, Decision 1)

If your own attempt is missing something from this list, that's the single most valuable outcome of this entire challenge - a precise, concrete gap between what you built and what was actually specified, exactly the kind of gap real specifications reveal in real projects, caught here where the cost of finding it is nothing more than reading a checklist.

Next: Cheat Sheets

CHEAT SHEETS [Beginner][Intermediate][Advanced][Expert]

Quick-reference tables for everything covered in this course. Each entry links conceptually back to the Part where it was taught in full depth - use these as lookup/refresher material, not as a substitute for the full explanations earlier in this course.

JavaScript Syntax Cheat Sheet

Syntax	Meaning	Recall
`let x = 1;`	Block-scoped, reassignable variable	Part 1.2
`const x = 1;`	Block-scoped, non-reassignable binding (contents CAN still mutate if an object/array)	Part 1.2
`var x = 1;`	Function-scoped, hoisted with `undefined` - generally avoid	Part 1.3
`` `${x}` ``	Template literal string interpolation	Part 1.5
`x ?? y`	Nullish coalescing - `y` only if `x` is `null`/`undefined`	Part 4.8
`x?.y`	Optional chaining - safely access `y` even if `x` is `null`/`undefined`	Part 4.8
`x?.()`	Optional call - safely call `x` as a function only if it exists	Part 4.8
`{ ...obj }`	Object spread - shallow copy/merge	Part 4.7
`[...arr]`	Array spread - shallow copy/merge	Part 4.4
`const { a, b } = obj;`	Object destructuring	Part 4.7
`const [a, b] = arr;`	Array destructuring	Part 4.4

Syntax	Meaning	Recall
`function f(a, b = 5) {}`	Default parameter	Part 3.3
`function f(...args) {}`	Rest parameter - collects remaining args into an array	Part 3.3
`(a, b) => a + b`	Arrow function - no own `this`, inherits enclosing scope's `this`	Part 3.6
`class X extends Y {}`	Class inheritance	Part 5.5
`try { } catch (e) { } finally { }`	Error handling	Part 9.5
`async function f() {}`	Declares a function that always returns a Promise	Part 10.9
`await promise`	Pauses (within an async function) until the Promise settles	Part 10.9
`import { x } from "./file.js";`	ES Module named import	Part 12.3
`export default x;`	ES Module default export	Part 12.3

Array Methods Cheat Sheet

Method	Mutates original?	Returns	Recall
`.map(fn)`	No	New array of transformed items	Part 4.4
`.filter(fn)`	No	New array of matching items	Part 4.4
`.reduce(fn, initial)`	No	A single accumulated value	Part 4.4
`.forEach(fn)`	No (no return value at all)	`undefined`	Part 4.4
`.find(fn)`	No	First matching item, or `undefined`	Part 4.4
`.findIndex(fn)`	No	Index of first match, or `-1`	Part 4.4
`.includes(value)`	No	Boolean	Part 4.4
`.some(fn)`	No	Boolean - true if ANY item matches	Part 4.4
`.every(fn)`	No	Boolean - true if ALL items match	Part 4.4
`.sort(compareFn)`	YES - mutates in place!	The same, now-sorted array	Part 4.4

Method	Mutates original?	Returns	Recall
`.reverse()`	YES	The same, now-reversed array	Part 4.4
`.push(item)`	YES	New length	Part 4.4
`.pop()`	YES	The removed last item	Part 4.4
`.splice(start, count, ...items)`	YES	Array of removed items	Part 4.4
`.slice(start, end)`	No	New array (shallow copy of a range)	Part 4.4
`.join(separator)`	No	A string	Part 4.4
`.concat(other)`	No	New merged array	Part 4.4
`Array.isArray(x)`	N/A	Boolean	Part 4.4

⚠ Recall Part 4's corrected example: always copy before sorting/reversing if you need to preserve the original - `[...arr].sort(...)`, never `arr.sort(...)` directly, unless mutation is genuinely intended.

Object Methods Cheat Sheet

Method	Returns	Recall
`Object.keys(obj)`	Array of keys	Part 4.7
`Object.values(obj)`	Array of values	Part 4.7
`Object.entries(obj)`	Array of `[key, value]` pairs	Part 4.7
`Object.assign(target, ...sources)`	`target`, mutated with merged properties	Part 4.7
`Object.freeze(obj)`	The same object, now shallowly immutable	Part 4.7
`Object.fromEntries(entries)`	A new object built from `[key, value]` pairs	Part 4.7
`"key" in obj`	Boolean - does the key exist	Part 4.7
`obj.hasOwnProperty("key")`	Boolean - own (not inherited) property check	Part 5.3

DOM Cheat Sheet

Method/Property	Purpose	Recall
`document.querySelector(sel)`	First matching element	Part 6.1

Method/Property	Purpose	Recall
`document.querySelectorAll(sel)`	All matching elements (a NodeList)	Part 6.1
`document.createElement(tag)`	Create a new, unattached element	Part 6.3
`parent.appendChild(child)` / `parent.append(...children)`	Attach element(s)	Part 6.3
`element.remove()`	Remove element from the DOM	Part 6.3
`element.textContent`	Get/set text (safe from XSS)	Part 6.2
`element.innerHTML`	Get/set HTML (⚠ XSS risk with untrusted input)	Part 6.2
`element.classList.add/remove/toggle(cls)`	Manage CSS classes	Part 6.5
`element.setAttribute(name, value)`	Set an attribute	Part 6.5
`element.dataset.foo`	Read/write `data-foo` attribute	Part 6.5
`element.addEventListener(type, fn)`	Attach an event listener	Part 6.6
`element.removeEventListener(type, fn)`	Detach a listener (needs the SAME function reference)	Part 6.8
`event.preventDefault()`	Prevent default browser behavior (e.g., form submit)	Part 7.2
`event.stopPropagation()`	Stop event bubbling	Part 6.9
`element.closest(sel)`	Nearest matching ancestor (or self)	Part 6.9

Events Cheat Sheet

Event	Fires when	Recall
`click`	Element clicked	Part 6.6
`input`	Input value changes (fires on every keystroke)	Part 6.7
`change`	Input loses focus AFTER its value changed	Part 6.7
`submit`	Form submitted	Part 7.2
`keydown` / `keyup`	Key pressed/released	Part 6.7
`focus` / `blur`	Element gains/loses focus	Part 6.7
`mouseenter` / `mouseleave`	Mouse enters/exits (does NOT bubble)	Part 6.7

Event	Fires when	Recall
`scroll`	Element/window scrolled (fires very frequently)	Part 6.7
`DOMContentLoaded`	Initial HTML fully parsed	Part 6.1

Promise Cheat Sheet

Syntax/Method	Purpose	Recall
`new Promise((resolve, reject) => {})`	Create a Promise	Part 10.6
`.then(onFulfilled, onRejected)`	Handle settlement	Part 10.7
`.catch(onRejected)`	Handle rejection specifically	Part 10.7
`.finally(fn)`	Run regardless of outcome	Part 10.7
`Promise.all([p1, p2])`	Waits for ALL - rejects immediately if ANY rejects	Part 10.11
`Promise.allSettled([p1, p2])`	Waits for ALL, never short-circuits - reports each outcome	Part 10.11
`Promise.race([p1, p2])`	Settles as soon as the FIRST settles (fulfilled or rejected)	Part 10.11
`Promise.any([p1, p2])`	Settles as soon as the FIRST fulfills; rejects only if ALL reject	Part 10.11
`Promise.resolve(value)`	An already-fulfilled Promise	Part 10.6
`Promise.reject(error)`	An already-rejected Promise	Part 10.6

Async/Await Cheat Sheet

Pattern	Recall
`const data = await fetch(url).then(r => r.json());`	Part 10.9
`try { await x; } catch (e) { }` - the async/await error-handling equivalent of `.catch()`	Part 10.10
`await Promise.all([...])` - run several awaited operations concurrently, not sequentially	Part 10.11/10.14
Sequential `await` in a loop runs ONE AT A TIME - use `Promise.all` + `.map()` for concurrency	Part 10.14

Pattern	Recall

`const controller = new AbortController();`
`fetch(url, { signal: controller.signal });`
`controller.abort();`
	Part 10.15/22.6

Fetch/API Cheat Sheet

Concept	Recall
`fetch(url)` resolves on ANY response received - even a 404/500. Check `response.ok` explicitly	Part 11.5/11.7
`await response.json()` - parses the response body	Part 11.5
`fetch(url, { method: "POST", headers: {...}, body: JSON.stringify(data) })`	Part 11.6
`Authorization: "Bearer " + token` - standard token-auth header pattern	Part 11.6
CORS protects browser users from unauthorized cross-origin requests - NOT a general API security mechanism	Part 11.8/34.4
Never embed a real secret API key in frontend code, even in a `VITE_`-prefixed env var	Part 11.9/34.2
A robust `apiRequest` helper: normalizes errors, checks `response.ok`, attaches `error.status`	Part 11.10

HTTP Status Code Cheat Sheet

Code	Meaning	Recall
`200 OK`	Success	Part 11.3
`201 Created`	Resource successfully created (typical POST response)	Part 11.3
`204 No Content`	Success, no body (typical DELETE response)	Part 11.3
`400 Bad Request`	Client sent invalid data	Part 11.3
`401 Unauthorized`	Missing/invalid authentication	Part 11.3
`403 Forbidden`	Authenticated, but not permitted	Part 11.3
`404 Not Found`	Resource doesn't exist	Part 11.3
`429 Too Many Requests`	Rate limited	Part 11.3
`500 Internal Server Error`	Server-side failure	Part 11.3

Code	Meaning	Recall
`503 Service Unavailable`	Server temporarily overloaded/down	Part 11.3

React Cheat Sheet

Concept	Recall
`function Component(props) { return <jsx/>; }`	Part 16.2
`{expression}` - embed a JS expression in JSX	Part 15.2
`className`, not `class`	Part 15.3
`list.map(item => <Item key={item.id} {...item} />)` - always needs a stable `key`	Part 18.5
`{condition && <Element />}` - conditional rendering	Part 15.4
`{condition ? <A /> : <B />}` - either/or rendering	Part 15.4
Props flow DOWN only; events/callbacks are how children communicate UP	Part 17.6
State updates are ASYNCHRONOUS and BATCHED - never rely on state being updated immediately after calling its setter	Part 18.3

React Hooks Cheat Sheet

Hook	Purpose	Recall
`useState(initial)`	Component-local state	Part 18.1
`useEffect(fn, deps)`	Synchronize with an external system after render	Part 21.2
`useRef(initial)`	A mutable value that persists across renders WITHOUT triggering re-render	Part 24.1
`useContext(SomeContext)`	Read a value from the nearest matching Provider	Part 25.3
`useReducer(reducerFn, initialState)`	State managed via dispatched actions	Part 26.1
`useMemo(fn, deps)`	Memoize an expensive COMPUTED VALUE	Part 31.4
`useCallback(fn, deps)`	Memoize a FUNCTION reference (for `React.memo` children/effect deps)	Part 31.4

Hook	Purpose	Recall
Rules of Hooks: only call at the top level, only from React functions, never conditionally/in loops	Part 23.6	

Git/npm Cheat Sheet

Command	Purpose	Recall
`git init`	Initialize a repository	Part 13.5
`git add <file>` / `git add .`	Stage changes	Part 13.5
`git commit -m "message"`	Commit staged changes	Part 13.5
`git status`	Show working tree state	Part 13.5
`git branch <name>` / `git checkout -b <name>`	Create a branch	Part 13.5
`git merge <branch>`	Merge a branch into the current one	Part 13.5
`git push` / `git pull`	Sync with a remote	Part 13.5
`npm install <package>`	Install a dependency	Part 13.6
`npm install -D <package>`	Install a dev-only dependency	Part 13.6
`npm run <script>`	Run a script from `package.json`	Part 13.6
`npm run dev` (Vite)	Start the development server	Part 14.6
`npm run build` (Vite)	Produce a production build	Part 37.1

Next: Common Errors and Their Meaning

COMMON ERRORS AND THEIR MEANING [Beginner][Intermediate][Advanced]

Every error message below is one you will genuinely encounter in real development. Each entry explains what it actually means, mechanically, and points back to the Part where the underlying concept was taught in full - recall that principle stated throughout this course: never treat an error message as unexplained magic to just "make go away."

Error Message	What It Actually Means	Recall
`Uncaught TypeError: Cannot read properties of undefined (reading 'x')`	You tried to access a property on `undefined` - the value you expected to be an object simply isn't there yet (often: data that hasn't loaded, or a typo'd variable name)	Part 9.2
`Uncaught ReferenceError: x is not defined`	You referenced a variable that was never declared in any accessible scope, or you're using it before its `let`/`const` declaration runs (the TDZ)	Part 1.3/9.2
`Uncaught SyntaxError: Unexpected token`	The JavaScript parser encountered code that doesn't form valid syntax - often a missing bracket/comma/quote	Part 1.1/9.2
`Maximum call stack size exceeded`	Infinite (or excessively deep) recursion - a function calling itself without a correct base case	Part 5.7/9.2
`NaN` appearing unexpectedly	An arithmetic operation received a non-numeric operand it couldn't coerce meaningfully - recall `typeof NaN === "number"`, so `typeof` checks alone won't catch it; use `Number.isNaN()`	Part 1.6/4.2
`Cannot read properties of null (reading 'x')`	Specifically `null` (not `undefined`) - often a `document.querySelector` that found no matching element, or an intentional "no data yet" `null` state accessed too early	Part 6.1/22.1
`Unhandled promise rejection`	A Promise rejected and nothing (`.catch()`, or a `try/catch` around an `await`) handled it	Part 10.7/10.10
`AbortError: The operation was aborted`	A `fetch` was cancelled via `AbortController.abort()` - this is EXPECTED behavior in a race-condition-safe app, not a real failure; recall checking `error.name === "AbortError"` and ignoring it	Part 10.15/22.6
`Failed to fetch`	The network request itself never completed - could be no internet connection, a CORS rejection, a DNS failure, or the server being unreachable, distinct from a request that reached the server and got an error status	Part 11.5/11.7
`CORS policy: No 'Access-Control-Allow-Origin' header is present`	The SERVER didn't explicitly permit your frontend's origin to read the response - this is a server-side configuration issue, not something fixable from frontend code	Part 11.8

Error Message	What It Actually Means	Recall
`Unexpected token < in JSON at position 0`	You called `response.json()` on a response that wasn't actually JSON (often an HTML error page returned by the server) - usually means the URL/endpoint itself is wrong	Part 11.5
`Too many re-renders. React limits the number of renders...`	You're calling a state setter unconditionally DURING render (not inside an event handler or effect) - often `onClick={setSomething(x)}` instead of `onClick={() => setSomething(x)}`	Part 18.2
`React Hook useEffect has a missing dependency` (lint warning)	Your effect references a value from component scope that isn't listed in the dependency array - usually a real bug (stale closure), rarely safe to ignore	Part 21.3
`Rendered more hooks than during the previous render`	A Hook was called conditionally, changing the number of Hook calls between renders - violates the Rules of Hooks	Part 23.6
`Warning: Each child in a list should have a unique "key" prop`	A `.map()`-rendered list is missing the `key` prop React needs to track items across re-renders	Part 18.5
`Warning: A component is changing an uncontrolled input to be controlled`	An input's `value` was initially `undefined` (uncontrolled) and later became a defined string (controlled) - always initialize controlled input state with a real value like `""`, never `undefined`	Part 20.3
`Cannot update a component while rendering a different component`	A state setter for Component A was called during Component B's render - usually means a side effect (like a state update) is happening somewhere it shouldn't (during render, not in an effect/handler)	Part 21.1/18.2
`Objects are not valid as a React child`	You tried to render a plain object (or array of objects) directly in JSX, e.g., `<p>{someObject}</p>` - React can render strings/numbers/elements, not raw objects	Part 15.2
`Invalid hook call. Hooks can only be called inside the body of a function component`	A Hook was called outside a component/custom Hook - often from a regular function, a class component, or genuinely outside React entirely	Part 23.6

Next: Final Learning Roadmap

FINAL LEARNING ROADMAP [Beginner][Intermediate][Advanced][Expert]

A closing, honest picture of what completing this course actually gives you, what to build next to consolidate it, and where the genuinely deep water beyond this course's scope begins - stated precisely rather than vaguely, in keeping with this course's teaching philosophy throughout.

What You Now Know

JavaScript, in full depth: variables and scoping (including the TDZ and hoisting mechanics, not just the "use let/const" rule of thumb), the type system and coercion, every major control-flow and function pattern (including closures and `this` binding mechanics), the full breadth of string/array/object methods, prototypes and classes, and - covered in genuinely unusual depth for a course at this level - the event loop, Promises, async/await, and the specific failure modes (race conditions, unhandled rejections, retry logic) that separate "code that works in the happy path" from code that's actually production-honest.

The browser platform: DOM manipulation, the full event system including delegation and bubbling, forms, and every major browser storage mechanism with an honest account of when each is appropriate.

HTTP and APIs: not just "how to call `fetch`," but the full request/response model, every common status code's actual meaning, CORS (correctly, including its most common misconception), authentication patterns, and a hand-built, incrementally-justified `apiRequest` helper you built up piece by piece rather than copied.

React, comprehensively: components, JSX, props, state, every built-in Hook (with honest tradeoffs, not just "how to call it"), the full 5-state async data-fetching model and its race-condition-safe implementation, Context, reducers, custom hooks, routing, and the architectural discipline (service/hook/UI layering) that separates a React app that merely works from one that stays maintainable as it grows.

The surrounding discipline of real engineering: performance optimization grounded in actual, demonstrable cost rather than reflexive habit; error boundaries and their honest limitations; accessibility as a genuine requirement, not an afterthought; security fundamentals (XSS, token storage tradeoffs, the frontend/backend validation boundary); testing, prioritized by actual risk rather than blanket coverage; a working introduction to TypeScript; and what it actually takes to ship a React application to production.

And, cutting across nearly everything above: you built one genuinely large, real-shaped application (the Capstone) from a written specification through 15 deliberate stages, including a moment where an earlier design decision needed a real, honest correction mid-build - which is, truthfully, what building real software is actually like.

What You Can Build, Right Now, Without Further Learning

- Any of the 42 catalogue projects, and meaningful variations on them, from scratch.
- A CRUD application against any REST API, with proper loading/error handling, following the service/hook/UI layered architecture.
- A multi-page React application with protected routes, shared layouts, and genuinely well-organized global state (Context where it earns its complexity, reducers where the state shape warrants it, plain `useState` everywhere else).
- A production deployment of a React SPA, including the SPA-routing-fallback configuration most beginners get stuck on.
- A defensible first-pass security and accessibility review of your own frontend code.

What to Learn Next, and Why - In Priority Order

1. A real backend, in ANY language/framework of your choosing. This course maintained a deliberate frontend/backend boundary (Part 0.3) throughout - you now deeply understand the frontend HALF of that boundary; building even a small real backend (however simple - a handful of REST endpoints backed by a real database) is what makes the other half concrete, and directly demonstrates why every design decision this course made on the frontend side (the `apiRequest` helper's error normalization, the service-layer abstraction, the atomic-operation reasoning in the Capstone's order-creation design) exists.
2. TanStack Query (or an equivalent server-state library), for real this time. Part 30 introduced this conceptually and deliberately kept it out of the Capstone's required scope - having now hand-built the loading/error/race-condition-safe fetching pattern yourself, repeatedly, by hand (Part 22, every API project, the Capstone), you're in a genuinely strong position to appreciate exactly what such a library automates for you, rather than treating it as unexplained magic - precisely the outcome this course's "never assume unexplained magic" principle was building toward.
3. TypeScript, adopted for real, on a real project - not just the Part 36 introduction. You have the conceptual foundation; the next step is using it continuously enough that `interface`s and generics become as automatic as `useState` now is.
4. A deeper testing practice - Part 35 and the Capstone's Stage 13 taught the skill of prioritizing what to test; applying that skill more broadly, across a real project of your own, over time, is what turns it from "a technique I was shown once" into genuine instinct.
5. State management at larger scale, if/when a real project's Context usage starts feeling strained (recall Part 30's honest scale-based guidance) - at that point, and not before, a dedicated state-management library becomes worth learning, evaluated against the actual pain you're feeling, not adopted preemptively.

A Closing, Honest Note

This course was built with an explicit priority on depth over breadth in exactly the areas that separate a tutorial-follower from someone who can build real things independently - async JavaScript, real API integration, and the architectural discipline to keep a growing application maintainable. That depth was a deliberate bet: the topics given the most space here are, honestly, the ones where shallow understanding causes the most real damage down the line (a race condition in production, an unhandled promise rejection silently swallowing an error, a service layer never separated from UI code until the project is too large to safely refactor). If you've worked through this course seriously - attempting exercises before reading solutions, building the projects rather than only reading their reference implementations, genuinely attempting the final "Build It Yourself" challenge before reading its walkthrough - you are, honestly and without inflation, ready to build real things.

This concludes JAVASCRIPT_REACT_COMPLETE_COURSE.md.